

从 C++ 到 AI 计算：第三册实践与代码卷

本地量化推理引擎、完整源码、KV Cache、后端优化与验收

CPU Performance Study

本卷是《从 C++ 到 AI 计算：第三册》的配套实践与代码卷，围绕 `linux_cpu_inference` 贯穿项目展开：模型资产、tokenizer、张量布局、int8 kernel、reference decoder、KV cache、trace、7B 账本、服务调度、CPU/GPU 后端边界和工程验收都要落到可构建代码、完整源码、测试和报告。

前言

第三册原理卷把 AI 推理系统拆成模型资产、typed graph、张量布局、量化、KV cache、后端、服务运行时和验收体系。本实践与代码卷只做一件事：把这些概念压到同一个可运行项目里，并把完整源码阅读路径并入同一目录，让读者知道每个概念对应哪段代码、哪条命令、哪种输入、哪份报告和哪条失败路径。

贯穿项目是 books/cpu-volume-3/labs/linux_cpu_inference。它不是玩具目录，而是第三册已经持续维护的本地 CPU 量化推理切片：包含教学模型格式、tokenizer、safetensors manifest、gate、int8 linear、packed/AVX2 路径、tiny reference decoder、GPT-2 reference path、KV cache trace、7B compute ledger、serving SLO probe、自研 GPT-2 smoke、真实 small 开源模型 smoke、CLI、benchmark 和 CTest。本卷不维护第二份实现；源码章节通过 [lstinputlisting](#) 直接读取同一份工程文件，所有练习、讲解和完整代码都回到同一份源码。

本卷的学习顺序是从小证据到大系统：先让一个 tiny MLP 算对，再解释张量地址流和 int8 累加；再让 reference decoder、sampler 和 KV cache trace 固定语义；然后接模型导入、shape verifier、memory planner 和 IR lowering；最后用 CPU/GPU 后端边界、工业专项和回归门禁收束。代码走读不再被放到最后当附录，而是插入到相同主题部分中：讲完项目入口就读构建文件和调用链，讲完量化就读公共合同、tiny MLP 和 int8 kernel，讲到真实模型时读 tokenizer、safetensors 和 GPT-2，收束验收时读工具、benchmark、smoke 和测试。每章都要求读者交付代码运行结果或报告字段，而不是只抄概念定义。

核心思想

第三册实践的目标不是“写一个看起来像大模型的 demo”，而是建立一条能被复查的推理证据链：资产能校验，shape 能验证，kernel 能对照，trace 能定位，性能能降级，服务指标能解释，回归门禁能长期维护。

常见缩写和术语先导

缩写	全称	本卷中的含义
AI	Artificial Intelligence	这里特指本地推理工程，不讨论泛泛应用口号。
LLM	Large Language Model	decoder-only 文本生成模型是本卷主线。
KV Cache	Key/Value Cache	attention 历史 K/V 状态，属于请求生命周期。
GEMM	General Matrix Multiply	矩阵乘，是预填充和训练常见核心算子。
GEMV	General Matrix Vector Multiply	矩阵向量乘，是单 token decode 的核心形态。
SIMD	Single Instruction Multiple Data	一条指令处理多个 lane 的 CPU 执行方式。
AVX2	Advanced Vector Extensions 2	x86-64 上的 256-bit SIMD 指令集之一。
FMA	Fused Multiply-Add	乘加融合指令，常出现在浮点 kernel 中。
SLO	Service Level Objective	服务目标，例如 TTFT、TPOT、p95/p99。
TTFT	Time To First Token	从请求进入到首 token 输出的时间。
TPOT	Time Per Output Token	decode 阶段每个输出 token 的平均或尾部时间。
IR	Intermediate Representation	编译器内部表示，用于 pass、lowering 和计划生成。
RAG	Retrieval-Augmented Generation	检索增强生成，需要把外部文档和 prompt 编排起来。
MoE	Mixture of Experts	多专家模型，会改变路由、负载和内存账本。
LoRA	Low-Rank Adaptation	低秩适配权重，常用于微调和多租户加载。

这些缩写不要死记。每个缩写在正文里都要落到一个代码对象或报告字段：KV cache 对应 trace 里的 slot 和 position，SLO 对应 serving probe 的 CSV，IR 对应 executable plan，SIMD/AVX2 对应 kernel 反汇编和 benchmark。

本卷结构

本卷按第三册原书的四个大块组织，但目录不再采用“前面做实践、后面贴源码”的割裂方式。每个大块都把实践任务、代码走读和验收证据放在同一条主线里：[linux_cpu_inference](#)。

第一部分从模型资产到量化 kernel。你会运行 tiny MLP，先读构建入口、项目边界和调用链总图，再检查 tokenizer 和 safetensors manifest，手算张量地址，比较 scalar、packed 和 AVX2 路径，并在公共合同、tiny MLP 与 int8 kernel 源码里解释 scale、accumulator 和 requantize 的边界。

第二部分进入 Transformer、KV cache、sampler 和服务运行时。你会跑 reference decoder trace，解释每个 checkpoint 的 shape、stride、dtype 和 layout，构造 KV cache 状态表，分析 admission、queue wait、TTFT、TPOT、取消和拒绝。

第三部分处理模型导入、真实模型加载闭环、memory planner 和 IR lowering。你会把 manifest、shape verifier、tensor role、state edge、workspace、liveness 和 executable plan 写成可检查报告，并在同一部分中读 reference decoder、tokenizer、safetensors 和 GPT-2 reference path 的完整实现，显式运行 LCQI 自研 GPT-2 safetensors smoke 和一个 small 级别开源 GGUF 模型 smoke，而不是只停留在 tiny fixture。

第四部分收束到后端优化和验收。你会比较 CPU kernel 证据、GPU 后端边界、工业专项能力和回归门禁，并直接读工具、smoke、benchmark、ledger、serving probe 和测试源码。最终交付不是“跑通一次”，而是可复查的工程档案：命令、输入、输出、trace、benchmark、不可验证边界和回归规则。

目录

前言	ii
常见缩写和术语先导	iii
本卷结构	iv
第一部分 推理链路、张量账本与量化	
第 1 章实践：端到端推理链路和项目总入口	2
一、对应原书问题：概念必须落到对象和命令	2
二、从特例推到规律：先抓一个能手算的切片	2
三、实践任务：把观察固定成报告字段	2
四、验收和递进大作业	2
代码走读：构建入口、项目边界与调用链总图	3
源码阅读覆盖范围	3
根构建入口	3
LCQI 目标定义	6
项目说明与模型资产	9
为什么要先讲调用链	17
一、全工程入口：哪些文件决定能不能复现	17
二、Tiny MLP 调用链：最短闭环怎样跑通	17
三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照	18
四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门	19
五、GPT-2 调用链：从 HuggingFace 资产到生成 token	19
六、报告、服务探针、外部对标和测试	20
第 2 章实践：张量布局、地址流与第一个 MatVec	22
一、对应原书问题：概念必须落到对象和命令	22
二、从特例推到规律：先抓一个能手算的切片	22
三、实践任务：把观察固定成报告字段	22
四、验收和递进大作业	22
第 3 章实践：GEMM、packing 与 SIMD 证据	23
一、对应原书问题：概念必须落到对象和命令	23
二、从特例推到规律：先抓一个能手算的切片	23
三、实践任务：把观察固定成报告字段	23
四、验收和递进大作业	23
第 4 章实践：int8 量化、累加器与重新量化	24
一、对应原书问题：概念必须落到对象和命令	24
二、从特例推到规律：先抓一个能手算的切片	24
三、实践任务：把观察固定成报告字段	24
四、验收和递进大作业	24
代码走读：公共合同、Tiny MLP 与 Int8 Kernel	25
为什么先读头文件	25
Tiny MLP 与推理入口	25
Int8 Kernel、Tokenizer 与 Safetensors	26
Reference Decoder 与 GPT-2 合同	30
最短闭环：从模型文件到 logits	38
模型加载与基础推理	38
Int8 Kernel 基础实现	45
第二部分 Transformer、KV Cache 与服务运行时	
第 5 章实践：Transformer 切片与 KV Cache 状态	57
一、对应原书问题：概念必须落到对象和命令	57
二、从特例推到规律：先抓一个能手算的切片	57
三、实践任务：把观察固定成报告字段	57
四、验收和递进大作业	57
第 6 章实践：Reference Decoder、logits 与采样	58
一、对应原书问题：概念必须落到对象和命令	58
二、从特例推到规律：先抓一个能手算的切片	58
三、实践任务：把观察固定成报告字段	58

四、验收和递进大作业	58
第 7 章实践：服务调度、SLO 与资源预算	59
一、对应原书问题：概念必须落到对象和命令	59
二、从特例推到规律：先抓一个能手算的切片	59
三、实践任务：把观察固定成报告字段	59
四、验收和递进大作业	59
第 8 章实践：RAG、Agent 与状态图边界	60
一、对应原书问题：概念必须落到对象和命令	60
二、从特例推到规律：先抓一个能手算的切片	60
三、实践任务：把观察固定成报告字段	60
四、验收和递进大作业	60
第三部分 模型导入、AI 编译器与内存计划	
第 9 章实践：模型导入、manifest 与 Shape Verifier	62
一、对应原书问题：概念必须落到对象和命令	62
二、从特例推到规律：先抓一个能手算的切片	62
三、实践任务：把观察固定成报告字段	62
四、验收和递进大作业	62
第 10 章实践：真实模型加载闭环与首 Token Trace	63
一、对应原书问题：概念必须落到对象和命令	63
二、从特例推到规律：先抓一个能手算的切片	63
三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通	63
四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来	64
五、实践任务：把观察固定成报告字段	65
六、验收和递进大作业	65
代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2	66
从 tiny MLP 走向 decoder-only 模型	66
Tiny Reference Decoder	66
Tokenizer 与 Safetensors 实现	83
GPT-2 Reference Path	99
一次可审查的 cached-KV 优化	99
第 11 章实践：运行时内存计划、liveness 与 Workspace	165
一、对应原书问题：概念必须落到对象和命令	165
二、从特例推到规律：先抓一个能手算的切片	165
三、实践任务：把观察固定成报告字段	165
四、验收和递进大作业	165
第 12 章实践：IR、Pass、Lowering 与 Executable Plan	166
一、对应原书问题：概念必须落到对象和命令	166
二、从特例推到规律：先抓一个能手算的切片	166
三、实践任务：把观察固定成报告字段	166
四、验收和递进大作业	166
第四部分 后端优化、工业专项与验收	
第 13 章实践：CPU 后端、微内核与性能证据	168
一、对应原书问题：概念必须落到对象和命令	168
二、从特例推到规律：先抓一个能手算的切片	168
三、实践任务：把观察固定成报告字段	168
四、验收和递进大作业	168
第 14 章实践：GPU 后端边界、copy 与同步成本	169
一、对应原书问题：概念必须落到对象和命令	169
二、从特例推到规律：先抓一个能手算的切片	169
三、实践任务：把观察固定成报告字段	169
四、验收和递进大作业	169
第 15 章实践：工业 LLM 专项能力对照	170
一、对应原书问题：概念必须落到对象和命令	170
二、从特例推到规律：先抓一个能手算的切片	170
三、实践任务：把观察固定成报告字段	170
四、验收和递进大作业	170
第 16 章实践：工程验收、回归门禁与最终项目	171
一、对应原书问题：概念必须落到对象和命令	171
二、从特例推到规律：先抓一个能手算的切片	171
三、实践任务：把观察固定成报告字段	171

四、验收和递进大作业	171
代码走读：工具、Smoke、Benchmark 与回归测试	172
工具不是附属品	172
Benchmark、Trace、Ledger 与 Serving Probe	172
外部模型 Smoke 与对比脚本	187
完整测试	225
实践与代码总索引	245
一、实践主题到代码走读的合并位置	245
二、最小复现命令	245
术语表	246

第零部分

推理链路、张量账本与量化

第 1 章实践：端到端推理链路和项目总入口

本章对应原书中的第三册“端到端链路：从模型文件到下一个 token”。不要把它当作概念复述，本章要把模型目录、manifest、tokenizer、typed graph、backend plan、request session、KV cache、oracle 和 profiler 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `tiny_mlp_i8.txt`，核心命令或测试入口是 `lcqi_cli`。

Listing 1.1 第 1 章实践：端到端推理链路和项目总入口的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / tiny_mlp_i8.txt
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：只跑一个 tiny MLP 推理请求。读者要先手写预期结果，再运行项目观察输出。动作是：把 `readfiles->tokenize->run_inference->logits->predicted_class` 串起来。如果输出里没有 `predicted_class` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型文件存在不等于推理链路可信；只有 CLI 输出、测试断言、输入合同和失败路径同时成立，才算完成入口闭环这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：最终大作业的总目录：每章报告都放入 `reports/volume3-practice/`，第一章提交项目地图、命令记录和一次成功/失败输入对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：构建入口、项目边界与调用链总图

源码阅读覆盖范围

LCQI 的源码边界由三个层次组成。第一层是仓库构建入口，它决定 CMake 怎样把第三册的实验代码纳入构建和测试。第二层是 `labs/linux_cpu_inference`，它包含真正的库、CLI、benchmark、trace、ledger、serving probe 和测试目标。第三层是工具脚本，它们下载或复用外部模型资产，生成 smoke 和 benchmark 报告。

源码阅读任务

先读本章的 CMake 文件，再读 README。读者要能回答三个问题：一是 `lcqi` 静态库包含哪些实现文件；二是哪些可执行文件只依赖自研代码，哪些依赖可选外部库；三是真实 GPT-2 和 SmolLM2 smoke 为什么不进入默认 CTest。

根构建入口

第三册根目录的 CMake 文件只做项目级选择：是否构建第三册 labs，以及如何把 `labs/linux_cpu_inference` 接入。它应该保持薄层，避免把具体目标定义分散到多个地方。

Listing 1.2 cpu-volume-3/README.md

```

1 # 从 C++ 到 AI 计算：第三册
2
3 第三册用于承接 AI、AI 编译器、算子开发和本地大模型推理引擎。第二册只讨论硬件原
  ↳ 理、多核、高性能计算、并发、锁、无锁数据结构、并行算法和分布式计算，不再
  ↳ 直接进入 AI。
4
5 本册贯穿项目是一台能推理开源 7B 左右 decoder-only 大模型的本地引擎，并把 AI 编
  ↳ 译器作为引擎内部主线：模型图导入、typed tensor IR、算子融合、内存规划、
  ↳ CPU/GPU lowering、kernel selection 和 runtime dispatch 都要能落回编译原理
  ↳ 与底层机器账本。工程路线从 CPU 上可解释的 reference 和量化切片开始，逐步
  ↳ 扩展到 tokenizer、模型加载、Transformer 层、KV cache、CPU SIMD/多线程后
  ↳ 端、GPU 后端、正确性报告和性能对标。
6
7 正式书稿工程位于：
8
9 ```text
10 source/latex/
11 ```
12
13 构建命令：
14
15 ```bash
16 make -C source/latex check
17 make -C source/latex pdf
18 make -C source/latex epub
19 ```
20
21 生成文件：
22
23 ```text

```

```

24 source/latex/main.pdf
25 source/latex/main.epub
26 ```
27
28 当前保留的第一阶段切片：
29
30 ```text
31 labs/linux_cpu_inference/
32 ```
33
34 该目录目前包含最小 MLP/int8 权重教学切片、tiny reference decoder、GPT-2 ↵
    ↵ reference path、int8 scalar baseline、packed layout、AVX2 SIMD 路径和 ↵
    ↵ shape sweep benchmark，用来验证张量、线性层、量化权重、tokenizer、↵
    ↵ safetensors、GPT-2 forward、KV cache、oracle 和 benchmark 的基本边界。当↵
    ↵ 前 x86-64 AVX2 结果见：
35
36 ```text
37 results/lcqi-x86-avx2-benchmark-2026-06-25.csv
38 results/lcqi-x86-avx2-decode4096-2026-06-25.csv
39 ```
40
41 自研 GPT-2 smoke 入口如下：
42
43 ```bash
44 make cpu3-gpt2-smoke
45 make cpu3-gpt2-benchmark-compare
46 ```
47
48 第一条命令下载 `openai-community/gpt2` 的标准 safetensors 资产到用户缓存目录，↵
    ↵ 用 LCQI C++ reference path 生成 1 个 token，并把报告写入：
49
50 ```text
51 results/lcqi-gpt2-smoke.txt
52 ```
53
54 第二条命令用同一个 GPT-2 F32 模型对比 LCQI full-prefix 和 cached-KV 生成路径，↵
    ↵ 并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考，报告写入：
55
56 ```text
57 results/lcqi-gpt2-benchmark-compare.txt
58 ```
59
60 它仍然不是生产级 AI Infra。生产级门禁见：
61
62 ```text
63 reports/ai-infra-maturity-audit.md
64 ```
65
66 第三册后续必须把 lab 推进到“小型 CPU 推理 runtime + 手写高性能算子 + 严格 ↵
    ↵ benchmark 报告 + 真实系统对标”，否则只能证明系统学习，不能证明生产级能↵
    ↵ 力。

```

Listing 1.3 cpu-volume-3/CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3 LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 option(CPU_VOLUME_3_BUILD_LABS "Build CPU Volume 3 labs" ON)
10
11 if(CPU_VOLUME_3_BUILD_LABS)
12     enable_testing()
13     add_subdirectory(labs/linux_cpu_inference)
14 endif()

```

实践与代码卷也提供一个薄 CMake 入口，它把同一份 LCQI 源码作为可测试对象接入。这样本卷不会复制实现，只复用第三册主工程。

Listing 1.4 cpu-volume-3-practice/CMakeLists.txt

```

1 cmake_minimum_required(VERSION 3.31)
2 project(CPUVolume3Practice LANGUAGES CXX)
3
4 set(CMAKE_CXX_STANDARD 20)
5 set(CMAKE_CXX_STANDARD_REQUIRED ON)
6 set(CMAKE_CXX_EXTENSIONS OFF)
7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
8
9 enable_testing()
10 add_subdirectory(..cpu-volume-3/labs/linux_cpu_inference linux_cpu_inference)

```

本卷自身的 Makefile 负责构建 PDF/EPUB，并通过 `test` 目标回到第三册真实 LCQI 工程做 CMake/CTest 验证。实践与代码卷如果只排版、不测试，就不能宣称“完整源码可运行”。

Listing 1.5 cpu-volume-3-practice/Makefile

```

1 LATEX_DIR := source/latex
2
3 .PHONY: all check pdf epub test clean
4
5 all: pdf epub
6
7 check:
8     $(MAKE) -C $(LATEX_DIR) check
9
10 pdf:
11     $(MAKE) -C $(LATEX_DIR) pdf
12
13 epub:
14     $(MAKE) -C $(LATEX_DIR) epub
15

```

```

16 test:
17     cmake -S . -B build/lcqi-debug -DCMAKE_BUILD_TYPE=Debug
18     cmake --build build/lcqi-debug
19     ctest --test-dir build/lcqi-debug --output-on-failure
20
21 clean:
22     $(MAKE) -C $(LATEX_DIR) clean

```

Listing 1.6 cpu-volume-3-practice/source/latex/Makefile

```

1 LATEXMK ?= latexmk
2 ENGINE ?= -xelatex
3 MAIN ?= main.tex
4 EPUB_BUILDER ?= ../../../../cpu-volume-1/source/latex/scripts/build_epub.py
5
6 .PHONY: all pdf epub clean check
7
8 all: pdf epub
9
10 pdf:
11     $(LATEXMK) $(ENGINE) -interaction=nonstopmode -halt-on-error $(MAIN)
12
13 epub:
14     @BOOK_TITLE="从 C++ 到 AI 计算：第三册实践与代码卷" \
15     BOOK_IMPORT_TITLE="CPU Volume 3 Practice Code" \
16     BOOK_ID_SEED="cpu-volume-3-practice-code-weread-20260629" \
17     EPUB_ASCII_TITLES=1 \
18     BOOK_SUBTITLE="本地量化推理引擎、完整源码、KV Cache、后端优化与验收" \
19     BOOK_AUTHOR="CPU Performance Study" \
20     python3 $(EPUB_BUILDER) --book-dir . --output main.epub --no-cover-page --↵
    ↵ forbid-images --linearize-tables --wechat-compatible --no-embed-fonts
21
22 clean:
23     $(LATEXMK) -C
24
25 check:
26     @python3 scripts/check_inputs.py
27     @command -v xelatex >/dev/null 2>&1 || { echo "missing xelatex"; exit 1; }
28     @command -v latexmk >/dev/null 2>&1 || { echo "missing latexmk"; exit 1; }

```

LCQI 目标定义

`linux_cpu_inference/CMakeLists.txt` 是代码走读中的第一份核心文件。这里能看到 `lcqi` 库、`lcqi_cli`、`lcqi_bench`、`lcqi_trace`、`lcqi_gpt2`、`lcqi_ledger`、`lcqi_serving_probe`、可选 `lcqi_onednn_bench` 和 `lcqi_tests` 的完整关系。

Listing 1.7 linux_cpu_inference/CMakeLists.txt

```

1 find_package(Threads REQUIRED)
2
3 add_library(lcqi STATIC
4     src/f32_kernels.cpp

```

```

5     src/gpt2_reference.cpp
6     src/inference.cpp
7     src/int8_kernels.cpp
8     src/model.cpp
9     src/reference_decoder.cpp
10    src/safetensors.cpp
11    src/tokenizer.cpp
12 )
13
14 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
15     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
16         target_sources(lcqi PRIVATE src/f32_kernels_avx2.cpp)
17         target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
18         set_source_files_properties(src/f32_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
19         set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
↵ COMPILE_OPTIONS "-mavx2;-mfma")
20         target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
21     endif()
22 endif()
23
24 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
25     message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
↵ current system is ${CMAKE_SYSTEM_NAME}.")
26 endif()
27
28 target_include_directories(lcqi
29     PUBLIC
30     ${CMAKE_CURRENT_SOURCE_DIR}/include
31 )
32
33 target_compile_features(lcqi PUBLIC cxx_std_20)
34 target_link_libraries(lcqi PUBLIC Threads::Threads)
35
36 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
37     target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
38 endif()
39
40 add_executable(lcqi_cli
41     src/main.cpp
42 )
43
44 target_link_libraries(lcqi_cli PRIVATE lcqi)
45
46 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
47     target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
48 endif()
49
50 add_executable(lcqi_bench
51     src/bench.cpp
52 )
53

```

```
54 target_link_libraries(lcqi_bench PRIVATE lcqi)
55
56 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
57     target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
58 endif()
59
60 add_executable(lcqi_trace
61     src/trace.cpp
62 )
63
64 target_link_libraries(lcqi_trace PRIVATE lcqi)
65
66 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
67     target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
68 endif()
69
70 add_executable(lcqi_gpt2
71     src/gpt2_cli.cpp
72 )
73
74 target_link_libraries(lcqi_gpt2 PRIVATE lcqi)
75
76 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
77     target_compile_options(lcqi_gpt2 PRIVATE -Wall -Wextra -Wpedantic)
78 endif()
79
80 add_executable(lcqi_ledger
81     src/ledger.cpp
82 )
83
84 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
85     target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
86 endif()
87
88 add_executable(lcqi_serving_probe
89     src/serving_probe.cpp
90 )
91
92 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
93     target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
94 endif()
95
96 find_package(dnnl QUIET)
97 if(dnnl_FOUND)
98     add_executable(lcqi_onednn_bench
99         src/onednn_bench.cpp
100     )
101     target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
102     target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
103     if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
104         target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra ↵
↵ Wpedantic)
```



```

105     endif()
106 endif()
107
108 add_executable(lcqi_tests
109     tests/lcqi_tests.cpp
110 )
111
112 target_link_libraries(lcqi_tests PRIVATE lcqi)
113
114 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
115     target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
116 endif()
117
118 add_test(NAME lcqi_tests COMMAND ${TARGET_FILE:lcqi_tests})
119 add_test(NAME lcqi_gpt2_tiny COMMAND ${TARGET_FILE:lcqi_gpt2})
120 set_tests_properties(lcqi_gpt2_tiny PROPERTIES
121     PASS_REGULAR_EXPRESSION "generated_ids 1 2 3 3"
122 )
123 add_test(NAME lcqi_trace COMMAND ${TARGET_FILE:lcqi_trace})
124 set_tests_properties(lcqi_trace PROPERTIES
125     PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
126 )
127 add_test(NAME lcqi_ledger COMMAND ${TARGET_FILE:lcqi_ledger})
128 set_tests_properties(lcqi_ledger PROPERTIES
129     PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
130 )
131 add_test(NAME lcqi_serving_probe COMMAND ${TARGET_FILE:lcqi_serving_probe})
132 set_tests_properties(lcqi_serving_probe PROPERTIES
133     PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
134 )

```

项目说明与模型资产

README 是复现实验的入口。它写清楚当前能力、后续边界、构建命令、tiny 推理命令、benchmark 字段、reference trace 字段、GPT-2 smoke、7B ledger、serving SLO probe 和 SmolLM2 small smoke。读源码前先读 README，可以避免把 tiny fixture、GPT-2 reference path 和外部 llama.cpp smoke 混成同一个能力等级。

Listing 1.8 linux_cpu_inference/README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三↵
  ↵ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大↵
  ↵ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache↵
  ↵ 、CPU/GPU 后端、正确性报告和性能对标。
4
5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器↵
  ↵ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线↵
  ↵ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源↵
  ↵ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变↵
  ↵ 当前切片的语义合同。

```

```

6
7 当前版本支持：
8
9 - 教学文本模型格式 `LCQI_MODEL_V1`
10 - 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
11 - safetensors header manifest 解析：metadata、tensor name、dtype、shape、data ←
    ↳ offsets、offset 边界和 dtype/shape 字节数校验
12 - 两层 MLP 教学切片
13 - int8 权重加 per-layer scale
14 - float 输入和激活
15 - 量化线性层、ReLU、分类 argmax
16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
17 - GPT-2 F32 dot kernel：cached generation 热路径使用可替换 dot 边界，x86-64/GNU ←
    ↳ /Clang 构建接入 AVX2/FMA 实现，其他平台保留标量 fallback
18 - tiny reference decoder：RMSNorm、float linear、RoPE、GQA KV cache、decode ←
    ↳ attention、SwiGLU、lm head
19 - GPT-2 reference path：byte-level BPE tokenizer、`config.json`、F32/F16/BF16 ←
    ↳ safetensors 张量读取、HuggingFace GPT-2 name mapping、LayerNorm、绝对位置 ←
    ↳ 嵌入、packed QKV、causal attention、GELU、tied lm head、full-prefix ←
    ↳ greedy generation、KV cache greedy generation 和分阶段 benchmark
20 - reference trace CSV：从 prompt/tokenizer 合同到 logits/sampler/token，逐 ←
    ↳ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values ←
    ↳ 摘要
21 - 7B ledger CSV：输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth ←
    ↳ only tokens/s 上限
22 - serving SLO probe CSV：输出 admission、batch state、KV 预算、queue wait、TTFT ←
    ↳ 、TPOT、取消回收和拒绝率
23 - 自研 GPT-2 冒烟：加载 `openai-community/gpt2` 的 `config.json`、`vocab.json` ←
    ↳ 、`merges.txt`、`model.safetensors`，在 LCQI C++ reference path 上生成 1 ←
    ↳ 个 token，并把文件 hash、prompt ids、generated ids 和文本输出写入报告
24 - 自研 GPT-2 benchmark 对比：同一个 GPT-2 F32 safetensors 模型分别跑 full- ←
    ↳ prefix 和 cached-KV，并在可用时附带 llama.cpp/SmolLM2 Q4_K_M 作为成熟引擎 ←
    ↳ 参考
25 - 自研 GPT-2 优化 A/B：从指定 baseline commit 建临时 worktree，当前实现和 ←
    ↳ baseline 都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建， ←
    ↳ 再用同一个 GPT-2 F32 模型多轮测量 full-prefix 与 cached-KV
26 - 真实 small 开源模型冒烟：通过 llama.cpp 加载 SmolLM2-135M-Instruct 的 GGUF 量 ←
    ↳ 化文件，在本地 CPU 上生成一段 assistant 文本，并把模型 hash、命令、输出和 ←
    ↳ 性能行写入报告
27 - CLI 推理和简单 benchmark
28 - CTest 正确性测试
29
30 后续扩展方向：
31
32 - 真实 7B 级模型 metadata、tokenizer 和量化权重导入
33 - SentencePiece tokenizer、GGUF 权重导入、safetensors 分片加载和更多模型方言 ←
    ↳ name mapping
34 - GPT-2 reference path 的 Transformers/PyTorch golden logits 对齐报告
35 - KV cache 分页、内存规划和可选量化
36 - CPU packed weight、SIMD、NUMA 和线程亲和优化
37 - GPU backend adapter 和 device kernel
38 - 与现有框架的 prefill/decode/内存/误差对标报告

```

```

39
40 构建：
41
42 ```bash
43 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↵
    ↪ DCMAKE_BUILD_TYPE=Debug
44 cmake --build books/cpu-volume-3/build/lcqi-debug
45 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
46 ```
47
48 运行：
49
50 ```bash
51 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
52   books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
53   1.0 2.0 -1.0 0.5 1000
54 ```
55
56 kernel benchmark:
57
58 ```bash
59 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
60 ```
61
62 输出 CSV 字段：
63
64 ```text
65 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
    ↪ average_us,max_abs_diff,checksum
66 ```
67
68 当前 benchmark 按 backend 逐行输出：`scalar`、`packed_scalar`、`avx2`。x86-64 ↵
    ↪ 构建会编译 AVX2 路径；不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
    ↪ available=0`，避免把源码存在误报成当前环境实测性能。
69
70 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
    ↪ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据，还必须继↵
    ↪ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
    ↪ 照、sanitizer、fuzz 和 CI。
71
72 safetensors manifest gate:
73
74 `lcqi_tests` 会运行时生成一个最小 safetensors fixture，验证 header 长度、↵
    ↪ `__metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
    ↪ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
    ↪ 字节数不匹配的坏文件，确认加载器会拒绝。这个 gate 只覆盖模型资产目录表，↵
    ↪ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行；后续真实模↵
    ↪ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
    ↪ descriptor 和 first token trace。
75
76 reference decoder trace:
77

```

```

78 ```bash
79 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
80 ```
81
82 输出 CSV 字段：
83
84 ```text
85 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
86 ```
87
88 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens`
    ↪ `=[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入
    ↪ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同，避免
    ↪ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/
    ↪ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、
    ↪ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`
    ↪ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、
    ↪ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。
89
90 当前 tokenizer fixture 的关键输出：
91
92 ```text
93 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
94 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-
    ↪ chat-template-v1
95 ```
96
97 GPT-2 自研 reference smoke:
98
99 ```bash
100 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
101 make cpu3-gpt2-smoke
102 make cpu3-gpt2-benchmark-compare
103 make cpu3-gpt2-hotspot-profile
104 python3 books/cpu-volume-3/tools/run_gpt2_optimization_ab.py --rounds 3
105 python3 books/cpu-volume-3/tools/run_gpt2_hotspot_profile.py --token-counts
    ↪ 4,16 --rounds 3
106 ```
107
108 第一条命令不需要下载模型，直接运行仓库内 tiny GPT-2 fixture，输出固定 prompt
    ↪ ids、logits、predicted token 和 greedy generated ids。`lcqi_tests` 会覆盖
    ↪ tiny GPT-2 forward/generation、byte-level BPE 编码解码、F32/F16/BF16
    ↪ safetensors 读取、HuggingFace 风格 tiny GPT-2 checkpoint 加载和坏 shape
    ↪ 拒绝。
109
110 第二条命令显式依赖网络，只下载 `openai-community/gpt2` 的 `config.json`、`vocab-
    ↪ .json`、`merges.txt`、`model.safetensors` 到 `~/cache/lcqi-gpt2-smoke/`
    ↪ `openai-community--gpt2`，模型文件不进入 Git。脚本会校验四个文件的 bytes
    ↪ 和 SHA256，构建 `lcqi_gpt2`，运行：
111
112 ```text
113 Hello, my name is

```

```

114 ```
115
116 当前验证报告写到：
117
118 ```text
119 books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
120 ```
121
122 最近一次本机报告的关键行是：
123
124 ```text
125 prompt_ids 15496 11 616 1438 318
126 generated_ids 15496 11 616 1438 318 1757
127 generated_text Hello, my name is John
128 validation=PASS
129 ```
130
131 这说明 LCQI 自研 C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer↵
    ↵ 、权重导入、forward 和 greedy generation。默认真实模型路径现在使用 `↵
    ↵ cached_kv`，也可以用 `--engine full` 复现旧的 full-prefix 重算路径：
132
133 ```bash
134 books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_gpt2 \
135 --benchmark --engine cached --threads 0 \
136 ~/.cache/lcqi-gpt2-smoke/openai-community--gpt2 \
137 "Hello, my name is" 4
138 ```
139
140 `--threads 0` 表示自动选择 cached decoder worker 数；`--threads 1` 强制串行；↵
    ↵ `--threads N` 固定 worker 数。当前 auto 上限是 16，避免在短 decode 上把线↵
    ↵ 程调度开销放大到超过收益。
141
142 `make cpu3-gpt2-benchmark-compare` 会生成：
143
144 ```text
145 books/cpu-volume-3/results/lcqi-gpt2-benchmark-compare.txt
146 ```
147
148 `run_gpt2_optimization_ab.py` 会生成：
149
150 ```text
151 books/cpu-volume-3/results/lcqi-gpt2-optimization-ab.txt
152 ```
153
154 这个 A/B 报告专门回答“当前 LCQI 代码相对指定 baseline 改快了吗”。脚本默认 ↵
    ↵ baseline 是 `4feb3f`，会临时创建 baseline worktree，分别构建 baseline 和↵
    ↵ 当前工作区的 Release `lcqi_gpt2`，再多轮运行同一条 prompt。上一轮工作区复↵
    ↵ 用优化的 2 轮 smoke 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-↵
    ↵ optimization-ab-summary.md`；本轮线程优化的正式 raw 报告保存在 `books/cpu↵
    ↵ -volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `↵
    ↵ books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。↵
    ↵ 以 `73f40c7` 作为 baseline，在 `caw` 上 5 轮中位数显示：4 token cached-KV↵

```

↪ 生成阶段从 `395.397 ms` 降到 `76.4141 ms`，16 token 从 `989.060 ms` 降到
 ↪ `185.479 ms`，生成吞吐分别提升到 `52.3464 token/s` 和 `86.2633 token/s`
 ↪ 。随后的 F32 dot kernel 优化以 `cb4d89f` 为 baseline，raw 报告保存在 `
 ↪ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `
 ↪ books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。在
 ↪ 同一台 `caw`、同一 GPT-2 F32 模型、同样 `--threads 0` 自动 16 workers
 ↪ 下，4 token 生成阶段中位数从 `74.4247 ms` 到 `62.3628 ms`，16 token 从
 ↪ `181.257 ms` 到 `157.230 ms`；decode token/s 分别提升到 `130.073` 和
 ↪ `127.405`。端到端 GPT-2 数字受调度、温度和共享机器状态影响很大，因此正式
 ↪ 结论必须看多轮中位数、min/max 和生成文本一致性，不能只挑一次最快结果。

155

156 `run\_gpt2\_hotspot\_profile.py` 专门回答“现在热点到底在哪”。它通过 `--profile-
 ↪ hotspots` 打开 cached decode 内部计时，字段包括 `hotspot_lm_head_pct`、`
 ↪ hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`
 ↪ hotspot_qkv_projection_pct` 和 `hotspot_attention_pct`。`caw` 上的 3 轮
 ↪ Release profile 摘要保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-
 ↪ profile-summary.md`，原始报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-
 ↪ -hotspot-profile-caw.txt`。关键结论是：4 token 与 16 token 两个口径下，`
 ↪ lm_head` 中位数约 `30.2%`，MLP 两个线性投影合计约 `46%`，QKV 投影约 `17%`
 ↪ ，cached attention 本体只有 `0.06%` 到 `0.11%`。所以当前短上下文 GPT-2 的
 ↪ 慢点不是 KV attention，而是标量 F32 matvec 和 `50257 x 768` vocab 扫描。

157

158 当前优化点集中在 cached-KV generation: `Gpt2CachedGreedyDecoder` 把模型级 shape
 ↪ validation、KV cache、临时 workspace 和 worker pool 的生命周期提升到整段
 ↪ 生成；`Gpt2ForwardWorkspace` 复用 hidden、normed、Q/K/V、packed QKV、
 ↪ attention、MLP、scores 和 logits 缓冲区；生成路径只需要 greedy token 时走
 ↪ logits-free argmax，避免把完整 logits vector 作为 API 结果反复分配；QKV
 ↪ 、attention output projection、MLP 两个 projection 和 tied `lm\_head` 都在
 ↪ cached session 内按输出并行；每一行内部再通过 `dot\_f32\_unchecked` 进入
 ↪ F32 dot kernel，x86-64 可用时使用 AVX2/FMA，其他平台走标量 fallback。KV
 ↪ cache 的 checked public API 仍保留，内部 cached attention 在一次边界校验
 ↪ 后使用私有 unchecked span 扫描热路径。它不改变 GPT-2 数学语义，只改变会话
 ↪ 生命周期、任务分片和热循环调度。

159

160 `make cpu3-gpt2-benchmark-compare` 仍用于和外部成熟引擎放在同一报告里看工程差
 ↪ 距。同机 llama.cpp/SmolLM2 Q4_K_M 是成熟引擎参照，不是同模型质量排名：
 ↪ LCQI 跑的是 GPT-2 F32 reference path，llama.cpp 跑的是 GGUF 量化 small
 ↪ instruct 模型。它揭示的是工程差距：LCQI 已经消除了“每步重算前缀”的主要算
 ↪ 法错误，补上 cached F32 row parallelism，并把热路径单行 dot 接到 AVX2/FMA
 ↪ ；但还缺 packed/quantized weight、GGUF 量化 block、mmap/lazy load、采样
 ↪ 器、分页 KV cache、NUMA/亲和和成熟调度。

161

162 它仍然不是生产级推理引擎：当前 GPT-2 路径还没有把 logits 和 Transformers/
 ↪ PyTorch 逐数值对齐成外部 golden 报告，也没有 GGUF 自研导入、分页 KV cache
 ↪ 、量化权重执行和高性能 SIMD/packed kernel。

163

164 7B ledger:

165

166 ```bash

167 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
 168 > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
 169 ```


```

170
171 输出 CSV 字段：
172
173 ```text
174 section,item,value,unit,notes
175 ```
176
177 这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`，报告 decode↵
    ↵ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s ↵
    ↵ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行，防止账本公↵
    ↵ 式漂移。
178
179 serving SLO probe:
180
181 ```bash
182 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↵
    ↵ \
183 > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
184 ```
185
186 输出 CSV 字段：
187
188 ```text
189 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↵
    ↵ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↵
    ↵ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↵
    ↵ metric_value
190 ```
191
192 probe 固定四个请求：短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按↵
    ↵ `prompt_tokens + max_new_tokens` 预留 KV，取消请求释放预算，超长请求返回↵
    ↵ `memory_budget_exceeded`，并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`↵
    ↵ rejection_rate` 等服务化指标。
193
194 真实 small 开源模型 smoke:
195
196 ```bash
197 make cpu3-smolllm2-smoke
198 ```
199
200 这条命令显式依赖网络和外部构建，因此不放进默认 CTest。脚本会把 llama.cpp 和模型↵
    ↵ 文件缓存到 `~/cache/lcqi-smolllm2-small-smoke`，模型不进入 Git。固定模型↵
    ↵ 是 `HuggingFaceTB/SmolLM2-135M-Instruct` 的 GGUF 量化版本，来源仓库为 `↵
    ↵ bartowski/SmolLM2-135M-Instruct-GGUF`，文件为 `SmolLM2-135M-Instruct-↵
    ↵ Q4_K_M.gguf`。脚本会校验文件大小 `105454432` 字节和 SHA256：
201
202 ```text
203 2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
204 ```
205
206 脚本构建固定 commit 的 `llama-simple-chat`，用 `-ngl 0` 强制 CPU 路径，输入短 ↵
    ↵ prompt，检查 assistant response 非空，并生成：

```

```

207
208 ```text
209 books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
210 ```
211
212 这份报告只能证明真实 GGUF small 模型已经完成加载、tokenizer/chat template、↵
    ↵ prefill、decode 和文本输出。它不能证明 LCQI 自研 C++ 引擎已经支持完整 ↵
    ↵ GGUF、不能证明回答质量正确，也不能替代 tiny fixture 的逐层 golden trace。↵
    ↵ tiny fixture 继续负责可手算语义和回归定位；SmolLM2 smoke 负责把“真实开源↵
    ↵ 模型能在本机跑起来”变成可复查证据。

```

tiny MLP 模型文件是最短闭环的资产。它决定输入维度、hidden 维度、输出维度、权重量化值、scale 和 bias；测试中的 logits golden 依赖这份文件。

Listing 1.9 models/tiny_mlp_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1
8 -4 6 1 -2
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4

```

tiny tokenizer fixture 固定 BOS、EOS、UNK、词表和模板 hash。tokenizer 合同如果漂移，后面的 decoder trace 和测试都会被污染。

Listing 1.10 models/tiny_tokenizer.txt

```

1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3
12 4 <eos>

```


为什么要先讲调用链

实践与代码卷如果只把 *.cpp 和 *.hpp 全部贴出来，读者仍然不知道从哪里下手。LCQI 的代码同时包含 tiny MLP、int8 kernel、tokenizer、safetensors、reference decoder、GPT-2、benchmark、trace、ledger、serving probe 和外部模型脚本；这些文件不是平铺关系，而是几条调用链。先讲调用链，再给完整源码，读者才能把每个文件放到正确位置。

核心思想

读 LCQI 的顺序不是“哪个文件长先读哪个”，而是“入口、合同、资产、执行、证据”。入口说明目标怎样构建，合同说明类型怎样连接，资产说明模型从哪里来，执行说明 token 或张量怎样流动，证据说明测试和报告守住了哪些边界。

一、全工程入口：哪些文件决定能不能复现

文件	负责什么	读源码时要确认什么
第三册README	第三册 AI 计算工程路线、当前 LCQI 能力、GPT-2 smoke 和结果报告位置。	能力声明必须能在 labs/linux_cpu_inference、tools 和 results 中找到证据。
第三册CMake	第三册根 CMake，负责把 linux_cpu_inference 接入。	根 CMake 应保持薄层；真正 target 定义必须在 lab 内部，避免入口文件承担实现细节。
本卷CMake	本卷复用同一份 LCQI lab。	本卷不复制源码，只把同一工程接入自己的测试构建，防止两份代码漂移。
本卷Makefile	本卷的书籍构建入口。	它构建 PDF/EPUB，也提供 test 目标去构建第三册 LCQI；实践与代码卷不能只排版不验证。
LCQICMake	lcqi 库、CLI、benchmark、trace、GPT-2、ledger、serving probe、CTest 目标。	读这里能看到哪些文件进入库，哪些是可执行程序，哪些依赖可选外部库。
LCQIREADME	本地构建、tiny 推理、benchmark、trace、GPT-2、SmolLM2 smoke 的命令。	先按 README 跑命令，再读源码；否则不知道某个函数是否真的能从命令行触达。

上表里的短标签对应下面这些真实路径：

- books/cpu-volume-3/README.md
- books/cpu-volume-3/CMakeLists.txt
- books/cpu-volume-3-practice/CMakeLists.txt
- books/cpu-volume-3-practice/Makefile
- books/cpu-volume-3/labs/linux_cpu_inference/CMakeLists.txt
- books/cpu-volume-3/labs/linux_cpu_inference/README.md

这些入口文件回答“代码在哪里”和“怎样运行”。读者进入本卷代码走读章时，先不要直接跳到 gpt2_reference.cpp。先确认 lcqi 静态库包含 model.cpp、inference.cpp、int8_kernels.cpp、f32_kernels.cpp、reference_decoder.cpp、tokenizer.cpp、safetensors.cpp 和 gpt2_reference.cpp；再确认 lcqi_cli、lcqi_bench、lcqi_trace、lcqi_gpt2、lcqi_ledger、lcqi_serving_probe 和 lcqi_tests 分别从哪条路径进入。

二、Tiny MLP 调用链：最短闭环怎样跑通

Tiny MLP 是最小推理闭环，用来证明“模型资产、shape 检查、int8 linear、激活函数、logits、argmax、CLI 输出”可以完整连起来。它不是大模型能力本身，但它是后面所有复杂路径的可手算底座。

文件	关键类型或函数	这段代码的意思
<code>models/tiny_mlp_i8.txt</code>	输入维度、hidden 维度、输出维度、int8 权重、scale、bias。	这是模型资产。测试中的 <code>golden logits</code> 来自这里，资产改动必须带着测试改动。
<code>include/lcqi/model.hpp</code>	<code>QuantizedLinearLayer</code> 、 <code>TinyMlpModel</code> 、 <code>load_model</code> 。	定义 tiny MLP 如何在内存中表示；只负责模型结构和加载入口，不做推理。
<code>src/model.cpp</code>	<code>read_i8_vector</code> 、 <code>validate_layer</code> 、 <code>load_model</code> 。	解析文本模型，拒绝坏 key、坏维度、权重数量不匹配和非法 scale，把错误挡在执行前。
<code>include/lcqi/inference.hpp</code>	<code>InferenceResult</code> 、 <code>linear_i8</code> 、 <code>relu_inplace</code> 、 <code>run_inference</code> 。	定义推理输出要包含 hidden、logits 和 predicted class，方便测试定位错误层。
<code>src/inference.cpp</code>	输入 shape 检查、两层 int8 linear、ReLU、argmax。	最短执行路径：模型加输入变成 logits。这里的代码应保持清楚，不能混入 CLI 或文件解析。
<code>src/main.cpp</code>	解析命令行输入并打印结果。	CLI 是薄层；它只接模型路径和输入，不复制 <code>run_inference</code> 逻辑。

这条链路按下面顺序读：`main()` 取得模型路径和输入向量，调用 `load_model()` 把 `tiny_mlp_i8.txt` 变成 `TinyMlpModel`，再调用 `run_inference()`。在 `run_inference()` 内部，第一层 `linear_i8()` 把 float 输入和 int8 权重相乘，乘以 scale 后加 bias；`relu_inplace()` 把负 hidden 截断；第二层 `linear_i8()` 得到 logits；最后 `argmax()` 给出 predicted class。

读这个路径时要抓住边界条件。输入长度不等于 `input_size` 要立即拒绝；权重数量不是 `rows*columns` 要在加载时拒绝；logits 为空不能取 `argmax`。这些错误如果拖到 SIMD kernel 或 benchmark 里才暴露，定位成本会高很多。

三、Int8 Kernel 调用链：reference、packed、AVX2 和 benchmark 怎样对照

文件	关键代码	这段代码的意思
<code>include/lcqi/int8_kernels.hpp</code>	<code>PackedLinearI8</code> 、 <code>KernelBenchmarkCase</code> 、 <code>benchmark_linear_i8_case</code> 。	定义 kernel 对标口径：shape、repeat、backend、平均耗时、checksum、max diff。
<code>src/int8_kernels.cpp</code>	<code>pack_linear_i8</code> 、 <code>linear_i8_packed</code> 、 <code>linear_i8_packed_with_workspace</code> 、 <code>deterministic fixture</code> 。	scalar 和 packed 路径是正确性基线，workspace 路径减少重复分配，fixture 让 benchmark 和测试可复现。
<code>src/int8_kernels_avx2.cpp</code>	<code>linear_i8_packed_avx2_available</code> 、 <code>linear_i8_packed_avx2</code> 。	平台优化路径。必须先判断机器能力，再执行 AVX2/FMA；不可用时报告 unavailable，而不是假装测过。
<code>include/lcqi/f32_kernels.hpp</code> 与 <code>src/f32_kernels.cpp</code>	<code>dot_f32</code> 、 <code>dot_f32_unchecked</code> 、 <code>dot_f32_scalar_unchecked</code> 。	GPT-2 F32 hot path 的单行点积边界。checked API 负责 size guard，unchecked API 给已经验证过 shape 的热循环使用。
<code>src/f32_kernels_avx2.cpp</code>	<code>dot_f32_avx2_available</code> 、 <code>dot_f32_avx2_unchecked</code> 。	x86 平台优化路径。它只替换一行 dot 的内核，不改变 GPT-2 层结构、线程分片或 greedy tie-breaking。

<code>src/bench.cpp</code>	shape 列表、repeat 参数、back-end 输出。	benchmark 程序只负责测 kernel 入口。性能结论必须和 <code>max_abs_diff</code> 、CPU、编译器、构建类型一起报告。
----------------------------	---------------------------------	---

kernel 文件要按“可信基线到优化路径”的顺序读。先看 deterministic layer 和 input 怎么生成，保证每次运行输入相同；再看 `linear_i8_packed()` 怎样把 int8 权重、scale、bias 和 float 输入算成输出；再看 workspace 路径怎样复用临时缓冲；最后看 AVX2 路径怎样在支持平台上替换内层循环。测试用 packed 与 scalar 的 `max_abs_diff` 对照，benchmark 再比较耗时。

四、Tokenizer、Safetensors 与 Reference Decoder：真实模型前的三道门

文件	关键类型或函数	这段代码的意思
<code>models/tiny_tokenizer.txt</code> <code>include/lcqi/tokenizer.hpp</code> 与 <code>src/tokenizer.cpp</code>	BOS、EOS、UNK、词表、模板 hash。 <code>TokenizerModel</code> 、 <code>load_tokenizer</code> 、 <code>encode_prompt</code> 、 <code>tokenizer_contract_hash</code> 。	固定 tokenizer 合同，避免 prompt 到 token id 的规则漂移。 把 prompt 切成稳定 token id；未知 token 是否允许，必须由 UNK 合同决定。
<code>include/lcqi/safetensors.hpp</code> 与 <code>src/safetensors.cpp</code>	<code>SafeTensorManifest</code> 、 <code>SafeTensorFile</code> 、manifest parser、dtype 转换。	解析模型字节之前先验证 header、dtype、shape、offset 和 payload 边界，防止坏资产进入数学层。
<code>include/lcqi/reference_decoder.hpp</code> 与 <code>src/reference_decoder.cpp</code> <code>src/trace.cpp</code>	embedding、RMSNorm、RoPE、KV cache、attention、SwiGLU、trace checkpoint。 trace CSV 输出。	Tiny decoder-only reference path。它用 checkpoint 解释每一步张量状态，不只返回最终 token。 把 reference decoder 的 checkpoint 打印成可复查表格，定位错误发生在哪个阶段。

这三道门分别解决三个问题。Tokenizer 解决“人写的 prompt 如何变成 token id”；Safetensors 解决“磁盘里的模型字节如何变成带 shape 的张量”；Reference Decoder 解决“token 和权重如何经过 decoder-only 层生成 logits”。任何一层含糊，后面的 GPT-2 路径都会变成看似能跑、实则无法定位的黑盒。

五、GPT-2 调用链：从 HuggingFace 资产到生成 token

GPT-2 路径是本卷代码走读里最长的一条链，因为它要处理真实模型格式。读 `gpt2_reference.cpp` 时按模块读，不要从第一行一路滚到最后。

文件	关键代码	这段代码的意思
<code>include/lcqi/gpt2_reference.hpp</code>	<code>Gpt2Config</code> 、 <code>Gpt2ReferenceModel</code> 、 <code>Gpt2KvCache</code> 、 <code>Gpt2Tokenizer</code> 、forward/generate API。	公开 GPT-2 reference path 的合同：配置、权重、KV cache、tokenizer、full-prefix 和 cached generation。

<code>src/gpt2_reference.cpp</code>	JSON parser、byte-level BPE、safetensors loader、LayerNorm、QKV、causal attention、GELU、lm head、KV cache。	正确性优先实现。它把 HuggingFace GPT-2 权重名、Conv1D 转置、tokenizer 和 forward 规则全部显式写出来。
<code>src/gpt2_cli.cpp</code>	tiny mode、真实模型目录模式、generation、benchmark 输出。	命令行入口。无参数跑 tiny fixture；传模型目录时加载真实 GPT-2 资产；benchmark 对比 full-prefix 和 cached-KV。
<code>tools/run_gpt2_smoke.py</code>	下载标准 GPT-2 资产，调用 <code>lcqi_gpt2</code> 。	证明自研 safetensors、BPE、forward、KV cache 和 greedy generation 能跑真实开源模型。
<code>tools/run_gpt2_benchmark_compare.py</code>	运行 LCQI full-prefix、cached-KV，并可附带成熟引擎参考。	用同一模型口径比较算法差距和 kernel 差距，不把不同模型的速度混成一个结论。

GPT-2 代码按五段读。第一段是配置和权重加载：`load_gpt2_config()` 读 `config.json`，`load_gpt2_reference_model()` 从 safetensors 中取 embedding、每层 LayerNorm、QKV、MLP 和 final norm。HuggingFace GPT-2 的 Conv1D 权重形状和普通线性层方向不同，所以代码显式调用转置函数，不能跳过。

第二段是 tokenizer：`load_gpt2_tokenizer()` 读 `vocab.json` 和 `merges.txt`，`gpt2_encode()` 执行 byte-level BPE，`gpt2_decode()` 把 id 还原为文本。这里的重点是字节到 Unicode 映射、BPE merge rank 和 unknown token 边界。

第三段是 full-prefix forward：`run_gpt2_forward()` 每生成一个新 token 都用完整 token 前缀重新跑所有层。它慢，但容易验证。流程是 token embedding 加 position embedding，逐层执行 LayerNorm、packed QKV、causal attention、残差、MLP、残差，最后 final LayerNorm 和 tied lm head 得到 logits。

第四段是 cached-KV forward：`Gpt2KvCache` 保存每层每个 position 的 key/value，`run_gpt2_forward_cached()` 每次只处理最新 token，并让 attention 读取过去 cache。这里要特别看 `validate_written()`：读取未写入 cache slot 必须报错，否则生成错误会很难定位。

第五段是 greedy generation：`gpt2_generate_greedy()` 用 full-prefix 路径，`gpt2_generate_greedy_cached()` 用 KV cache 路径。两个函数都要检查 prompt 非空、`max_new_tokens` 非负、不会超过 `max_positions`，并在生成 EOS 时停止。

六、报告、服务探针、外部对标和测试

文件	关键代码	这段代码的意思
<code>src/ledger.cpp</code>	7B 参数量、KV cache、bandwidth-only 上限。	输出的是工程账本，不是实测速度；它帮助读者估算权重和 KV 内存压力。
<code>src/serving_probe.cpp</code>	短请求、RAG 请求、取消请求、超长请求，TTFT/TPOT 字段。	把服务层边界变成字段：哪些请求接受，哪些拒绝，取消后资源是否回收。
<code>src/onednn_bench.cpp</code>	oneDNN matmul benchmark。	可选外部库对标入口。没有 oneDNN 环境时不能声称已验证。
<code>tools/run_smolllm2_small_smoke.py</code>	下载 SmolLM2-135M-Instruct GGUF，构建固定 commit 的 llama.cpp smoke。	这是成熟外部引擎上的真实 small 模型 smoke，用来建立外部参照，不代表 LCQI 已具备同等能力。
<code>tools/run_lcqi_benchmark.sh</code>	收集 LCQI 本地 benchmark。	在固定机器上重复运行同一组 shape，形成可复查 CSV 或文本报告。

<code>tests/lcqi_tests. cpp</code>	<code>tiny MLP、tokenizer、safeten- sors、GPT-2 tiny、HF-style checkpoint、bad shape、de- coder、trace、kernel。</code>	这是本卷最重要的代码证据。新增功能如果没有进入这里，就不能写成已经守住的能力。
--	--	---

读测试文件时，不要只看最后是否返回 0。它里面的 fixture 本身就是源码讲解的一部分：测试会手写 safetensors header，构造 F16/BF16/F32 payload，生成 tiny GPT-2 权重，验证 bad shape 会被拒绝，比较 packed int8 kernel 和 scalar 路径。这些测试告诉读者“代码真正承诺了什么”。如果一个能力只出现在 README 或 CLI 输出里，却没有进入测试或 smoke 报告，就只能算实验入口，不能算稳定能力。

源码阅读任务

读完本章后，再去读相邻主题中的完整源码。每看到一个文件，都把它归到五类之一：入口、合同、资产解析、执行路径、证据。归不进去的文件，就说明工程边界需要重新解释或重构。

第2章实践：张量布局、地址流与第一个 MatVec

本章对应原书中的第三册“张量布局、地址流与第一个矩阵向量内核”。不要把它当作概念复述，本章要把 TensorView、dtype、shape、stride、row-major、地址公式、bias、ReLU、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `linear_i8`，核心命令或测试入口是 `lcqi_tests`。

Listing 2.1 第2章实践：张量布局、地址流与第一个 MatVec 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / linear_i8
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：手算一个 2 行 4 列权重矩阵的地址偏移。读者要先手写预期结果，再运行项目观察输出。动作是：把 `weight[out,k]` 展开成 `base+(out*K+k)*sizeof(i8)`，再对应到 `linear_i8` 的循环。如果输出里没有 `logit` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：张量不是裸数组；每个 kernel 入口都要能解释 dtype、shape、stride、scale 和输出缓冲这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一页地址流账本：输入向量、hidden 权重、输出 logits 各自的 shape、连续性、读写次数和错误边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 3 章实践：GEMM、packing 与 SIMD 证据

本章对应原书中的第三册“从矩阵向量乘走向 GEMM、packing 与 SIMD”。不要把它当作概念复述，本章要把 packed weight、scalar baseline、AVX2 backend、output block、shape sweep、反汇编和降级放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `output_block`，核心命令或测试入口是 `lcqi_bench`。

Listing 3.1 第 3 章实践：GEMM、packing 与 SIMD 证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / output_block
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：同一组输入同时跑 `scalar`、`packed_scalar` 和 `avx2`。读者要先手写预期结果，再运行项目观察输出。动作是：比较 CSV 中 `backend`、`available`、`average_us`、`max_abs_diff` 和 `checksum`。如果输出里没有 `max_abs_diff` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：SIMD 路径只有在结果和 baseline 对齐后才有资格谈速度；`available=0` 也是证据，不是失败这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 shape sweep 表，至少解释一个 SIMD 有收益、一个收益不明显、一个当前环境不可验证的场景。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第4章实践：int8 量化、累加器与重新量化

本章对应原书中的第三册“量化系统总览”和“int8 量化、累加器与重新量化”。不要把它当作概念复述，本章要把 int8 weight、float activation、scale、accumulator、requantize、oracle tolerance 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `TinyMlpModel`，核心命令或测试入口是 `lcqi_tests`。

Listing 4.1 第4章实践：int8 量化、累加器与重新量化的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / TinyMlpModel
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：构造一行 int8 权重 `[2, -1, 3, 0]` 和输入 `[1, 2, -1, 0.5]`。读者要先手写预期结果，再运行项目观察输出。动作是：先手算 int32/float accumulator，再和 `linear_i8` 输出比较。如果输出里没有 `scale` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：量化不是把 float 强转成 int8；必须记录 scale 作用位置、累加宽度、误差阈值和 golden 输出这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交量化合同卡：每个 tensor 的 dtype、scale 粒度、accumulator 类型、误差门限和拒绝条件。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：公共合同、Tiny MLP 与 Int8 Kernel

为什么先读头文件

头文件是系统对外承诺的最小合同。实现可以重写，kernel 可以替换，benchmark 可以补充平台路径，但头文件里暴露的数据结构、函数入口和返回字段决定了测试、CLI、工具脚本和章节讲解怎样调用这套工程。

核心思想

读 LCQI 时不要从最长的 `gpt2_reference.cpp` 开始。先读公共头文件，明确“模型如何表示”“推理返回什么”“trace 记录什么”“KV cache 怎样寻址”“tokenizer 的坏输入怎样处理”，再进入实现。这样读到长循环和 shape 检查时，知道它们在维护哪份合同。

Tiny MLP 与推理入口

`model.hpp` 定义 tiny MLP 的模型对象、权重、bias、scale 和加载入口。它的责任是把文本模型资产解释成 C++ 对象，不负责执行推理。

Listing 4.2 `include/lcqi/model.hpp`

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     float scale = 1.0F;
14     std::vector<std::int8_t> weights;
15     std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19     std::int32_t input_size = 0;
20     std::int32_t hidden_size = 0;
21     std::int32_t output_size = 0;
22     QuantizedLinearLayer hidden;
23     QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
27
28 } // namespace lcqi

```

`inference.hpp` 定义推理结果和 `run_inference` 入口。返回值包含 hidden、logits 和 pre-

dicted class，测试可以用这些字段定位是加载错、kernel 错，还是 argmax 错。

Listing 4.3 include/lcqi/inference.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <span>
5  #include <vector>
6
7  #include <lcqi/model.hpp>
8
9  namespace lcqi {
10
11  struct InferenceResult {
12      std::vector<float> logits;
13      std::int32_t predicted_class = 0;
14  };
15
16  void linear_i8(const QuantizedLinearLayer& layer,
17               std::span<const float> input,
18               std::span<float> output);
19
20  void relu_inplace(std::span<float> values);
21
22  InferenceResult run_inference(const TinyMlpModel& model,
23                               std::span<const float> input);
24
25 } // namespace lcqi

```

Int8 Kernel、Tokenizer 与 Safetensors

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。注意 AVX2 是运行时能力，不是源码存在就等于当前机器已验证。

Listing 4.4 include/lcqi/int8_kernels.hpp

```

1  #pragma once
2
3  #include <cstdint>
4  #include <span>
5  #include <vector>
6
7  #include <lcqi/model.hpp>
8
9  namespace lcqi {
10
11  struct PackedLinearI8 {
12      std::int32_t input_size = 0;
13      std::int32_t output_size = 0;
14      std::int32_t output_block_size = 0;
15      float scale = 1.0F;
16      std::vector<std::int8_t> packed_weights;

```

```

17     std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21     std::int32_t input_size = 0;
22     std::int32_t output_size = 0;
23     std::int32_t output_block_size = 0;
24     std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28     const char* name = "";
29     bool available = false;
30     double average_us = 0.0;
31     float max_abs_diff = 0.0F;
32     float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36     KernelBenchmarkCase benchmark_case;
37     std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41                               std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
44                      std::span<const float> input,
45                      std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48                                      std::span<const float> input,
49                                      std::span<float> output,
50                                      std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55                             std::span<const float> input,
56                             std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59                                                  std::int32_t output_size,
60                                                  float scale);
61
62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63
64 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);
65
66 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ←
67         ↪ benchmark_case);
68

```

```
68 } // namespace lcqi
```

F32 kernel 头文件定义 GPT-2 热路径使用的点积边界。读这里要区分三层：`dot_f32()` 是 checked API, 先确认两个 span 长度一致; `dot_f32_unchecked()` 是热路径入口, 由上层 shape validation 保证长度正确; `dot_f32_scalar_unchecked()` 是永远可用的正确性基线。AVX2/FMA 是否可用是运行时能力, 不是 CMake 把文件编进来就等于一定执行。

Listing 4.5 include/lcqi/f32_kernels.hpp

```
1 #pragma once
2
3 #include <cstddef>
4 #include <span>
5
6 namespace lcqi {
7
8 [[nodiscard]] float dot_f32(std::span<const float> lhs, std::span<const float> ↵
    ↵ rhs);
9
10 [[nodiscard]] float dot_f32_scalar_unchecked(const float* lhs,
11                                             const float* rhs,
12                                             std::size_t size) noexcept;
13
14 [[nodiscard]] float dot_f32_unchecked(const float* lhs,
15                                       const float* rhs,
16                                       std::size_t size) noexcept;
17
18 [[nodiscard]] bool dot_f32_avx2_available() noexcept;
19
20 [[nodiscard]] float dot_f32_avx2_unchecked(const float* lhs,
21                                             const float* rhs,
22                                             std::size_t size) noexcept;
23
24 } // namespace lcqi
```

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。prompt 进入模型前, 必须先变成稳定 token id; 未知 token 行为也必须被写进合同。

Listing 4.6 include/lcqi/tokenizer.hpp

```
1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12     std::string text;
13     std::int32_t id = 0;
```

```

14 };
15
16 struct TokenizerModel {
17     std::int32_t bos_id = -1;
18     std::int32_t eos_id = -1;
19     std::int32_t unk_id = -1;
20     std::string chat_template_hash;
21     std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27     const TokenizerModel& tokenizer,
28     std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
    ↵ tokenizer);
31
32 } // namespace lcqi

```

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责模型字节怎样被解释，不应该和数学推理逻辑混在一起。

Listing 4.7 include/lcqi/safetensors.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <span>
6 #include <string>
7 #include <string_view>
8 #include <utility>
9 #include <vector>
10
11 namespace lcqi {
12
13 struct SafeTensorEntry {
14     std::string name;
15     std::string dtype;
16     std::vector<std::int64_t> shape;
17     std::uint64_t data_begin = 0;
18     std::uint64_t data_end = 0;
19
20     [[nodiscard]] std::uint64_t byte_size() const;
21 };
22
23 struct SafeTensorManifest {
24     std::uint64_t header_size = 0;
25     std::uint64_t data_start_offset = 0;
26     std::vector<std::pair<std::string, std::string>> metadata;
27     std::vector<SafeTensorEntry> tensors;

```

```

28
29 [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
    ↵ const;
30 };
31
32 struct SafeTensorFile {
33     std::filesystem::path path;
34     SafeTensorManifest manifest;
35     std::vector<std::uint8_t> bytes;
36 };
37
38 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
39     const std::filesystem::path& path);
40
41 [[nodiscard]] SafeTensorFile load_safetensors_file(const std::filesystem::path& ↵
    ↵ path);
42
43 [[nodiscard]] std::vector<float> read_safetensor_f32_tensor(
44     const SafeTensorFile& file,
45     std::string_view name);
46
47 [[nodiscard]] std::span<const std::uint8_t> read_safetensor_tensor_bytes(
48     const SafeTensorFile& file,
49     std::string_view name);
50
51 } // namespace lcqi

```

Reference Decoder 与 GPT-2 合同

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing 4.8 include/lcqi/reference_decoder.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <span>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11     std::int32_t input_size = 0;
12     std::int32_t output_size = 0;
13     std::vector<float> weights;
14     std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18     std::int32_t hidden_size = 0;

```

```

19     std::int32_t query_heads = 0;
20     std::int32_t kv_heads = 0;
21     std::int32_t head_dim = 0;
22     std::int32_t intermediate_size = 0;
23     std::int32_t vocab_size = 0;
24     std::int32_t max_context = 0;
25     float rms_epsilon = 1.0e-5F;
26     float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30     std::vector<float> rms_attention_weight;
31     LinearWeightsF32 wq;
32     LinearWeightsF32 wk;
33     LinearWeightsF32 wv;
34     LinearWeightsF32 wo;
35     std::vector<float> rms_mlp_weight;
36     LinearWeightsF32 w_gate;
37     LinearWeightsF32 w_up;
38     LinearWeightsF32 w_down;
39 };
40
41 struct ReferenceDecoderModel {
42     DecoderConfig config;
43     std::vector<float> token_embedding;
44     std::vector<DecoderLayerWeightsF32> layers;
45     std::vector<float> final_norm_weight;
46     LinearWeightsF32 lm_head;
47 };
48
49 struct ReferenceDecodeResult {
50     std::vector<float> logits;
51     std::int32_t predicted_token = 0;
52 };
53
54 struct ReferenceTensorCheckpoint {
55     std::string name;
56     std::int32_t token_position = 0;
57     std::int32_t layer_id = 0;
58     std::vector<std::int32_t> shape;
59     std::vector<std::int32_t> stride;
60     std::string dtype;
61     std::string layout;
62     float checksum = 0.0F;
63     float max_abs = 0.0F;
64     std::vector<float> values;
65 };
66
67 struct ReferenceDecodeTraceResult {
68     ReferenceDecodeResult result;
69     std::vector<ReferenceTensorCheckpoint> checkpoints;
70 };

```



```

71
72 class ReferenceKVCache {
73 public:
74     ReferenceKVCache(const DecoderConfig& config, std::int32_t layer_count);
75
76     void append(std::int32_t layer_id,
77                std::int32_t model_position,
78                std::span<const float> key,
79                std::span<const float> value);
80
81     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
82                                             std::int32_t model_position,
83                                             std::int32_t kv_head) const;
84
85     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
86                                              std::int32_t model_position,
87                                              std::int32_t kv_head) const;
88
89     [[nodiscard]] std::int32_t layer_count() const noexcept;
90     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
91
92 private:
93     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
94                                           std::int32_t model_position,
95                                           std::int32_t kv_head) const;
96
97     void validate_address(std::int32_t layer_id,
98                         std::int32_t model_position,
99                         std::int32_t kv_head) const;
100
101     DecoderConfig config_;
102     std::int32_t layer_count_ = 0;
103     std::int32_t filled_tokens_ = 0;
104     std::vector<float> keys_;
105     std::vector<float> values_;
106     std::vector<std::uint8_t> written_;
107 };
108
109 void rms_norm(std::span<const float> input,
110              std::span<const float> weight,
111              float epsilon,
112              std::span<float> output);
113
114 void linear_f32(const LinearWeightsF32& layer,
115                std::span<const float> input,
116                std::span<float> output);
117
118 void apply_rope(std::span<float> heads,
119                std::int32_t head_count,
120                std::int32_t head_dim,
121                std::int32_t model_position,
122                float rope_base);

```

```

123 void attention_decode(const DecoderConfig& config,
124                       const ReferenceKVCache& cache,
125                       std::int32_t layer_id,
126                       std::int32_t model_position,
127                       std::span<const float> query,
128                       std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131                                           std::span<const std::int32_t> ↵
132                                           ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135     const ReferenceDecoderModel& model,
136     std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139 } // namespace lcqi

```

GPT-2 reference 头文件定义 GPT-2 配置、byte-level BPE tokenizer、HuggingFace safetensors 加载、forward、cached forward 和 greedy generation。它单独存在，是因为 GPT-2 使用 LayerNorm、绝对位置嵌入、packed QKV、GELU 和 tied lm head，不能直接塞进 LLaMA-like reference decoder。

Listing 4.9 include/lcqi/gpt2_reference.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <memory>
6 #include <span>
7 #include <string>
8 #include <string_view>
9 #include <unordered_map>
10 #include <vector>
11
12 namespace lcqi {
13
14 struct Gpt2Config {
15     std::int32_t hidden_size = 0;
16     std::int32_t head_count = 0;
17     std::int32_t layer_count = 0;
18     std::int32_t max_positions = 0;
19     std::int32_t vocab_size = 0;
20     std::int32_t intermediate_size = 0;
21     std::int32_t bos_token_id = -1;
22     std::int32_t eos_token_id = -1;
23     float layer_norm_epsilon = 1.0e-5F;
24     std::string activation_function = "gelu_new";
25 };
26
27 struct Gpt2LinearF32 {
28     std::int32_t input_size = 0;

```

```

29     std::int32_t output_size = 0;
30     std::vector<float> weights;
31     std::vector<float> bias;
32 };
33
34 struct Gpt2LayerWeightsF32 {
35     std::vector<float> ln_1_weight;
36     std::vector<float> ln_1_bias;
37     Gpt2LinearF32 c_attn;
38     Gpt2LinearF32 c_proj;
39     std::vector<float> ln_2_weight;
40     std::vector<float> ln_2_bias;
41     Gpt2LinearF32 c_fc;
42     Gpt2LinearF32 mlp_c_proj;
43 };
44
45 struct Gpt2ReferenceModel {
46     Gpt2Config config;
47     std::vector<float> token_embedding;
48     std::vector<float> position_embedding;
49     std::vector<Gpt2LayerWeightsF32> layers;
50     std::vector<float> final_ln_weight;
51     std::vector<float> final_ln_bias;
52     std::vector<float> lm_head_weight;
53     bool tie_lm_head_to_embedding = true;
54 };
55
56 struct Gpt2ForwardResult {
57     std::vector<float> logits;
58     std::int32_t predicted_token = 0;
59 };
60
61 struct Gpt2HotspotProfile {
62     std::int64_t decoder_steps = 0;
63     std::int64_t layer_steps = 0;
64     double total_step_ms = 0.0;
65     double embedding_ms = 0.0;
66     double layer_norm_ms = 0.0;
67     double final_norm_ms = 0.0;
68     double qkv_projection_ms = 0.0;
69     double kv_append_ms = 0.0;
70     double attention_ms = 0.0;
71     double attention_projection_ms = 0.0;
72     double residual_add_ms = 0.0;
73     double mlp_fc_ms = 0.0;
74     double gelu_ms = 0.0;
75     double mlp_projection_ms = 0.0;
76     double lm_head_ms = 0.0;
77     double logits_result_ms = 0.0;
78 };
79
80 struct Gpt2ExecutionOptions {

```

```

81     std::int32_t worker_count = 0;
82     std::int32_t parallel_min_rows = 512;
83 };
84
85 class Gpt2KvCache;
86
87 namespace detail {
88     class Gpt2ParallelWorkerPool;
89
90     void attend_cached_position(const Gpt2KvCache& cache,
91                               std::int32_t layer_id,
92                               std::int32_t model_position,
93                               std::span<const float> query,
94                               std::span<float> scores,
95                               std::span<float> output);
96 } // namespace detail
97
98 class Gpt2ForwardWorkspace {
99 public:
100     explicit Gpt2ForwardWorkspace(const Gpt2Config& config);
101
102     void reset_for_config(const Gpt2Config& config);
103
104     [[nodiscard]] std::span<float> hidden() noexcept;
105     [[nodiscard]] std::span<float> normed() noexcept;
106     [[nodiscard]] std::span<float> query() noexcept;
107     [[nodiscard]] std::span<float> key() noexcept;
108     [[nodiscard]] std::span<float> value() noexcept;
109     [[nodiscard]] std::span<float> attention() noexcept;
110     [[nodiscard]] std::span<float> projected() noexcept;
111     [[nodiscard]] std::span<float> qkv_packed() noexcept;
112     [[nodiscard]] std::span<float> mlp_fc() noexcept;
113     [[nodiscard]] std::span<float> mlp_out() noexcept;
114     [[nodiscard]] std::span<float> logits() noexcept;
115     [[nodiscard]] std::span<float> scores_prefix(std::int32_t count);
116
117 private:
118     Gpt2Config config_;
119     std::vector<float> hidden_;
120     std::vector<float> normed_;
121     std::vector<float> query_;
122     std::vector<float> key_;
123     std::vector<float> value_;
124     std::vector<float> attention_;
125     std::vector<float> projected_;
126     std::vector<float> qkv_packed_;
127     std::vector<float> mlp_fc_;
128     std::vector<float> mlp_out_;
129     std::vector<float> logits_;
130     std::vector<float> scores_;
131 };
132

```

```

133 class Gpt2KvCache {
134 public:
135     explicit Gpt2KvCache(const Gpt2Config& config);
136
137     void append(std::int32_t layer_id,
138                std::int32_t model_position,
139                std::span<const float> key,
140                std::span<const float> value);
141
142     [[nodiscard]] std::span<const float> key(std::int32_t layer_id,
143                                              std::int32_t model_position,
144                                              std::int32_t head) const;
145
146     [[nodiscard]] std::span<const float> value(std::int32_t layer_id,
147                                                std::int32_t model_position,
148                                                std::int32_t head) const;
149
150     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
151     [[nodiscard]] std::size_t byte_size() const noexcept;
152
153 private:
154     friend void detail::attend_cached_position(const Gpt2KvCache& cache,
155                                              std::int32_t layer_id,
156                                              std::int32_t model_position,
157                                              std::span<const float> query,
158                                              std::span<float> scores,
159                                              std::span<float> output);
160
161     [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
162                                           std::int32_t model_position,
163                                           std::int32_t head) const;
164     [[nodiscard]] std::size_t base_offset_unchecked(std::int32_t layer_id,
165                                                    std::int32_t model_position ←
166
167     ↪ ,
168
169     ↪ std::int32_t head) const ↪
170     ↪ noexcept;
171     [[nodiscard]] std::span<const float> key_unchecked(std::int32_t layer_id,
172                                                         std::int32_t ↪
173     ↪ model_position,
174
175     ↪ std::int32_t head) const ↪
176     ↪ noexcept;
177     [[nodiscard]] std::span<const float> value_unchecked(std::int32_t layer_id,
178                                                         std::int32_t ↪
179     ↪ model_position,
180
181     ↪ std::int32_t head) ↪
182     ↪ const noexcept;
183     void validate_address(std::int32_t layer_id,
184                          std::int32_t model_position,
185                          std::int32_t head) const;
186     void validate_written(std::int32_t layer_id,
187                          std::int32_t model_position) const;
188
189 private:
190     void validate_address(const Gpt2KvCache& cache,
191                          std::int32_t layer_id,
192                          std::int32_t model_position,
193                          std::int32_t head) const;
194     void validate_written(const Gpt2KvCache& cache,
195                          std::int32_t layer_id,
196                          std::int32_t model_position) const;
197
198 private:
199     void validate_address(const Gpt2KvCache& cache,
200                          std::int32_t layer_id,
201                          std::int32_t model_position,
202                          std::int32_t head) const;
203     void validate_written(const Gpt2KvCache& cache,
204                          std::int32_t layer_id,
205                          std::int32_t model_position) const;
206
207 private:
208     void validate_address(const Gpt2KvCache& cache,
209                          std::int32_t layer_id,
210                          std::int32_t model_position,
211                          std::int32_t head) const;
212     void validate_written(const Gpt2KvCache& cache,
213                          std::int32_t layer_id,
214                          std::int32_t model_position) const;
215
216 private:
217     void validate_address(const Gpt2KvCache& cache,
218                          std::int32_t layer_id,
219                          std::int32_t model_position,
220                          std::int32_t head) const;
221     void validate_written(const Gpt2KvCache& cache,
222                          std::int32_t layer_id,
223                          std::int32_t model_position) const;
224
225 private:
226     void validate_address(const Gpt2KvCache& cache,
227                          std::int32_t layer_id,
228                          std::int32_t model_position,
229                          std::int32_t head) const;
230     void validate_written(const Gpt2KvCache& cache,
231                          std::int32_t layer_id,
232                          std::int32_t model_position) const;
233
234 private:
235     void validate_address(const Gpt2KvCache& cache,
236                          std::int32_t layer_id,
237                          std::int32_t model_position,
238                          std::int32_t head) const;
239     void validate_written(const Gpt2KvCache& cache,
240                          std::int32_t layer_id,
241                          std::int32_t model_position) const;
242
243 private:
244     void validate_address(const Gpt2KvCache& cache,
245                          std::int32_t layer_id,
246                          std::int32_t model_position,
247                          std::int32_t head) const;
248     void validate_written(const Gpt2KvCache& cache,
249                          std::int32_t layer_id,
250                          std::int32_t model_position) const;
251
252 private:
253     void validate_address(const Gpt2KvCache& cache,
254                          std::int32_t layer_id,
255                          std::int32_t model_position,
256                          std::int32_t head) const;
257     void validate_written(const Gpt2KvCache& cache,
258                          std::int32_t layer_id,
259                          std::int32_t model_position) const;
260
261 private:
262     void validate_address(const Gpt2KvCache& cache,
263                          std::int32_t layer_id,
264                          std::int32_t model_position,
265                          std::int32_t head) const;
266     void validate_written(const Gpt2KvCache& cache,
267                          std::int32_t layer_id,
268                          std::int32_t model_position) const;
269
270 private:
271     void validate_address(const Gpt2KvCache& cache,
272                          std::int32_t layer_id,
273                          std::int32_t model_position,
274                          std::int32_t head) const;
275     void validate_written(const Gpt2KvCache& cache,
276                          std::int32_t layer_id,
277                          std::int32_t model_position) const;
278
279 private:
280     void validate_address(const Gpt2KvCache& cache,
281                          std::int32_t layer_id,
282                          std::int32_t model_position,
283                          std::int32_t head) const;
284     void validate_written(const Gpt2KvCache& cache,
285                          std::int32_t layer_id,
286                          std::int32_t model_position) const;
287
288 private:
289     void validate_address(const Gpt2KvCache& cache,
290                          std::int32_t layer_id,
291                          std::int32_t model_position,
292                          std::int32_t head) const;
293     void validate_written(const Gpt2KvCache& cache,
294                          std::int32_t layer_id,
295                          std::int32_t model_position) const;
296
297 private:
298     void validate_address(const Gpt2KvCache& cache,
299                          std::int32_t layer_id,
300                          std::int32_t model_position,
301                          std::int32_t head) const;
302     void validate_written(const Gpt2KvCache& cache,
303                          std::int32_t layer_id,
304                          std::int32_t model_position) const;
305
306 private:
307     void validate_address(const Gpt2KvCache& cache,
308                          std::int32_t layer_id,
309                          std::int32_t model_position,
310                          std::int32_t head) const;
311     void validate_written(const Gpt2KvCache& cache,
312                          std::int32_t layer_id,
313                          std::int32_t model_position) const;
314
315 private:
316     void validate_address(const Gpt2KvCache& cache,
317                          std::int32_t layer_id,
318                          std::int32_t model_position,
319                          std::int32_t head) const;
320     void validate_written(const Gpt2KvCache& cache,
321                          std::int32_t layer_id,
322                          std::int32_t model_position) const;
323
324 private:
325     void validate_address(const Gpt2KvCache& cache,
326                          std::int32_t layer_id,
327                          std::int32_t model_position,
328                          std::int32_t head) const;
329     void validate_written(const Gpt2KvCache& cache,
330                          std::int32_t layer_id,
331                          std::int32_t model_position) const;
332
333 private:
334     void validate_address(const Gpt2KvCache& cache,
335                          std::int32_t layer_id,
336                          std::int32_t model_position,
337                          std::int32_t head) const;
338     void validate_written(const Gpt2KvCache& cache,
339                          std::int32_t layer_id,
340                          std::int32_t model_position) const;
341
342 private:
343     void validate_address(const Gpt2KvCache& cache,
344                          std::int32_t layer_id,
345                          std::int32_t model_position,
346                          std::int32_t head) const;
347     void validate_written(const Gpt2KvCache& cache,
348                          std::int32_t layer_id,
349                          std::int32_t model_position) const;
350
351 private:
352     void validate_address(const Gpt2KvCache& cache,
353                          std::int32_t layer_id,
354                          std::int32_t model_position,
355                          std::int32_t head) const;
356     void validate_written(const Gpt2KvCache& cache,
357                          std::int32_t layer_id,
358                          std::int32_t model_position) const;
359
360 private:
361     void validate_address(const Gpt2KvCache& cache,
362                          std::int32_t layer_id,
363                          std::int32_t model_position,
364                          std::int32_t head) const;
365     void validate_written(const Gpt2KvCache& cache,
366                          std::int32_t layer_id,
367                          std::int32_t model_position) const;
368
369 private:
370     void validate_address(const Gpt2KvCache& cache,
371                          std::int32_t layer_id,
372                          std::int32_t model_position,
373                          std::int32_t head) const;
374     void validate_written(const Gpt2KvCache& cache,
375                          std::int32_t layer_id,
376                          std::int32_t model_position) const;
377
378 private:
379     void validate_address(const Gpt2KvCache& cache,
380                          std::int32_t layer_id,
381                          std::int32_t model_position,
382                          std::int32_t head) const;
383     void validate_written(const Gpt2KvCache& cache,
384                          std::int32_t layer_id,
385                          std::int32_t model_position) const;
386
387 private:
388     void validate_address(const Gpt2KvCache& cache,
389                          std::int32_t layer_id,
390                          std::int32_t model_position,
391                          std::int32_t head) const;
392     void validate_written(const Gpt2KvCache& cache,
393                          std::int32_t layer_id,
394                          std::int32_t model_position) const;
395
396 private:
397     void validate_address(const Gpt2KvCache& cache,
398                          std::int32_t layer_id,
399                          std::int32_t model_position,
400                          std::int32_t head) const;
401     void validate_written(const Gpt2KvCache& cache,
402                          std::int32_t layer_id,
403                          std::int32_t model_position) const;
404
405 private:
406     void validate_address(const Gpt2KvCache& cache,
407                          std::int32_t layer_id,
408                          std::int32_t model_position,
409                          std::int32_t head) const;
410     void validate_written(const Gpt2KvCache& cache,
411                          std::int32_t layer_id,
412                          std::int32_t model_position) const;
413
414 private:
415     void validate_address(const Gpt2KvCache& cache,
416                          std::int32_t layer_id,
417                          std::int32_t model_position,
418                          std::int32_t head) const;
419     void validate_written(const Gpt2KvCache& cache,
420                          std::int32_t layer_id,
421                          std::int32_t model_position) const;
422
423 private:
424     void validate_address(const Gpt2KvCache& cache,
425                          std::int32_t layer_id,
426                          std::int32_t model_position,
427                          std::int32_t head) const;
428     void validate_written(const Gpt2KvCache& cache,
429                          std::int32_t layer_id,
430                          std::int32_t model_position) const;
431
432 private:
433     void validate_address(const Gpt2KvCache& cache,
434                          std::int32_t layer_id,
435                          std::int32_t model_position,
436                          std::int32_t head)
```

```

179     Gpt2Config config_;
180     std::int32_t filled_tokens_ = 0;
181     std::vector<float> keys_;
182     std::vector<float> values_;
183     std::vector<std::uint8_t> written_;
184 };
185
186 class Gpt2CachedGreedyDecoder {
187 public:
188     explicit Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model);
189     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
190                             Gpt2HotspotProfile* hotspot_profile);
191     Gpt2CachedGreedyDecoder(const Gpt2ReferenceModel& model,
192                             Gpt2HotspotProfile* hotspot_profile,
193                             const Gpt2ExecutionOptions& execution_options);
194     ~Gpt2CachedGreedyDecoder();
195
196     Gpt2CachedGreedyDecoder(const Gpt2CachedGreedyDecoder&) = delete;
197     Gpt2CachedGreedyDecoder& operator=(const Gpt2CachedGreedyDecoder&) = delete; ↵
198     ↵ ;
199     Gpt2CachedGreedyDecoder(Gpt2CachedGreedyDecoder&&) noexcept;
200     Gpt2CachedGreedyDecoder& operator=(Gpt2CachedGreedyDecoder&&) noexcept;
201
202     [[nodiscard]] std::int32_t step(std::int32_t token_id);
203     [[nodiscard]] Gpt2ForwardResult step_with_logits(std::int32_t token_id);
204     [[nodiscard]] const Gpt2KvCache& cache() const noexcept;
205     [[nodiscard]] std::int32_t filled_tokens() const noexcept;
206     [[nodiscard]] std::size_t kv_cache_bytes() const noexcept;
207     [[nodiscard]] std::int32_t worker_count() const noexcept;
208 private:
209     const Gpt2ReferenceModel* model_;
210     Gpt2HotspotProfile* hotspot_profile_;
211     Gpt2ExecutionOptions execution_options_;
212     Gpt2KvCache cache_;
213     Gpt2ForwardWorkspace workspace_;
214     std::unique_ptr<detail::Gpt2ParallelWorkerPool> worker_pool_;
215 };
216
217 struct Gpt2Tokenizer {
218     std::int32_t bos_token_id = -1;
219     std::int32_t eos_token_id = -1;
220     std::unordered_map<std::string, std::int32_t> vocab;
221     std::vector<std::string> id_to_token;
222     std::unordered_map<std::string, std::int32_t> bpe_ranks;
223 };
224
225 [[nodiscard]] Gpt2Tokenizer load_gpt2_tokenizer(
226     const std::filesystem::path& vocab_json,
227     const std::filesystem::path& merges_txt,
228     std::int32_t bos_token_id,
229     std::int32_t eos_token_id);

```

```

230
231 [[nodiscard]] std::vector<std::int32_t> gpt2_encode(
232     const Gpt2Tokenizer& tokenizer,
233     std::string_view text);
234
235 [[nodiscard]] std::string gpt2_decode(
236     const Gpt2Tokenizer& tokenizer,
237     std::span<const std::int32_t> token_ids);
238
239 [[nodiscard]] Gpt2Config load_gpt2_config(const std::filesystem::path& ↵
    ↵ config_json);
240
241 [[nodiscard]] Gpt2ReferenceModel load_gpt2_reference_model(
242     const std::filesystem::path& config_json,
243     const std::filesystem::path& safetensors_path);
244
245 [[nodiscard]] Gpt2ReferenceModel load_gpt2_from_directory(
246     const std::filesystem::path& model_directory);
247
248 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward(
249     const Gpt2ReferenceModel& model,
250     std::span<const std::int32_t> token_ids);
251
252 [[nodiscard]] Gpt2ForwardResult run_gpt2_forward_cached(
253     const Gpt2ReferenceModel& model,
254     Gpt2KvCache& cache,
255     std::int32_t token_id);
256
257 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy(
258     const Gpt2ReferenceModel& model,
259     std::span<const std::int32_t> prompt_token_ids,
260     std::int32_t max_new_tokens);
261
262 [[nodiscard]] std::vector<std::int32_t> gpt2_generate_greedy_cached(
263     const Gpt2ReferenceModel& model,
264     std::span<const std::int32_t> prompt_token_ids,
265     std::int32_t max_new_tokens);
266
267 [[nodiscard]] Gpt2ReferenceModel make_tiny_gpt2_reference_model();
268
269 } // namespace lcqi

```

最短闭环：从模型文件到 logits

tiny MLP 路径不是为了证明 LCQI 已经是完整大模型引擎，而是为了建立最短的、可手算的、可测试的推理闭环。读者要把模型加载、输入检查、int8 linear、ReLU、第二层 logits、argmax 和 CLI 输出连起来看。

模型加载与基础推理

`model.cpp` 负责解析 tiny 文本模型、权重、scale、bias 和 shape。坏格式、坏维度和坏数值都应该在这里被拒绝，不能拖到 kernel 内部才崩溃。

Listing 4.10 src/model.cpp

```

1  #include <lcqi/model.hpp>
2
3  #include <fstream>
4  #include <limits>
5  #include <stdexcept>
6  #include <string>
7
8  namespace lcqi {
9      namespace {
10
11         constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12         constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14         std::string read_key(std::istream& input) {
15             std::string key;
16             if (!(input >> key)) {
17                 throw std::runtime_error("unexpected end of model file");
18             }
19             return key;
20         }
21
22         void expect_key(std::istream& input, const std::string& expected) {
23             const std::string key = read_key(input);
24             if (key != expected) {
25                 throw std::runtime_error("expected key '" + expected + "', got '" + key +
26                 ↵ + "'");
27             }
28
29             std::int32_t read_i32(std::istream& input, const std::string& key) {
30                 expect_key(input, key);
31                 std::int32_t value = 0;
32                 if (!(input >> value)) {
33                     throw std::runtime_error("invalid int32 value for key '" + key + "'");
34                 }
35                 return value;
36             }
37
38             float read_float(std::istream& input, const std::string& key) {
39                 expect_key(input, key);
40                 float value = 0.0F;
41                 if (!(input >> value)) {
42                     throw std::runtime_error("invalid float value for key '" + key + "'");
43                 }
44                 return value;
45             }
46
47             std::vector<std::int8_t> read_i8_vector(std::istream& input,
48                                                     const std::string& key,
49                                                     std::int32_t count) {
50                 expect_key(input, key);

```

```

51     if (count < 0) {
52         throw std::runtime_error("negative vector size for key '" + key + "'");
53     }
54     std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55     for (std::int32_t i = 0; i < count; ++i) {
56         std::int32_t raw = 0;
57         if (!(input >> raw)) {
58             throw std::runtime_error("invalid int8 payload for key '" + key + "'");
59         }
60         if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61             throw std::runtime_error("int8 payload out of range for key '" + key + "'");
62         }
63         values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
64     }
65     return values;
66 }
67
68 std::vector<float> read_float_vector(std::istream& input,
69                                     const std::string& key,
70                                     std::int32_t count) {
71     expect_key(input, key);
72     if (count < 0) {
73         throw std::runtime_error("negative vector size for key '" + key + "'");
74     }
75     std::vector<float> values(static_cast<std::size_t>(count));
76     for (std::int32_t i = 0; i < count; ++i) {
77         if (!(input >> values[static_cast<std::size_t>(i)])) {
78             throw std::runtime_error("invalid float payload for key '" + key + "'");
79         }
80     }
81     return values;
82 }
83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
85     if (layer.input_size <= 0 || layer.output_size <= 0) {
86         throw std::runtime_error(name + " layer has invalid shape");
87     }
88     const std::size_t expected_weights =
89         static_cast<std::size_t>(layer.input_size) *
90         static_cast<std::size_t>(layer.output_size);
91     if (layer.weights.size() != expected_weights) {
92         throw std::runtime_error(name + " layer weight size mismatch");
93     }
94     if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95         throw std::runtime_error(name + " layer bias size mismatch");
96     }
97 }
98

```

```

99 std::int32_t checked_element_count(std::int32_t rows,
100                                   std::int32_t columns,
101                                   const std::string& name) {
102     if (rows <= 0 || columns <= 0) {
103         throw std::runtime_error(name + " layer has invalid shape");
104     }
105     const std::int64_t count =
106         static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107     if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::↵
↵ max())) {
108         throw std::runtime_error(name + " layer element count is too large");
109     }
110     return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116     std::ifstream input(path);
117     if (!input) {
118         throw std::runtime_error("cannot open model file: " + path.string());
119     }
120
121     expect_key(input, "LCQI_MODEL_V1");
122
123     TinyMlpModel model;
124     model.input_size = read_i32(input, "input_size");
125     model.hidden_size = read_i32(input, "hidden_size");
126     model.output_size = read_i32(input, "output_size");
127
128     const std::int32_t hidden_weight_count =
129         checked_element_count(model.input_size, model.hidden_size, "hidden");
130     const std::int32_t output_weight_count =
131         checked_element_count(model.hidden_size, model.output_size, "output");
132
133     model.hidden.input_size = model.input_size;
134     model.hidden.output_size = model.hidden_size;
135     model.hidden.scale = read_float(input, "hidden_scale");
136     model.hidden.weights = read_i8_vector(
137         input,
138         "hidden_weights",
139         hidden_weight_count);
140     model.hidden.bias = read_float_vector(input, "hidden_bias", model.↵
↵ hidden_size);
141
142     model.output.input_size = model.hidden_size;
143     model.output.output_size = model.output_size;
144     model.output.scale = read_float(input, "output_scale");
145     model.output.weights = read_i8_vector(
146         input,
147         "output_weights",
148         output_weight_count);

```

```

149     model.output.bias = read_float_vector(input, "output_bias", model.↵
↵     output_size);
150
151     validate_layer(model.hidden, "hidden");
152     validate_layer(model.output, "output");
153     return model;
154 }
155
156 } // namespace lcqi

```

`inference.cpp` 执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是系统中最短的“模型资产到输出”路径。

Listing 4.11 `src/inference.cpp`

```

1  #include <lcqi/inference.hpp>
2
3  #include <algorithm>
4  #include <cmath>
5  #include <stdexcept>
6
7  namespace lcqi {
8  namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11     if (input.size() != static_cast<std::size_t>(expected)) {
12         throw std::runtime_error("input size does not match model shape");
13     }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17     if (values.empty()) {
18         throw std::runtime_error("cannot take argmax of empty logits");
19     }
20     std::int32_t best_index = 0;
21     float best_value = values[0];
22     for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i)↵
↵     {
23         const float candidate = values[static_cast<std::size_t>(i)];
24         if (candidate > best_value) {
25             best_value = candidate;
26             best_index = i;
27         }
28     }
29     return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35               std::span<const float> input,
36               std::span<float> output) {
37     if (input.size() != static_cast<std::size_t>(layer.input_size)) {

```

```

38     throw std::runtime_error("linear_i8 input size mismatch");
39 }
40 if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41     throw std::runtime_error("linear_i8 output size mismatch");
42 }
43
44 for (std::int32_t out = 0; out < layer.output_size; ++out) {
45     const std::size_t row_base =
46         static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.↵
↵ input_size);
47     float acc = layer.bias[static_cast<std::size_t>(out)];
48     for (std::int32_t in = 0; in < layer.input_size; ++in) {
49         const std::int8_t weight =
50             layer.weights[row_base + static_cast<std::size_t>(in)];
51         acc += static_cast<float>(weight) * layer.scale *
52             input[static_cast<std::size_t>(in)];
53     }
54     output[static_cast<std::size_t>(out)] = acc;
55 }
56 }
57
58 void relu_inplace(std::span<float> values) {
59     for (float& value : values) {
60         value = std::max(0.0F, value);
61     }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65                               std::span<const float> input) {
66     validate_input_size(input, model.input_size);
67
68     std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F↵
↵ );
69     std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F↵
↵ );
70
71     linear_i8(model.hidden, input, hidden);
72     relu_inplace(hidden);
73     linear_i8(model.output, hidden, logits);
74
75     InferenceResult result;
76     result.predicted_class = argmax(logits);
77     result.logits = std::move(logits);
78     return result;
79 }
80
81 } // namespace lcqi

```

`main.cpp` 把模型路径和输入接到 `run_inference`，并打印结果。CLI 应保持薄层，不复制模型解析和推理逻辑。

Listing 4.12 src/main.cpp

```

1 #include <lcqi/inference.hpp>
2 #include <lcqi/model.hpp>
3
4 #include <chrono>
5 #include <cstdint>
6 #include <cstdlib>
7 #include <filesystem>
8 #include <iostream>
9 #include <span>
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
    ↵ expected_size) {
21     if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22         throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
    ↵ [repeat]");
23     }
24     std::vector<float> input(static_cast<std::size_t>(expected_size));
25     for (std::int32_t i = 0; i < expected_size; ++i) {
26         input[static_cast<std::size_t>(i)] =
27             std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28     }
29     return input;
30 }
31
32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33     const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
34     if (argc <= repeat_index) {
35         return LCQI_DEFAULT_REPEAT;
36     }
37     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
    ↵ repeat_index]));
38     if (repeat <= 0) {
39         throw std::runtime_error("repeat must be positive");
40     }
41     return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47     try {
48         if (argc < LCQI_EXPECTED_ARGC_MIN) {
49             throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats↵

```

```

↪ ...> [repeat]);
50     }
51
52     const std::filesystem::path model_path = argv[1];
53     const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54     const std::vector<float> input = parse_input(argc, argv, model.↪
↪ input_size);
55     const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58     std::cout << "predicted_class " << result.predicted_class << "\n";
59     std::cout << "logits";
60     for (const float value : result.logits) {
61         std::cout << ' ' << value;
62     }
63     std::cout << "\n";
64
65     const auto begin = std::chrono::steady_clock::now();
66     std::int32_t checksum = 0;
67     for (std::int32_t i = 0; i < repeat; ++i) {
68         checksum += lcqi::run_inference(model, input).predicted_class;
69     }
70     const auto end = std::chrono::steady_clock::now();
71     const std::chrono::duration<double> elapsed = end - begin;
72     const double average_us =
73         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↪
↪ >(repeat);
74     std::cout << "repeat " << repeat << "\n";
75     std::cout << "average_us " << average_us << "\n";
76     std::cout << "checksum " << checksum << "\n";
77     return EXIT_SUCCESS;
78 } catch (const std::exception& error) {
79     std::cerr << "error: " << error.what() << "\n";
80     return EXIT_FAILURE;
81 }
82 }

```

Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照，也是 benchmark 的正确性底座。

Listing 4.13 src/int8_kernels.cpp

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #include <lcqi/inference.hpp>
4
5 #include <algorithm>
6 #include <chrono>
7 #include <cmath>
8 #include <stdexcept>

```



```

9  #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15 constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16 constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17 constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27     if (value <= 0) {
28         throw std::runtime_error(std::string(name) + " must be positive");
29     }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33     if (value < 0) {
34         throw std::runtime_error(std::string(name) + " cannot be negative");
35     }
36     return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40     require_positive(layer.input_size, "input_size");
41     require_positive(layer.output_size, "output_size");
42     const std::size_t expected_weights =
43         checked_size(layer.input_size, "input_size") *
44         checked_size(layer.output_size, "output_size");
45     if (layer.weights.size() != expected_weights) {
46         throw std::runtime_error("quantized layer weight size mismatch");
47     }
48     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("quantized layer bias size mismatch");
50     }
51 }
52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54     require_positive(value, "value");
55     require_positive(block_size, "block_size");
56     return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60     float sum = 0.0F;

```

```

61     for (const float value : values) {
62         sum += value;
63     }
64     return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68     result.backends.push_back(KernelBackendBenchmark{
69         .name = name,
70         .available = false,
71         .average_us = 0.0,
72         .max_abs_diff = 0.0F,
73         .checksum = 0.0F,
74     });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79     require_positive(repeat, "repeat");
80     const auto begin = std::chrono::steady_clock::now();
81     for (std::int32_t index = 0; index < repeat; ++index) {
82         callable();
83     }
84     const auto end = std::chrono::steady_clock::now();
85     const std::chrono::duration<double> elapsed = end - begin;
86     return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>
↵ >(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92                               std::int32_t output_block_size) {
93     validate_quantized_layer(layer);
94     require_positive(output_block_size, "output_block_size");
95
96     PackedLinearI8 packed;
97     packed.input_size = layer.input_size;
98     packed.output_size = layer.output_size;
99     packed.output_block_size = output_block_size;
100    packed.scale = layer.scale;
101    packed.bias = layer.bias;
102
103    const std::int32_t padded_outputs =
104        round_up_to_block(layer.output_size, output_block_size);
105    packed.packed_weights.assign(
106        checked_size(padded_outputs, "padded_outputs") *
107        checked_size(layer.input_size, "input_size"),
108        0);
109
110    std::size_t packed_index = 0;
111    for (std::int32_t output_block = 0; output_block < padded_outputs;

```

```

112     output_block += output_block_size) {
113         for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
114             for (std::int32_t offset = 0; offset < output_block_size; ++offset) {
115                 const std::int32_t output_index = output_block + offset;
116                 if (output_index < layer.output_size) {
117                     const std::size_t source =
118                         checked_size(output_index, "output_index") *
119                         checked_size(layer.input_size, "input_size") +
120                         checked_size(input_index, "input_index");
121                     packed.packed_weights[packed_index] = layer.weights[source];
122                 }
123                 ++packed_index;
124             }
125         }
126     }
127     return packed;
128 }
129
130 void linear_i8_packed(const PackedLinearI8& layer,
131                      std::span<const float> input,
132                      std::span<float> output) {
133     std::vector<float> accumulators(
134         checked_size(layer.output_block_size, "output_block_size"),
135         0.0F);
136     linear_i8_packed_with_workspace(layer, input, output, accumulators);
137 }
138
139 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
140                                     std::span<const float> input,
141                                     std::span<float> output,
142                                     std::span<float> accumulator_workspace) {
143     require_positive(layer.input_size, "packed input_size");
144     require_positive(layer.output_size, "packed output_size");
145     require_positive(layer.output_block_size, "packed output_block_size");
146     if (input.size() != checked_size(layer.input_size, "input_size")) {
147         throw std::runtime_error("linear_i8_packed input size mismatch");
148     }
149     if (output.size() != checked_size(layer.output_size, "output_size")) {
150         throw std::runtime_error("linear_i8_packed output size mismatch");
151     }
152     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153         throw std::runtime_error("linear_i8_packed bias size mismatch");
154     }
155
156     const std::int32_t padded_outputs =
157         round_up_to_block(layer.output_size, layer.output_block_size);
158     const std::size_t expected_packed_size =
159         checked_size(padded_outputs, "padded_outputs") *
160         checked_size(layer.input_size, "input_size");

```

```

161     if (layer.packed_weights.size() != expected_packed_size) {
162         throw std::runtime_error("linear_i8_packed packed weight size mismatch"
163     ↪ );
164     }
165     if (accumulator_workspace.size() != checked_size(layer.output_block_size, "
166     ↪ output_block_size")) {
167         throw std::runtime_error("linear_i8_packed accumulator workspace size
168     ↪ mismatch");
169     }
170
171     std::size_t packed_index = 0;
172     for (std::int32_t output_block = 0; output_block < padded_outputs;
173         output_block += layer.output_block_size) {
174         std::fill(accumulator_workspace.begin(), accumulator_workspace.end(),
175     ↪ 0.0F);
176         for (std::int32_t offset = 0; offset < layer.output_block_size; ++
177     ↪ offset) {
178             const std::int32_t output_index = output_block + offset;
179             if (output_index < layer.output_size) {
180                 accumulator_workspace[checked_size(offset, "offset")] =
181                 layer.bias[checked_size(output_index, "output_index")];
182             }
183         }
184
185         for (std::int32_t input_index = 0; input_index < layer.input_size; ++
186     ↪ input_index) {
187             const float input_value = input[checked_size(input_index, "
188     ↪ input_index")];
189             for (std::int32_t offset = 0; offset < layer.output_block_size; ++
190     ↪ offset) {
191                 const std::int32_t output_index = output_block + offset;
192                 const std::int8_t weight = layer.packed_weights[packed_index
193     ↪ ++];
194                 if (output_index < layer.output_size) {
195                     accumulator_workspace[checked_size(offset, "offset")] +=
196                     static_cast<float>(weight) * layer.scale * input_value;
197                 }
198             }
199         }
200     }
201
202     QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,

```

```

203         std::int32_t output_size,
204         float scale) {
205     require_positive(input_size, "input_size");
206     require_positive(output_size, "output_size");
207     QuantizedLinearLayer layer;
208     layer.input_size = input_size;
209     layer.output_size = output_size;
210     layer.scale = scale;
211     const std::size_t weight_count =
212         checked_size(input_size, "input_size") * checked_size(output_size, "↵
↵ output_size");
213     layer.weights.resize(weight_count);
214     layer.bias.resize(checked_size(output_size, "output_size"));
215     for (std::int32_t output_index = 0; output_index < output_size; ++↵
↵ output_index) {
216         layer.bias[checked_size(output_index, "output_index")] =
217             static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218                               LCQI_INPUT_PATTERN_CENTER) *
219             LCQI_INPUT_PATTERN_SCALE;
220         for (std::int32_t input_index = 0; input_index < input_size; ++↵
↵ input_index) {
221             const std::int32_t raw =
222                 ((output_index * input_size + input_index) % ↵
↵ LCQI_I8_PATTERN_MODULUS) -
223                 LCQI_I8_PATTERN_CENTER;
224             layer.weights[checked_size(output_index, "output_index") *
225                           checked_size(input_size, "input_size") +
226                           checked_size(input_index, "input_index")] =
227                 static_cast<std::int8_t>(raw);
228         }
229     }
230     return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {
234     require_positive(input_size, "input_size");
235     std::vector<float> input(checked_size(input_size, "input_size"));
236     for (std::int32_t input_index = 0; input_index < input_size; ++input_index)↵
↵ {
237         input[checked_size(input_index, "input_index")] =
238             static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239                               LCQI_INPUT_PATTERN_CENTER) *
240             LCQI_INPUT_PATTERN_SCALE;
241     }
242     return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246     if (lhs.size() != rhs.size()) {
247         throw std::runtime_error("max_abs_diff size mismatch");
248     }
249     float diff = 0.0F;

```

```

250     for (std::size_t index = 0; index < lhs.size(); ++index) {
251         diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252     }
253     return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ←
    ↪ benchmark_case) {
257     require_positive(benchmark_case.input_size, "benchmark input_size");
258     require_positive(benchmark_case.output_size, "benchmark output_size");
259     require_positive(benchmark_case.output_block_size, "benchmark ←
    ↪ output_block_size");
260     require_positive(benchmark_case.repeat, "benchmark repeat");
261
262     const QuantizedLinearLayer layer =
263         make_deterministic_i8_layer(benchmark_case.input_size,
264                                     benchmark_case.output_size,
265                                     0.015625F);
266     const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.↪
    ↪ output_block_size);
267     const PackedLinearI8 packed_simd = pack_linear_i8(layer, ↪
    ↪ LCQI_SIMD_OUTPUT_BLOCK);
268     const std::vector<float> input = make_deterministic_input(benchmark_case.↪
    ↪ input_size);
269     std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "↪
    ↪ output_size"),
270                                     0.0F);
271     std::vector<float> packed_output(scalar_output.size(), 0.0F);
272     std::vector<float> simd_output(scalar_output.size(), 0.0F);
273     std::vector<float> packed_workspace(
274         checked_size(benchmark_case.output_block_size, "output_block_size"),
275         0.0F);
276
277     linear_i8(layer, input, scalar_output);
278     linear_i8_packed_with_workspace(packed, input, packed_output, ↪
    ↪ packed_workspace);
279
280     KernelBenchmarkResult result;
281     result.benchmark_case = benchmark_case;
282     result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283     result.backends.push_back(KernelBackendBenchmark{
284         .name = LCQI_BACKEND_SCALAR,
285         .available = true,
286         .average_us = measure_average_us(benchmark_case.repeat,
287                                         [&]() { linear_i8(layer, input, ↪
    ↪ scalar_output); }),
288         .max_abs_diff = 0.0F,
289         .checksum = checksum(scalar_output),
290     });
291     result.backends.push_back(KernelBackendBenchmark{
292         .name = LCQI_BACKEND_PACKED_SCALAR,
293         .available = true,

```

```

294     .average_us = measure_average_us(
295         benchmark_case.repeat,
296         [&]() { linear_i8_packed_with_workspace(packed,
297                                                     input,
298                                                     packed_output,
299                                                     packed_workspace); }),
300     .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301     .checksum = checksum(packed_output),
302 );
303 if (linear_i8_packed_avx2_available()) {
304     linear_i8_packed_avx2(packed_simd, input, simd_output);
305     result.backends.push_back(KernelBackendBenchmark{
306         .name = LCQI_BACKEND_AVX2,
307         .available = true,
308         .average_us = measure_average_us(
309             benchmark_case.repeat,
310             [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); ↵
↵ }),
311         .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312         .checksum = checksum(simd_output),
313     });
314 } else {
315     add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316 }
317 return result;
318 }
319
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326     return false;
327 }
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
↵ span<float>) {
330     throw std::runtime_error("AVX2 packed kernel is not enabled in this build") ↵
↵ ;
331 }
332
333 } // namespace lcqi
334 #endif

```

`int8_kernels_avx2.cpp` 是平台优化路径。读者要先看可用性判断，再看向量化实现；如果机器不支持 AVX2/FMA，报告必须写 `unavailable`。

Listing 4.14 `src/int8_kernels_avx2.cpp`

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)

```

```

4
5 #include <immintrin.h>
6
7 #include <algorithm>
8 #include <cstdint>
9 #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {
15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19     if (value <= 0) {
20         throw std::runtime_error(std::string(name) + " must be positive");
21     }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25     if (value < 0) {
26         throw std::runtime_error(std::string(name) + " cannot be negative");
27     }
28     return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32     require_positive(value, "value");
33     require_positive(block_size, "block_size");
34     return ((value + block_size - 1) / block_size) * block_size;
35 }
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38                             std::span<const float> input,
39                             std::span<float> output) {
40     require_positive(layer.input_size, "packed input_size");
41     require_positive(layer.output_size, "packed output_size");
42     if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43         throw std::runtime_error("AVX2 packed kernel requires output_block_size↵
↵ == 8");
44     }
45     if (input.size() != checked_size(layer.input_size, "input_size")) {
46         throw std::runtime_error("AVX2 packed kernel input size mismatch");
47     }
48     if (output.size() != checked_size(layer.output_size, "output_size")) {
49         throw std::runtime_error("AVX2 packed kernel output size mismatch");
50     }
51     if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
52         throw std::runtime_error("AVX2 packed kernel bias size mismatch");
53     }
54     const std::int32_t padded_outputs =

```



```

55     round_up_to_block(layer.output_size, layer.output_block_size);
56     const std::size_t expected_packed_size =
57         checked_size(padded_outputs, "padded_outputs") *
58         checked_size(layer.input_size, "input_size");
59     if (layer.packed_weights.size() != expected_packed_size) {
60         throw std::runtime_error("AVX2 packed kernel packed weight size ↵
↵ mismatch");
61     }
62 }
63
64 } // namespace
65
66 bool linear_i8_packed_avx2_available() noexcept {
67     #if defined(__GNUC__) || defined(__clang__)
68         __builtin_cpu_init();
69         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
70     #else
71         return true;
72     #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76                             std::span<const float> input,
77                             std::span<float> output) {
78     if (!linear_i8_packed_avx2_available()) {
79         throw std::runtime_error("AVX2/FMA is not available on this CPU");
80     }
81     validate_avx2_contract(layer, input, output);
82
83     const std::int32_t padded_outputs =
84         round_up_to_block(layer.output_size, layer.output_block_size);
85     std::size_t packed_index = 0;
86     alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88     for (std::int32_t output_block = 0; output_block < padded_outputs;
89         output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90         _mm256 accumulator = _mm256_setzero_ps();
91
92         for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset ↵
↵ ) {
93             const std::int32_t output_index = output_block + offset;
94             block_output[checked_size(offset, "offset")] =
95                 output_index < layer.output_size
96                     ? layer.bias[checked_size(output_index, "output_index")]
97                     : 0.0F;
98         }
99         accumulator = _mm256_load_ps(block_output);
100
101         for (std::int32_t input_index = 0; input_index < layer.input_size; ++ ↵
↵ input_index) {
102             std::uint64_t packed_bytes = 0;
103             std::memcpy(&packed_bytes,

```

```

104         layer.packed_weights.data() + packed_index,
105         sizeof(packed_bytes));
106     const __m128i packed_i8 =
107         _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
108     packed_index += LCQI_AVX2_OUTPUT_BLOCK;
109     const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
110     const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
111     const __m256 input_scaled =
112         _mm256_set1_ps(input[checked_size(input_index, "input_index")] ←
↵ * layer.scale);
113     accumulator = _mm256_fmadd_ps(weights_f32, input_scaled, ←
↵ accumulator);
114     }
115
116     _mm256_store_ps(block_output, accumulator);
117     for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset↵
↵ ) {
118         const std::int32_t output_index = output_block + offset;
119         if (output_index < layer.output_size) {
120             output[checked_size(output_index, "output_index")] =
121                 block_output[checked_size(offset, "offset")];
122         }
123     }
124 }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

第零部分

Transformer、KV Cache 与服务运行时

第 5 章实践：Transformer 切片与 KV Cache 状态

本章对应原书中的第三册“AI 推理原理、KV cache 与计算账本”和“KV cache 实现细节”。不要把它当作概念复述，本章要把 RMSNorm、Q/K/V、RoPE、GQA、KV slot、position、state edge 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `kv_cache`，核心命令或测试入口是 `lcqi_trace`。

Listing 5.1 第 5 章实践：Transformer 切片与 KV Cache 状态的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / kv_cache
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：固定 prompt `tok1tok2tok3`，观察第一个 decode step。读者要先手写预期结果，再运行项目观察输出。动作是：把 trace 中 tokenizer、q/k/v、`kv_cache_slot`、attention 和 logits 串成状态表。如果输出里没有 `token_position` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：KV cache 是请求状态，不是临时 activation；position、layer、head、layout 和生命周期必须进入 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 KV cache 状态表：每层每个 token 写入哪里、何时读取、何时禁止复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第6章实践：Reference Decoder、logits 与采样

本章对应原书中的第三册“Reference decoder 实现”和“推理算法：logits、softmax、采样”。不要把它当作概念复述，本章要把 reference decoder、logits golden、argmax sampler、tokenizer contract hash、BOS/EOS 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 **argmax**，核心命令或测试入口是 **lcqi_trace**。

Listing 6.1 第6章实践：Reference Decoder、logits 与采样的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / argmax
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：让 trace 固定输出 tokenizer contract 和 logits golden。读者要先手写预期结果，再运行项目观察输出。动作是：比较 generated token、logits 数组和 tokenizer hash。如果输出里没有 **tokenizer_contract** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：采样前必须先证明 logits 可复查；sampler 的随机性、温度和 top-k 只能建立在 reference 通过之后这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 **include/lcqi** 头文件和一个 **src** 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交采样报告：argmax、top-k、temperature 三种策略各自需要哪些额外字段才能复现。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 7 章实践：服务调度、SLO 与资源预算

本章对应原书中的第三册“业务服务实战：请求、调度、队列、取消与资源预算”。不要把它当作概念复述，本章要把 admission、KV budget、queue wait、TTFT、TPOT、cancel、reject、p95 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `ttft_ms`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 7.1 第 7 章实践：服务调度、SLO 与资源预算的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / ttft_ms
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：运行 serving probe 中短请求、RAG 请求、取消请求和超长请求。读者要先手写预期结果，再运行项目观察输出。动作是：解释 `memory_budget_exceeded`、cancel reclaim、`queue_wait_p95_ms` 和 `rejection_rate`。如果输出里没有 `rejection_rate` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：推理服务不是只看 tokens/s；是否接收请求、如何预留 KV、取消后是否回收，决定系统是否可运营这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 SLO 表：每类请求的 prompt/new token、预算、是否接受、TTFT、TPOT、拒绝原因和未验证边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 8 章实践：RAG、Agent 与状态图边界

本章对应原书中的第三册“上层 AI 应用编排：RAG、Agent、工具和状态图”。不要把它当作概念复述，本章要把 retrieval、tool call、prompt assembly、state graph、budget、trace id 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `req_rag`，核心命令或测试入口是 `lcqi_serving_probe`。

Listing 8.1 第 8 章实践：RAG、Agent 与状态图边界的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_serving_probe / req_rag
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 RAG 请求拆成 retrieve、rerank、prompt build、prefill、decode 五段。读者要先手写预期结果，再运行项目观察输出。动作是：为每段写输入输出和失败后能否重试。如果输出里没有 `prompt_tokens` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：上层编排不能把外部工具结果和模型状态混成字符串；每个边界都要有身份、预算和可重放记录这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交一个 RAG 状态图：节点、边、超时、重试、缓存键、prompt 版本和审计字段。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

模型导入、AI 编译器与内存计划

第9章实践：模型导入、manifest 与 Shape Verifier

本章对应原书中的第三册“模型导入、manifest 与 shape verifier”。不要把它当作概念复述，本章要把 safetensors header、metadata、tensor name、dtype、shape、`data_offsets`、payload boundary 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `load_safetensors_manifest`，核心命令或测试入口是 `lcqi_tests`。

Listing 9.1 第9章实践：模型导入、manifest 与 Shape Verifier 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_tests / load_safetensors_manifest
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：加载测试临时 safetensors fixture。读者要先手写预期结果，再运行项目观察输出。动作是：故意构造 offset 越界和 dtype/shape 字节数不匹配。如果输出里没有 `data_offsets` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：模型导入的第一职责是早拒绝：名字、shape、dtype、offset 和 payload 都必须在 kernel 运行前被验证这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 manifest gate 报告：合法 fixture 和两个坏 fixture 的字段、错误原因和拒绝位置。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 10 章实践：真实模型加载闭环与首 Token Trace

本章对应原书中的第三册“真实模型加载闭环：Tokenizer、权重格式与首 token trace”。不要把它当作概念复述，本章要把 tokenizer、weight mapping、chat template、first token trace、checksum 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

本章新增一个明确边界：tiny fixture 负责手算和逐层 oracle；LCQI GPT-2 reference path 负责证明自研代码已经能加载标准 GPT-2 safetensors、执行 byte-level BPE、跑 GPT-2 forward 并 greedy 生成；SmolLM2 small smoke 负责证明外部 llama.cpp 路径能加载 GGUF small 模型。三者不能互相冒充。tiny MLP 算对，不等于 GPT-2 完成；GPT-2 reference 能出一个 token，不等于高性能 KV cache runtime 完成；SmolLM2 能输出文本，也不等于 LCQI 自研 GGUF 后端完成。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里有三组核心对象。第一组是 `tiny_tokenizer.txt`、reference decoder 和 `lcqi_trace`，用于手算 token 合同和逐 checkpoint 追踪。第二组是 `gpt2_reference.hpp/.cpp`、`lcqi_gpt2` 和 `tools/run_gpt2_smoke.py`，用于自研 GPT-2 safetensors 路径。第三组是 `tools/run_smolllm2_small_smoke.py`，用于下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，再通过 llama.cpp 在 CPU 上实际生成 assistant 文本。

Listing 10.1 第 10 章实践：真实模型加载闭环与首 Token Trace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / tiny_tokenizer.txt
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
make cpu3-gpt2-smoke
make cpu3-smolllm2-smoke
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：从 tokenizer fixture 开始，不跳到真实 7B。读者要先手写预期结果，再运行项目观察输出。动作是：解释 BOS/EOS、UNK、template hash 和 decoder 输入 tokens 的边界。如果输出里没有 `tokens=[0,1,2,3,4]` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：真实模型加载不是下载权重；必须先让 tokenizer 合同、权重 role、shape verifier 和首 token trace 形成闭环。这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、自研 GPT-2 reference smoke：先把标准 safetensors 跑通

GPT-2 不是 LLaMA。它使用 LayerNorm，不是 RMSNorm；使用 learned absolute position embedding，不是 RoPE；attention 里的 Q/K/V 在 HuggingFace 权重中是 packed Conv1D 权重；MLP 使用 GELU，不是 SwiGLU；`lm_head` 通常和 token embedding 共享权重。把 GPT-2 硬塞进原来的 LLaMA-like tiny decoder，会让概念混乱，所以 LCQI 新增了独立的 `gpt2_reference` 模块。

仓库内最小验收分两层。第一层不下载模型，运行 tiny GPT-2 fixture：

Listing 10.2 仓库内 tiny GPT-2 数学闭环

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_gpt2
```

它固定 prompt ids 为 **123**，检查 logits、predicted token 和 greedy generated ids。对应测试还会临时写出一个 HuggingFace 风格 **model.safetensors**，再用 loader 读回来，覆盖 tensor name mapping、shape verifier、Conv1D 转置和 tied lm head。坏 shape 必须在 loader 阶段被拒绝。

第二层下载真实 GPT-2 资产：

Listing 10.3 LCQI 自研 GPT-2 smoke

```
make cpu3-gpt2-smoke
```

脚本下载 **openai-community/gpt2** 的 **config.json**、**vocab.json**、**merges.txt** 和 **model.safetensors** 到用户缓存目录，不提交模型文件。最近一次报告写到：

Listing 10.4 GPT-2 smoke 报告关键字段

```
books/cpu-volume-3/results/lcqi-gpt2-smoke.txt
prompt_ids 15496 11 616 1438 318
generated_ids 15496 11 616 1438 318 1757
generated_text Hello, my name is John
validation=PASS
```

这说明自研 LCQI C++ reference path 已经能跑标准 GPT-2 safetensors 的 tokenizer、权重导入、forward 和 greedy generation。边界也要写清：当前实现为了可读和可验证，会重跑完整前缀，没有 KV cache、没有 packed weight、没有多线程，也没有把 logits 与 PyTorch/Transformers 的逐数值 golden 报告纳入仓库。因此它是正确性闭环，不是性能结论。

四、真实 small 模型 smoke：不用 tiny，先让开源模型实际跑起来

本章的真实模型 smoke 固定为 **SmolLM2-135M-Instruct**，规模是 135M 参数，不是 tiny 随机夹具。脚本使用 GGUF 文件 **SmolLM2-135M-Instruct-Q4_K_M.gguf**，来源仓库是 **bartowski/SmolLM2-135M-Instruct-GGUF**，原模型组织是 **HuggingFaceTB**，license 字段为 **apache-2.0**。脚本不会把模型文件提交到仓库，而是缓存到用户目录；验收时检查文件大小 **105454432** 字节和 SHA256：

Listing 10.5 SmolLM2 small GGUF 文件身份

```
model_source_id=HuggingFaceTB/SmolLM2-135M-Instruct
model_gguf_repo_id=bartowski/SmolLM2-135M-Instruct-GGUF
model_file=SmolLM2-135M-Instruct-Q4_K_M.gguf
model_expected_bytes=105454432
model_expected_sha256=2↵
↵ e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d
```

运行命令是：

Listing 10.6 真实 small 模型 CPU 冒烟

```
make cpu3-smolllm2-smoke
```

这条命令会做四步。第一，下载或复用 GGUF 文件，并校验 SHA256，防止“文件名对了但内容不对”。第二，拉取固定 commit 的 `llama.cpp` 并构建 `llama-simple-chat`，避免依赖本机临时安装。第三，用 `-ngl0` 强制 CPU 路径，输入一个短 prompt，确认程序能走过 tokenizer、chat template、prefill、decode 和 sampler。第四，把报告写到：

Listing 10.7 真实模型 smoke 报告路径

```
books/cpu-volume-3/results/lcqi-smolllm2-small-model-smoke.txt
```

报告里必须至少出现这些字段：`model_expected_sha256`、`llama_cpp_commit`、`command`、`exit_code=0`、`validation=PASS` 和 `assistant_response`。如果只有下载日志，没有 assistant response，不能算通过；如果 assistant response 内容答错了，也不能改写成“质量通过”。本章只验证真实模型资产可加载、可 tokenize、可 decode、可生成非空文本。

五、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：运行 GPT-2 smoke。报告不许只贴“生成了 John”，必须解释 `config.json`、`vocab.json`、`merges.txt`、`model.safetensors` 各自对应哪段代码，为什么 GPT-2 需要 byte-level BPE，为什么 HuggingFace Conv1D 权重要转置，为什么 `lm_head` 可以 tied 到 embedding。

任务四：运行真实 small 模型 smoke。报告不许只贴“命令执行成功”，必须解释脚本为什么校验模型 hash、为什么固定 `llama.cpp` commit、为什么使用 `-ngl0`，以及为什么 assistant response 非空只是 smoke，不是质量评测。

任务五：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。GPT-2 反例可以是：tensor name 缺失、shape 和 config 不一致、vocab 中缺少 BPE merge 后 token、`max_new_tokens` 超过 context。真实模型 smoke 的反例可以是：模型 SHA256 不匹配、网络不可用、`cmake` 缺失、assistant response 为空、命令超时。每个反例都要写清期望处理方式。

六、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。新增 GPT-2 smoke 后，最低验收还包括：真实 GPT-2 文件身份有 hash，prompt ids 可解释，generated ids 可解释，文本反解码可解释，并明确声明这不是高性能推理结论。新增 SmolLM2 smoke 后，最低验收还包括：模型身份有 hash，模型规模不是 tiny，CPU 路径实际运行，报告里有非空 assistant response，并明确声明这不是自研 LCQI 完整 GGUF 支持。

递进大作业：提交 GPT-2 reference path 的下一阶段迁移清单：KV cache、prefill/decode 分离、packed weight、线程并行、Transformers golden logits、更多模型方言 name mapping、分片 safetensors 和性能报告。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读: Reference Decoder、Tokenizer、Safetensors 与 GPT-2

从 tiny MLP 走向 decoder-only 模型

Reference decoder 和 GPT-2 reference path 是 LCQI 从教学闭环走向真实模型导入的关键层。这里的代码更长，是因为真实模型不只有矩阵乘法：它还需要 tokenizer、位置、归一化、QKV 拆分、attention mask、KV cache、激活函数、lm head、权重命名映射、dtype 转换和坏资产拒绝。

Tiny Reference Decoder

`reference_decoder.cpp` 实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing 10.8 src/reference_decoder.cpp

```

1 #include <lcqi/reference_decoder.hpp>
2
3 #include <algorithm>
4 #include <array>
5 #include <cmath>
6 #include <limits>
7 #include <stdexcept>
8 #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19
20 void require_positive(std::int32_t value, const char* name) {
21     if (value <= 0) {
22         throw std::runtime_error(std::string(name) + " must be positive");
23     }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27     require_positive(config.hidden_size, "hidden_size");
28     require_positive(config.query_heads, "query_heads");
29     require_positive(config.kv_heads, "kv_heads");
30     require_positive(config.head_dim, "head_dim");
31     require_positive(config.intermediate_size, "intermediate_size");
32     require_positive(config.vocab_size, "vocab_size");

```



```

33     require_positive(config.max_context, "max_context");
34     if (config.query_heads % config.kv_heads != 0) {
35         throw std::runtime_error("query_heads must be divisible by kv_heads");
36     }
37     if (config.hidden_size != config.query_heads * config.head_dim) {
38         throw std::runtime_error("hidden_size must equal query_heads * head_dim↵
↵ ");
39     }
40     if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41         throw std::runtime_error("head_dim must be even for RoPE");
42     }
43     if (config.rope_base <= 1.0F) {
44         throw std::runtime_error("rope_base must be greater than 1");
45     }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49     if (value < 0) {
50         throw std::runtime_error(std::string(name) + " cannot be negative");
51     }
52     return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56     return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60     require_positive(layer.input_size, "linear input_size");
61     require_positive(layer.output_size, "linear output_size");
62     const std::size_t expected_weights =
63         checked_size(layer.input_size, name) * checked_size(layer.output_size, ↵
↵ name);
64     if (layer.weights.size() != expected_weights) {
65         throw std::runtime_error(std::string(name) + " weight size mismatch");
66     }
67     if (!layer.bias.empty() &&
68         layer.bias.size() != checked_size(layer.output_size, name)) {
69         throw std::runtime_error(std::string(name) + " bias size mismatch");
70     }
71 }
72
73 void validate_span_size(std::size_t actual, std::int32_t expected, const char* ↵
↵ name) {
74     if (actual != checked_size(expected, name)) {
75         throw std::runtime_error(std::string(name) + " size mismatch");
76     }
77 }
78
79 std::int32_t argmax(std::span<const float> values) {
80     if (values.empty()) {
81         throw std::runtime_error("cannot take argmax of empty values");

```

```

82     }
83     std::int32_t best_index = 0;
84     float best_value = values[0];
85     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
↵    ↵ )); ++index) {
86         const float candidate = values[static_cast<std::size_t>(index)];
87         if (candidate > best_value) {
88             best_value = candidate;
89             best_index = index;
90         }
91     }
92     return best_index;
93 }
94
95 float silu(float value) {
96     return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100                                std::int32_t row,
101                                std::int32_t row_width) {
102     const std::size_t begin = checked_size(row, "row") * checked_size(row_width,
↵    ↵ , "row_width");
103     return values.subspan(begin, checked_size(row_width, "row_width"));
104 }
105
106 void add_inplace(std::span<float> target, std::span<const float> source) {
107     if (target.size() != source.size()) {
108         throw std::runtime_error("residual add size mismatch");
109     }
110     for (std::size_t index = 0; index < target.size(); ++index) {
111         target[index] += source[index];
112     }
113 }
114
115 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
116                std::span<const float> input,
117                std::span<float> output) {
118     std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F
↵    ↵ );
119     std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
120     std::vector<float> hidden(gate.size(), 0.0F);
121     linear_f32(layer.w_gate, input, gate);
122     linear_f32(layer.w_up, input, up);
123     if (gate.size() != up.size()) {
124         throw std::runtime_error("SwiGLU gate/up size mismatch");
125     }
126     for (std::size_t index = 0; index < hidden.size(); ++index) {
127         hidden[index] = silu(gate[index]) * up[index];
128     }
129     linear_f32(layer.w_down, hidden, output);
130 }

```

```

131
132 float checksum(std::span<const float> values) {
133     float sum = 0.0F;
134     for (const float value : values) {
135         sum += value;
136     }
137     return sum;
138 }
139
140 float max_abs(std::span<const float> values) {
141     float result = 0.0F;
142     for (const float value : values) {
143         result = std::max(result, std::fabs(value));
144     }
145     return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape ←
    ↪ ) {
149     std::vector<std::int32_t> stride(shape.size(), 1);
150     std::int32_t running = 1;
151     for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ←
    ↪ index >= 0; --index) {
152         stride[checked_size(index, "stride index")] = running;
153         running *= shape[checked_size(index, "shape index")];
154     }
155     return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159                    const char* name,
160                    std::int32_t token_position,
161                    std::int32_t layer_id,
162                    std::span<const std::int32_t> shape,
163                    const char* layout,
164                    std::span<const float> values) {
165     if (checkpoints == nullptr) {
166         return;
167     }
168     ReferenceTensorCheckpoint checkpoint;
169     checkpoint.name = name;
170     checkpoint.token_position = token_position;
171     checkpoint.layer_id = layer_id;
172     checkpoint.shape.assign(shape.begin(), shape.end());
173     checkpoint.stride = contiguous_stride(shape);
174     checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175     checkpoint.layout = layout;
176     checkpoint.checksum = checksum(values);
177     checkpoint.max_abs = max_abs(values);
178     checkpoint.values.assign(values.begin(), values.end());
179     checkpoints->push_back(std::move(checkpoint));
180 }

```



```

181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183                             std::span<const float> input,
184                             std::span<float> output,
185                             std::int32_t token_position,
186                             std::int32_t layer_id,
187                             std::vector<ReferenceTensorCheckpoint>* checkpoints)↵
    ↵ {
188     std::vector<float> gate(check_size(layer.w_gate.output_size, "gate"), 0.0↵
    ↵ F);
189     std::vector<float> up(check_size(layer.w_up.output_size, "up"), 0.0F);
190     std::vector<float> hidden(gate.size(), 0.0F);
191     linear_f32(layer.w_gate, input, gate);
192     linear_f32(layer.w_up, input, up);
193     if (gate.size() != up.size()) {
194         throw std::runtime_error("SwiGLU gate/up size mismatch");
195     }
196     const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↵
    ↵ output_size};
197     add_checkpoint(checkpoints,
198                     "mlp_gate",
199                     token_position,
200                     layer_id,
201                     intermediate_shape,
202                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
203                     gate);
204     add_checkpoint(checkpoints,
205                     "mlp_up",
206                     token_position,
207                     layer_id,
208                     intermediate_shape,
209                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
210                     up);
211     for (std::size_t index = 0; index < hidden.size(); ++index) {
212         hidden[index] = silu(gate[index]) * up[index];
213     }
214     add_checkpoint(checkpoints,
215                     "mlp_hidden",
216                     token_position,
217                     layer_id,
218                     intermediate_shape,
219                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
220                     hidden);
221     linear_f32(layer.w_down, hidden, output);
222 }
223
224 void forward_layer(const DecoderConfig& config,
225                   const DecoderLayerWeightsF32& layer,
226                   std::int32_t layer_id,
227                   std::int32_t model_position,
228                   ReferenceKVCache& cache,
229                   std::span<float> hidden,

```

```

230         std::vector<ReferenceTensorCheckpoint>* checkpoints = ↵
↵ nullptr) {
231     validate_span_size(hidden.size(), config.hidden_size, "hidden");
232
233     std::vector<float> normed(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
234     std::vector<float> query(check_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
235     std::vector<float> key(check_size(kv_width(config), "kv_width"), 0.0F);
236     std::vector<float> value(check_size(kv_width(config), "kv_width"), 0.0F);
237     std::vector<float> attention(check_size(config.hidden_size, "hidden_size"↵
↵ ), 0.0F);
238     std::vector<float> attention_projected(check_size(config.hidden_size, "↵
↵ hidden_size"), 0.0F);
239     std::vector<float> mlp(check_size(config.hidden_size, "hidden_size"), 0.0↵
↵ F);
240
241     const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242     const std::array<std::int32_t, 2> query_shape{config.query_heads, config.↵
↵ head_dim};
243     const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim↵
↵ };
244     const std::array<std::int32_t, 4> kv_slot_shape{
245         1, 1, config.kv_heads, config.head_dim};
246
247     rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248     add_checkpoint(checkpoints,
249         "attn_norm",
250         model_position,
251         layer_id,
252         hidden_shape,
253         LCQI_TRACE_LAYOUT_CONTIGUOUS,
254         normed);
255     linear_f32(layer.wq, normed, query);
256     linear_f32(layer.wk, normed, key);
257     linear_f32(layer.wv, normed, value);
258     add_checkpoint(checkpoints,
259         "q_before_rope",
260         model_position,
261         layer_id,
262         query_shape,
263         LCQI_TRACE_LAYOUT_CONTIGUOUS,
264         query);
265     add_checkpoint(checkpoints,
266         "k_before_rope",
267         model_position,
268         layer_id,
269         kv_shape,
270         LCQI_TRACE_LAYOUT_CONTIGUOUS,
271         key);
272     add_checkpoint(checkpoints,
273         "v",

```

```

274         model_position,
275         layer_id,
276         kv_shape,
277         LCQI_TRACE_LAYOUT_CONTIGUOUS,
278         value);
279     apply_rope(query, config.query_heads, config.head_dim, model_position, ↵
↵ config.rope_base);
280     apply_rope(key, config.kv_heads, config.head_dim, model_position, config.↵
↵ rope_base);
281     add_checkpoint(checkpoints,
282         "q_after_rope",
283         model_position,
284         layer_id,
285         query_shape,
286         LCQI_TRACE_LAYOUT_CONTIGUOUS,
287         query);
288     add_checkpoint(checkpoints,
289         "k_after_rope",
290         model_position,
291         layer_id,
292         kv_shape,
293         LCQI_TRACE_LAYOUT_CONTIGUOUS,
294         key);
295
296     cache.append(layer_id, model_position, key, value);
297     add_checkpoint(checkpoints,
298         "kv_cache_key_slot",
299         model_position,
300         layer_id,
301         kv_slot_shape,
302         LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303         key);
304     attention_decode(config, cache, layer_id, model_position, query, attention)↵
↵ ;
305     add_checkpoint(checkpoints,
306         "attention_out",
307         model_position,
308         layer_id,
309         hidden_shape,
310         LCQI_TRACE_LAYOUT_CONTIGUOUS,
311         attention);
312     linear_f32(layer.wo, attention, attention_projected);
313     add_inplace(hidden, attention_projected);
314     add_checkpoint(checkpoints,
315         "post_attention_residual",
316         model_position,
317         layer_id,
318         hidden_shape,
319         LCQI_TRACE_LAYOUT_CONTIGUOUS,
320         hidden);
321
322     rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);

```

```

323     add_checkpoint(checkpoints,
324                     "mlp_norm",
325                     model_position,
326                     layer_id,
327                     hidden_shape,
328                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
329                     normed);
330     if (checkpoints == nullptr) {
331         swiglu_mlp(layer, normed, mlp);
332     } else {
333         swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ←
↪ checkpoints);
334     }
335     add_inplace(hidden, mlp);
336     add_checkpoint(checkpoints,
337                     "layer_out",
338                     model_position,
339                     layer_id,
340                     hidden_shape,
341                     LCQI_TRACE_LAYOUT_CONTIGUOUS,
342                     hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346                               std::int32_t output_size,
347                               std::vector<float> weights,
348                               std::vector<float> bias = {}) {
349     LinearWeightsF32 layer;
350     layer.input_size = input_size;
351     layer.output_size = output_size;
352     layer.weights = std::move(weights);
353     layer.bias = std::move(bias);
354     validate_linear(layer, "tiny linear");
355     return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365     const ReferenceDecoderModel& model,
366     std::span<const std::int32_t> token_ids) {
367     validate_config(model.config);
368     if (token_ids.empty()) {
369         throw std::runtime_error("reference decode needs at least one token");
370     }
371     if (token_ids.size() > checked_size(model.config.max_context, "max_context" ←
↪ )) {
372         throw std::runtime_error("token_ids exceed max_context");

```

```

373     }
374     if (model.token_embedding.size() !=
375         checked_size(model.config.vocab_size, "vocab_size") *
376         checked_size(model.config.hidden_size, "hidden_size")) {
377         throw std::runtime_error("token embedding size mismatch");
378     }
379     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size,
380 ↪ , "hidden_size")) {
381         throw std::runtime_error("final norm weight size mismatch");
382     }
383     validate_linear(model.lm_head, "lm_head");
384
385     ReferenceDecodeTraceResult trace;
386     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers.
387 ↪ .size()));
388     std::vector<float> hidden(checked_size(model.config.hidden_size, "
389 ↪ hidden_size"), 0.0F);
390     const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
391     for (std::int32_t position = 0; position < static_cast<std::int32_t>(
392 ↪ token_ids.size());
393         ++position) {
394         const std::int32_t token_id = token_ids[checked_size(position, "
395 ↪ position")];
396         if (token_id < 0 || token_id >= model.config.vocab_size) {
397             throw std::runtime_error("token id out of vocabulary");
398         }
399         const std::span<const float> embedding =
400             row_span(model.token_embedding, token_id, model.config.hidden_size)
401 ↪ ;
402         std::copy(embedding.begin(), embedding.end(), hidden.begin());
403         add_checkpoint(&trace.checkpoints,
404             "embedding",
405             position,
406             -1,
407             hidden_shape,
408             LCQI_TRACE_LAYOUT_ROW_MAJOR,
409             hidden);
410         for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(
411 ↪ model.layers.size());
412             ++layer_id) {
413             forward_layer(model.config,
414                 model.layers[checked_size(layer_id, "layer_id")],
415                 layer_id,
416                 position,
417                 cache,
418                 hidden,
419                 &trace.checkpoints);
420         }
421     }
422
423     std::vector<float> normed(checked_size(model.config.hidden_size, "
424 ↪ hidden_size"), 0.0F);

```

```

417     std::vector<float> logits(checkered_size(model.config.vocab_size, "vocab_size" ←
    ↪ "), 0.0F);
418     const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419     const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size ←
    ↪ ()) - 1;
420     rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed) ←
    ↪ ;
421     add_checkpoint(&trace.checkpoints,
422                   "final_norm",
423                   last_position,
424                   -1,
425                   hidden_shape,
426                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
427                   normed);
428     linear_f32(model.lm_head, normed, logits);
429     add_checkpoint(&trace.checkpoints,
430                   "logits",
431                   last_position,
432                   -1,
433                   logits_shape,
434                   LCQI_TRACE_LAYOUT_CONTIGUOUS,
435                   logits);
436
437     trace.result.predicted_token = argmax(logits);
438     trace.result.logits = std::move(logits);
439     return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t ←
    ↪ layer_count)
443 : config_(config),
444   layer_count_(layer_count) {
445     validate_config(this->config_);
446     require_positive(this->layer_count_, "layer_count");
447     const std::size_t element_count =
448         checked_size(this->layer_count_, "layer_count") *
449         checked_size(this->config_.max_context, "max_context") *
450         checked_size(this->config_.kv_heads, "kv_heads") *
451         checked_size(this->config_.head_dim, "head_dim");
452     this->keys_.assign(element_count, 0.0F);
453     this->values_.assign(element_count, 0.0F);
454     this->written_.assign(
455         checked_size(this->layer_count_, "layer_count") *
456         checked_size(this->config_.max_context, "max_context"),
457         0);
458 }
459
460 void ReferenceKVCache::append(std::int32_t layer_id,
461                               std::int32_t model_position,
462                               std::span<const float> key,
463                               std::span<const float> value) {
464     validate_span_size(key.size(), kv_width(this->config_), "KV key");

```

```

465     validate_span_size(value.size(), kv_width(this->config_), "KV value");
466     this->validate_address(layer_id, model_position, 0);
467     for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) {
468         const std::size_t target = this->base_offset(layer_id, model_position,
469         kv_head);
469         const std::size_t source =
470             checked_size(kv_head, "kv_head") * checked_size(this->config_.
471         head_dim, "head_dim");
471         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
472             this->config_.head_dim,
473             this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
474         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
475             this->config_.head_dim,
476             this->values_.begin() + static_cast<std::ptrdiff_t>(target));
477     };
478     const std::size_t written_index =
479         checked_size(layer_id, "layer_id") * checked_size(this->config_.
480         max_context, "max_context") +
481         checked_size(model_position, "model_position");
481     this->written_[written_index] = 1;
482     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
483 }
484
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486                                             std::int32_t model_position,
487                                             std::int32_t kv_head) const {
488     this->validate_address(layer_id, model_position, kv_head);
489     const std::size_t offset = this->base_offset(layer_id, model_position,
490     kv_head);
491     return std::span<const float>(this->keys_.data() + offset,
492     checked_size(this->config_.head_dim, "
493     head_dim"));
494 }
495
496 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
497                                             std::int32_t model_position,
498                                             std::int32_t kv_head) const {
499     this->validate_address(layer_id, model_position, kv_head);
500     const std::size_t offset = this->base_offset(layer_id, model_position,
501     kv_head);
502     return std::span<const float>(this->values_.data() + offset,
503     checked_size(this->config_.head_dim, "
504     head_dim"));
505 }
506
507 std::int32_t ReferenceKVCache::layer_count() const noexcept {
508     return this->layer_count_;
509 }
510
511 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {

```



```

508     return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512                                           std::int32_t model_position,
513                                           std::int32_t kv_head) const {
514     return (((checked_size(layer_id, "layer_id") *
515                    checked_size(this->config.max_context, "max_context")) +
516            checked_size(model_position, "model_position")) *
517            checked_size(this->config.kv_heads, "kv_heads") +
518            checked_size(kv_head, "kv_head")) *
519            checked_size(this->config.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523                                         std::int32_t model_position,
524                                         std::int32_t kv_head) const {
525     if (layer_id < 0 || layer_id >= this->layer_count_) {
526         throw std::runtime_error("KV layer_id out of range");
527     }
528     if (model_position < 0 || model_position >= this->config.max_context) {
529         throw std::runtime_error("KV model_position out of range");
530     }
531     if (kv_head < 0 || kv_head >= this->config.kv_heads) {
532         throw std::runtime_error("KV head out of range");
533     }
534 }
535
536 void rms_norm(std::span<const float> input,
537              std::span<const float> weight,
538              float epsilon,
539              std::span<float> output) {
540     if (input.size() != weight.size() || input.size() != output.size()) {
541         throw std::runtime_error("RMSNorm size mismatch");
542     }
543     if (input.empty()) {
544         throw std::runtime_error("RMSNorm input is empty");
545     }
546     float sum_square = 0.0F;
547     for (const float value : input) {
548         sum_square += value * value;
549     }
550     const float mean_square = sum_square / static_cast<float>(input.size());
551     const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552     for (std::size_t index = 0; index < input.size(); ++index) {
553         output[index] = input[index] * scale * weight[index];
554     }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558                std::span<const float> input,
559                std::span<float> output) {

```

```

560     validate_linear(layer, "linear_f32");
561     validate_span_size(input.size(), layer.input_size, "linear input");
562     validate_span_size(output.size(), layer.output_size, "linear output");
563     for (std::int32_t out = 0; out < layer.output_size; ++out) {
564         const std::size_t row_base =
565             checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");
566         float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size↵
↵ (out, "out")];
567         for (std::int32_t in = 0; in < layer.input_size; ++in) {
568             accumulator += layer.weights[row_base + checked_size(in, "in")] *
569                 input[checked_size(in, "in")];
570         }
571         output[checked_size(out, "out")] = accumulator;
572     }
573 }
574
575 void apply_rope(std::span<float> heads,
576                std::int32_t head_count,
577                std::int32_t head_dim,
578                std::int32_t model_position,
579                float rope_base) {
580     require_positive(head_count, "head_count");
581     require_positive(head_dim, "head_dim");
582     validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583     if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584         throw std::runtime_error("RoPE head_dim must be even");
585     }
586     if (model_position < 0) {
587         throw std::runtime_error("RoPE model_position cannot be negative");
588     }
589     if (rope_base <= 1.0F) {
590         throw std::runtime_error("RoPE base must be greater than 1");
591     }
592     for (std::int32_t head = 0; head < head_count; ++head) {
593         for (std::int32_t offset = 0; offset < head_dim; offset += ↵
↵ LCQI_ROPE_PAIR_WIDTH) {
594             const float exponent = static_cast<float>(offset) / static_cast<↵
↵ float>(head_dim);
595             const float theta = 1.0F / std::pow(rope_base, exponent);
596             const float angle = static_cast<float>(model_position) * theta;
597             const float cosine = std::cos(angle);
598             const float sine = std::sin(angle);
599             const std::size_t first =
600                 checked_size(head, "head") * checked_size(head_dim, "head_dim")↵
↵ +
601                 checked_size(offset, "offset");
602             const float x0 = heads[first];
603             const float x1 = heads[first + 1];
604             heads[first] = x0 * cosine - x1 * sine;
605             heads[first + 1] = x0 * sine + x1 * cosine;
606         }

```

```

607     }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611                     const ReferenceKVCache& cache,
612                     std::int32_t layer_id,
613                     std::int32_t model_position,
614                     std::span<const float> query,
615                     std::span<float> output) {
616     validate_config(config);
617     validate_span_size(query.size(), config.hidden_size, "attention query");
618     validate_span_size(output.size(), config.hidden_size, "attention output");
619     if (model_position < 0 || model_position >= cache.filled_tokens()) {
620         throw std::runtime_error("attention model_position has not been written
621     ↪ ");
622     }
623     std::fill(output.begin(), output.end(), 0.0F);
624
625     const std::int32_t group_size = config.query_heads / config.kv_heads;
626     const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.kv_
627     ↪ head_dim));
628     std::vector<float> scores(check_size(model_position + 1, "score count"),
629                             LCQI_SOFTMAX_NEGATIVE_INFINITY);
630     std::vector<float> probabilities(scores.size(), 0.0F);
631
632     for (std::int32_t query_head = 0; query_head < config.query_heads; ++
633     ↪ query_head) {
634         const std::int32_t kv_head = query_head / group_size;
635         const std::size_t query_base =
636         ↪ checked_size(query_head, "query_head") * checked_size(config.kv_
637         ↪ head_dim, "head_dim");
638         float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
639         for (std::int32_t position = 0; position <= model_position; ++position)
640         ↪ {
641             const std::span<const float> key = cache.key(layer_id, position,
642             ↪ kv_head);
643             float dot = 0.0F;
644             for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
645                 dot += query[query_base + checked_size(dim, "dim")] *
646                 ↪ key[checked_size(dim, "dim")];
647             }
648             const float score = dot * score_scale;
649             scores[checked_size(position, "position")] = score;
650             max_score = std::max(max_score, score);
651         }
652
653         float probability_sum = 0.0F;
654         for (std::int32_t position = 0; position <= model_position; ++position)
655         ↪ {
656             const std::size_t index = checked_size(position, "position");
657             const float unnormalized = std::exp(scores[index] - max_score);
658             probabilities[index] = unnormalized;

```

```

652         probability_sum += unnormalized;
653     }
654     if (probability_sum <= 0.0F) {
655         throw std::runtime_error("attention probability sum is invalid");
656     }
657     for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {
658         const float probability =
659             probabilities[checked_size(position, "position")] / ↵
↵ probability_sum;
660         const std::span<const float> value = cache.value(layer_id, position↵
↵ , kv_head);
661         for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662             output[query_base + checked_size(dim, "dim")] +=
663                 probability * value[checked_size(dim, "dim")];
664         }
665     }
666 }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670                                           std::span<const std::int32_t> ↵
↵ token_ids) {
671     validate_config(model.config);
672     if (token_ids.empty()) {
673         throw std::runtime_error("reference decode needs at least one token");
674     }
675     if (token_ids.size() > checked_size(model.config.max_context, "max_context"↵
↵ )) {
676         throw std::runtime_error("token_ids exceed max_context");
677     }
678     if (model.token_embedding.size() !=
679         checked_size(model.config.vocab_size, "vocab_size") *
680         checked_size(model.config.hidden_size, "hidden_size")) {
681         throw std::runtime_error("token embedding size mismatch");
682     }
683     if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↵
↵ , "hidden_size")) {
684         throw std::runtime_error("final norm weight size mismatch");
685     }
686     validate_linear(model.lm_head, "lm_head");
687
688     ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↵
↵ .size()));
689     std::vector<float> hidden(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
690     for (std::int32_t position = 0; position < static_cast<std::int32_t>(↵
↵ token_ids.size());
691         ++position) {
692         const std::int32_t token_id = token_ids[checked_size(position, "↵
↵ position")];
693         if (token_id < 0 || token_id >= model.config.vocab_size) {

```

```

694         throw std::runtime_error("token id out of vocabulary");
695     }
696     const std::span<const float> embedding =
697         row_span(model.token_embedding, token_id, model.config.hidden_size)↵
↵ ;
698     std::copy(embedding.begin(), embedding.end(), hidden.begin());
699     for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↵
↵ model.layers.size());
700         ++layer_id) {
701         forward_layer(model.config,
702                       model.layers[checked_size(layer_id, "layer_id")],
703                       layer_id,
704                       position,
705                       cache,
706                       hidden);
707     }
708 }
709
710 std::vector<float> normed(checked_size(model.config.hidden_size, "↵
↵ hidden_size"), 0.0F);
711 std::vector<float> logits(checked_size(model.config.vocab_size, "vocab_size↵
↵ "), 0.0F);
712 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↵
↵ ;
713 linear_f32(model.lm_head, normed, logits);
714
715 ReferenceDecodeResult result;
716 result.predicted_token = argmax(logits);
717 result.logits = std::move(logits);
718 return result;
719 }
720
721 ReferenceDecoderModel make_tiny_reference_decoder_model() {
722     ReferenceDecoderModel model;
723     model.config.hidden_size = 4;
724     model.config.query_heads = 2;
725     model.config.kv_heads = 1;
726     model.config.head_dim = 2;
727     model.config.intermediate_size = 6;
728     model.config.vocab_size = 5;
729     model.config.max_context = 8;
730     model.config.rms_epsilon = 1.0e-5F;
731     model.config.rope_base = 10000.0F;
732
733     model.token_embedding = {
734         0.20F, -0.10F, 0.05F, 0.30F,
735         -0.40F, 0.10F, 0.25F, -0.20F,
736         0.15F, 0.35F, -0.30F, 0.05F,
737         0.50F, -0.25F, 0.10F, -0.15F,
738         -0.05F, 0.20F, 0.40F, -0.35F,
739     };
740

```

```

741 DecoderLayerWeightsF32 layer;
742 layer.rms_attention_weight = {1.0F, 0.9F, 1.1F, 1.0F};
743 layer.wq = make_linear(4,
744                        4,
745                        {
746                            0.20F, -0.10F, 0.05F, 0.30F,
747                            -0.15F, 0.25F, 0.10F, -0.05F,
748                            0.05F, 0.10F, -0.20F, 0.15F,
749                            0.30F, -0.05F, 0.20F, 0.10F,
750                        });
751 layer.wk = make_linear(4,
752                        2,
753                        {
754                            0.10F, 0.20F, -0.10F, 0.05F,
755                            -0.05F, 0.15F, 0.25F, -0.20F,
756                        });
757 layer.wv = make_linear(4,
758                        2,
759                        {
760                            0.25F, -0.05F, 0.10F, 0.15F,
761                            -0.10F, 0.30F, 0.05F, 0.20F,
762                        });
763 layer.wo = make_linear(4,
764                        4,
765                        {
766                            0.20F, 0.10F, -0.05F, 0.15F,
767                            -0.10F, 0.25F, 0.20F, -0.05F,
768                            0.05F, -0.20F, 0.30F, 0.10F,
769                            0.15F, 0.05F, -0.10F, 0.25F,
770                        });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773                            6,
774                            {
775                                0.20F, -0.10F, 0.05F, 0.10F,
776                                -0.05F, 0.15F, 0.20F, -0.10F,
777                                0.10F, 0.05F, -0.15F, 0.25F,
778                                -0.20F, 0.10F, 0.15F, 0.05F,
779                                0.05F, -0.25F, 0.10F, 0.20F,
780                                0.15F, 0.05F, 0.05F, -0.15F,
781                            });
782 layer.w_up = make_linear(4,
783                            6,
784                            {
785                                0.10F, 0.20F, -0.05F, 0.05F,
786                                -0.15F, 0.05F, 0.25F, 0.10F,
787                                0.20F, -0.10F, 0.05F, 0.15F,
788                                0.05F, 0.10F, -0.20F, 0.25F,
789                                -0.05F, 0.15F, 0.10F, -0.10F,
790                                0.25F, -0.05F, 0.05F, 0.10F,
791                            });
792 layer.w_down = make_linear(6,

```

```

793         4,
794         {
795             0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796             -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797             0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798             0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799         });
800     model.layers.push_back(std::move(layer));
801     model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802     model.lm_head = make_linear(4,
803                                 5,
804                                 {
805                 0.20F, -0.10F, 0.05F, 0.15F,
806                 -0.05F, 0.25F, 0.10F, -0.20F,
807                 0.15F, 0.05F, -0.25F, 0.10F,
808                 -0.10F, 0.20F, 0.15F, 0.05F,
809                 0.05F, -0.15F, 0.20F, 0.25F,
810             },
811             zeros(5));
812     return model;
813 }
814
815 } // namespace lcqi

```

Tokenizer 与 Safetensors 实现

`tokenizer.cpp` 把 prompt 变成 token id, 并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入, UNK 是否按合同处理。

Listing 10.9 src/tokenizer.cpp

```

1  #include <lcqi/tokenizer.hpp>
2
3  #include <cstdint>
4  #include <fstream>
5  #include <stdexcept>
6  #include <string>
7  #include <string_view>
8
9  namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17     std::string key;
18     if (!(input >> key)) {
19         throw std::runtime_error("unexpected end of tokenizer file");
20     }
21     return key;

```



```

22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25     const std::string key = read_key(input);
26     if (key != expected) {
27         throw std::runtime_error("expected tokenizer key '" + expected + "', ←
↪ got '" + key + "'");
28     }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {
32     expect_key(input, key);
33     std::int32_t value = 0;
34     if (!(input >> value)) {
35         throw std::runtime_error("invalid tokenizer int32 value for key '" + ←
↪ key + "'");
36     }
37     return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41     expect_key(input, key);
42     std::string value;
43     if (!(input >> value)) {
44         throw std::runtime_error("invalid tokenizer string value for key '" + ←
↪ key + "'");
45     }
46     return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ←
↪ text) {
50     for (const TokenEntry& entry : tokenizer.vocab) {
51         if (entry.text == text) {
52             return entry.id;
53         }
54     }
55     return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59     if (tokenizer.vocab.empty()) {
60         throw std::runtime_error("tokenizer vocab is empty");
61     }
62     for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63         const TokenEntry& left = tokenizer.vocab[i];
64         if (left.id < 0) {
65             throw std::runtime_error("tokenizer token id must be non-negative")↪
↪ ;
66         }
67         for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
68             const TokenEntry& right = tokenizer.vocab[j];

```

```

69         if (left.id == right.id) {
70             throw std::runtime_error("duplicate tokenizer token id");
71         }
72         if (left.text == right.text) {
73             throw std::runtime_error("duplicate tokenizer token text");
74         }
75     }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78     lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79     throw std::runtime_error("tokenizer BOS id does not match vocab");
80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82     lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83     throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88     hash ^= static_cast<std::uint64_t>(value);
89     hash *= LCQI_FNV_PRIME;
90     return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94     for (const unsigned char value : text) {
95         hash = hash_byte(hash, value);
96     }
97     return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101     const std::uint32_t bits = static_cast<std::uint32_t>(value);
102     for (std::int32_t shift = 0; shift < 32; shift += 8) {
103         hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0x
104         ↪ xFFU));
105     }
106     return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111     std::ifstream input(path);
112     if (!input) {
113         throw std::runtime_error("cannot open tokenizer file: " + path.string()
114         ↪ );
115     }
116     expect_key(input, "LCQI_TOKENIZER_V1");
117
118     TokenizerModel tokenizer;

```

```

119     tokenizer.bos_id = read_i32(input, "bos_id");
120     tokenizer.eos_id = read_i32(input, "eos_id");
121     tokenizer.unk_id = read_i32(input, "unk_id");
122     tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
123     const std::int32_t vocab_size = read_i32(input, "vocab_size");
124     if (vocab_size <= 0) {
125         throw std::runtime_error("tokenizer vocab_size must be positive");
126     }
127
128     tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
129     expect_key(input, "vocab");
130     for (std::int32_t i = 0; i < vocab_size; ++i) {
131         TokenEntry entry;
132         if (!(input >> entry.id >> entry.text)) {
133             throw std::runtime_error("invalid tokenizer vocab entry");
134         }
135         tokenizer.vocab.push_back(entry);
136     }
137
138     validate_tokenizer(tokenizer);
139     return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143                                         std::string_view prompt) {
144     std::vector<std::int32_t> ids;
145     if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146         ids.push_back(tokenizer.bos_id);
147     }
148
149     std::size_t begin = 0;
150     while (begin < prompt.size()) {
151         while (begin < prompt.size() && prompt[begin] == ' ') {
152             ++begin;
153         }
154         if (begin >= prompt.size()) {
155             break;
156         }
157         std::size_t end = begin;
158         while (end < prompt.size() && prompt[end] != ' ') {
159             ++end;
160         }
161         const std::string_view token = prompt.substr(begin, end - begin);
162         const std::int32_t token_id = lookup_token_id(tokenizer, token);
163         if (token_id == LCQI_INVALID_TOKEN_ID) {
164             if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165                 throw std::runtime_error("tokenizer encountered unknown token");
166             }
167             ids.push_back(tokenizer.unk_id);
168         } else {
169             ids.push_back(token_id);

```

```

170     }
171     begin = end;
172 }
173
174 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
175     ids.push_back(tokenizer.eos_id);
176 }
177 return ids;
178 }
179
180 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {
181     std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182     hash = hash_string(hash, tokenizer.chat_template_hash);
183     hash = hash_i32(hash, tokenizer.bos_id);
184     hash = hash_i32(hash, tokenizer.eos_id);
185     hash = hash_i32(hash, tokenizer.unk_id);
186     for (const TokenEntry& entry : tokenizer.vocab) {
187         hash = hash_string(hash, entry.text);
188         hash = hash_i32(hash, entry.id);
189     }
190     return hash;
191 }
192
193 } // namespace lcqi

```

safetensors.cpp 解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing 10.10 src/safetensors.cpp

```

1 #include <lcqi/safetensors.hpp>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cstdint>
6 #include <cmath>
7 #include <cstring>
8 #include <fstream>
9 #include <limits>
10 #include <stdexcept>
11 #include <string>
12 #include <string_view>
13 #include <vector>
14
15 namespace lcqi {
16 namespace {
17
18 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
19 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U↵
    ↵ ;
20 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
21 constexpr std::uint64_t LCQI_BYTE_BITS = 8;
22 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;

```

```

23 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
24 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
30 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
31 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
32 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
33 constexpr std::uint32_t LCQI_F32_EXPONENT_MASK = 0x7F800000U;
34 constexpr std::uint32_t LCQI_F16_SIGN_MASK = 0x8000U;
35 constexpr std::uint32_t LCQI_F16_EXPONENT_MASK = 0x7C00U;
36 constexpr std::uint32_t LCQI_F16_MANTISSA_MASK = 0x03FFU;
37 constexpr std::uint32_t LCQI_F16_EXPONENT_BIAS = 15U;
38 constexpr std::uint32_t LCQI_F32_EXPONENT_BIAS = 127U;
39 constexpr std::uint32_t LCQI_F32_MANTISSA_BITS = 23U;
40 constexpr std::uint32_t LCQI_F16_MANTISSA_BITS = 10U;
41 constexpr std::uint32_t LCQI_F16_TO_F32_SIGN_SHIFT = 16U;
42
43 class HeaderCursor {
44 public:
45     explicit HeaderCursor(std::string_view text) : text_(text) {}
46
47     void expect(char expected) {
48         this->skip_ws();
49         if (this->eof() || this->text_[this->position_] != expected) {
50             throw std::runtime_error("invalid safetensors header syntax");
51         }
52         ++this->position_;
53     }
54
55     [[nodiscard]] bool consume(char expected) {
56         this->skip_ws();
57         if (!this->eof() && this->text_[this->position_] == expected) {
58             ++this->position_;
59             return true;
60         }
61         return false;
62     }
63
64     [[nodiscard]] std::string parse_string() {
65         this->skip_ws();
66         this->expect('\"');
67         std::string result;
68         while (!this->eof()) {
69             const char ch = this->text_[this->position_++];
70             if (ch == '\"') {
71                 return result;
72             }
73             if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ←
↪ {

```

```

74         throw std::runtime_error("control character in safetensors ↵
↵ string");
75     }
76     if (ch == '\\') {
77         result.push_back(this->parse_escape());
78     } else {
79         result.push_back(ch);
80     }
81 }
82 throw std::runtime_error("unterminated safetensors string");
83 }
84
85 [[nodiscard]] std::uint64_t parse_u64() {
86     this->skip_ws();
87     if (this->eof() ||
88 ↵ !std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
89         throw std::runtime_error("expected unsigned integer in safetensors ↵
↵ header");
90     }
91     std::uint64_t value = 0;
92     while (!this->eof() &&
93 ↵ std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
94         const std::uint64_t digit =
95             static_cast<std::uint64_t>(this->text_[this->position_] - '0');
96         if (value >
97 ↵ (std::numeric_limits<std::uint64_t>::max() - digit) / ↵
↵ LCQI_DECIMAL_BASE) {
98             throw std::runtime_error("safetensors integer overflow");
99         }
100         value = value * LCQI_DECIMAL_BASE + digit;
101         ++this->position_;
102     }
103     return value;
104 }
105
106 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
107     std::vector<std::int64_t> values;
108     this->expect('[');
109     if (this->consume(' ')) {
110         return values;
111     }
112     while (true) {
113         const std::uint64_t raw = this->parse_u64();
114         if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↵
↵ int64_t>::max())) {
115             throw std::runtime_error("safetensors shape value is too large"↵
↵ );
116         }
117         values.push_back(static_cast<std::int64_t>(raw));
118         if (this->consume(' ')) {

```

```

119         return values;
120     }
121     this->expect(',');
122 }
123 }
124
125 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
126     std::vector<std::uint64_t> values;
127     this->expect('[');
128     if (this->consume(' ')) {
129         return values;
130     }
131     while (true) {
132         values.push_back(this->parse_u64());
133         if (this->consume(' ')) {
134             return values;
135         }
136         this->expect(',');
137     }
138 }
139
140 [[nodiscard]] bool eof_after_ws() {
141     this->skip_ws();
142     return this->eof();
143 }
144
145 private:
146     std::string_view text_;
147     std::size_t position_ = 0;
148
149     [[nodiscard]] bool eof() const {
150         return this->position_ >= this->text_.size();
151     }
152
153     void skip_ws() {
154         while (!this->eof() &&
155             std::isspace(static_cast<unsigned char>(this->text_[this->position_])) {
156             ++this->position_;
157         }
158     }
159
160     [[nodiscard]] char parse_escape() {
161         if (this->eof()) {
162             throw std::runtime_error("unterminated safetensors escape");
163         }
164         const char escaped = this->text_[this->position_++];
165         switch (escaped) {
166             case '"':
167             case '\\':
168             case '/':
169             return escaped;

```



```

170         case 'b':
171             return '\\b';
172         case 'f':
173             return '\\f';
174         case 'n':
175             return '\\n';
176         case 'r':
177             return '\\r';
178         case 't':
179             return '\\t';
180         default:
181             throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
182     }
183 }
184 };
185
186 std::uint64_t read_le_u64(std::span<const std::uint8_t> bytes) {
187     if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
188         throw std::runtime_error("safetensors file is too small");
189     }
190     std::uint64_t value = 0;
191     for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
192         value |= static_cast<std::uint64_t>(bytes[i]) << (LCQI_BYTE_BITS * i);
193     }
194     return value;
195 }
196
197 std::string read_file_bytes(const std::filesystem::path& path) {
198     std::ifstream input(path, std::ios::binary);
199     if (!input) {
200         throw std::runtime_error("cannot open safetensors file: " + path.string↵
↵ ());
201     }
202     input.seekg(0, std::ios::end);
203     const std::streamoff size = input.tellg();
204     if (size < 0) {
205         throw std::runtime_error("cannot determine safetensors file size");
206     }
207     input.seekg(0, std::ios::beg);
208     std::string bytes(static_cast<std::size_t>(size), '\\0');
209     if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize↵
↵ >(bytes.size())) {
210         throw std::runtime_error("cannot read safetensors file");
211     }
212     return bytes;
213 }
214
215 std::vector<std::uint8_t> read_file_u8(const std::filesystem::path& path) {
216     std::ifstream input(path, std::ios::binary);
217     if (!input) {
218         throw std::runtime_error("cannot open safetensors file: " + path.string↵

```

```

    ↪ ();
219 }
220 input.seekg(0, std::ios::end);
221 const std::streamoff size = input.tellg();
222 if (size < 0) {
223     throw std::runtime_error("cannot determine safetensors file size");
224 }
225 input.seekg(0, std::ios::beg);
226 std::vector<std::uint8_t> bytes(static_cast<std::size_t>(size), 0);
227 if (!bytes.empty() &&
228     !input.read(reinterpret_cast<char*>(bytes.data()),
229                 static_cast<std::streamsize>(bytes.size()))) {
230     throw std::runtime_error("cannot read safetensors file");
231 }
232 return bytes;
233 }
234
235 void parse_metadata_value(HeaderCursor& cursor,
236                           SafeTensorManifest& manifest,
237                           const std::string& key) {
238     if (key == "__metadata__") {
239         cursor.expect('{');
240         if (cursor.consume('}')) {
241             return;
242         }
243         while (true) {
244             const std::string metadata_key = cursor.parse_string();
245             cursor.expect(':');
246             const std::string metadata_value = cursor.parse_string();
247             manifest.metadata.emplace_back(metadata_key, metadata_value);
248             if (cursor.consume('}')) {
249                 return;
250             }
251             cursor.expect(',');
252         }
253     }
254     throw std::runtime_error("unexpected safetensors metadata value");
255 }
256
257 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ↪
    ↪ name) {
258     SafeTensorEntry entry;
259     entry.name = name;
260     bool saw_dtype = false;
261     bool saw_shape = false;
262     bool saw_offsets = false;
263
264     cursor.expect('{');
265     if (cursor.consume('}')) {
266         throw std::runtime_error("empty safetensors tensor entry");
267     }
268     while (true) {

```

```

269     const std::string field = cursor.parse_string();
270     cursor.expect(':');
271     if (field == "dtype") {
272         entry.dtype = cursor.parse_string();
273         saw_dtype = true;
274     } else if (field == "shape") {
275         entry.shape = cursor.parse_i64_array();
276         saw_shape = true;
277     } else if (field == "data_offsets") {
278         const std::vector<std::uint64_t> offsets = cursor.parse_u64_array();
279         ↵ ;
280         if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
281             throw std::runtime_error("safetensors data_offsets must contain ↵
282             ↵ two values");
283         }
284         entry.data_begin = offsets[0];
285         entry.data_end = offsets[1];
286         saw_offsets = true;
287     } else {
288         throw std::runtime_error("unsupported safetensors tensor field: " + ↵
289         ↵ field);
290     }
291     if (cursor.consume('}')) {
292         break;
293     }
294     cursor.expect(',');
295 }
296
297 if (!saw_dtype || !saw_shape || !saw_offsets) {
298     throw std::runtime_error("safetensors tensor entry is missing required ↵
299     ↵ fields");
300 }
301 if (entry.data_end < entry.data_begin) {
302     throw std::runtime_error("safetensors tensor offset range is invalid");
303 }
304 return entry;
305 }
306
307 void parse_header(std::string_view header, SafeTensorManifest& manifest) {
308     HeaderCursor cursor(header);
309     cursor.expect('{');
310     if (cursor.consume('}')) {
311         throw std::runtime_error("safetensors header is empty");
312     }
313
314     while (true) {
315         const std::string key = cursor.parse_string();
316         cursor.expect(':');
317         if (key == "__metadata__") {
318             parse_metadata_value(cursor, manifest, key);
319         } else {
320             manifest.tensors.push_back(parse_tensor_entry(cursor, key));

```

```

317     }
318     if (cursor.consume('}')) {
319         break;
320     }
321     cursor.expect(',');
322 }
323
324 if (!cursor.eof_after_ws()) {
325     throw std::runtime_error("trailing data after safetensors header");
326 }
327 }
328
329 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
330
331 void validate_manifest(const SafeTensorManifest& manifest,
332                       std::uint64_t file_size) {
333     const std::uint64_t payload_size = file_size - manifest.data_start_offset;
334     for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
335         const SafeTensorEntry& left = manifest.tensors[i];
336         if (left.name.empty() || left.dtype.empty()) {
337             throw std::runtime_error("safetensors tensor has empty name or ↵
↵ dtype");
338         }
339         if (left.data_end > payload_size) {
340             throw std::runtime_error("safetensors tensor data_offsets exceed ↵
↵ payload size");
341         }
342         const std::uint64_t expected_bytes = expected_tensor_bytes(left);
343         if (left.byte_size() != expected_bytes) {
344             throw std::runtime_error("safetensors tensor byte size does not ↵
↵ match dtype and shape");
345         }
346         for (const std::int64_t dim : left.shape) {
347             if (dim < 0) {
348                 throw std::runtime_error("safetensors shape dimension is ↵
↵ negative");
349             }
350         }
351         for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {
352             const SafeTensorEntry& right = manifest.tensors[j];
353             if (left.name == right.name) {
354                 throw std::runtime_error("duplicate safetensors tensor name");
355             }
356             const bool overlaps =
357                 left.data_begin < right.data_end && right.data_begin < left.↵
↵ data_end;
358             if (overlaps) {
359                 throw std::runtime_error("overlapping safetensors tensor data ↵
↵ range");
360             }
361         }
362     }

```

```

363 }
364
365 std::uint64_t dtype_byte_size(std::string_view dtype) {
366     if (dtype == "F64") {
367         return LCQI_DTYPE_BYTES_F64;
368     }
369     if (dtype == "F32") {
370         return LCQI_DTYPE_BYTES_F32;
371     }
372     if (dtype == "F16" || dtype == "BF16") {
373         return LCQI_DTYPE_BYTES_F16;
374     }
375     if (dtype == "I64" || dtype == "U64") {
376         return LCQI_DTYPE_BYTES_I64;
377     }
378     if (dtype == "I32" || dtype == "U32") {
379         return LCQI_DTYPE_BYTES_I32;
380     }
381     if (dtype == "I16" || dtype == "U16") {
382         return LCQI_DTYPE_BYTES_I16;
383     }
384     if (dtype == "I8") {
385         return LCQI_DTYPE_BYTES_I8;
386     }
387     if (dtype == "U8" || dtype == "B00L") {
388         return LCQI_DTYPE_BYTES_U8;
389     }
390     throw std::runtime_error("unsupported safetensors dtype: " + std::string(
391         dtype));
392 }
393
394 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
395     std::uint64_t element_count = 1;
396     for (const std::int64_t dim : entry.shape) {
397         if (dim < 0) {
398             throw std::runtime_error("safetensors shape dimension is negative");
399         }
400         const std::uint64_t extent = static_cast<std::uint64_t>(dim);
401         if (extent != 0 &&
402             element_count > std::numeric_limits<std::uint64_t>::max() / extent) {
403             throw std::runtime_error("safetensors shape element count overflow");
404         }
405         element_count *= extent;
406     }
407     const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
408     if (dtype_bytes != 0 &&
409         element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes) {
410         throw std::runtime_error("safetensors tensor byte size overflow");
411     }
412 }

```

```

410     }
411     return element_count * dtype_bytes;
412 }
413
414 SafeTensorManifest parse_manifest_from_bytes(std::span<const std::uint8_t> ↵
    ↵ bytes) {
415     const std::uint64_t header_size = read_le_u64(bytes);
416     if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
417         throw std::runtime_error("safetensors header size is invalid");
418     }
419     const std::uint64_t data_start_offset =
420         static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↵
    ↵ ;
421     if (data_start_offset > bytes.size()) {
422         throw std::runtime_error("safetensors header extends beyond file size")↵
    ↵ ;
423     }
424
425     SafeTensorManifest manifest;
426     manifest.header_size = header_size;
427     manifest.data_start_offset = data_start_offset;
428     const char* header_begin =
429         reinterpret_cast<const char*>(bytes.data() + ↵
    ↵ LCQI_SAFETENSORS_PREFIX_BYTES);
430     const std::string_view header(header_begin, static_cast<std::size_t>(↵
    ↵ header_size));
431     parse_header(header, manifest);
432     validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
433     return manifest;
434 }
435
436 std::uint16_t read_le_u16(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
437     return static_cast<std::uint16_t>(
438         static_cast<std::uint16_t>(bytes[offset]) |
439         (static_cast<std::uint16_t>(bytes[offset + 1]) << LCQI_BYTE_BITS));
440 }
441
442 std::uint32_t read_le_u32(std::span<const std::uint8_t> bytes, std::size_t ↵
    ↵ offset) {
443     std::uint32_t value = 0;
444     for (std::size_t i = 0; i < LCQI_DTYPE_BYTES_F32; ++i) {
445         value |= static_cast<std::uint32_t>(bytes[offset + i]) << (↵
    ↵ LCQI_BYTE_BITS * i);
446     }
447     return value;
448 }
449
450 float bits_to_float(std::uint32_t bits) {
451     float value = 0.0F;
452     std::memcpy(&value, &bits, sizeof(value));
453     return value;

```

```

454 }
455
456 float f16_to_f32(std::uint16_t bits) {
457     const std::uint32_t sign = (static_cast<std::uint32_t>(bits) & ↵
    ↵ LCQI_F16_SIGN_MASK)
458         << LCQI_F16_TO_F32_SIGN_SHIFT;
459     const std::uint32_t exponent = static_cast<std::uint32_t>(bits) & ↵
    ↵ LCQI_F16_EXPONENT_MASK;
460     const std::uint32_t mantissa = static_cast<std::uint32_t>(bits) & ↵
    ↵ LCQI_F16_MANTISSA_MASK;
461
462     if (exponent == 0U) {
463         if (mantissa == 0U) {
464             return bits_to_float(sign);
465         }
466         float value = static_cast<float>(mantissa) /
467             static_cast<float>(1U << LCQI_F16_MANTISSA_BITS);
468         value = std::ldexp(value, 1 - static_cast<int>(LCQI_F16_EXPONENT_BIAS)); ↵
    ↵ ;
469         return (sign == 0U) ? value : -value;
470     }
471
472     if (exponent == LCQI_F16_EXPONENT_MASK) {
473         const std::uint32_t f32_mantissa =
474             mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
475         return bits_to_float(sign | LCQI_F32_EXPONENT_MASK | f32_mantissa);
476     }
477
478     const std::uint32_t f32_exponent =
479         ((exponent >> LCQI_F16_MANTISSA_BITS) - LCQI_F16_EXPONENT_BIAS +
480         LCQI_F32_EXPONENT_BIAS)
481         << LCQI_F32_MANTISSA_BITS;
482     const std::uint32_t f32_mantissa =
483         mantissa << (LCQI_F32_MANTISSA_BITS - LCQI_F16_MANTISSA_BITS);
484     return bits_to_float(sign | f32_exponent | f32_mantissa);
485 }
486
487 float bf16_to_f32(std::uint16_t bits) {
488     return bits_to_float(static_cast<std::uint32_t>(bits) << 16U);
489 }
490
491 std::span<const std::uint8_t> tensor_payload_span(const SafeTensorFile& file,
492     const SafeTensorEntry& entry) ↵
    ↵ {
493     const std::uint64_t begin = file.manifest.data_start_offset + entry. ↵
    ↵ data_begin;
494     const std::uint64_t end = file.manifest.data_start_offset + entry.data_end;
495     if (begin > end || end > file.bytes.size()) {
496         throw std::runtime_error("safetensors tensor byte range is invalid");
497     }
498     return std::span<const std::uint8_t>(
499         file.bytes.data() + static_cast<std::size_t>(begin),

```

```

500     static_cast<std::size_t>(end - begin));
501 }
502
503 } // namespace
504
505 std::uint64_t SafeTensorEntry::byte_size() const {
506     return this->data_end - this->data_begin;
507 }
508
509 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ←
    ↪ const {
510     const auto found = std::find_if(
511         this->tensors.begin(),
512         this->tensors.end(),
513         [name](const SafeTensorEntry& entry) {
514             return entry.name == name;
515         });
516     if (found == this->tensors.end()) {
517         return nullptr;
518     }
519     return &(*found);
520 }
521
522 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path) ←
    ↪ {
523     const std::string bytes = read_file_bytes(path);
524     return parse_manifest_from_bytes(
525         std::span<const std::uint8_t>(
526             reinterpret_cast<const std::uint8_t*>(bytes.data()),
527             bytes.size()));
528 }
529
530 SafeTensorFile load_safetensors_file(const std::filesystem::path& path) {
531     SafeTensorFile file;
532     file.path = path;
533     file.bytes = read_file_u8(path);
534     file.manifest = parse_manifest_from_bytes(file.bytes);
535     return file;
536 }
537
538 std::span<const std::uint8_t> read_safetensor_tensor_bytes(
539     const SafeTensorFile& file,
540     std::string_view name) {
541     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
542     if (entry == nullptr) {
543         throw std::runtime_error("missing safetensors tensor: " + std::string(←
    ↪ name));
544     }
545     return tensor_payload_span(file, *entry);
546 }
547
548 std::vector<float> read_safetensor_f32_tensor(const SafeTensorFile& file,

```



```

549         std::string_view name) {
550     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
551     if (entry == nullptr) {
552         throw std::runtime_error("missing safetensors tensor: " + std::string(↵
↵ name));
553     }
554     const std::span<const std::uint8_t> bytes = tensor_payload_span(file, *↵
↵ entry);
555     std::vector<float> values;
556     if (entry->dtype == "F32") {
557         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F32);
558         for (std::size_t index = 0; index < values.size(); ++index) {
559             values[index] = bits_to_float(
560                 read_le_u32(bytes, index * LCQI_DTYPE_BYTES_F32));
561         }
562         return values;
563     }
564     if (entry->dtype == "F16") {
565         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
566         for (std::size_t index = 0; index < values.size(); ++index) {
567             values[index] = f16_to_f32(
568                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
569         }
570         return values;
571     }
572     if (entry->dtype == "BF16") {
573         values.resize(bytes.size() / LCQI_DTYPE_BYTES_F16);
574         for (std::size_t index = 0; index < values.size(); ++index) {
575             values[index] = bf16_to_f32(
576                 read_le_u16(bytes, index * LCQI_DTYPE_BYTES_F16));
577         }
578         return values;
579     }
580     throw std::runtime_error("safetensors tensor is not float-compatible: " + ↵
↵ entry->name);
581 }
582
583 } // namespace lcqi

```

GPT-2 Reference Path

`gpt2_reference.cpp` 包括 JSON 配置读取、byte-level BPE、safetensors F32/F16/BF16 张量读取、HuggingFace 权重名前缀解析、LayerNorm、packed QKV、causal attention、GELU、tied lm head、full-prefix greedy generation 和 cached-KV generation。它仍然是正确性优先的 reference path，但现在已经开始承接前三册的优化方法：先建立 baseline，再改 hot path，再用测试和 A/B benchmark 决定改动是否能进入书。

一次可审查的 cached-KV 优化

优化不能从“感觉慢”开始。LCQI 先用 `makecpu3-gpt2-benchmark-compare` 证明 full-prefix 每生成一个 token 都重算前缀，而 cached-KV 已经把前缀复用下来；随后再看 cached path 内部。旧实现每处理一个 token、每经过一层 Transformer，都会重新分配 `normed`、`query`、`key`、

`value`、`attention`、`projected`、`mlp_fc`、`mlp_out` 和 `attention scores`。同时，生成路径只需要下一个 greedy token，却仍然每步构造完整 `Gpt2ForwardResult.logits`。这些问题不是算法语义错误，但会在真实 GPT-2 的 decode loop 里形成额外分配、重复 shape validation、重复边界转换和不必要的结果写回。

本轮优化对应三册里的三条原则。第一册要求从热循环和二进制可观察行为出发，所以先量 `lm_head`、MLP、QKV、attention projection 和 attention 本体各占多少，再决定改哪里。第二册要求减少重复分配和内存扰动，所以新增 `Gpt2ForwardWorkspace`，把 hidden、QKV、MLP、scores 和 logits 缓冲区的生命周期提升到整段 cached generation。第三册要求按 decoder runtime 的状态组织代码，所以新增 `Gpt2CachedGreedyDecoder`，让 KV cache、workspace、worker pool 和模型验证跟随一次生成会话，而不是散落在每个 token 的临时对象里。

代码上有四处关键边界。第一，旧的 `run_gpt2_forward_cached` 仍然保留完整 logits API，测试和源码阅读可以继续用它和 full-prefix logits 做逐项对照。第二，CLI 的 `cached_kv` 生成路径改用 `Gpt2CachedGreedyDecoder::step`，只返回 greedy token；需要完整 logits 的测试使用 `step_with_logits`。第三，KV cache 对外仍保留 checked `key/value` API，内部 attention 在一次地址和 workspace 校验后使用私有 unchecked span 扫描热路径，避免把“不检查”的能力暴露给普通调用者。第四，线程池只属于 cached decoder 会话；`--threads0` 自动选择 worker 数，`--threads1` 强制串行，`--threadsN` 固定线程数，避免把全局可变线程服务塞进 reference model。

正确性证据在 `tests/lcqi_tests.cpp`。tiny GPT-2 同一个 prompt 同时覆盖 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token。它要求 predicted token、logits size、逐项 logit 误差、KV cache filled token 数和 greedy generated ids 全部一致。这样优化后的代码不是“跑得快但不知道还对不对”，而是先通过 tiny golden，再进入真实 GPT-2 benchmark。

性能证据不只看一次运行。新增 `tools/run_gpt2_optimization_ab.py` 会从指定 baseline commit 建临时 worktree，baseline 和当前工作区都通过 `books/cpu-volume-3-practice` 的 Release CMake 入口构建，再用同一份 GPT-2 F32 safetensors 模型跑多轮 full-prefix 与 cached-KV。后来远端 `caw` 不能直接复用本机 SSH remote 别名时，本轮采用等价的源码包 A/B：本机用 `gitarchive73f40c7` 导出 baseline，用当前工作区导出 current，二者在 `caw` 上分别 Release 构建并运行同一模型。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-manual-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-optimization-summary.md`。5 轮中位数显示，4 token cached-KV 生成阶段从 395.397ms 到 76.4141ms，16 token 从 989.060ms 到 185.479ms；生成吞吐分别到 52.3464token/s 和 86.2633token/s。

用户看到上一轮提升幅度后，正确反应不是继续猜，而是重新量热点。新增 `--profile-hotspots` 和 `tools/run_gpt2_hotspot_profile.py` 后，可以把 cached decode 的时间拆成 embedding、LayerNorm、QKV projection、KV append、attention、attention projection、MLP fc、GELU、MLP projection 和 lm head。远端 `caw` 使用 Intel Core Ultra 7 265KF、GCC 15.2.1、Release 构建、GPT-2 F32 safetensors，跑 `Hello, mynameis` 这个 prompt，优化前 3 轮中位数保存在 `books/cpu-volume-3/reports/lcqi-gpt2-hotspot-profile-summary.md`：生成 4 个 token 时，`lm_head` 约占 30.2423%，MLP 的 `c_fc` 和 `c_proj` 合计约 46.4228%，QKV projection 约 17.2127%，cached attention 本体只有 0.0558254%。这说明慢点不是 attention，而是大量互相独立的行点积。

这一轮优化据此把 `lm_head` greedy argmax 和 GPT-2 F32 `linear` 的输出行切给持久 worker pool。每一行仍然是同一个 dot product；不同线程只负责不同输出行。完整 logits 路径把 logits buffer 分片写入，logits-free greedy 路径每个分片先找本地 best token，再按分片顺序合并，用严格大于比较保留较小 token id 的 tie-breaking。这样并行改变调度，不改变数学定义。

还有一个细节说明为什么优化必须带着 profiler 走。第一次 auto 阈值只按“一百万次乘加”判断是否并行，结果 `768\times768` 的 attention output projection 仍走串行。中间 profile 立即显示 attention projection 从约 6% 反弹到约 25%，成为新瓶颈。最终阈值改成：输出行数至少 512，且输入列数至少 512 时也允许并行。最终 profile 保存在 `books/cpu-volume-3/reports/lcqi-gpt2-threaded-hotspot-caw.txt`：4 token 生成中位数 75.9807ms，16 token 184.780ms；`lm_head` 约 20%，MLP 两个投影合计约 45%，QKV 约 19%，attention projection 约 12%，

attention 本体仍低于 1%。

用户继续问和 llama.cpp 的差距时，下一步就不能再停留在“行并行”。LCQI 新增 `include/lcqi/f32_kernels.hpp`、`src/f32_kernels.cpp` 和 `src/f32_kernels_avx2.cpp`，把 GPT-2 cached 热路径的单行点积抽成 `dot_f32_unchecked()`。checked `dot_f32()` 负责普通 span 长度检查，unchecked 入口给已经完成 shape validation 的 `linear`、`logits` 和 `logits-free lm_head` 使用；x86-64/GNU/Clang 构建会编译 AVX2/FMA 文件，运行时再用 `dot_f32_avx2_available()` 判断是否能执行。这样 AVX intrinsics 不进入 GPT-2 层逻辑，后续替换 AVX-512、int8 或 int4 block kernel 时，边界仍然清楚。

`tests/lcqi_tests.cpp` 也随之增加 F32 dot kernel 对照：同一组固定输入先跑 scalar，再跑 dispatch；如果当前 CPU 支持 AVX2/FMA，再显式跑 AVX2 实现，并要求误差落在 $1.0e-4$ 内。测试还会传入长度不一致的 span，确认 checked API 会拒绝。这样优化路径不是“编译过就算”，而是有 scalar oracle、运行时能力判断和错误输入边界。

这轮 F32 dot 优化以 `cb4d89f` 为 baseline，在 `caw` 上用同一 GPT-2 F32 模型、Release 构建和 `--threads0` 自动 16 workers 跑 5 轮。raw 报告保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-ab-caw.txt`，汇总保存在 `books/cpu-volume-3/reports/lcqi-gpt2-f32-dot-optimization-summary.md`。4 token 生成中位数从 74.4247ms 到 62.3628ms，约 1.19x；16 token 从 181.257ms 到 157.230ms，约 1.15x。decode 中位数分别从 26.1887ms 到 23.0640ms，从 132.201ms 到 117.734ms。这个提升比线程化小，因为它只缩短每一行内部 dot，而没有减少行数、层数、vocab 扫描或内存流量。

这个结论必须保守阅读。真实端到端 GPT-2 benchmark 受调度、频率、温度和共享机器状态影响明显，单次结果可能反向波动。因此报告里必须同时看 `median/min/max`、生成文本是否一致、baseline 和 current 是否同一构建入口。更大的差距仍然不在这轮 F32 AVX2 dot 里：LCQI 还没有 GGUF 量化导入、packed/quantized lm head、量化 block 权重、分页 KV cache、NUMA/亲和和成熟调度。也就是说，本轮把 reference path 从“cached 会话内多线程行点积”推进到“行点积有 SIMD 内核边界”，但还没有把它变成 llama.cpp 级推理引擎。

Listing 10.11 `src/f32_kernels.cpp`

```
1 #include <lcqi/f32_kernels.hpp>
2
3 #include <stdexcept>
4
5 namespace lcqi {
6 namespace {
7
8 constexpr std::size_t LCQI_F32_AVX2_MIN_DOT_SIZE = 32;
9
10 } // namespace
11
12 float dot_f32(std::span<const float> lhs, std::span<const float> rhs) {
13     if (lhs.size() != rhs.size()) {
14         throw std::runtime_error("dot_f32 size mismatch");
15     }
16     return dot_f32_unchecked(lhs.data(), rhs.data(), lhs.size());
17 }
18
19 float dot_f32_scalar_unchecked(const float* lhs,
20                                const float* rhs,
21                                std::size_t size) noexcept {
22     float sum = 0.0F;
23     for (std::size_t index = 0; index < size; ++index) {
24         sum += lhs[index] * rhs[index];
25     }
```

```

25     }
26     return sum;
27 }
28
29 float dot_f32_unchecked(const float* lhs,
30                         const float* rhs,
31                         std::size_t size) noexcept {
32     static const bool AVX2_AVAILABLE = dot_f32_avx2_available();
33     if (AVX2_AVAILABLE && size >= LCQI_F32_AVX2_MIN_DOT_SIZE) {
34         return dot_f32_avx2_unchecked(lhs, rhs, size);
35     }
36     return dot_f32_scalar_unchecked(lhs, rhs, size);
37 }
38
39 } // namespace lcqi
40
41 #if !defined(LCQI_ENABLE_AVX2)
42 namespace lcqi {
43
44 bool dot_f32_avx2_available() noexcept {
45     return false;
46 }
47
48 float dot_f32_avx2_unchecked(const float* lhs,
49                             const float* rhs,
50                             std::size_t size) noexcept {
51     return dot_f32_scalar_unchecked(lhs, rhs, size);
52 }
53
54 } // namespace lcqi
55 #endif

```

Listing 10.12 src/f32_kernels_avx2.cpp

```

1 #include <lcqi/f32_kernels.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <immintrin.h>
6
7 #include <cstdint>
8
9 namespace lcqi {
10 namespace {
11
12 constexpr std::size_t LCQI_F32_AVX2_LANE_COUNT = 8;
13 constexpr std::size_t LCQI_F32_AVX2_UNROLL_COUNT = 4;
14 constexpr std::size_t LCQI_F32_AVX2_UNROLLED_FLOATS =
15     LCQI_F32_AVX2_LANE_COUNT * LCQI_F32_AVX2_UNROLL_COUNT;
16
17 [[nodiscard]] float horizontal_sum(__m256 values) noexcept {
18     const __m128 low = _mm256_castps256_ps128(values);

```

```

19     const __m128 high = _mm256_extractf128_ps(values, 1);
20     const __m128 pair_sum = _mm_add_ps(low, high);
21     const __m128 first_fold = _mm_hadd_ps(pair_sum, pair_sum);
22     const __m128 second_fold = _mm_hadd_ps(first_fold, first_fold);
23     return _mm_cvtss_f32(second_fold);
24 }
25
26 } // namespace
27
28 bool dot_f32_avx2_available() noexcept {
29     #if defined(__GNUC__) || defined(__clang__)
30         __builtin_cpu_init();
31         return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
32     #else
33         return true;
34     #endif
35 }
36
37 float dot_f32_avx2_unchecked(const float* lhs,
38                             const float* rhs,
39                             std::size_t size) noexcept {
40     std::size_t index = 0;
41     __m256 accumulator0 = _mm256_setzero_ps();
42     __m256 accumulator1 = _mm256_setzero_ps();
43     __m256 accumulator2 = _mm256_setzero_ps();
44     __m256 accumulator3 = _mm256_setzero_ps();
45
46     for (; index + LCQI_F32_AVX2_UNROLLED_FLOATS <= size;
47          index += LCQI_F32_AVX2_UNROLLED_FLOATS) {
48         const __m256 lhs0 = _mm256_loadu_ps(lhs + index);
49         const __m256 rhs0 = _mm256_loadu_ps(rhs + index);
50         const __m256 lhs1 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT);
51         const __m256 rhs1 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT);
52         const __m256 lhs2 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 2);
53         const __m256 rhs2 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 2);
54         const __m256 lhs3 = _mm256_loadu_ps(lhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 3);
55         const __m256 rhs3 = _mm256_loadu_ps(rhs + index + ↵
↵ LCQI_F32_AVX2_LANE_COUNT * 3);
56
57         accumulator0 = _mm256_fmadd_ps(lhs0, rhs0, accumulator0);
58         accumulator1 = _mm256_fmadd_ps(lhs1, rhs1, accumulator1);
59         accumulator2 = _mm256_fmadd_ps(lhs2, rhs2, accumulator2);
60         accumulator3 = _mm256_fmadd_ps(lhs3, rhs3, accumulator3);
61     }
62
63     __m256 accumulator = _mm256_add_ps(
64         _mm256_add_ps(accumulator0, accumulator1),

```

```

65     _mm256_add_ps(accumulator2, accumulator3));
66     for (; index + LCQI_F32_AVX2_LANE_COUNT <= size;
67         index += LCQI_F32_AVX2_LANE_COUNT) {
68         const __m256 lhs_values = _mm256_loadu_ps(lhs + index);
69         const __m256 rhs_values = _mm256_loadu_ps(rhs + index);
70         accumulator = _mm256_fmadd_ps(lhs_values, rhs_values, accumulator);
71     }
72
73     float sum = horizontal_sum(accumulator);
74     for (; index < size; ++index) {
75         sum += lhs[index] * rhs[index];
76     }
77     return sum;
78 }
79
80 } // namespace lcqi
81
82 #endif

```

Listing 10.13 src/gpt2_reference.cpp

```

1  #include <lcqi/gpt2_reference.hpp>
2
3  #include <lcqi/f32_kernels.hpp>
4  #include <lcqi/safetensors.hpp>
5
6  #include <algorithm>
7  #include <array>
8  #include <chrono>
9  #include <condition_variable>
10 #include <cmath>
11 #include <cstdint>
12 #include <fstream>
13 #include <functional>
14 #include <limits>
15 #include <mutex>
16 #include <optional>
17 #include <sstream>
18 #include <stdexcept>
19 #include <string>
20 #include <string_view>
21 #include <thread>
22 #include <unordered_set>
23 #include <utility>
24
25 namespace lcqi {
26 namespace {
27
28 constexpr std::int32_t LCQI_GPT2_DEFAULT_BOS_EOS = 50256;
29 constexpr std::int32_t LCQI_GPT2_ATTENTION_PROJECTIONS = 3;
30 constexpr std::int32_t LCQI_GPT2_PAIR_TOKEN_COUNT = 2;
31 constexpr std::int32_t LCQI_GPT2_BYTE_COUNT = 256;

```



```

32 constexpr std::int32_t LCQI_GPT2_UNICODE_PRIVATE_BASE = 256;
33 constexpr std::int32_t LCQI_GPT2_ASCII_EXCLAMATION = 33;
34 constexpr std::int32_t LCQI_GPT2_ASCII_TILDE = 126;
35 constexpr std::int32_t LCQI_GPT2_LATIN_INVERTED_EXCLAMATION = 161;
36 constexpr std::int32_t LCQI_GPT2_LATIN_NOT_SIGN = 172;
37 constexpr std::int32_t LCQI_GPT2_LATIN_REGISTERED = 174;
38 constexpr std::int32_t LCQI_GPT2_LATIN_Y_DIAERESIS = 255;
39 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_MASK = 0x3F;
40 constexpr std::int32_t LCQI_GPT2_UTF8_CONTINUATION_BITS = 0x80;
41 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_HEAD = 0xC0;
42 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_HEAD = 0xE0;
43 constexpr std::int32_t LCQI_GPT2_UTF8_FOUR_BYTE_HEAD = 0xF0;
44 constexpr std::int32_t LCQI_GPT2_UTF8_ONE_BYTE_LIMIT = 0x80;
45 constexpr std::int32_t LCQI_GPT2_UTF8_TWO_BYTE_LIMIT = 0x800;
46 constexpr std::int32_t LCQI_GPT2_UTF8_THREE_BYTE_LIMIT = 0x10000;
47 constexpr unsigned char LCQI_GPT2_ASCII_APOSTROPHE = 0x27U;
48 constexpr float LCQI_GPT2_GELU_SQRT_2_OVER_PI = 0.7978845608028654F;
49 constexpr float LCQI_GPT2_GELU_CUBIC_COEFF = 0.044715F;
50 constexpr float LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY =
51     -std::numeric_limits<float>::infinity();
52 constexpr std::int32_t LCQI_GPT2_MAX_AUTO_WORKERS = 16;
53 constexpr std::int32_t LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS = 512;
54 constexpr std::int32_t LCQI_GPT2_PARALLEL_MIN_COLUMNS = 512;
55 constexpr std::int64_t LCQI_GPT2_PARALLEL_MIN_OPS = 1'000'000;
56 constexpr std::size_t LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER = 4;
57
58 using Gpt2ProfileClock = std::chrono::steady_clock;
59
60 } // namespace
61
62 namespace detail {
63
64 class Gpt2ParallelWorkerPool {
65 public:
66     explicit Gpt2ParallelWorkerPool(std::int32_t requested_workers)
67         : worker_count_(Gpt2ParallelWorkerPool::normalize_worker_count(↵
↵ requested_workers)) {
68         if (this->worker_count_ <= 1) {
69             return;
70         }
71         const std::size_t background_count =
72             static_cast<std::size_t>(this->worker_count_ - 1);
73         this->threads_.reserve(background_count);
74         for (std::size_t index = 0; index < background_count; ++index) {
75             this->threads_.emplace_back([this]() { this->worker_loop(); });
76         }
77     }
78
79     ~Gpt2ParallelWorkerPool() {
80     {
81         std::lock_guard<std::mutex> lock(this->mutex_);
82         this->stopping_ = true;

```

```

83         ++this->generation_;
84     }
85     this->condition_.notify_all();
86     for (std::thread& thread : this->threads_) {
87         if (thread.joinable()) {
88             thread.join();
89         }
90     }
91 }
92
93 Gpt2ParallelWorkerPool(const Gpt2ParallelWorkerPool&) = delete;
94 Gpt2ParallelWorkerPool& operator=(const Gpt2ParallelWorkerPool&) = delete;
95
96 [[nodiscard]] std::int32_t worker_count() const noexcept {
97     return this->worker_count_;
98 }
99
100 void parallel_for_rows(std::size_t begin,
101                       std::size_t end,
102                       std::size_t grain,
103                       const std::function<void(std::size_t, std::size_t)>&
↪ function) {
104     if (end <= begin) {
105         return;
106     }
107     const std::size_t row_count = end - begin;
108     if (this->worker_count_ <= 1 || row_count <= grain) {
109         function(begin, end);
110         return;
111     }
112
113     const std::size_t task_count =
114     ↪ std::min<std::size_t>(static_cast<std::size_t>(this->worker_count_)
↪ ,
115                          (row_count + grain - 1) / grain);
116     if (task_count <= 1) {
117         function(begin, end);
118         return;
119     }
120
121     {
122         std::lock_guard<std::mutex> lock(this->mutex_);
123         this->task_begin_ = begin;
124         this->task_end_ = end;
125         this->task_count_ = task_count;
126         this->next_worker_index_ = 0;
127         this->active_workers_ = task_count - 1;
128         this->task_function_ = &function;
129         ++this->generation_;
130     }
131     this->condition_.notify_all();
132

```



```

133     const std::size_t main_worker = task_count - 1;
134     const auto [main_begin, main_end] = this->chunk_bounds(begin, end, ↵
↵ task_count, main_worker);
135     function(main_begin, main_end);
136
137     std::unique_lock<std::mutex> lock(this->mutex_);
138     this->finished_.wait(lock, [this]() { return this->active_workers_ == ↵
↵ 0; });
139     this->task_function_ = nullptr;
140 }
141
142 private:
143     std::int32_t worker_count_ = 1;
144     std::vector<std::thread> threads_;
145     std::mutex mutex_;
146     std::condition_variable condition_;
147     std::condition_variable finished_;
148     bool stopping_ = false;
149     std::uint64_t generation_ = 0;
150     std::size_t task_begin_ = 0;
151     std::size_t task_end_ = 0;
152     std::size_t task_count_ = 0;
153     std::size_t next_worker_index_ = 0;
154     std::size_t active_workers_ = 0;
155     const std::function<void(std::size_t, std::size_t)>* task_function_ = ↵
↵ nullptr;
156
157     [[nodiscard]] static std::int32_t normalize_worker_count(std::int32_t ↵
↵ requested_workers) {
158         if (requested_workers > 0) {
159             return requested_workers;
160         }
161         const unsigned int hardware_workers = std::thread::hardware_concurrency↵
↵ ();
162         if (hardware_workers == 0) {
163             return 1;
164         }
165         const std::int32_t capped =
166             std::min<std::int32_t>(static_cast<std::int32_t>(hardware_workers),
167                                   LCQI_GPT2_MAX_AUTO_WORKERS);
168         return std::max(capped, 1);
169     }
170
171     [[nodiscard]] std::pair<std::size_t, std::size_t> chunk_bounds(
172         std::size_t begin,
173         std::size_t end,
174         std::size_t task_count,
175         std::size_t worker_index) const {
176         const std::size_t row_count = end - begin;
177         const std::size_t chunk_begin = begin + row_count * worker_index / ↵
↵ task_count;
178         const std::size_t chunk_end = begin + row_count * (worker_index + 1) / ↵

```

```

179     ↪ task_count;
180     return {chunk_begin, chunk_end};
181 }
182
183 void worker_loop() {
184     std::uint64_t observed_generation = 0;
185     while (true) {
186         std::size_t worker_index = 0;
187         std::size_t begin = 0;
188         std::size_t end = 0;
189         ↪ const std::function<void(std::size_t, std::size_t)>* function = ↪
190         ↪ nullptr;
191         {
192             std::unique_lock<std::mutex> lock(this->mutex_);
193             this->condition_.wait(lock, [this, observed_generation]() {
194                 ↪ return this->stopping_ || this->generation_ != ↪
195                 ↪ observed_generation;
196             });
197             if (this->stopping_) {
198                 return;
199             }
200             observed_generation = this->generation_;
201             if (this->task_function_ == nullptr ||
202                 this->next_worker_index_ >= this->task_count_ - 1) {
203                 ↪ continue;
204             }
205             worker_index = this->next_worker_index_;
206             ++this->next_worker_index_;
207             const auto bounds =
208                 this->chunk_bounds(this->task_begin_,
209                                     this->task_end_,
210                                     this->task_count_,
211                                     worker_index);
212             begin = bounds.first;
213             end = bounds.second;
214             function = this->task_function_;
215         }
216         (*function)(begin, end);
217
218         {
219             std::lock_guard<std::mutex> lock(this->mutex_);
220             --this->active_workers_;
221             if (this->active_workers_ == 0) {
222                 ↪ this->finished_.notify_one();
223             }
224         }
225     }
226 }
227 } // namespace detail

```

```

228
229 namespace {
230
231 class JsonCursor {
232 public:
233     explicit JsonCursor(std::string_view text) : text_(text) {}
234
235     void expect(char expected) {
236         this->skip_ws();
237         if (this->eof() || this->text_[this->position_] != expected) {
238             throw std::runtime_error("invalid JSON syntax");
239         }
240         ++this->position_;
241     }
242
243     [[nodiscard]] bool consume(char expected) {
244         this->skip_ws();
245         if (!this->eof() && this->text_[this->position_] == expected) {
246             ++this->position_;
247             return true;
248         }
249         return false;
250     }
251
252     [[nodiscard]] std::string parse_string() {
253         this->skip_ws();
254         this->expect('"');
255         std::string result;
256         while (!this->eof()) {
257             const char ch = this->text_[this->position_++];
258             if (ch == '"') {
259                 return result;
260             }
261             if (ch == '\\') {
262                 result += this->parse_escape();
263             } else {
264                 result.push_back(ch);
265             }
266         }
267         throw std::runtime_error("unterminated JSON string");
268     }
269
270     [[nodiscard]] std::int64_t parse_i64() {
271         this->skip_ws();
272         bool negative = false;
273         if (this->consume('-')) {
274             negative = true;
275         }
276         if (this->eof() || this->text_[this->position_] < '0' ||
277             this->text_[this->position_] > '9') {
278             throw std::runtime_error("expected JSON integer");
279         }

```

```

280     std::int64_t value = 0;
281     while (!this->eof() && this->text_[this->position_] >= '0' &&
282            this->text_[this->position_] <= '9') {
283         const std::int64_t digit = this->text_[this->position_] - '0';
284         if (value >
285             (std::numeric_limits<std::int64_t>::max() - digit) / 10) {
286             throw std::runtime_error("JSON integer overflow");
287         }
288         value = value * 10 + digit;
289         ++this->position_;
290     }
291     return negative ? -value : value;
292 }
293
294 [[nodiscard]] double parse_number() {
295     this->skip_ws();
296     const std::size_t begin = this->position_;
297     if (!this->eof() && this->text_[this->position_] == '-') {
298         ++this->position_;
299     }
300     while (!this->eof() && this->text_[this->position_] >= '0' &&
301            this->text_[this->position_] <= '9') {
302         ++this->position_;
303     }
304     if (!this->eof() && this->text_[this->position_] == '.') {
305         ++this->position_;
306         while (!this->eof() && this->text_[this->position_] >= '0' &&
307                this->text_[this->position_] <= '9') {
308             ++this->position_;
309         }
310     }
311     if (!this->eof() &&
312         (this->text_[this->position_] == 'e' || this->text_[this->position_ ←
↵ ] == 'E')) {
313         ++this->position_;
314         if (!this->eof() &&
315             (this->text_[this->position_] == '+' || this->text_[this-> ←
↵ position_] == '-')) {
316             ++this->position_;
317         }
318         while (!this->eof() && this->text_[this->position_] >= '0' &&
319                this->text_[this->position_] <= '9') {
320             ++this->position_;
321         }
322     }
323     if (begin == this->position_) {
324         throw std::runtime_error("expected JSON number");
325     }
326     return std::stod(std::string(this->text_.substr(begin, this->position_ ←
↵ - begin)));
327 }
328

```

```

329 void skip_value() {
330     this->skip_ws();
331     if (this->eof()) {
332         throw std::runtime_error("unexpected end of JSON");
333     }
334     const char ch = this->text_[this->position_];
335     if (ch == '"') {
336         static_cast<void>(this->parse_string());
337         return;
338     }
339     if (ch == '{') {
340         this->expect('{');
341         if (this->consume('}') {
342             return;
343         }
344         while (true) {
345             static_cast<void>(this->parse_string());
346             this->expect(':');
347             this->skip_value();
348             if (this->consume('}')) {
349                 return;
350             }
351             this->expect(',');
352         }
353     }
354     if (ch == '[') {
355         this->expect('[');
356         if (this->consume(']')) {
357             return;
358         }
359         while (true) {
360             this->skip_value();
361             if (this->consume(']')) {
362                 return;
363             }
364             this->expect(',');
365         }
366     }
367     if (ch == 't') {
368         this->expect_literal("true");
369         return;
370     }
371     if (ch == 'f') {
372         this->expect_literal("false");
373         return;
374     }
375     if (ch == 'n') {
376         this->expect_literal("null");
377         return;
378     }
379     static_cast<void>(this->parse_number());
380 }

```

```

381
382 [[nodiscard]] bool eof_after_ws() {
383     this->skip_ws();
384     return this->eof();
385 }
386
387 private:
388     std::string_view text_;
389     std::size_t position_ = 0;
390
391 [[nodiscard]] bool eof() const {
392     return this->position_ >= this->text_.size();
393 }
394
395 void skip_ws() {
396     while (!this->eof() &&
397           (this->text_[this->position_] == ' ' ||
398            this->text_[this->position_] == '\n' ||
399            this->text_[this->position_] == '\r' ||
400            this->text_[this->position_] == '\t')) {
401         ++this->position_;
402     }
403 }
404
405 void expect_literal(std::string_view literal) {
406     for (const char expected : literal) {
407         if (this->eof() || this->text_[this->position_] != expected) {
408             throw std::runtime_error("invalid JSON literal");
409         }
410         ++this->position_;
411     }
412 }
413
414 [[nodiscard]] std::string parse_escape() {
415     if (this->eof()) {
416         throw std::runtime_error("unterminated JSON escape");
417     }
418     const char escaped = this->text_[this->position_++];
419     switch (escaped) {
420     case '"':
421     case '\\':
422     case '/':
423         return std::string(1, escaped);
424     case 'b':
425         return "\b";
426     case 'f':
427         return "\f";
428     case 'n':
429         return "\n";
430     case 'r':
431         return "\r";
432     case 't':

```

```

433         return "\\t";
434     case 'u':
435         return this->parse_unicode_escape();
436     default:
437         throw std::runtime_error("unsupported JSON escape");
438     }
439 }
440
441 [[nodiscard]] std::string parse_unicode_escape() {
442     std::uint32_t value = 0;
443     for (std::int32_t i = 0; i < 4; ++i) {
444         if (this->eof()) {
445             throw std::runtime_error("unterminated JSON unicode escape");
446         }
447         const char ch = this->text_[this->position_++];
448         value <= 4U;
449         if (ch >= '0' && ch <= '9') {
450             value += static_cast<std::uint32_t>(ch - '0');
451         } else if (ch >= 'a' && ch <= 'f') {
452             value += static_cast<std::uint32_t>(10 + ch - 'a');
453         } else if (ch >= 'A' && ch <= 'F') {
454             value += static_cast<std::uint32_t>(10 + ch - 'A');
455         } else {
456             throw std::runtime_error("invalid JSON unicode escape");
457         }
458     }
459     std::string encoded;
460     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
461         encoded.push_back(static_cast<char>(value));
462     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
463         encoded.push_back(static_cast<char>(
464             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
465         encoded.push_back(static_cast<char>(
466             LCQI_GPT2_UTF8_CONTINUATION_BITS |
467             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
468     } else {
469         encoded.push_back(static_cast<char>(
470             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
471         encoded.push_back(static_cast<char>(
472             LCQI_GPT2_UTF8_CONTINUATION_BITS |
473             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
474         encoded.push_back(static_cast<char>(
475             LCQI_GPT2_UTF8_CONTINUATION_BITS |
476             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
477     }
478     return encoded;
479 }
480 };
481
482 std::string read_text_file(const std::filesystem::path& path) {
483     std::ifstream input(path);
484     if (!input) {

```



```

485     throw std::runtime_error("cannot open text file: " + path.string());
486 }
487 std::ostringstream buffer;
488 buffer << input.rdbuf();
489 return buffer.str();
490 }
491
492 std::size_t checked_size(std::int64_t value, const char* name) {
493     if (value < 0) {
494         throw std::runtime_error(std::string(name) + " cannot be negative");
495     }
496     return static_cast<std::size_t>(value);
497 }
498
499 std::string encode_codepoint(std::uint32_t value) {
500     std::string encoded;
501     if (value < LCQI_GPT2_UTF8_ONE_BYTE_LIMIT) {
502         encoded.push_back(static_cast<char>(value));
503     } else if (value < LCQI_GPT2_UTF8_TWO_BYTE_LIMIT) {
504         encoded.push_back(static_cast<char>(
505             LCQI_GPT2_UTF8_TWO_BYTE_HEAD | (value >> 6U)));
506         encoded.push_back(static_cast<char>(
507             LCQI_GPT2_UTF8_CONTINUATION_BITS |
508             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
509     } else if (value < LCQI_GPT2_UTF8_THREE_BYTE_LIMIT) {
510         encoded.push_back(static_cast<char>(
511             LCQI_GPT2_UTF8_THREE_BYTE_HEAD | (value >> 12U)));
512         encoded.push_back(static_cast<char>(
513             LCQI_GPT2_UTF8_CONTINUATION_BITS |
514             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
515         encoded.push_back(static_cast<char>(
516             LCQI_GPT2_UTF8_CONTINUATION_BITS |
517             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
518     } else {
519         encoded.push_back(static_cast<char>(
520             LCQI_GPT2_UTF8_FOUR_BYTE_HEAD | (value >> 18U)));
521         encoded.push_back(static_cast<char>(
522             LCQI_GPT2_UTF8_CONTINUATION_BITS |
523             ((value >> 12U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
524         encoded.push_back(static_cast<char>(
525             LCQI_GPT2_UTF8_CONTINUATION_BITS |
526             ((value >> 6U) & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
527         encoded.push_back(static_cast<char>(
528             LCQI_GPT2_UTF8_CONTINUATION_BITS |
529             (value & LCQI_GPT2_UTF8_CONTINUATION_MASK)));
530     }
531     return encoded;
532 }
533
534 void require_positive(std::int32_t value, const char* name) {
535     if (value <= 0) {
536         throw std::runtime_error(std::string(name) + " must be positive");

```

```

537     }
538 }
539
540 void validate_config(const Gpt2Config& config) {
541     require_positive(config.hidden_size, "hidden_size");
542     require_positive(config.head_count, "head_count");
543     require_positive(config.layer_count, "layer_count");
544     require_positive(config.max_positions, "max_positions");
545     require_positive(config.vocab_size, "vocab_size");
546     require_positive(config.intermediate_size, "intermediate_size");
547     if (config.hidden_size % config.head_count != 0) {
548         throw std::runtime_error("hidden_size must be divisible by head_count")↵
↵ ;
549     }
550     if (config.layer_norm_epsilon <= 0.0F) {
551         throw std::runtime_error("layer_norm_epsilon must be positive");
552     }
553 }
554
555 std::int32_t head_dim(const Gpt2Config& config) {
556     return config.hidden_size / config.head_count;
557 }
558
559 std::size_t matrix_size(std::int32_t rows, std::int32_t columns) {
560     return checked_size(rows, "rows") * checked_size(columns, "columns");
561 }
562
563 Gpt2ProfileClock::time_point profile_start(const Gpt2HotspotProfile* profile) {
564     if (profile == nullptr) {
565         return Gpt2ProfileClock::time_point{};
566     }
567     return Gpt2ProfileClock::now();
568 }
569
570 void add_profile_ms(Gpt2HotspotProfile* profile,
571                   double Gpt2HotspotProfile::*field,
572                   Gpt2ProfileClock::time_point start) {
573     if (profile == nullptr) {
574         return;
575     }
576     (*profile).*field +=
577         std::chrono::duration<double, std::milli>(Gpt2ProfileClock::now() - ↵
↵ start).count();
578 }
579
580 void validate_vector_size(std::size_t actual, std::int32_t expected, const char↵
↵ * name) {
581     if (actual != checked_size(expected, name)) {
582         throw std::runtime_error(std::string(name) + " size mismatch");
583     }
584 }
585

```

```

586 void validate_linear(const Gpt2LinearF32& linear, const char* name) {
587     require_positive(linear.input_size, "linear input_size");
588     require_positive(linear.output_size, "linear output_size");
589     if (linear.weights.size() != matrix_size(linear.output_size, linear.↵
↵ input_size)) {
590         throw std::runtime_error(std::string(name) + " weight size mismatch");
591     }
592     if (!linear.bias.empty() &&
593         linear.bias.size() != checked_size(linear.output_size, "linear ↵
↵ output_size")) {
594         throw std::runtime_error(std::string(name) + " bias size mismatch");
595     }
596 }
597
598 void validate_model(const Gpt2ReferenceModel& model) {
599     validate_config(model.config);
600     if (model.layers.size() != checked_size(model.config.layer_count, "↵
↵ layer_count")) {
601         throw std::runtime_error("GPT-2 layer count mismatch");
602     }
603     if (model.token_embedding.size() !=
604         matrix_size(model.config.vocab_size, model.config.hidden_size)) {
605         throw std::runtime_error("GPT-2 token embedding size mismatch");
606     }
607     if (model.position_embedding.size() !=
608         matrix_size(model.config.max_positions, model.config.hidden_size)) {
609         throw std::runtime_error("GPT-2 position embedding size mismatch");
610     }
611     validate_vector_size(model.final_ln_weight.size(), model.config.hidden_size↵
↵ , "ln_f weight");
612     validate_vector_size(model.final_ln_bias.size(), model.config.hidden_size, ↵
↵ "ln_f bias");
613     if (model.tie_lm_head_to_embedding) {
614         if (!model.lm_head_weight.empty()) {
615             throw std::runtime_error("tied GPT-2 lm_head should not carry ↵
↵ separate weights");
616         }
617     } else if (model.lm_head_weight.size() !=
618         matrix_size(model.config.vocab_size, model.config.hidden_size)) ↵
↵ {
619         throw std::runtime_error("GPT-2 lm_head size mismatch");
620     }
621     for (const Gpt2LayerWeightsF32& layer : model.layers) {
622         validate_vector_size(layer.ln_1_weight.size(), model.config.hidden_size↵
↵ , "ln_1 weight");
623         validate_vector_size(layer.ln_1_bias.size(), model.config.hidden_size, ↵
↵ "ln_1 bias");
624         validate_linear(layer.c_attn, "c_attn");
625         validate_linear(layer.c_proj, "c_proj");
626         validate_vector_size(layer.ln_2_weight.size(), model.config.hidden_size↵
↵ , "ln_2 weight");
627         validate_vector_size(layer.ln_2_bias.size(), model.config.hidden_size, ↵

```

```

    ↪ "ln_2 bias");
628     validate_linear(layer.c_fc, "c_fc");
629     validate_linear(layer.mlp_c_proj, "mlp_c_proj");
630 }
631 }
632
633 std::span<const float> row_span(std::span<const float> values,
634                                std::int32_t row,
635                                std::int32_t row_width) {
636     return values.subspan(checked_size(row, "row") * checked_size(row_width, "↪
    ↪ row_width"),
637                           checked_size(row_width, "row_width"));
638 }
639
640 void add_inplace(std::span<float> target, std::span<const float> source) {
641     if (target.size() != source.size()) {
642         throw std::runtime_error("GPT-2 residual add size mismatch");
643     }
644     for (std::size_t index = 0; index < target.size(); ++index) {
645         target[index] += source[index];
646     }
647 }
648
649 void layer_norm(std::span<const float> input,
650                std::span<const float> weight,
651                std::span<const float> bias,
652                float epsilon,
653                std::span<float> output) {
654     if (input.empty() || input.size() != weight.size() || input.size() != bias.↪
    ↪ size() ||
655         input.size() != output.size()) {
656         throw std::runtime_error("GPT-2 LayerNorm size mismatch");
657     }
658     float mean = 0.0F;
659     for (const float value : input) {
660         mean += value;
661     }
662     mean /= static_cast<float>(input.size());
663
664     float variance = 0.0F;
665     for (const float value : input) {
666         const float centered = value - mean;
667         variance += centered * centered;
668     }
669     variance /= static_cast<float>(input.size());
670     const float scale = 1.0F / std::sqrt(variance + epsilon);
671
672     for (std::size_t index = 0; index < input.size(); ++index) {
673         output[index] = (input[index] - mean) * scale * weight[index] + bias[↪
    ↪ index];
674     }
675 }

```

```

676
677 void linear_f32(const Gpt2LinearF32& linear,
678               std::span<const float> input,
679               std::span<float> output) {
680     validate_linear(linear, "GPT-2 linear");
681     if (input.size() != checked_size(linear.input_size, "linear input") ||
682         output.size() != checked_size(linear.output_size, "linear output")) {
683         throw std::runtime_error("GPT-2 linear input/output size mismatch");
684     }
685     const std::size_t input_size = checked_size(linear.input_size, "linear ↵
↵ input");
686     const std::size_t output_size = checked_size(linear.output_size, "linear ↵
↵ output");
687     for (std::size_t out = 0; out < output_size; ++out) {
688         float sum = linear.bias.empty() ? 0.0F : linear.bias[out];
689         const std::size_t row_base = out * input_size;
690         for (std::size_t in = 0; in < input_size; ++in) {
691             sum += linear.weights[row_base + in] * input[in];
692         }
693         output[out] = sum;
694     }
695 }
696
697 void linear_f32_unchecked(const Gpt2LinearF32& linear,
698                          std::span<const float> input,
699                          std::span<float> output) {
700     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
701     const std::size_t output_size = static_cast<std::size_t>(linear.output_size↵
↵ );
702     const bool has_bias = !linear.bias.empty();
703     const float* weights = linear.weights.data();
704     const float* bias = linear.bias.data();
705     const float* input_data = input.data();
706     for (std::size_t out = 0; out < output_size; ++out) {
707         const float* row = weights + out * input_size;
708         output[out] =
709             dot_f32_unchecked(row, input_data, input_size) + (has_bias ? bias[↵
↵ out] : 0.0F);
710     }
711 }
712
713 [[nodiscard]] bool should_parallelize_rows(const Gpt2ExecutionOptions& options,
714                                           std::size_t rows,
715                                           std::size_t columns,
716                                           const detail::Gpt2ParallelWorkerPool↵
↵ * worker_pool) {
717     if (worker_pool == nullptr || worker_pool->worker_count() <= 1) {
718         return false;
719     }
720     const std::int32_t min_rows =
721         options.parallel_min_rows > 0 ? options.parallel_min_rows
722         : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;

```

```

723     if (rows < checked_size(min_rows, "parallel_min_rows")) {
724         return false;
725     }
726     if (options.worker_count > 1) {
727         return true;
728     }
729     return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS, "↵
↵ parallel_min_columns") ||
730         rows * columns >= static_cast<std::size_t>(↵
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
731 }
732
733 [[nodiscard]] std::size_t parallel_row_grain(std::size_t rows,
734                                             const detail::↵
↵ Gpt2ParallelWorkerPool& worker_pool) {
735     const std::size_t workers = checked_size(worker_pool.worker_count(), "↵
↵ worker_count");
736     const std::size_t target_tasks = workers * ↵
↵ LCQI_GPT2_PARALLEL_ROW_BLOCK_MULTIPLIER;
737     return std::max<std::size_t>(1, (rows + target_tasks - 1) / target_tasks);
738 }
739
740 [[nodiscard]] bool model_has_parallel_work(const Gpt2ReferenceModel& model,
741                                             const Gpt2ExecutionOptions& options)↵
↵ {
742     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
↵ hidden_size");
743     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
↵ vocab_size");
744     const std::size_t intermediate_size =
745         checked_size(model.config.intermediate_size, "intermediate_size");
746     const std::size_t qkv_rows =
747         checked_size(model.config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS↵
↵ ,
748                     "qkv output rows");
749     const std::int32_t min_rows =
750         options.parallel_min_rows > 0 ? options.parallel_min_rows
751         : LCQI_GPT2_DEFAULT_PARALLEL_MIN_ROWS;
752     const auto large_enough = [min_rows](std::size_t rows, std::size_t columns)↵
↵ {
753         if (rows < checked_size(min_rows, "parallel_min_rows")) {
754             return false;
755         }
756         return columns >= checked_size(LCQI_GPT2_PARALLEL_MIN_COLUMNS,
757                                         "parallel_min_columns") ||
758             rows * columns >= static_cast<std::size_t>(↵
↵ LCQI_GPT2_PARALLEL_MIN_OPS);
759     };
760     if (options.worker_count > 1) {
761         return vocab_size >= checked_size(min_rows, "parallel_min_rows") ||
762             qkv_rows >= checked_size(min_rows, "parallel_min_rows") ||
763             intermediate_size >= checked_size(min_rows, "parallel_min_rows")↵

```

```

↪ ||
764         hidden_size >= checked_size(min_rows, "parallel_min_rows");
765     }
766     if (options.worker_count == 1) {
767         return false;
768     }
769     return large_enough(vocab_size, hidden_size) ||
770            large_enough(qkv_rows, hidden_size) ||
771            large_enough(hidden_size, hidden_size) ||
772            large_enough(intermediate_size, hidden_size) ||
773            large_enough(hidden_size, intermediate_size);
774 }
775
776 [[nodiscard]] std::unique_ptr<detail::Gpt2ParallelWorkerPool> make_worker_pool(
777     const Gpt2ReferenceModel& model,
778     const Gpt2ExecutionOptions& options) {
779     if (!model_has_parallel_work(model, options)) {
780         return nullptr;
781     }
782     return std::make_unique<detail::Gpt2ParallelWorkerPool>(options.↪
↪ worker_count);
783 }
784
785 void linear_f32_unchecked_parallel(const Gpt2LinearF32& linear,
786                                   std::span<const float> input,
787                                   std::span<float> output,
788                                   const Gpt2ExecutionOptions& options,
789                                   detail::Gpt2ParallelWorkerPool* worker_pool)↪
↪ {
790     const std::size_t input_size = static_cast<std::size_t>(linear.input_size);
791     const std::size_t output_size = static_cast<std::size_t>(linear.output_size↪
↪ );
792     if (!should_parallelize_rows(options, output_size, input_size, worker_pool)↪
↪ ) {
793         linear_f32_unchecked(linear, input, output);
794         return;
795     }
796
797     const bool has_bias = !linear.bias.empty();
798     const float* weights = linear.weights.data();
799     const float* bias = linear.bias.data();
800     const float* input_data = input.data();
801     float* output_data = output.data();
802     const std::function<void(std::size_t, std::size_t)> rows_fn =
803         [input_size, has_bias, weights, bias, input_data, output_data](
804             std::size_t begin,
805             std::size_t end) {
806         for (std::size_t out = begin; out < end; ++out) {
807             const float* row = weights + out * input_size;
808             output_data[out] =
809                 dot_f32_unchecked(row, input_data, input_size) +
810                 (has_bias ? bias[out] : 0.0F);

```

```

811     }
812 };
813 worker_pool->parallel_for_rows(0, output_size, parallel_row_grain(↵
↵ output_size, *worker_pool), rows_fn);
814 }
815
816 float gelu_new(float value) {
817     const float cubic = value * value * value;
818     return 0.5F * value *
819         (1.0F + std::tanh(LCQI_GPT2_GELU_SQRT_2_OVER_PI *
820             (value + LCQI_GPT2_GELU_CUBIC_COEFF * cubic)));
821 }
822
823 std::int32_t argmax(std::span<const float> values) {
824     if (values.empty()) {
825         throw std::runtime_error("cannot take argmax of empty logits");
826     }
827     std::int32_t best_index = 0;
828     float best_value = values.front();
829     for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size(↵
↵ )); ++index) {
830         const float value = values[checked_size(index, "argmax index")];
831         if (value > best_value) {
832             best_value = value;
833             best_index = index;
834         }
835     }
836     return best_index;
837 }
838
839 void project_qkv(const Gpt2Config& config,
840                 const Gpt2LayerWeightsF32& layer,
841                 std::span<const float> hidden,
842                 std::span<float> q,
843                 std::span<float> k,
844                 std::span<float> v) {
845     std::vector<float> packed(matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS,
846                                           config.hidden_size),
847                               0.0F);
848     linear_f32(layer.c_attn, hidden, packed);
849     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
850     std::copy_n(packed.begin(), config.hidden_size, q.begin());
851     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
852                 config.hidden_size,
853                 k.begin());
854     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
855                 config.hidden_size,
856                 v.begin());
857 }
858
859 void project_qkv_reuse_workspace(const Gpt2Config& config,
860                                 const Gpt2LayerWeightsF32& layer,

```



```

861         std::span<const float> hidden,
862         std::span<float> packed,
863         std::span<float> q,
864         std::span<float> k,
865         std::span<float> v,
866         const Gpt2ExecutionOptions& options,
867         detail::Gpt2ParallelWorkerPool* worker_pool) {
868     linear_f32_unchecked_parallel(layer.c_attn, hidden, packed, options, ↵
↵ worker_pool);
869     const std::size_t width = checked_size(config.hidden_size, "hidden_size");
870     std::copy_n(packed.begin(), config.hidden_size, q.begin());
871     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width),
872               config.hidden_size,
873               k.begin());
874     std::copy_n(packed.begin() + static_cast<std::ptrdiff_t>(width * 2),
875               config.hidden_size,
876               v.begin());
877 }
878
879 void attention_full_sequence(const Gpt2Config& config,
880                             std::span<const float> q_by_pos,
881                             std::span<const float> k_by_pos,
882                             std::span<const float> v_by_pos,
883                             std::int32_t sequence_length,
884                             std::span<float> output_by_pos) {
885     const std::int32_t local_head_dim = head_dim(config);
886     const float score_scale = 1.0F / std::sqrt(static_cast<float>(↵
↵ local_head_dim));
887     std::fill(output_by_pos.begin(), output_by_pos.end(), 0.0F);
888     std::vector<float> scores(checked_size(sequence_length, "sequence_length"),
889                             LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY);
890
891     for (std::int32_t position = 0; position < sequence_length; ++position) {
892         for (std::int32_t head = 0; head < config.head_count; ++head) {
893             float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
894             for (std::int32_t past = 0; past <= position; ++past) {
895                 float dot = 0.0F;
896                 for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
897                     const std::size_t q_index =
898                         matrix_size(position, config.hidden_size) +
899                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
900                     const std::size_t k_index =
901                         matrix_size(past, config.hidden_size) +
902                         matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
903                     dot += q_by_pos[q_index] * k_by_pos[k_index];
904                 }
905                 const float score = dot * score_scale;
906                 scores[checked_size(past, "past")] = score;
907                 max_score = std::max(max_score, score);
908             }

```

```

909
910     float denominator = 0.0F;
911     for (std::int32_t past = 0; past <= position; ++past) {
912         const std::size_t index = checked_size(past, "past");
913         scores[index] = std::exp(scores[index] - max_score);
914         denominator += scores[index];
915     }
916     if (denominator <= 0.0F) {
917         throw std::runtime_error("GPT-2 attention denominator is ↵
↵ invalid");
918     }
919     for (std::int32_t past = 0; past <= position; ++past) {
920         const float probability = scores[checked_size(past, "past")] / ↵
↵ denominator;
921         for (std::int32_t dim = 0; dim < local_head_dim; ++dim) {
922             const std::size_t output_index =
923                 matrix_size(position, config.hidden_size) +
924                 matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
925             const std::size_t value_index =
926                 matrix_size(past, config.hidden_size) +
927                 matrix_size(head, local_head_dim) + checked_size(dim, "↵
↵ head dim");
928             output_by_pos[output_index] += probability * v_by_pos[↵
↵ value_index];
929         }
930     }
931 }
932 }
933 }
934
935 void forward_gpt2_layer(const Gpt2Config& config,
936                         const Gpt2LayerWeightsF32& layer,
937                         std::int32_t sequence_length,
938                         std::vector<float>& hidden_by_pos) {
939     std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↵
↵ 0.0F);
940     std::vector<float> q_by_pos(matrix_size(sequence_length, config.hidden_size)↵
↵ ), 0.0F);
941     std::vector<float> k_by_pos(q_by_pos.size(), 0.0F);
942     std::vector<float> v_by_pos(q_by_pos.size(), 0.0F);
943     std::vector<float> attention_by_pos(q_by_pos.size(), 0.0F);
944     std::vector<float> projected(checked_size(config.hidden_size, "hidden_size"↵
↵ ), 0.0F);
945     std::vector<float> mlp_fc(checked_size(config.intermediate_size, "↵
↵ intermediate_size"), 0.0F);
946     std::vector<float> mlp_out(checked_size(config.hidden_size, "hidden_size"),↵
↵ 0.0F);
947
948     for (std::int32_t position = 0; position < sequence_length; ++position) {
949         const std::span<float> hidden = std::span<float>(
950             hidden_by_pos.data() + matrix_size(position, config.hidden_size),

```



```

1001         std::span<float> hidden,
1002         Gpt2HotspotProfile* hotspot_profile,
1003         const Gpt2ExecutionOptions& options,
1004         detail::Gpt2ParallelWorkerPool* worker_pool) {
1005     if (hotspot_profile != nullptr) {
1006         ++hotspot_profile->layer_steps;
1007     }
1008     std::span<float> normed = workspace.normed();
1009     std::span<float> query = workspace.query();
1010     std::span<float> key = workspace.key();
1011     std::span<float> value = workspace.value();
1012     std::span<float> attention = workspace.attention();
1013     std::span<float> projected = workspace.projected();
1014     std::span<float> qkv_packed = workspace.qkv_packed();
1015     std::span<float> mlp_fc = workspace.mlp_fc();
1016     std::span<float> mlp_out = workspace.mlp_out();
1017     std::span<float> scores = workspace.scores_prefix(model_position + 1);
1018
1019     Gpt2ProfileClock::time_point start =
1020         profile_start(hotspot_profile);
1021     layer_norm(hidden,
1022               layer.ln_1_weight,
1023               layer.ln_1_bias,
1024               config.layer_norm_epsilon,
1025               normed);
1026     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1027
1028     start = profile_start(hotspot_profile);
1029     project_qkv_reuse_workspace(config,
1030                                layer,
1031                                normed,
1032                                qkv_packed,
1033                                query,
1034                                key,
1035                                value,
1036                                options,
1037                                worker_pool);
1038     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::qkv_projection_ms, ←
↪ start);
1039
1040     start = profile_start(hotspot_profile);
1041     cache.append(layer_id, model_position, key, value);
1042     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::kv_append_ms, start);
1043
1044     start = profile_start(hotspot_profile);
1045     detail::attend_cached_position(cache,
1046                                   layer_id,
1047                                   model_position,
1048                                   query,
1049                                   scores,
1050                                   attention);
1051     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::attention_ms, start);

```

```

1052
1053     start = profile_start(hotspot_profile);
1054     linear_f32_unchecked_parallel(layer.c_proj, attention, projected, options, ↵
↵ worker_pool);
1055     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::↵
↵ attention_projection_ms, start);
1056
1057     start = profile_start(hotspot_profile);
1058     add_inplace(hidden, projected);
1059     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↵
↵ );
1060
1061     start = profile_start(hotspot_profile);
1062     layer_norm(hidden,
1063                 layer.ln_2_weight,
1064                 layer.ln_2_bias,
1065                 config.layer_norm_epsilon,
1066                 normed);
1067     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::layer_norm_ms, start);
1068
1069     start = profile_start(hotspot_profile);
1070     linear_f32_unchecked_parallel(layer.c_fc, normed, mlp_fc, options, ↵
↵ worker_pool);
1071     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_fc_ms, start);
1072
1073     start = profile_start(hotspot_profile);
1074     for (float& value_ref : mlp_fc) {
1075         value_ref = gelu_new(value_ref);
1076     }
1077     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::gelu_ms, start);
1078
1079     start = profile_start(hotspot_profile);
1080     linear_f32_unchecked_parallel(layer.mlp_c_proj, mlp_fc, mlp_out, options, ↵
↵ worker_pool);
1081     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::mlp_projection_ms, ↵
↵ start);
1082
1083     start = profile_start(hotspot_profile);
1084     add_inplace(hidden, mlp_out);
1085     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::residual_add_ms, start↵
↵ );
1086 }
1087
1088 void compute_logits(const Gpt2ReferenceModel& model,
1089                    std::span<const float> hidden,
1090                    std::span<float> logits,
1091                    const Gpt2ExecutionOptions& options,
1092                    detail::Gpt2ParallelWorkerPool* worker_pool) {
1093     if (logits.size() != checked_size(model.config.vocab_size, "vocab_size")) {
1094         throw std::runtime_error("GPT-2 logits size mismatch");
1095     }
1096     const std::span<const float> weight =

```

```

1097     model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
↵ token_embedding)
1098         : std::span<const float>(model.↵
↵ lm_head_weight);
1099     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
↵ hidden_size");
1100     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
↵ vocab_size");
1101     const float* weight_data = weight.data();
1102     const float* hidden_data = hidden.data();
1103     const std::function<void(std::size_t, std::size_t)> rows_fn =
1104         [hidden_size, weight_data, hidden_data, logits_data = logits.data()](
1105             std::size_t begin,
1106             std::size_t end) {
1107         for (std::size_t token = begin; token < end; ++token) {
1108             const float* row = weight_data + token * hidden_size;
1109             logits_data[token] = dot_f32_unchecked(row, hidden_data, ↵
↵ hidden_size);
1110         }
1111     };
1112     if (should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)↵
↵ {
1113         worker_pool->parallel_for_rows(0,
1114             vocab_size,
1115             parallel_row_grain(vocab_size, *↵
↵ worker_pool),
1116             rows_fn);
1117     } else {
1118         rows_fn(0, vocab_size);
1119     }
1120 }
1121
1122 void compute_logits(const Gpt2ReferenceModel& model,
1123     std::span<const float> hidden,
1124     std::span<float> logits) {
1125     Gpt2ExecutionOptions options;
1126     options.worker_count = 1;
1127     compute_logits(model, hidden, logits, options, nullptr);
1128 }
1129
1130 struct Gpt2TokenScore {
1131     std::int32_t token = 0;
1132     float value = -std::numeric_limits<float>::infinity();
1133 };
1134
1135 [[nodiscard]] Gpt2TokenScore compute_predicted_token_range(const float* ↵
↵ weight_data,
1136     const float* ↵
↵ hidden_data,
1137     std::size_t ↵
↵ hidden_size,
1138     std::size_t begin,

```

```

1139                                     std::size_t end) {
1140     Gpt2TokenScore best;
1141     best.token = static_cast<std::int32_t>(begin);
1142     for (std::size_t token = begin; token < end; ++token) {
1143         const float* row = weight_data + token * hidden_size;
1144         const float sum = dot_f32_unchecked(row, hidden_data, hidden_size);
1145         if (token == begin || sum > best.value) {
1146             best.value = sum;
1147             best.token = static_cast<std::int32_t>(token);
1148         }
1149     }
1150     return best;
1151 }
1152
1153 std::int32_t compute_predicted_token(const Gpt2ReferenceModel& model,
1154                                     std::span<const float> hidden,
1155                                     const Gpt2ExecutionOptions& options,
1156                                     detail::Gpt2ParallelWorkerPool* ↵
1157                                     ↵ worker_pool) {
1158     const std::span<const float> weight =
1159         model.tie_lm_head_to_embedding ? std::span<const float>(model.↵
1160         ↵ token_embedding)
1161         : std::span<const float>(model.↵
1162         ↵ lm_head_weight);
1163     const std::size_t hidden_size = checked_size(model.config.hidden_size, "↵
1164     ↵ hidden_size");
1165     const std::size_t vocab_size = checked_size(model.config.vocab_size, "↵
1166     ↵ vocab_size");
1167     const float* weight_data = weight.data();
1168     const float* hidden_data = hidden.data();
1169     if (!should_parallelize_rows(options, vocab_size, hidden_size, worker_pool)↵
1170     ↵ ) {
1171         return compute_predicted_token_range(weight_data, hidden_data, ↵
1172         ↵ hidden_size, 0, vocab_size)
1173             .token;
1174     }
1175
1176     const std::size_t worker_count = checked_size(worker_pool->worker_count(), ↵
1177     ↵ "worker_count");
1178     const std::size_t chunk_count = std::min(worker_count, vocab_size);
1179     std::vector<Gpt2TokenScore> partials(chunk_count);
1180     const std::function<void(std::size_t, std::size_t)> rows_fn =
1181         [chunk_count, hidden_size, vocab_size, weight_data, hidden_data, &↵
1182         ↵ partials](
1183         std::size_t begin,
1184         std::size_t end) {
1185         for (std::size_t chunk = begin; chunk < end; ++chunk) {
1186             const std::size_t token_begin = vocab_size * chunk / ↵
1187             ↵ chunk_count;
1188             const std::size_t token_end = vocab_size * (chunk + 1) / ↵
1189             ↵ chunk_count;
1190             partials[chunk] = compute_predicted_token_range(

```

```

1180         weight_data,
1181         hidden_data,
1182         hidden_size,
1183         token_begin,
1184         token_end);
1185     }
1186 };
1187 worker_pool->parallel_for_rows(0, chunk_count, 1, rows_fn);
1188
1189 Gpt2TokenScore best = partials.front();
1190 for (std::size_t index = 1; index < partials.size(); ++index) {
1191     if (partials[index].value > best.value) {
1192         best = partials[index];
1193     }
1194 }
1195 return best.token;
1196 }
1197
1198 Gpt2ForwardResult run_gpt2_cached_step_unchecked(const Gpt2ReferenceModel& ←
1199     ↪ model,
1200     Gpt2KvCache& cache,
1201     Gpt2ForwardWorkspace& ↪
1202     ↪ workspace,
1203     std::int32_t token_id,
1204     bool need_logits,
1205     Gpt2HotspotProfile* ↪
1206     ↪ hotspot_profile,
1207     const Gpt2ExecutionOptions& ↪
1208     ↪ options,
1209     detail::Gpt2ParallelWorkerPool↪
1210     ↪ * worker_pool) {
1211     const Gpt2ProfileClock::time_point step_start = profile_start(↪
1212     ↪ hotspot_profile);
1213     if (hotspot_profile != nullptr) {
1214         ++hotspot_profile->decoder_steps;
1215     }
1216     if (token_id < 0 || token_id >= model.config.vocab_size) {
1217         throw std::runtime_error("GPT-2 token id out of range");
1218     }
1219     const std::int32_t model_position = cache.filled_tokens();
1220     if (model_position >= model.config.max_positions) {
1221         throw std::runtime_error("GPT-2 cached forward would exceed ↪
1222     ↪ max_positions");
1223     }
1224
1225     std::span<float> hidden = workspace.hidden();
1226     const std::span<const float> token =
1227         row_span(model.token_embedding, token_id, model.config.hidden_size);
1228     const std::span<const float> position_embedding =
1229         row_span(model.position_embedding, model_position, model.config.↪
1230     ↪ hidden_size);
1231     Gpt2ProfileClock::time_point start = profile_start(hotspot_profile);

```



```

1224     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
1225         hidden[checked_size(dim, "dim")] =
1226             token[checked_size(dim, "dim")] + position_embedding[checked_size(↵
↵ dim, "dim")]);
1227     }
1228     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::embedding_ms, start);
1229
1230     for (std::int32_t layer_id = 0;
1231          layer_id < static_cast<std::int32_t>(model.layers.size()));
1232          ++layer_id) {
1233         forward_gpt2_layer_cached(model.config,
1234                                   model.layers[checked_size(layer_id, "layer_id"↵
↵ ")],
1235                                   layer_id,
1236                                   model_position,
1237                                   cache,
1238                                   workspace,
1239                                   hidden,
1240                                   hotspot_profile,
1241                                   options,
1242                                   worker_pool);
1243     }
1244
1245     std::span<float> normed = workspace.normed();
1246     start = profile_start(hotspot_profile);
1247     layer_norm(hidden,
1248                model.final_ln_weight,
1249                model.final_ln_bias,
1250                model.config.layer_norm_epsilon,
1251                normed);
1252     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::final_norm_ms, start);
1253
1254     Gpt2ForwardResult result;
1255     if (need_logits) {
1256         std::span<float> logits = workspace.logits();
1257         start = profile_start(hotspot_profile);
1258         compute_logits(model, normed, logits, options, worker_pool);
1259         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↵
↵ ;
1260         start = profile_start(hotspot_profile);
1261         result.logits.assign(logits.begin(), logits.end());
1262         result.predicted_token = argmax(result.logits);
1263         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::logits_result_ms, ↵
↵ start);
1264     } else {
1265         start = profile_start(hotspot_profile);
1266         result.predicted_token = compute_predicted_token(model, normed, options↵
↵ , worker_pool);
1267         add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::lm_head_ms, start)↵
↵ ;
1268     }
1269     add_profile_ms(hotspot_profile, &Gpt2HotspotProfile::total_step_ms, ↵

```

```

    ↪ step_start);
1270     return result;
1271 }
1272
1273 std::vector<std::string> gpt2_bytes_to_unicode() {
1274     std::vector<std::int32_t> bytes;
1275     for (std::int32_t value = LCQI_GPT2_ASCII_EXCLAMATION;
1276         value <= LCQI_GPT2_ASCII_TILDE;
1277         ++value) {
1278         bytes.push_back(value);
1279     }
1280     for (std::int32_t value = LCQI_GPT2_LATIN_INVERTED_EXCLAMATION;
1281         value <= LCQI_GPT2_LATIN_NOT_SIGN;
1282         ++value) {
1283         bytes.push_back(value);
1284     }
1285     for (std::int32_t value = LCQI_GPT2_LATIN_REGISTERED;
1286         value <= LCQI_GPT2_LATIN_Y_DIAERESIS;
1287         ++value) {
1288         bytes.push_back(value);
1289     }
1290
1291     std::unordered_set<std::int32_t> byte_set(bytes.begin(), bytes.end());
1292     std::vector<std::int32_t> chars = bytes;
1293     std::int32_t private_offset = 0;
1294     for (std::int32_t byte = 0; byte < LCQI_GPT2_BYTE_COUNT; ++byte) {
1295         if (!byte_set.contains(byte)) {
1296             bytes.push_back(byte);
1297             chars.push_back(LCQI_GPT2_UNICODE_PRIVATE_BASE + private_offset);
1298             ++private_offset;
1299         }
1300     }
1301
1302     std::vector<std::string> mapping(LCQI_GPT2_BYTE_COUNT);
1303     for (std::size_t index = 0; index < bytes.size(); ++index) {
1304         mapping[checked_size(bytes[index], "byte unicode index")] =
1305             encode_codepoint(static_cast<std::uint32_t>(chars[index]));
1306     }
1307     return mapping;
1308 }
1309
1310 std::unordered_map<std::string, unsigned char> gpt2_unicode_to_bytes(
1311     const std::vector<std::string>& byte_encoder) {
1312     std::unordered_map<std::string, unsigned char> result;
1313     for (std::size_t byte = 0; byte < byte_encoder.size(); ++byte) {
1314         result.emplace(byte_encoder[byte], static_cast<unsigned char>(byte));
1315     }
1316     return result;
1317 }
1318
1319 std::string bytes_to_bpe_alphabet(std::string_view text) {
1320     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↪

```

```

    ↪ ();
1321     std::string result;
1322     for (const unsigned char byte : text) {
1323         result += BYTE_ENCODER[byte];
1324     }
1325     return result;
1326 }
1327
1328 std::vector<std::string> utf8_symbols(std::string_view text) {
1329     std::vector<std::string> symbols;
1330     std::size_t offset = 0;
1331     while (offset < text.size()) {
1332         const unsigned char lead = static_cast<unsigned char>(text[offset]);
1333         std::size_t width = 1;
1334         if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1335             width = 2;
1336         } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1337             width = 3;
1338         } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1339             width = 4;
1340         }
1341         if (offset + width > text.size()) {
1342             throw std::runtime_error("invalid UTF-8/BPE symbol boundary");
1343         }
1344         symbols.emplace_back(text.substr(offset, width));
1345         offset += width;
1346     }
1347     return symbols;
1348 }
1349
1350 std::string pair_key(std::string_view left, std::string_view right) {
1351     std::string key(left);
1352     key.push_back('\n');
1353     key += right;
1354     return key;
1355 }
1356
1357 std::vector<std::string> bpe_apply(const Gpt2Tokenizer& tokenizer, std::↪
    ↪ string_view token) {
1358     std::vector<std::string> word = utf8_symbols(token);
1359     if (word.size() <= 1) {
1360         return word;
1361     }
1362
1363     while (true) {
1364         std::optional<std::int32_t> best_rank;
1365         std::string best_left;
1366         std::string best_right;
1367         for (std::size_t index = 0; index + 1 < word.size(); ++index) {
1368             const auto rank = tokenizer.bpe_ranks.find(pair_key(word[index], ↪
    ↪ word[index + 1]));
1369             if (rank != tokenizer.bpe_ranks.end() &&

```

```

1370         (!best_rank.has_value() || rank->second < *best_rank)) {
1371             best_rank = rank->second;
1372             best_left = word[index];
1373             best_right = word[index + 1];
1374         }
1375     }
1376     if (!best_rank.has_value()) {
1377         return word;
1378     }
1379
1380     std::vector<std::string> merged;
1381     std::size_t index = 0;
1382     while (index < word.size()) {
1383         if (index + 1 < word.size() && word[index] == best_left &&
1384             word[index + 1] == best_right) {
1385             merged.push_back(best_left + best_right);
1386             index += LCQI_GPT2_PAIR_TOKEN_COUNT;
1387         } else {
1388             merged.push_back(word[index]);
1389             ++index;
1390         }
1391     }
1392     word = std::move(merged);
1393     if (word.size() == 1) {
1394         return word;
1395     }
1396 }
1397 }
1398
1399 bool is_ascii_letter(unsigned char value) {
1400     return (value >= 'A' && value <= 'Z') || (value >= 'a' && value <= 'z');
1401 }
1402
1403 bool is_ascii_digit(unsigned char value) {
1404     return value >= '0' && value <= '9';
1405 }
1406
1407 bool is_ascii_space(unsigned char value) {
1408     return value == ' ' || value == '\t' || value == '\n' ||
1409         value == '\r' || value == '\f' || value == '\v';
1410 }
1411
1412 bool is_gpt2_punctuation(unsigned char value) {
1413     return !is_ascii_letter(value) && !is_ascii_digit(value) &&
1414         !is_ascii_space(value);
1415 }
1416
1417 std::size_t consume_utf8_codepoint(std::string_view text, std::size_t offset) {
1418     const unsigned char lead = static_cast<unsigned char>(text[offset]);
1419     std::size_t width = 1;
1420     if ((lead & 0xE0U) == LCQI_GPT2_UTF8_TWO_BYTE_HEAD) {
1421         width = 2;

```

```

1422     } else if ((lead & 0xF0U) == LCQI_GPT2_UTF8_THREE_BYTE_HEAD) {
1423         width = 3;
1424     } else if ((lead & 0xF8U) == LCQI_GPT2_UTF8_FOUR_BYTE_HEAD) {
1425         width = 4;
1426     }
1427     if (offset + width > text.size()) {
1428         throw std::runtime_error("invalid UTF-8 text for GPT-2 tokenizer");
1429     }
1430     return offset + width;
1431 }
1432
1433 bool starts_with_contraction(std::string_view text,
1434                             std::size_t offset,
1435                             std::string_view contraction) {
1436     if (offset + contraction.size() > text.size()) {
1437         return false;
1438     }
1439     for (std::size_t index = 0; index < contraction.size(); ++index) {
1440         char left = text[offset + index];
1441         const char right = contraction[index];
1442         if (left >= 'A' && left <= 'Z') {
1443             left = static_cast<char>(left - 'A' + 'a');
1444         }
1445         if (left != right) {
1446             return false;
1447         }
1448     }
1449     return true;
1450 }
1451
1452 std::vector<std::string> gpt2_pretokenize(std::string_view text) {
1453     std::vector<std::string> pieces;
1454     std::size_t offset = 0;
1455     const std::array<std::string_view, 7> contractions{
1456         "'s", "'t", "'re", "'ve", "'m", "'ll", "'d",
1457     };
1458     while (offset < text.size()) {
1459         bool emitted_contraction = false;
1460         for (const std::string_view contraction : contractions) {
1461             if (starts_with_contraction(text, offset, contraction)) {
1462                 pieces.emplace_back(text.substr(offset, contraction.size()));
1463                 offset += contraction.size();
1464                 emitted_contraction = true;
1465                 break;
1466             }
1467         }
1468         if (emitted_contraction) {
1469             continue;
1470         }
1471
1472         const std::size_t begin = offset;
1473         bool has_prefix_space = false;

```

```

1474     if (text[offset] == ' ' && offset + 1 < text.size() &&
1475         !is_ascii_space(static_cast<unsigned char>(text[offset + 1]))) {
1476         has_prefix_space = true;
1477         ++offset;
1478     }
1479
1480     if (offset >= text.size()) {
1481         pieces.emplace_back(text.substr(begin, offset - begin));
1482         continue;
1483     }
1484
1485     const unsigned char ch = static_cast<unsigned char>(text[offset]);
1486     if (is_ascii_letter(ch)) {
1487         while (offset < text.size() &&
1488             is_ascii_letter(static_cast<unsigned char>(text[offset]))) {
1489             ++offset;
1490         }
1491     } else if (is_ascii_digit(ch)) {
1492         while (offset < text.size() &&
1493             is_ascii_digit(static_cast<unsigned char>(text[offset]))) {
1494             ++offset;
1495         }
1496     } else if (is_gpt2_punctuation(ch) && ch != LCQI_GPT2_ASCII_APOSTROPHE) {
1497         while (offset < text.size() &&
1498             is_gpt2_punctuation(static_cast<unsigned char>(text[offset]))) {
1499             ++offset;
1500         }
1501     } else if (is_ascii_space(ch)) {
1502         while (offset < text.size() &&
1503             is_ascii_space(static_cast<unsigned char>(text[offset]))) {
1504             ++offset;
1505         }
1506     } else {
1507         offset = consume_utf8_codepoint(text, offset);
1508     }
1509
1510     if (has_prefix_space || offset > begin) {
1511         pieces.emplace_back(text.substr(begin, offset - begin));
1512     } else {
1513         ++offset;
1514     }
1515 }
1516 return pieces;
1517 }
1518
1519 std::int32_t parse_i32_field(JsonCursor& cursor) {
1520     const std::int64_t value = cursor.parse_i64();
1521     if (value < std::numeric_limits<std::int32_t>::min() ||

```

```

1523     value > std::numeric_limits<std::int32_t>::max()) {
1524         throw std::runtime_error("JSON int32 field out of range");
1525     }
1526     return static_cast<std::int32_t>(value);
1527 }
1528
1529 std::unordered_map<std::string, std::int32_t> parse_vocab_json(std::string_view ↵
    ↵ text) {
1530     JsonCursor cursor(text);
1531     std::unordered_map<std::string, std::int32_t> vocab;
1532     cursor.expect('{');
1533     if (cursor.consume('}')) {
1534         return vocab;
1535     }
1536     while (true) {
1537         const std::string key = cursor.parse_string();
1538         cursor.expect(':');
1539         const std::int32_t id = parse_i32_field(cursor);
1540         vocab.emplace(key, id);
1541         if (cursor.consume('}')) {
1542             break;
1543         }
1544         cursor.expect(',');
1545     }
1546     if (!cursor.eof_after_ws()) {
1547         throw std::runtime_error("trailing data after vocab JSON");
1548     }
1549     return vocab;
1550 }
1551
1552 std::vector<std::string> build_id_to_token(
1553     const std::unordered_map<std::string, std::int32_t>& vocab) {
1554     std::int32_t max_id = -1;
1555     for (const auto& [token, id] : vocab) {
1556         if (id < 0) {
1557             throw std::runtime_error("GPT-2 vocab id cannot be negative");
1558         }
1559         max_id = std::max(max_id, id);
1560     }
1561     std::vector<std::string> result(check_size(max_id + 1, "max vocab id"));
1562     for (const auto& [token, id] : vocab) {
1563         std::string& slot = result[checked_size(id, "vocab id")];
1564         if (!slot.empty()) {
1565             throw std::runtime_error("duplicate GPT-2 vocab id");
1566         }
1567         slot = token;
1568     }
1569     return result;
1570 }
1571
1572 std::unordered_map<std::string, std::int32_t> parse_merges(std::string_view ↵
    ↵ text) {

```

```

1573     std::unordered_map<std::string, std::int32_t> ranks;
1574     std::istringstream input{std::string(text)};
1575     std::string line;
1576     std::int32_t rank = 0;
1577     while (std::getline(input, line)) {
1578         if (line.empty() || line[0] == '#') {
1579             continue;
1580         }
1581         std::istringstream line_stream(line);
1582         std::string left;
1583         std::string right;
1584         if (!(line_stream >> left >> right)) {
1585             throw std::runtime_error("invalid GPT-2 merges line");
1586         }
1587         ranks.emplace(pair_key(left, right), rank);
1588         ++rank;
1589     }
1590     return ranks;
1591 }
1592
1593 void transpose_hf_conv1d(std::vector<float>& values,
1594                         std::int32_t input_size,
1595                         std::int32_t output_size) {
1596     if (values.size() != matrix_size(input_size, output_size)) {
1597         throw std::runtime_error("GPT-2 Conv1D tensor size mismatch");
1598     }
1599     std::vector<float> transposed(matrix_size(output_size, input_size), 0.0F);
1600     for (std::int32_t input = 0; input < input_size; ++input) {
1601         for (std::int32_t output = 0; output < output_size; ++output) {
1602             transposed[matrix_size(output, input_size) + checked_size(input, "↵
↵ input")] =
1603                 values[matrix_size(input, output_size) + checked_size(output, "↵
↵ output")];
1604         }
1605     }
1606     values = std::move(transposed);
1607 }
1608
1609 std::vector<float> require_tensor(const SafeTensorFile& file,
1610                                  std::string_view name,
1611                                  std::span<const std::int64_t> expected_shape)↵
↵ {
1612     const SafeTensorEntry* entry = file.manifest.find_tensor(name);
1613     if (entry == nullptr) {
1614         throw std::runtime_error("missing GPT-2 tensor: " + std::string(name));
1615     }
1616     if (entry->shape.size() != expected_shape.size()) {
1617         throw std::runtime_error("GPT-2 tensor rank mismatch: " + std::string(↵
↵ name));
1618     }
1619     for (std::size_t index = 0; index < expected_shape.size(); ++index) {
1620         if (entry->shape[index] != expected_shape[index]) {

```



```

1621         throw std::runtime_error("GPT-2 tensor shape mismatch: " + std::string(
1622             ↪ string(name)));
1623     }
1624     return read_safetensor_f32_tensor(file, name);
1625 }
1626
1627 std::vector<float> require_first_tensor(const SafeTensorFile& file,
1628                                         std::span<const std::string_view> names
1629                                         ↪ ,
1630                                         std::span<const std::int64_t> ↪
1631                                         ↪ expected_shape) {
1632     for (const std::string_view name : names) {
1633         if (file.manifest.find_tensor(name) != nullptr) {
1634             return require_tensor(file, name, expected_shape);
1635         }
1636     }
1637     throw std::runtime_error("missing GPT-2 tensor: " + std::string(names.front(
1638         ↪ )))
1639     ↪ ();
1640 }
1641
1642 Gpt2LinearF32 make_linear(std::int32_t input_size,
1643                           std::int32_t output_size,
1644                           std::vector<float> weights,
1645                           std::vector<float> bias = {}) {
1646     Gpt2LinearF32 linear;
1647     linear.input_size = input_size;
1648     linear.output_size = output_size;
1649     linear.weights = std::move(weights);
1650     linear.bias = std::move(bias);
1651     validate_linear(linear, "make GPT-2 linear");
1652     return linear;
1653 }
1654
1655 std::vector<float> zeros(std::int32_t count) {
1656     return std::vector<float>(checked_size(count, "zero count"), 0.0F);
1657 }
1658
1659 std::vector<float> ones(std::int32_t count) {
1660     return std::vector<float>(checked_size(count, "one count"), 1.0F);
1661 }
1662
1663 std::string tensor_name(std::int32_t layer, std::string_view suffix) {
1664     return "transformer.h." + std::to_string(layer) + "." + std::string(suffix)
1665     ↪ ;
1666 }
1667
1668 std::string bare_tensor_name(std::int32_t layer, std::string_view suffix) {
1669     return "h." + std::to_string(layer) + "." + std::string(suffix);
1670 }
1671
1672 std::array<std::string_view, 2> embedding_names(std::string_view bare) {

```

[illegible]

```

1716 Gpt2Tokenizer tokenizer;
1717 tokenizer.bos_token_id = bos_token_id;
1718 tokenizer.eos_token_id = eos_token_id;
1719 tokenizer.vocab = parse_vocab_json(read_text_file(vocab_json));
1720 tokenizer.id_to_token = build_id_to_token(tokenizer.vocab);
1721 tokenizer.bpe_ranks = parse_merges(read_text_file(merges_txt));
1722 if (tokenizer.vocab.empty() || tokenizer.bpe_ranks.empty()) {
1723     throw std::runtime_error("GPT-2 tokenizer assets are empty");
1724 }
1725 return tokenizer;
1726 }
1727
1728 std::vector<std::int32_t> gpt2_encode(const Gpt2Tokenizer& tokenizer,
1729                                     std::string_view text) {
1730     std::vector<std::int32_t> ids;
1731     for (const std::string& piece : gpt2_pretokenize(text)) {
1732         const std::string byte_piece = bytes_to_bpe_alphabet(piece);
1733         const std::vector<std::string> bpe_tokens = bpe_apply(tokenizer, ↵
↵ byte_piece);
1734         for (const std::string& token : bpe_tokens) {
1735             const auto found = tokenizer.vocab.find(token);
1736             if (found == tokenizer.vocab.end()) {
1737                 throw std::runtime_error("GPT-2 BPE token is missing from vocab↵
↵ ");
1738             }
1739             ids.push_back(found->second);
1740         }
1741     }
1742     return ids;
1743 }
1744
1745 std::string gpt2_decode(const Gpt2Tokenizer& tokenizer,
1746                        std::span<const std::int32_t> token_ids) {
1747     static const std::vector<std::string> BYTE_ENCODER = gpt2_bytes_to_unicode↵
↵ ();
1748     static const std::unordered_map<std::string, unsigned char> BYTE_DECODER =
1749         gpt2_unicode_to_bytes(BYTE_ENCODER);
1750
1751     std::string token_text;
1752     for (const std::int32_t token_id : token_ids) {
1753         if (token_id < 0 || token_id >= static_cast<std::int32_t>(tokenizer.↵
↵ id_to_token.size())) {
1754             throw std::runtime_error("GPT-2 token id out of range");
1755         }
1756         token_text += tokenizer.id_to_token[checked_size(token_id, "token id")↵
↵ ];
1757     }
1758
1759     std::string result;
1760     const std::vector<std::string> symbols = utf8_symbols(token_text);
1761     for (const std::string& symbol : symbols) {
1762         const auto found = BYTE_DECODER.find(symbol);

```

```

1763     if (found == BYTE_DECODER.end()) {
1764         throw std::runtime_error("GPT-2 token cannot be decoded to byte");
1765     }
1766     result.push_back(static_cast<char>(found->second));
1767 }
1768 return result;
1769 }
1770
1771 Gpt2Config load_gpt2_config(const std::filesystem::path& config_json) {
1772     const std::string config_text = read_text_file(config_json);
1773     JsonCursor cursor(config_text);
1774     Gpt2Config config;
1775     config.bos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1776     config.eos_token_id = LCQI_GPT2_DEFAULT_BOS_EOS;
1777
1778     cursor.expect('{');
1779     if (cursor.consume('}')) {
1780         throw std::runtime_error("GPT-2 config is empty");
1781     }
1782     while (true) {
1783         const std::string key = cursor.parse_string();
1784         cursor.expect(':');
1785         if (key == "n_embd") {
1786             config.hidden_size = parse_i32_field(cursor);
1787         } else if (key == "n_head") {
1788             config.head_count = parse_i32_field(cursor);
1789         } else if (key == "n_layer") {
1790             config.layer_count = parse_i32_field(cursor);
1791         } else if (key == "n_positions" || key == "n_ctx") {
1792             const std::int32_t value = parse_i32_field(cursor);
1793             config.max_positions = std::max(config.max_positions, value);
1794         } else if (key == "vocab_size") {
1795             config.vocab_size = parse_i32_field(cursor);
1796         } else if (key == "n_inner") {
1797             config.intermediate_size = parse_i32_field(cursor);
1798         } else if (key == "layer_norm_epsilon") {
1799             config.layer_norm_epsilon = static_cast<float>(cursor.parse_number↵
↵ ();
1800         } else if (key == "activation_function") {
1801             config.activation_function = cursor.parse_string();
1802         } else if (key == "bos_token_id") {
1803             config.bos_token_id = parse_i32_field(cursor);
1804         } else if (key == "eos_token_id") {
1805             config.eos_token_id = parse_i32_field(cursor);
1806         } else {
1807             cursor.skip_value();
1808         }
1809         if (cursor.consume('}')) {
1810             break;
1811         }
1812         cursor.expect(',');
1813     }

```

```

1814     if (!cursor.eof_after_ws()) {
1815         throw std::runtime_error("trailing data after GPT-2 config JSON");
1816     }
1817     if (config.intermediate_size == 0 && config.hidden_size > 0) {
1818         config.intermediate_size = config.hidden_size * 4;
1819     }
1820     validate_config(config);
1821     return config;
1822 }
1823
1824 Gpt2ReferenceModel load_gpt2_reference_model(const std::filesystem::path& ↵
    ↵ config_json,
1825                                             const std::filesystem::path& ↵
    ↵ safetensors_path) {
1826     Gpt2ReferenceModel model;
1827     model.config = load_gpt2_config(config_json);
1828     const SafeTensorFile file = load_safetensors_file(safetensors_path);
1829     const Gpt2Config& config = model.config;
1830     const std::array<std::int64_t, 2> wte_shape{config.vocab_size, config.↵
    ↵ hidden_size};
1831     const std::array<std::int64_t, 2> wpe_shape{config.max_positions, config.↵
    ↵ hidden_size};
1832     const std::array<std::int64_t, 1> hidden_shape{config.hidden_size};
1833
1834     const std::array<std::string_view, 2> wte_names = embedding_names("wte.↵
    ↵ weight");
1835     const std::array<std::string_view, 2> wpe_names = embedding_names("wpe.↵
    ↵ weight");
1836     model.token_embedding = require_first_tensor(file, wte_names, wte_shape);
1837     model.position_embedding = require_first_tensor(file, wpe_names, wpe_shape)↵
    ↵ ;
1838     model.layers.resize(check_size(config.layer_count, "layer_count"));
1839
1840     for (std::int32_t layer_id = 0; layer_id < config.layer_count; ++layer_id) ↵
    ↵ {
1841         Gpt2LayerWeightsF32& layer = model.layers[check_size(layer_id, "↵
    ↵ layer_id")];
1842         layer.ln_1_weight = require_layer_tensor(file, layer_id, "ln_1.weight",↵
    ↵ hidden_shape);
1843         layer.ln_1_bias = require_layer_tensor(file, layer_id, "ln_1.bias", ↵
    ↵ hidden_shape);
1844         layer.ln_2_weight = require_layer_tensor(file, layer_id, "ln_2.weight",↵
    ↵ hidden_shape);
1845         layer.ln_2_bias = require_layer_tensor(file, layer_id, "ln_2.bias", ↵
    ↵ hidden_shape);
1846
1847         const std::array<std::int64_t, 2> c_attn_shape{
1848             config.hidden_size,
1849             config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1850         std::vector<float> c_attn_weight =
1851             require_layer_tensor(file, layer_id, "attn.c_attn.weight", ↵
    ↵ c_attn_shape);

```

```

1852     transpose_hf_conv1d(c_attn_weight,
1853                        config.hidden_size,
1854                        config.hidden_size * ←
↪ LCQI_GPT2_ATTENTION_PROJECTIONS);
1855     const std::array<std::int64_t, 1> c_attn_bias_shape{
1856         config.hidden_size * LCQI_GPT2_ATTENTION_PROJECTIONS};
1857     layer.c_attn = make_linear(config.hidden_size,
1858                              config.hidden_size * ←
↪ LCQI_GPT2_ATTENTION_PROJECTIONS,
1859                              std::move(c_attn_weight),
1860                              require_layer_tensor(file,
1861                                                    layer_id,
1862                                                    "attn.c_attn.bias",
1863                                                    c_attn_bias_shape));
1864
1865     const std::array<std::int64_t, 2> c_proj_shape{config.hidden_size, ←
↪ config.hidden_size};
1866     std::vector<float> c_proj_weight =
1867         require_layer_tensor(file, layer_id, "attn.c_proj.weight", ←
↪ c_proj_shape);
1868     transpose_hf_conv1d(c_proj_weight, config.hidden_size, config.↪
↪ hidden_size);
1869     layer.c_proj = make_linear(config.hidden_size,
1870                              config.hidden_size,
1871                              std::move(c_proj_weight),
1872                              require_layer_tensor(file,
1873                                                    layer_id,
1874                                                    "attn.c_proj.bias",
1875                                                    hidden_shape));
1876
1877     const std::array<std::int64_t, 2> c_fc_shape{
1878         config.hidden_size,
1879         config.intermediate_size};
1880     std::vector<float> c_fc_weight =
1881         require_layer_tensor(file, layer_id, "mlp.c_fc.weight", c_fc_shape)↪
↪ ;
1882     transpose_hf_conv1d(c_fc_weight, config.hidden_size, config.↪
↪ intermediate_size);
1883     const std::array<std::int64_t, 1> intermediate_shape{config.↪
↪ intermediate_size};
1884     layer.c_fc = make_linear(config.hidden_size,
1885                              config.intermediate_size,
1886                              std::move(c_fc_weight),
1887                              require_layer_tensor(file,
1888                                                    layer_id,
1889                                                    "mlp.c_fc.bias",
1890                                                    intermediate_shape));
1891
1892     const std::array<std::int64_t, 2> mlp_proj_shape{
1893         config.intermediate_size,
1894         config.hidden_size};
1895     std::vector<float> mlp_proj_weight =

```

```

1896         require_layer_tensor(file, layer_id, "mlp.c_proj.weight", ←
↪ mlp_proj_shape);
1897         transpose_hf_conv1d(mlp_proj_weight, config.intermediate_size, config.↪
↪ hidden_size);
1898         layer.mlp_c_proj = make_linear(config.intermediate_size,
1899                                     config.hidden_size,
1900                                     std::move(mlp_proj_weight),
1901                                     require_layer_tensor(file,
1902                                                         layer_id,
1903                                                         "mlp.c_proj.bias",
1904                                                         hidden_shape));
1905     }
1906
1907     const std::array<std::string_view, 2> ln_weight_names = embedding_names("↪
↪ ln_f.weight");
1908     const std::array<std::string_view, 2> ln_bias_names = embedding_names("ln_f↪
↪ .bias");
1909     model.final_ln_weight = require_first_tensor(file, ln_weight_names, ←
↪ hidden_shape);
1910     model.final_ln_bias = require_first_tensor(file, ln_bias_names, ←
↪ hidden_shape);
1911     if (file.manifest.find_tensor("lm_head.weight") != nullptr) {
1912         model.tie_lm_head_to_embedding = false;
1913         model.lm_head_weight = require_tensor(file, "lm_head.weight", wte_shape↪
↪ );
1914     } else {
1915         model.tie_lm_head_to_embedding = true;
1916     }
1917     validate_model(model);
1918     return model;
1919 }
1920
1921 Gpt2ReferenceModel load_gpt2_from_directory(const std::filesystem::path& ←
↪ model_directory) {
1922     return load_gpt2_reference_model(model_directory / "config.json",
1923                                     find_gpt2_weight_file(model_directory));
1924 }
1925
1926 Gpt2ForwardWorkspace::Gpt2ForwardWorkspace(const Gpt2Config& config) : config_(↪
↪ config) {
1927     this->reset_for_config(config);
1928 }
1929
1930 void Gpt2ForwardWorkspace::reset_for_config(const Gpt2Config& config) {
1931     validate_config(config);
1932     this->config_ = config;
1933     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "↪
↪ hidden_size");
1934     const std::size_t intermediate_size =
1935         checked_size(this->config_.intermediate_size, "intermediate_size");
1936     this->hidden_.assign(hidden_size, 0.0F);
1937     this->normed_.assign(hidden_size, 0.0F);

```

```

1938     this->query_.assign(hidden_size, 0.0F);
1939     this->key_.assign(hidden_size, 0.0F);
1940     this->value_.assign(hidden_size, 0.0F);
1941     this->attention_.assign(hidden_size, 0.0F);
1942     this->projected_.assign(hidden_size, 0.0F);
1943     this->qkv_packed_.assign(
1944         matrix_size(LCQI_GPT2_ATTENTION_PROJECTIONS, this->config_.hidden_size)↵
        ↵ ,
1945         0.0F);
1946     this->mlp_fc_.assign(intermediate_size, 0.0F);
1947     this->mlp_out_.assign(hidden_size, 0.0F);
1948     this->logits_.assign(checked_size(this->config_.vocab_size, "vocab_size"), ↵
        ↵ 0.0F);
1949     this->scores_.assign(checked_size(this->config_.max_positions, "↵
        ↵ max_positions"), 0.0F);
1950 }
1951
1952 std::span<float> Gpt2ForwardWorkspace::hidden() noexcept {
1953     return this->hidden_;
1954 }
1955
1956 std::span<float> Gpt2ForwardWorkspace::normed() noexcept {
1957     return this->normed_;
1958 }
1959
1960 std::span<float> Gpt2ForwardWorkspace::query() noexcept {
1961     return this->query_;
1962 }
1963
1964 std::span<float> Gpt2ForwardWorkspace::key() noexcept {
1965     return this->key_;
1966 }
1967
1968 std::span<float> Gpt2ForwardWorkspace::value() noexcept {
1969     return this->value_;
1970 }
1971
1972 std::span<float> Gpt2ForwardWorkspace::attention() noexcept {
1973     return this->attention_;
1974 }
1975
1976 std::span<float> Gpt2ForwardWorkspace::projected() noexcept {
1977     return this->projected_;
1978 }
1979
1980 std::span<float> Gpt2ForwardWorkspace::qkv_packed() noexcept {
1981     return this->qkv_packed_;
1982 }
1983
1984 std::span<float> Gpt2ForwardWorkspace::mlp_fc() noexcept {
1985     return this->mlp_fc_;
1986 }

```



```

1987
1988 std::span<float> Gpt2ForwardWorkspace::mlp_out() noexcept {
1989     return this->mlp_out_;
1990 }
1991
1992 std::span<float> Gpt2ForwardWorkspace::logits() noexcept {
1993     return this->logits_;
1994 }
1995
1996 std::span<float> Gpt2ForwardWorkspace::scores_prefix(std::int32_t count) {
1997     if (count < 0 || count > this->config_.max_positions) {
1998         throw std::runtime_error("GPT-2 attention scores count out of range");
1999     }
2000     return std::span<float>(this->scores_.data(), checked_size(count, "scores ←
    ↪ count"));
2001 }
2002
2003 Gpt2KvCache::Gpt2KvCache(const Gpt2Config& config) : config_(config) {
2004     validate_config(this->config_);
2005     const std::size_t element_count =
2006         checked_size(this->config_.layer_count, "layer_count") *
2007         checked_size(this->config_.max_positions, "max_positions") *
2008         checked_size(this->config_.head_count, "head_count") *
2009         checked_size(head_dim(this->config_), "head_dim");
2010     this->keys_.assign(element_count, 0.0F);
2011     this->values_.assign(element_count, 0.0F);
2012     this->written_.assign(
2013         checked_size(this->config_.layer_count, "layer_count") *
2014         checked_size(this->config_.max_positions, "max_positions"),
2015         0);
2016 }
2017
2018 void Gpt2KvCache::append(std::int32_t layer_id,
2019                          std::int32_t model_position,
2020                          std::span<const float> key,
2021                          std::span<const float> value) {
2022     const std::size_t hidden_size = checked_size(this->config_.hidden_size, "←
    ↪ hidden_size");
2023     if (key.size() != hidden_size || value.size() != hidden_size) {
2024         throw std::runtime_error("GPT-2 KV key/value size mismatch");
2025     }
2026     this->validate_address(layer_id, model_position, 0);
2027     const std::int32_t local_head_dim = head_dim(this->config_);
2028     for (std::int32_t head = 0; head < this->config_.head_count; ++head) {
2029         const std::size_t source =
2030             checked_size(head, "head") * checked_size(local_head_dim, "head_dim←
    ↪ ");
2031         const std::size_t target = this->base_offset(layer_id, model_position, ←
    ↪ head);
2032         std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
2033                    local_head_dim,
2034                    this->keys_.begin() + static_cast<std::ptrdiff_t>(target));

```

```

2035         std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
2036                     local_head_dim,
2037                     this->values_.begin() + static_cast<std::ptrdiff_t>(target) ←
    ↪ );
2038     }
2039     const std::size_t written_index =
2040         checked_size(layer_id, "layer_id") *
2041         checked_size(this->config_.max_positions, "max_positions") +
2042         checked_size(model_position, "model_position");
2043     this->written_[written_index] = 1;
2044     this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
2045 }
2046
2047 std::span<const float> Gpt2KvCache::key(std::int32_t layer_id,
2048                                         std::int32_t model_position,
2049                                         std::int32_t head) const {
2050     this->validate_address(layer_id, model_position, head);
2051     this->validate_written(layer_id, model_position);
2052     return this->key_unchecked(layer_id, model_position, head);
2053 }
2054
2055 std::span<const float> Gpt2KvCache::value(std::int32_t layer_id,
2056                                           std::int32_t model_position,
2057                                           std::int32_t head) const {
2058     this->validate_address(layer_id, model_position, head);
2059     this->validate_written(layer_id, model_position);
2060     return this->value_unchecked(layer_id, model_position, head);
2061 }
2062
2063 void detail::attend_cached_position(const Gpt2KvCache& cache,
2064                                    std::int32_t layer_id,
2065                                    std::int32_t model_position,
2066                                    std::span<const float> query,
2067                                    std::span<float> scores,
2068                                    std::span<float> output) {
2069     cache.validate_address(layer_id, model_position, 0);
2070     cache.validate_written(layer_id, model_position);
2071     const std::int32_t local_head_dim = head_dim(cache.config_);
2072     const std::size_t hidden_size = checked_size(cache.config_.hidden_size, " ←
    ↪ hidden_size");
2073     const std::size_t local_head_dim_size = checked_size(local_head_dim, " ←
    ↪ head_dim");
2074     const std::size_t model_position_size =
2075         checked_size(model_position, "model_position");
2076     if (query.size() != hidden_size || output.size() != hidden_size ||
2077         scores.size() < model_position_size + 1) {
2078         throw std::runtime_error("GPT-2 cached attention workspace size ←
    ↪ mismatch");
2079     }
2080
2081     const float score_scale = 1.0F / std::sqrt(static_cast<float>( ←
    ↪ local_head_dim));

```

```

2082     std::fill(output.begin(), output.end(), 0.0F);
2083     for (std::int32_t head = 0; head < cache.config_.head_count; ++head) {
2084         const std::size_t head_base =
2085             checked_size(head, "head") * local_head_dim_size;
2086         float max_score = LCQI_GPT2_SOFTMAX_NEGATIVE_INFINITY;
2087         for (std::size_t past = 0; past <= model_position_size; ++past) {
2088             const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2089             const std::span<const float> key =
2090                 cache.key_unchecked(layer_id, past_i32, head);
2091             float dot = 0.0F;
2092             for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2093                 dot += query[head_base + dim] * key[dim];
2094             }
2095             const float score = dot * score_scale;
2096             scores[past] = score;
2097             max_score = std::max(max_score, score);
2098         }
2099
2100         float denominator = 0.0F;
2101         for (std::size_t past = 0; past <= model_position_size; ++past) {
2102             scores[past] = std::exp(scores[past] - max_score);
2103             denominator += scores[past];
2104         }
2105         if (denominator <= 0.0F) {
2106             throw std::runtime_error("GPT-2 cached attention denominator is ←
↪ invalid");
2107         }
2108
2109         for (std::size_t past = 0; past <= model_position_size; ++past) {
2110             const std::int32_t past_i32 = static_cast<std::int32_t>(past);
2111             const float probability = scores[past] / denominator;
2112             const std::span<const float> value =
2113                 cache.value_unchecked(layer_id, past_i32, head);
2114             for (std::size_t dim = 0; dim < local_head_dim_size; ++dim) {
2115                 output[head_base + dim] += probability * value[dim];
2116             }
2117         }
2118     }
2119 }
2120
2121 std::span<const float> Gpt2KvCache::key_unchecked(std::int32_t layer_id,
2122                                                    std::int32_t model_position,
2123                                                    std::int32_t head) const ←
↪ noexcept {
2124     const std::size_t offset = this->base_offset_unchecked(layer_id, ←
↪ model_position, head);
2125     return std::span<const float>(
2126         this->keys_.data() + offset,
2127         static_cast<std::size_t>(head_dim(this->config_)));
2128 }
2129
2130 std::span<const float> Gpt2KvCache::value_unchecked(std::int32_t layer_id,

```

```

2131                                     std::int32_t model_position←
    ↪ ,
2132                                     std::int32_t head) const ↪
    ↪ noexcept {
2133     const std::size_t offset = this->base_offset_unchecked(layer_id, ↪
    ↪ model_position, head);
2134     return std::span<const float>(
2135         this->values_.data() + offset,
2136         static_cast<std::size_t>(head_dim(this->config_)));
2137 }
2138
2139 std::int32_t Gpt2KvCache::filled_tokens() const noexcept {
2140     return this->filled_tokens;
2141 }
2142
2143 std::size_t Gpt2KvCache::byte_size() const noexcept {
2144     return (this->keys_.size() + this->values_.size()) * sizeof(float) +
2145         this->written_.size() * sizeof(std::uint8_t);
2146 }
2147
2148 std::size_t Gpt2KvCache::base_offset(std::int32_t layer_id,
2149                                     std::int32_t model_position,
2150                                     std::int32_t head) const {
2151     return (((checked_size(layer_id, "layer_id") *
2152         checked_size(this->config_.max_positions, "max_positions")) +
2153         checked_size(model_position, "model_position")) *
2154         checked_size(this->config_.head_count, "head_count") +
2155         checked_size(head, "head")) *
2156         checked_size(head_dim(this->config_), "head_dim");
2157 }
2158
2159 std::size_t Gpt2KvCache::base_offset_unchecked(std::int32_t layer_id,
2160                                               std::int32_t model_position,
2161                                               std::int32_t head) const ↪
    ↪ noexcept {
2162     return (((static_cast<std::size_t>(layer_id) *
2163         static_cast<std::size_t>(this->config_.max_positions)) +
2164         static_cast<std::size_t>(model_position)) *
2165         static_cast<std::size_t>(this->config_.head_count) +
2166         static_cast<std::size_t>(head)) *
2167         static_cast<std::size_t>(head_dim(this->config_)));
2168 }
2169
2170 void Gpt2KvCache::validate_address(std::int32_t layer_id,
2171                                   std::int32_t model_position,
2172                                   std::int32_t head) const {
2173     if (layer_id < 0 || layer_id >= this->config_.layer_count) {
2174         throw std::runtime_error("GPT-2 KV layer_id out of range");
2175     }
2176     if (model_position < 0 || model_position >= this->config_.max_positions) {
2177         throw std::runtime_error("GPT-2 KV model_position out of range");
2178     }

```



```

2227         this->workspace_,
2228         token_id,
2229         false,
2230         this->hotspot_profile_,
2231         this->execution_options_,
2232         this->worker_pool_.get())
2233     .predicted_token;
2234 }
2235
2236 Gpt2ForwardResult Gpt2CachedGreedyDecoder::step_with_logits(std::int32_t ↵
    ↵ token_id) {
2237     return run_gpt2_cached_step_unchecked(*this->model_,
2238         this->cache_,
2239         this->workspace_,
2240         token_id,
2241         true,
2242         this->hotspot_profile_,
2243         this->execution_options_,
2244         this->worker_pool_.get());
2245 }
2246
2247 const Gpt2KvCache& Gpt2CachedGreedyDecoder::cache() const noexcept {
2248     return this->cache_;
2249 }
2250
2251 std::int32_t Gpt2CachedGreedyDecoder::filled_tokens() const noexcept {
2252     return this->cache_.filled_tokens();
2253 }
2254
2255 std::size_t Gpt2CachedGreedyDecoder::kv_cache_bytes() const noexcept {
2256     return this->cache_.byte_size();
2257 }
2258
2259 std::int32_t Gpt2CachedGreedyDecoder::worker_count() const noexcept {
2260     return this->worker_pool_ == nullptr ? 1 : this->worker_pool_->worker_count↵
    ↵ ();
2261 }
2262
2263 Gpt2ForwardResult run_gpt2_forward(const Gpt2ReferenceModel& model,
2264     std::span<const std::int32_t> token_ids) {
2265     validate_model(model);
2266     if (token_ids.empty()) {
2267         throw std::runtime_error("GPT-2 forward needs at least one token");
2268     }
2269     if (token_ids.size() > checked_size(model.config.max_positions, "↵
    ↵ max_positions")) {
2270         throw std::runtime_error("GPT-2 token_ids exceed max_positions");
2271     }
2272
2273     const std::int32_t sequence_length = static_cast<std::int32_t>(token_ids.↵
    ↵ size());
2274     std::vector<float> hidden_by_pos(matrix_size(sequence_length, model.config.↵

```

```

    ↪ hidden_size), 0.0F);
2275 for (std::int32_t position = 0; position < sequence_length; ++position) {
2276     const std::int32_t token_id = token_ids[checked_size(position, "↪
    ↪ position")]);
2277     if (token_id < 0 || token_id >= model.config.vocab_size) {
2278         throw std::runtime_error("GPT-2 token id out of range");
2279     }
2280     const std::span<const float> token =
2281         row_span(model.token_embedding, token_id, model.config.hidden_size)↪
    ↪ ;
2282     const std::span<const float> position_embedding =
2283         row_span(model.position_embedding, position, model.config.↪
    ↪ hidden_size);
2284     std::span<float> hidden = std::span<float>(
2285         hidden_by_pos.data() + matrix_size(position, model.config.↪
    ↪ hidden_size),
2286         checked_size(model.config.hidden_size, "hidden_size"));
2287     for (std::int32_t dim = 0; dim < model.config.hidden_size; ++dim) {
2288         hidden[checked_size(dim, "dim")] =
2289             token[checked_size(dim, "dim")] + position_embedding[↪
    ↪ checked_size(dim, "dim")]);
2290     }
2291 }
2292
2293 for (const Gpt2LayerWeightsF32& layer : model.layers) {
2294     forward_gpt2_layer(model.config, layer, sequence_length, hidden_by_pos)↪
    ↪ ;
2295 }
2296
2297 std::vector<float> normed(checked_size(model.config.hidden_size, "↪
    ↪ hidden_size"), 0.0F);
2298 const std::span<const float> final_hidden = std::span<const float>(
2299     hidden_by_pos.data() + matrix_size(sequence_length - 1, model.config.↪
    ↪ hidden_size),
2300     checked_size(model.config.hidden_size, "hidden_size"));
2301 layer_norm(final_hidden,
2302     model.final_ln_weight,
2303     model.final_ln_bias,
2304     model.config.layer_norm_epsilon,
2305     normed);
2306
2307 Gpt2ForwardResult result;
2308 result.logits.assign(checked_size(model.config.vocab_size, "vocab_size"), ↪
    ↪ 0.0F);
2309 compute_logits(model, normed, result.logits);
2310 result.predicted_token = argmax(result.logits);
2311 return result;
2312 }
2313
2314 Gpt2ForwardResult run_gpt2_forward_cached(const Gpt2ReferenceModel& model,
2315     Gpt2KvCache& cache,
2316     std::int32_t token_id) {

```



```

2317 validate_model(model);
2318 Gpt2ForwardWorkspace workspace(model.config);
2319 Gpt2ExecutionOptions options;
2320 options.worker_count = 1;
2321 return run_gpt2_cached_step_unchecked(
2322     model,
2323     cache,
2324     workspace,
2325     token_id,
2326     true,
2327     nullptr,
2328     options,
2329     nullptr);
2330 }
2331
2332 std::vector<std::int32_t> gpt2_generate_greedy(const Gpt2ReferenceModel& model,
2333                                             std::span<const std::int32_t> ←
2334         prompt_token_ids,
2335                                             std::int32_t max_new_tokens) {
2336     if (max_new_tokens < 0) {
2337         throw std::runtime_error("max_new_tokens cannot be negative");
2338     }
2339     std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids ←
2340     ← .end());
2341     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2342         if (tokens.size() >= checked_size(model.config.max_positions, " ←
2343         ← max_positions")) {
2344             throw std::runtime_error("GPT-2 generation would exceed ←
2345             ← max_positions");
2346         }
2347         const Gpt2ForwardResult result = run_gpt2_forward(model, tokens);
2348         tokens.push_back(result.predicted_token);
2349         if (model.config.eos_token_id >= 0 && result.predicted_token == model. ←
2350         ← config.eos_token_id) {
2351             break;
2352         }
2353     }
2354     return tokens;
2355 }
2356
2357 std::vector<std::int32_t> gpt2_generate_greedy_cached(
2358     const Gpt2ReferenceModel& model,
2359     std::span<const std::int32_t> prompt_token_ids,
2360     std::int32_t max_new_tokens) {
2361     if (prompt_token_ids.empty()) {
2362         throw std::runtime_error("GPT-2 generation needs at least one prompt ←
2363         ← token");
2364     }
2365     if (max_new_tokens < 0) {
2366         throw std::runtime_error("max_new_tokens cannot be negative");
2367     }
2368     if (prompt_token_ids.size() > checked_size(model.config.max_positions, " ←

```



```

    ↪ max_positions")) {
2363     throw std::runtime_error("GPT-2 prompt exceeds max_positions");
2364 }
2365
2366 Gpt2CachedGreedyDecoder decoder(model);
2367 std::vector<std::int32_t> tokens(prompt_token_ids.begin(), prompt_token_ids↪
    ↪ .end());
2368 if (max_new_tokens == 0) {
2369     return tokens;
2370 }
2371 std::int32_t predicted_token = 0;
2372 for (const std::int32_t token_id : prompt_token_ids) {
2373     predicted_token = decoder.step(token_id);
2374 }
2375 for (std::int32_t step = 0; step < max_new_tokens; ++step) {
2376     if (tokens.size() >= checked_size(model.config.max_positions, "↪
    ↪ max_positions")) {
2377         throw std::runtime_error("GPT-2 generation would exceed ↪
    ↪ max_positions");
2378     }
2379     tokens.push_back(predicted_token);
2380     if (model.config.eos_token_id >= 0 && predicted_token == model.config.↪
    ↪ eos_token_id) {
2381         break;
2382     }
2383     if (step + 1 < max_new_tokens) {
2384         predicted_token = decoder.step(predicted_token);
2385     }
2386 }
2387 return tokens;
2388 }
2389
2390 Gpt2ReferenceModel make_tiny_gpt2_reference_model() {
2391     Gpt2ReferenceModel model;
2392     model.config.hidden_size = 4;
2393     model.config.head_count = 2;
2394     model.config.layer_count = 1;
2395     model.config.max_positions = 8;
2396     model.config.vocab_size = 6;
2397     model.config.intermediate_size = 8;
2398     model.config.bos_token_id = 0;
2399     model.config.eos_token_id = 5;
2400     model.config.layer_norm_epsilon = 1.0e-5F;
2401     model.config.activation_function = "gelu_new";
2402
2403     model.token_embedding = {
2404         0.20F, -0.10F, 0.00F, 0.10F,
2405         -0.10F, 0.30F, 0.20F, -0.20F,
2406         0.05F, 0.10F, -0.30F, 0.25F,
2407         0.40F, -0.20F, 0.15F, 0.05F,
2408         -0.30F, 0.05F, 0.35F, -0.15F,
2409         0.10F, 0.20F, -0.10F, 0.30F,

```

```

2410 };
2411 model.position_embedding = {
2412     0.01F, 0.02F, 0.03F, 0.04F,
2413     -0.02F, 0.01F, 0.00F, 0.03F,
2414     0.03F, -0.01F, 0.02F, 0.00F,
2415     0.00F, 0.02F, -0.02F, 0.01F,
2416     0.02F, 0.00F, 0.01F, -0.01F,
2417     -0.01F, 0.03F, 0.00F, 0.02F,
2418     0.01F, -0.02F, 0.03F, 0.00F,
2419     0.00F, 0.01F, -0.01F, 0.02F,
2420 };
2421
2422 Gpt2LayerWeightsF32 layer;
2423 layer.ln_1_weight = ones(4);
2424 layer.ln_1_bias = zeros(4);
2425 layer.ln_2_weight = {0.9F, 1.1F, 1.0F, 0.95F};
2426 layer.ln_2_bias = {0.01F, -0.02F, 0.00F, 0.03F};
2427 layer.c_attn = make_linear(
2428     4,
2429     12,
2430     {
2431         0.10F, -0.20F, 0.05F, 0.15F,
2432         -0.05F, 0.12F, 0.20F, -0.10F,
2433         0.18F, 0.02F, -0.08F, 0.11F,
2434         -0.12F, 0.06F, 0.14F, 0.04F,
2435         0.07F, 0.16F, -0.09F, 0.03F,
2436         -0.15F, 0.05F, 0.13F, 0.10F,
2437         0.04F, -0.11F, 0.19F, -0.02F,
2438         0.09F, 0.08F, -0.06F, 0.17F,
2439         0.20F, -0.04F, 0.01F, 0.06F,
2440         -0.08F, 0.15F, 0.07F, -0.03F,
2441         0.05F, 0.10F, -0.14F, 0.12F,
2442         0.11F, -0.07F, 0.16F, 0.02F,
2443     },
2444     {
2445         0.01F, -0.02F, 0.00F, 0.03F,
2446         -0.01F, 0.02F, 0.01F, -0.02F,
2447         0.00F, 0.01F, -0.01F, 0.02F,
2448     });
2449 layer.c_proj = make_linear(
2450     4,
2451     4,
2452     {
2453         0.10F, 0.05F, -0.08F, 0.12F,
2454         -0.04F, 0.11F, 0.09F, -0.02F,
2455         0.07F, -0.10F, 0.13F, 0.05F,
2456         0.02F, 0.14F, -0.06F, 0.08F,
2457     },
2458     {0.01F, 0.00F, -0.01F, 0.02F});
2459 layer.c_fc = make_linear(
2460     4,
2461     8,

```

```

2462     {
2463         0.20F, -0.10F, 0.05F, 0.03F,
2464         -0.05F, 0.14F, 0.12F, -0.08F,
2465         0.09F, 0.07F, -0.11F, 0.16F,
2466         -0.12F, 0.05F, 0.18F, 0.02F,
2467         0.04F, -0.09F, 0.13F, 0.10F,
2468         0.11F, 0.03F, -0.07F, 0.15F,
2469         -0.02F, 0.17F, 0.06F, -0.04F,
2470         0.08F, -0.06F, 0.10F, 0.12F,
2471     },
2472     {0.01F, -0.02F, 0.00F, 0.03F, -0.01F, 0.02F, 0.01F, 0.00F});
2473     layer.mlp_c_proj = make_linear(
2474         8,
2475         4,
2476         {
2477             0.10F, -0.08F, 0.06F, 0.12F, -0.04F, 0.07F, 0.03F, 0.11F,
2478             -0.05F, 0.13F, 0.02F, -0.09F, 0.15F, 0.04F, -0.03F, 0.08F,
2479             0.07F, 0.01F, -0.12F, 0.10F, 0.06F, -0.05F, 0.14F, 0.02F,
2480             0.03F, 0.09F, 0.11F, -0.06F, 0.05F, 0.12F, -0.08F, 0.04F,
2481         },
2482         {0.00F, 0.01F, -0.01F, 0.02F});
2483     model.layers.push_back(std::move(layer));
2484     model.final_ln_weight = {1.0F, 0.95F, 1.05F, 1.0F};
2485     model.final_ln_bias = {0.0F, 0.01F, -0.01F, 0.02F};
2486     model.tie_lm_head_to_embedding = true;
2487     validate_model(model);
2488     return model;
2489 }
2490
2491 } // namespace lcqi

```

`gpt2_cli.cpp` 负责把模型目录、prompt、`max_new_tokens` 和 `benchmark` 选项接到 GPT-2 reference path。无参数时运行 tiny GPT-2 fixture；传入 HuggingFace 模型目录时加载真实资产。

Listing 10.14 `src/gpt2_cli.cpp`

```

1 #include <lcqi/gpt2_reference.hpp>
2
3 #include <chrono>
4 #include <cstdlib>
5 #include <exception>
6 #include <filesystem>
7 #include <iostream>
8 #include <span>
9 #include <stdexcept>
10 #include <string>
11 #include <string_view>
12 #include <vector>
13
14 namespace {
15
16 constexpr int LCQI_GPT2_TINY_ARGC = 1;
17 constexpr int LCQI_GPT2_MIN_REAL_ARGC = 3;

```

```

18 constexpr std::int32_t LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS = 8;
19 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_0 = 1;
20 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_1 = 2;
21 constexpr std::int32_t LCQI_GPT2_TINY_PROMPT_2 = 3;
22
23 enum class Gpt2EngineMode {
24     full_prefix,
25     cached_kv,
26 };
27
28 struct Gpt2CliOptions {
29     Gpt2EngineMode engine = Gpt2EngineMode::cached_kv;
30     bool benchmark = false;
31     bool profile_hotspots = false;
32     std::filesystem::path model_dir;
33     std::string prompt;
34     std::int32_t max_new_tokens = LCQI_GPT2_DEFAULT_MAX_NEW_TOKENS;
35     std::int32_t worker_count = 0;
36 };
37
38 struct Gpt2BenchmarkTimings {
39     double load_ms = 0.0;
40     double tokenize_ms = 0.0;
41     double prefill_ms = 0.0;
42     double decode_ms = 0.0;
43     double generate_ms = 0.0;
44     double total_ms = 0.0;
45     std::size_t prefill_steps = 0;
46     std::size_t decode_steps = 0;
47     std::size_t kv_cache_bytes = 0;
48     std::int32_t worker_count = 1;
49 };
50
51 using Clock = std::chrono::steady_clock;
52
53 void print_ids(std::span<const std::int32_t> ids) {
54     for (std::size_t index = 0; index < ids.size(); ++index) {
55         if (index != 0) {
56             std::cout << ' ';
57         }
58         std::cout << ids[index];
59     }
60 }
61
62 [[nodiscard]] double elapsed_ms(Clock::time_point start, Clock::time_point end) ←
    ↪ {
63     return std::chrono::duration<double, std::milli>(end - start).count();
64 }
65
66 [[nodiscard]] std::size_t checked_size(std::int64_t value, const char* name) {
67     if (value < 0) {
68         throw std::runtime_error(std::string(name) + " cannot be negative");

```

```

69     }
70     return static_cast<std::size_t>(value);
71 }
72
73 [[nodiscard]] const char* engine_name(Gpt2EngineMode engine) {
74     switch (engine) {
75         case Gpt2EngineMode::full_prefix:
76             return "full_prefix";
77         case Gpt2EngineMode::cached_kv:
78             return "cached_kv";
79     }
80     return "unknown";
81 }
82
83 [[nodiscard]] Gpt2EngineMode parse_engine(std::string_view value) {
84     if (value == "full" || value == "full_prefix") {
85         return Gpt2EngineMode::full_prefix;
86     }
87     if (value == "cached" || value == "cached_kv") {
88         return Gpt2EngineMode::cached_kv;
89     }
90     throw std::runtime_error("unknown --engine value, expected cached or full")↵
    ↵ ;
91 }
92
93 [[nodiscard]] std::int32_t parse_non_negative_i32(const std::string& text,
94                                                    const char* name) {
95     const std::int32_t value = static_cast<std::int32_t>(std::stoi(text));
96     if (value < 0) {
97         throw std::runtime_error(std::string(name) + " cannot be negative");
98     }
99     return value;
100 }
101
102 [[nodiscard]] Gpt2CliOptions parse_options(int argc, char** argv) {
103     Gpt2CliOptions options;
104     std::vector<std::string> positional;
105     for (int index = 1; index < argc; ++index) {
106         const std::string arg = argv[index];
107         if (arg == "--benchmark") {
108             options.benchmark = true;
109         } else if (arg == "--profile-hotspots") {
110             options.profile_hotspots = true;
111             options.benchmark = true;
112         } else if (arg == "--engine") {
113             if (index + 1 >= argc) {
114                 throw std::runtime_error("--engine requires a value");
115             }
116             ++index;
117             options.engine = parse_engine(argv[index]);
118         } else if (arg.rfind("--engine=", 0) == 0) {
119             options.engine = parse_engine(std::string_view(arg).substr(std::string::npos - 10));

```

```

120     } else if (arg == "--threads") {
121         if (index + 1 >= argc) {
122             throw std::runtime_error("--threads requires a value");
123         }
124         ++index;
125         options.worker_count = parse_non_negative_i32(argv[index], "--↵
↵ threads");
126     } else if (arg.rfind("--threads=", 0) == 0) {
127         options.worker_count =
128             parse_non_negative_i32(arg.substr(std::string("--threads=").↵
↵ size()), "--threads");
129     } else if (arg == "--full") {
130         options.engine = Gpt2EngineMode::full_prefix;
131     } else if (arg == "--cached") {
132         options.engine = Gpt2EngineMode::cached_kv;
133     } else if (!arg.empty() && arg.front() == '-') {
134         throw std::runtime_error("unknown option: " + arg);
135     } else {
136         positional.push_back(arg);
137     }
138 }
139
140 if (positional.size() < 2 || positional.size() > 3) {
141     throw std::runtime_error(
142         "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↵
↵ ] "
143         "model_dir prompt [max_new_tokens]");
144 }
145 options.model_dir = positional[0];
146 options.prompt = positional[1];
147 if (positional.size() == 3) {
148     options.max_new_tokens = parse_non_negative_i32(positional[2], "↵
↵ max_new_tokens");
149 }
150 return options;
151 }
152
153 [[nodiscard]] std::vector<std::int32_t> generate_full_prefix(
154     const lcqi::Gpt2ReferenceModel& model,
155     std::span<const std::int32_t> prompt_ids,
156     std::int32_t max_new_tokens,
157     Gpt2BenchmarkTimings& timings) {
158     const Clock::time_point generate_start = Clock::now();
159     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
160     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
161         if (tokens.size() >= checked_size(model.config.max_positions, "↵
↵ max_positions")) {
162             throw std::runtime_error("GPT-2 generation would exceed ↵
↵ max_positions");
163         }
164         const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, ↵

```

```

    ↪ tokens);
165     ++timings.decode_steps;
166     tokens.push_back(result.predicted_token);
167     if (model.config.eos_token_id >= 0 && result.predicted_token == model.↪
    ↪ config.eos_token_id) {
168         break;
169     }
170 }
171 const Clock::time_point generate_end = Clock::now();
172 timings.generate_ms = elapsed_ms(generate_start, generate_end);
173 timings.decode_ms = timings.generate_ms;
174 return tokens;
175 }
176
177 [[nodiscard]] std::vector<std::int32_t> generate_cached(
178     const lcqi::Gpt2ReferenceModel& model,
179     std::span<const std::int32_t> prompt_ids,
180     std::int32_t max_new_tokens,
181     Gpt2BenchmarkTimings& timings,
182     lcqi::Gpt2HotspotProfile* hotspot_profile,
183     std::int32_t worker_count) {
184     if (prompt_ids.empty()) {
185         throw std::runtime_error("GPT-2 generation needs at least one prompt ↪
    ↪ token");
186     }
187     lcqi::Gpt2ExecutionOptions execution_options;
188     execution_options.worker_count = worker_count;
189     lcqi::Gpt2CachedGreedyDecoder decoder(model, hotspot_profile, ↪
    ↪ execution_options);
190     timings.kv_cache_bytes = decoder.kv_cache_bytes();
191     timings.worker_count = decoder.worker_count();
192     std::vector<std::int32_t> tokens(prompt_ids.begin(), prompt_ids.end());
193     if (max_new_tokens == 0) {
194         return tokens;
195     }
196
197     const Clock::time_point generate_start = Clock::now();
198     std::int32_t predicted_token = 0;
199     const Clock::time_point prefill_start = Clock::now();
200     for (const std::int32_t token_id : prompt_ids) {
201         predicted_token = decoder.step(token_id);
202         ++timings.prefill_steps;
203     }
204     const Clock::time_point prefill_end = Clock::now();
205     timings.prefill_ms = elapsed_ms(prefill_start, prefill_end);
206
207     const Clock::time_point decode_start = Clock::now();
208     for (std::int32_t step = 0; step < max_new_tokens; ++step) {
209         if (tokens.size() >= checked_size(model.config.max_positions, "↪
    ↪ max_positions")) {
210             throw std::runtime_error("GPT-2 generation would exceed ↪
    ↪ max_positions");

```

```

211     }
212     tokens.push_back(predicted_token);
213     if (model.config.eos_token_id >= 0 && predicted_token == model.config.eos_token_id) {
214         break;
215     }
216     if (step + 1 < max_new_tokens) {
217         predicted_token = decoder.step(predicted_token);
218         ++timings.decode_steps;
219     }
220 }
221 const Clock::time_point decode_end = Clock::now();
222 const Clock::time_point generate_end = Clock::now();
223 timings.decode_ms = elapsed_ms(decode_start, decode_end);
224 timings.generate_ms = elapsed_ms(generate_start, generate_end);
225 return tokens;
226 }
227
228 void print_benchmark(const Gpt2BenchmarkTimings& timings,
229                     std::size_t prompt_tokens,
230                     std::size_t generated_tokens) {
231     const double new_tokens = static_cast<double>(generated_tokens);
232     const double generated_tokens_per_second =
233         timings.generate_ms > 0.0 ? new_tokens * 1000.0 / timings.generate_ms :
234         0.0;
235     const double decode_tokens_per_second =
236         timings.decode_ms > 0.0
237         ? static_cast<double>(timings.decode_steps) * 1000.0 / timings.decode_ms
238         : 0.0;
239
240     std::cout << "benchmark_load_ms " << timings.load_ms << "\n";
241     std::cout << "benchmark_tokenize_ms " << timings.tokenize_ms << "\n";
242     std::cout << "benchmark_prefill_ms " << timings.prefill_ms << "\n";
243     std::cout << "benchmark_decode_ms " << timings.decode_ms << "\n";
244     std::cout << "benchmark_generate_ms " << timings.generate_ms << "\n";
245     std::cout << "benchmark_total_ms " << timings.total_ms << "\n";
246     std::cout << "benchmark_prompt_tokens " << prompt_tokens << "\n";
247     std::cout << "benchmark_generated_tokens " << generated_tokens << "\n";
248     std::cout << "benchmark_prefill_steps " << timings.prefill_steps << "\n";
249     std::cout << "benchmark_decode_steps " << timings.decode_steps << "\n";
250     std::cout << "benchmark_worker_count " << timings.worker_count << "\n";
251     std::cout << "benchmark_generate_tokens_per_second "
252         << generated_tokens_per_second << "\n";
253     std::cout << "benchmark_decode_tokens_per_second "
254         << decode_tokens_per_second << "\n";
255     std::cout << "benchmark_kv_cache_bytes " << timings.kv_cache_bytes << "\n";
256 }
257
258 void print_hotspot_profile(const lcqi::Gpt2HotspotProfile& profile) {
259     const double total = profile.total_step_ms > 0.0 ? profile.total_step_ms :
260         1.0;

```



```

259     const auto print_ms = [total](std::string_view name, double value) {
260         std::cout << "hotspot_" << name << "_ms " << value << "\n";
261         std::cout << "hotspot_" << name << "_pct " << (value * 100.0 / total) <<
        << "\n";
262     };
263     std::cout << "hotspot_decoder_steps " << profile.decoder_steps << "\n";
264     std::cout << "hotspot_layer_steps " << profile.layer_steps << "\n";
265     print_ms("total_step", profile.total_step_ms);
266     print_ms("embedding", profile.embedding_ms);
267     print_ms("layer_norm", profile.layer_norm_ms);
268     print_ms("final_norm", profile.final_norm_ms);
269     print_ms("qkv_projection", profile.qkv_projection_ms);
270     print_ms("kv_append", profile.kv_append_ms);
271     print_ms("attention", profile.attention_ms);
272     print_ms("attention_projection", profile.attention_projection_ms);
273     print_ms("residual_add", profile.residual_add_ms);
274     print_ms("mlp_fc", profile.mlp_fc_ms);
275     print_ms("gelu", profile.gelu_ms);
276     print_ms("mlp_projection", profile.mlp_projection_ms);
277     print_ms("lm_head", profile.lm_head_ms);
278     print_ms("logits_result", profile.logits_result_ms);
279 }
280
281 int run_tiny_mode() {
282     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model()
        << ();
283     const std::vector<std::int32_t> prompt{
284         LCQI_GPT2_TINY_PROMPT_0,
285         LCQI_GPT2_TINY_PROMPT_1,
286         LCQI_GPT2_TINY_PROMPT_2,
287     };
288     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, prompt)
        << );
289     const std::vector<std::int32_t> generated =
290         lcqi::gpt2_generate_greedy(model, prompt, 2);
291     std::cout << "mode tiny\n";
292     std::cout << "prompt_ids ";
293     print_ids(prompt);
294     std::cout << "\n";
295     std::cout << "predicted_token " << result.predicted_token << "\n";
296     std::cout << "logits";
297     for (const float value : result.logits) {
298         std::cout << ' ' << value;
299     }
300     std::cout << "\n";
301     std::cout << "generated_ids ";
302     print_ids(generated);
303     std::cout << "\n";
304     return EXIT_SUCCESS;
305 }
306
307 int run_real_model_mode(int argc, char** argv) {

```

```

308     if (argc < LCQI_GPT2_MIN_REAL_ARGC) {
309         throw std::runtime_error(
310             "usage: lcqi_gpt2 [--engine cached|full] [--threads N] [--benchmark↵
↵ ] "
311             "model_dir prompt [max_new_tokens]");
312     }
313     const Clock::time_point total_start = Clock::now();
314     const Gpt2CliOptions options = parse_options(argc, argv);
315
316     Gpt2BenchmarkTimings timings;
317     const Clock::time_point load_start = Clock::now();
318     const lcqi::Gpt2ReferenceModel model = lcqi::load_gpt2_from_directory(↵
↵ options.model_dir);
319     const lcqi::Gpt2Tokenizer tokenizer = lcqi::load_gpt2_tokenizer(
320         options.model_dir / "vocab.json",
321         options.model_dir / "merges.txt",
322         model.config.bos_token_id,
323         model.config.eos_token_id);
324     timings.load_ms = elapsed_ms(load_start, Clock::now());
325
326     const Clock::time_point tokenize_start = Clock::now();
327     const std::vector<std::int32_t> prompt_ids = lcqi::gpt2_encode(tokenizer, ↵
↵ options.prompt);
328     timings.tokenize_ms = elapsed_ms(tokenize_start, Clock::now());
329
330     std::vector<std::int32_t> generated;
331     lcqi::Gpt2HotspotProfile hotspot_profile;
332     if (options.engine == Gpt2EngineMode::full_prefix) {
333         if (options.profile_hotspots) {
334             throw std::runtime_error("--profile-hotspots currently supports ↵
↵ cached engine only");
335         }
336         generated = generate_full_prefix(model, prompt_ids, options.↵
↵ max_new_tokens, timings);
337     } else {
338         generated = generate_cached(model,
339                                     prompt_ids,
340                                     options.max_new_tokens,
341                                     timings,
342                                     options.profile_hotspots ? &hotspot_profile↵
↵ : nullptr,
343                                     options.worker_count);
344     }
345     timings.total_ms = elapsed_ms(total_start, Clock::now());
346
347     std::cout << "mode gpt2_directory\n";
348     std::cout << "engine " << engine_name(options.engine) << "\n";
349     std::cout << "model_dir " << options.model_dir.string() << "\n";
350     std::cout << "prompt_ids ";
351     print_ids(prompt_ids);
352     std::cout << "\n";
353     std::cout << "generated_ids ";

```

```

354     print_ids(generated);
355     std::cout << "\n";
356     std::cout << "generated_text " << lcqi::gpt2_decode(tokenizer, generated) <<
    << "\n";
357     if (options.benchmark) {
358         const std::size_t generated_tokens = generated.size() - prompt_ids.size()
    < ();
359         print_benchmark(timings, prompt_ids.size(), generated_tokens);
360     }
361     if (options.profile_hotspots) {
362         print_hotspot_profile(hotspot_profile);
363     }
364     return EXIT_SUCCESS;
365 }
366
367 } // namespace
368
369 int main(int argc, char** argv) {
370     try {
371         if (argc == LCQI_GPT2_TINY_ARGC) {
372             return run_tiny_mode();
373         }
374         return run_real_model_mode(argc, argv);
375     } catch (const std::exception& error) {
376         std::cerr << "error: " << error.what() << "\n";
377         return EXIT_FAILURE;
378     }
379 }

```

第 11 章实践：运行时内存计划、liveness 与 Workspace

本章对应原书中的第三册“推理运行时与内存规划”。不要把它当作概念复述，本章要把 activation arena、workspace、liveness、state edge、debug dump、buffer reuse 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `layout`，核心命令或测试入口是 `lcqi_trace`。

Listing 11.1 第 11 章实践：运行时内存计划、liveness 与 Workspace 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / layout
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：比较普通 activation 和 KV cache 的生命周期。读者要先手写预期结果，再运行项目观察输出。动作是：说明为什么 KV cache 不能被 activation planner 当作临时缓冲复用。如果输出里没有 `checksum` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：内存复用只在生命周期不重叠时成立；debug/oracle/materialization 会延长生命周期这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 liveness 表：每个 checkpoint 的 `defined_at`、`last_used_at`、是否 state、是否允许复用。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 12 章实践：IR、Pass、Lowering 与 Executable Plan

本章对应原书中的第三册“AI 编译器细化：IR、pass、lowering 与 executable plan”。不要把它当作概念复述，本章要把 typed graph、pass、fusion、layout propagation、backend capability、fallback reason 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `run_inference`，核心命令或测试入口是 `lcqi_cli`。

Listing 12.1 第 12 章实践：IR、Pass、Lowering 与 Executable Plan 的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↪ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_cli / run_inference
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把 tiny MLP 写成三节点 IR：linear、relu、linear。读者要先手写预期结果，再运行项目观察输出。动作是：说明哪些 pass 可以融合，哪些必须保留为 oracle checkpoint。如果输出里没有 `backend` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：IR 的价值不是画图，而是让 shape、layout、量化、后端选择和 fallback 诊断有稳定承载物这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 executable plan 草图：节点、输入输出 tensor、layout、kernel、fallback reason 和 profiler event。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第零部分

后端优化、工业专项与验收

第 13 章实践：CPU 后端、微内核与性能证据

本章对应原书中的第三册“CPU 后端优化细讲”和“CPU decode 实战”。不要把它当作概念复述，本章要把 scalar baseline、packed weight、AVX2、objdump、perf boundary、shape sweep 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 **avx2**，核心命令或测试入口是 **lcqi_bench**。

Listing 13.1 第 13 章实践：CPU 后端、微内核与性能证据的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↳ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_bench / avx2
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：对同一 kernel 保存 benchmark CSV 和反汇编摘要。读者要先手写预期结果，再运行项目观察输出。动作是：解释速度差来自地址流、SIMD lane、packing、load 或 reduction 哪一项。如果输出里没有 **average_us** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：CPU 后端优化必须同时有 oracle、反汇编、shape sweep 和降级边界；只有耗时表不够这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 **include/lcqi** 头文件和一个 **src** 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 CPU kernel evidence：命令、机器、编译器、输入 shape、backend、checksum、反汇编摘要和未验证 PMU 边界。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 14 章实践：GPU 后端边界、copy 与同步成本

本章对应原书中的第三册“GPU 硬件基础”和“GPU 后端优化细讲”。不要把它当作概念复述，本章要把 device buffer、stream、event、kernel launch、copy、sync、fallback 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 **StepTrace**，核心命令或测试入口是 **lcqi_trace**。

Listing 14.1 第 14 章实践：GPU 后端边界、copy 与同步成本的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  <- DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_trace / StepTrace
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：当前项目没有 GPU kernel，也要写清 adapter 合同。读者要先手写预期结果，再运行项目观察输出。动作是：把 CPU trace 字段扩展成 GPU StepTrace 需要哪些字段。如果输出里没有 **copy_time_ns** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：GPU 快不快不能只看 kernel；copy、sync、sampler、fallback 和 batch wait 必须进入端到端 trace 这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 **include/lcqi** 头文件和一个 **src** 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交 GPU adapter 设计：buffer owner、stream、event、H2D/D2H 字节、sync 点、fallback 和 oracle 对照。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 15 章实践：工业 LLM 专项能力对照

本章对应原书中的第三册“工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照”。不要把它当作概念复述，本章要把 speculative decoding、MoE routing、LoRA adapter、multi-GPU、source reading 放到同一个可运行项目里检查。贯穿代码仍然是 [books/cpu-volume-3/labs/linux_cpu_inference](#)；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `bandwidth_limit`，核心命令或测试入口是 `lcqi_ledger`。

Listing 15.1 第 15 章实践：工业 LLM 专项能力对照的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<↵
↵ DCMMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: lcqi_ledger / bandwidth_limit
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：选择一个专项，不写泛泛介绍。读者要先手写预期结果，再运行项目观察输出。动作是：把它接到现有 trace、memory、backend 或 serving 字段上。如果输出里没有 `tokens/s` 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：工业专项不是功能清单；必须说明它购买什么能力、增加什么状态、破坏哪些假设这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 [books/cpu-volume-3/labs/linux_cpu_inference/README.md](#)、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交专项设计卡：动机、状态新增、trace 字段、回归测试、性能账本和不能验证的部分。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

第 16 章实践：工程验收、回归门禁与最终项目

本章对应原书中的第三册“工程验收与回归体系”。不要把它当作概念复述，本章要把 reference、unit test、trace golden、benchmark gate、SLO gate、coverage、CI 放到同一个可运行项目里检查。贯穿代码仍然是 `books/cpu-volume-3/labs/linux_cpu_inference`；如果某个实验在当前机器上不能验证，报告必须写清降级原因，不能把源码存在当作实测证据。

一、对应原书问题：概念必须落到对象和命令

原书讲的是系统边界，本章实践要回答三个问题。第一，哪个代码对象承载这个概念；第二，哪个命令能产生可复查输出；第三，哪个坏输入或缺失字段能证明边界不是空话。这里的核心对象包括 `lcqi_tests`，核心命令或测试入口是 `ctest`。

Listing 16.1 第 16 章实践：工程验收、回归门禁与最终项目的最小命令入口

```
cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
  ↳ DCMAKE_BUILD_TYPE=Debug
cmake --build books/cpu-volume-3/build/lcqi-debug
ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
# chapter focus: ctest / lcqi_tests
```

二、从特例推到规律：先抓一个能手算的切片

本章先看特例：把所有前面报告合成最终验收包。读者要先手写预期结果，再运行项目观察输出。动作是：运行 `ctest` 并说明每个 test 保护哪条合同。如果输出里没有 **100% tests passed** 或对应字段无法解释，就不要继续优化；先回到 reference、输入合同和 trace 字段。

从这个特例推出的规律是：实践卷最终目标是长期维护：结果正确、trace 稳定、性能可解释、服务指标可回归、失败边界不伪装这条规律要写进报告，不要只写“运行成功”。运行成功只说明某条路径没崩，解释清楚输入、输出、错误路径和不可验证边界，才说明学会了这个知识点。

三、实践任务：把观察固定成报告字段

任务一：定位源码。至少阅读 `books/cpu-volume-3/labs/linux_cpu_inference/README.md`、相关 `include/lcqi` 头文件和一个 `src` 实现文件。报告要写出函数入口、输入对象、输出对象和错误处理方式。

任务二：运行命令。保存构建命令、测试命令、核心输出和环境字段。若命令依赖 AVX2、perf、GPU、NUMA 或外部库，必须记录当前机器是否支持；不支持时把结论降级为“接口已设计，性能未验证”。

任务三：构造反例。围绕本章知识点设计一个坏输入、缺失字段或不满足前提的场景，说明系统应拒绝、降级、fallback 还是继续执行。只给正常路径不合格。

四、验收和递进大作业

验收标准：报告能从一个具体输入推到工程规则；命令可以在仓库中复跑；输出字段能对应到源码；失败路径说得清；未验证边界不伪装成结论。

递进大作业：提交最终项目：README、命令记录、报告索引、测试结果、benchmark 表、trace 摘要、风险清单和下一阶段计划。这个作业会被后续章节继续引用，命名、目录和字段不要随意变化。

代码走读：工具、Smoke、Benchmark 与回归测试

工具不是附属品

LCQI 的工程证据不只在 C++ 源码里。smoke 脚本决定真实模型怎样下载、校验和运行；benchmark 程序决定性能口径；trace、ledger 和 serving probe 决定报告字段；CTest 决定哪些行为能长期回归。读者必须把这些文件当作工程合同的一部分。

Benchmark、Trace、Ledger 与 Serving Probe

`bench.cpp` 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing 16.2 src/bench.cpp

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #include <algorithm>
4 #include <cstdlib>
5 #include <iostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22     #if defined(__x86_64__) || defined(_M_X64)
23         return LCQI_TARGET_ARCH_X86_64;
24     #elif defined(__i386__) || defined(_M_IX86)
25         return LCQI_TARGET_ARCH_I386;
26     #else
27         return LCQI_TARGET_ARCH_UNKNOWN;
28     #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32     if (argc <= LCQI_REPEAT_ARG_INDEX) {
33         return LCQI_DEFAULT_REPEAT;
34     }
35     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
36     if (repeat <= 0) {

```

```

37     throw std::runtime_error("repeat must be positive");
38 }
39 return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43     return {
44         {128, 128, 16, repeat},
45         {256, 256, 16, repeat},
46         {512, 512, 16, repeat},
47         {513, 257, 16, repeat},
48         {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49                                     repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50         {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51          std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↵
52 ↵ LCQI_LARGE_CASE_REPEAT_DIVISOR)}},
53     };
54 }
55 } // namespace
56
57 int main(int argc, char** argv) {
58     try {
59         const std::int32_t repeat = parse_repeat(argc, argv);
60         std::cout
61             << "target_arch,input_size,output_size,output_block,repeat,backend,↵
62 ↵ available,"
63             << "average_us,max_abs_diff,checksum\n";
64         for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↵
65 ↵ repeat)) {
66             const lcqi::KernelBenchmarkResult result =
67                 lcqi::benchmark_linear_i8_case(benchmark_case);
68             for (const lcqi::KernelBackendBenchmark& backend : result.backends)↵
69 ↵ {
70                 std::cout << target_arch() << ','
71                     << result.benchmark_case.input_size << ','
72                     << result.benchmark_case.output_size << ','
73                     << result.benchmark_case.output_block_size << ','
74                     << result.benchmark_case.repeat << ','
75                     << backend.name << ','
76                     << (backend.available ? 1 : 0) << ','
77                     << backend.average_us << ','
78                     << backend.max_abs_diff << ','
79                     << backend.checksum << '\n';
80             }
81         }
82         return EXIT_SUCCESS;
83     } catch (const std::exception& error) {
84         std::cerr << "error: " << error.what() << '\n';
85         return EXIT_FAILURE;
86     }
87 }

```

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing 16.3 src/trace.cpp

```

1  #include <lcqi/reference_decoder.hpp>
2  #include <lcqi/tokenizer.hpp>
3
4  #include <algorithm>
5  #include <array>
6  #include <cstdlib>
7  #include <exception>
8  #include <filesystem>
9  #include <iostream>
10 #include <span>
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25     return std::filesystem::path(__FILE__).parent_path().parent_path() /
26         "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel&
    ↪ & tokenizer,
30                                                     std::string_view prompt) {
31     std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer,
    ↪ prompt);
32     if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33         full_tokens.front() != LCQI_TRACE_BOS_ID ||
34         full_tokens.back() != LCQI_TRACE_EOS_ID) {
35         throw std::runtime_error("unexpected tokenizer fixture ids");
36     }
37
38     std::vector<std::int32_t> decoder_tokens(
39         full_tokens.begin() + 1,
40         full_tokens.end() - 1);
41     if (!std::equal(decoder_tokens.begin(),
42                    decoder_tokens.end(),
43                    LCQI_TRACE_TOKEN_IDS.begin(),
44                    LCQI_TRACE_TOKEN_IDS.end())) {
45         throw std::runtime_error("unexpected decoder token ids");
46     }

```

```

47     return decoder_tokens;
48 }
49
50 std::string join_ints(std::span<const std::int32_t> values) {
51     std::ostringstream stream;
52     for (std::size_t index = 0; index < values.size(); ++index) {
53         if (index != 0) {
54             stream << 'x';
55         }
56         stream << values[index];
57     }
58     return stream.str();
59 }
60
61 std::string join_int_values(std::span<const std::int32_t> values) {
62     std::ostringstream stream;
63     for (std::size_t index = 0; index < values.size(); ++index) {
64         if (index != 0) {
65             stream << ';';
66         }
67         stream << values[index];
68     }
69     return stream.str();
70 }
71
72 std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73     std::int32_t result = 0;
74     for (const std::int32_t value : values) {
75         result += value;
76     }
77     return result;
78 }
79
80 std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81     std::int32_t result = 0;
82     for (const std::int32_t value : values) {
83         result = std::max(result, std::abs(value));
84     }
85     return result;
86 }
87
88 std::string join_floats(std::span<const float> values) {
89     std::ostringstream stream;
90     const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);
91     for (std::size_t index = 0; index < count; ++index) {
92         if (index != 0) {
93             stream << ';';
94         }
95         stream << values[index];
96     }
97     if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98         stream << "...";

```

```

99     }
100     return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104                    const lcqi::TokenizerModel& tokenizer,
105                    std::span<const std::int32_t> full_tokens) {
106     std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107     std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108         << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109     std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110         << ', ' << full_tokens.size() << ",1,i32,contiguous,"
111         << checksum_ints(full_tokens) << ', '
112         << max_abs_ints(full_tokens) << ', '
113         << join_int_values(full_tokens) << '\n';
114     std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115         << ",scalar,scalar,u64,fnv1a,"
116         << tokenizer_contract_hash(tokenizer) << ",0,"
117         << tokenizer.chat_template_hash << '\n';
118     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵     {
119         std::cout << checkpoint.name << ', '
120             << checkpoint.token_position << ', '
121             << checkpoint.layer_id << ', '
122             << join_ints(checkpoint.shape) << ', '
123             << join_ints(checkpoint.stride) << ', '
124             << checkpoint.dtype << ', '
125             << checkpoint.layout << ', '
126             << checkpoint.checksum << ', '
127             << checkpoint.max_abs << ', '
128             << join_floats(checkpoint.values) << '\n';
129     }
130     std::cout << "sampler_argmax,"
131         << LCQI_TRACE_TOKEN_IDS.size() - 1 << ', '
132         << LCQI_TRACE_METADATA_LAYER
133         << ', ' << trace.result.logits.size() << ",1,f32,argmax,"
134         << "0,0,token=" << trace.result.predicted_token << '\n';
135     std::cout << "generated_token,"
136         << LCQI_TRACE_TOKEN_IDS.size() << ', '
137         << LCQI_TRACE_METADATA_LAYER
138         << ",scalar,scalar,i32,generated,"
139         << trace.result.predicted_token << ', '
140         << trace.result.predicted_token << ', '
141         << trace.result.predicted_token << '\n';
142 }
143
144 } // namespace
145
146 int main() {
147     try {
148         const lcqi::TokenizerModel tokenizer =

```



```

149         lcqi::load_tokenizer(tokenizer_fixture_path());
150         const std::vector<std::int32_t> full_tokens =
151             lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152         const std::vector<std::int32_t> decoder_tokens =
153             decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154         const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155         const lcqi::ReferenceDecodeTraceResult trace =
156             lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157         print_trace_csv(trace, tokenizer, full_tokens);
158         return EXIT_SUCCESS;
159     } catch (const std::exception& error) {
160         std::cerr << "error: " << error.what() << '\n';
161         return EXIT_FAILURE;
162     }
163 }

```

ledger.cpp 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量、权重/KV 字节和 bandwidth-only tokens/s 上限。

Listing 16.4 src/ledger.cpp

```

1  #include <cstdlib>
2  #include <cstdint>
3  #include <exception>
4  #include <iomanip>
5  #include <iostream>
6  #include <string_view>
7
8  namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20
21 struct DTypeLedger {
22     std::string_view name;
23     double weight_gb_per_token = 0.0;
24     double kv_element_bytes = 0.0;
25 };
26
27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28     {"fp16", 14.0, 2.0},
29     {"int8", 7.0, 2.0},
30     {"int4", 3.7, 2.0},
31 };

```



```

32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34     50.0,
35     100.0,
36     200.0,
37     600.0,
38     1000.0,
39     1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43     const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44     const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45     const double qkv = hidden * (hidden + 2.0 * kv_width);
46     const double output_projection = hidden * hidden;
47     const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48     return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52     return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53           LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57     return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58           LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62     const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63           kv_bytes_per_token_all_layers(element_bytes);
64     return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68     ↵ noexcept {
69     const double bytes = static_cast<double>(context_length) *
70           kv_bytes_per_token_all_layers(element_bytes);
71     return bytes / LCQI_BYTES_PER_MIB;
72 }
73
74 void print_model_rows() {
75     const double linear_macs = linear_macs_per_layer();
76     const double attention_macs = attention_macs_per_layer();
77     const double total_flops =
78         (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ↵
79         ↵ LCQI_FLOPS_PER_MAC;
80     std::cout << "section,item,value,unit,notes\n";
81     std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ↵
82         ↵ fixture\n";
83     std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";

```

```

81     std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA" <<
    << query_heads << "\n";
82     std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV" <<
    << kv_heads << "\n";
83     std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per" <<
    << head_dim << "\n";
84     std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs," <<
    << decode_token << "\n";
85     std::cout << "compute,attention_macs_per_layer," << attention_macs
86         << ",macs,context=4096\n";
87     std::cout << "compute,total_decode_flops_per_token," << total_flops
88         << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92     for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93         std::cout << "kv,fp16_capacity_context_" << context << ', '
94             << kv_capacity_mib(2.0, context)
95             << ",MiB,single session\n";
96     }
97 }
98
99 void print_dtype_rows() {
100     for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101         const double kv_read_gb = kv_read_gb_per_decode_token(dtype.
    << kv_element_bytes);
102         const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103         std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104             << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105         std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106             << kv_read_gb << ",GB/token,context=4096\n";
107         std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108             << total_gb << ",GB/token,weight plus KV\n";
109         for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110             std::cout << "bandwidth_limit," << dtype.name << "_at_"
111                 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112                 << bandwidth / total_gb
113                 << ",tokens/s,bandwidth-only upper bound\n";
114         }
115     }
116 }
117
118 } // namespace
119
120 int main() {
121     try {
122         std::cout << std::fixed << std::setprecision(3);
123         print_model_rows();
124         print_kv_rows();
125         print_dtype_rows();
126         return EXIT_SUCCESS;
127     } catch (const std::exception& error) {

```

```

128         std::cerr << "error: " << error.what() << '\n';
129         return EXIT_FAILURE;
130     }
131 }

```

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

Listing 16.5 `src/serving_probe.cpp`

```

1  #include <algorithm>
2  #include <cstdlib>
3  #include <cstdint>
4  #include <exception>
5  #include <cmath>
6  #include <iomanip>
7  #include <iostream>
8  #include <string_view>
9  #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29     std::string_view request_id;
30     std::int32_t prompt_tokens = 0;
31     std::int32_t max_new_tokens = 0;
32     bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36     std::string_view request_id;
37     std::int32_t accepted = 0;
38     std::string_view rejection_reason;
39     std::int32_t prompt_tokens = 0;
40     std::int32_t reserved_tokens = 0;
41     double kv_reserved_mib = 0.0;
42     double queue_wait_ms = 0.0;
43     double prefill_ms = 0.0;

```

```

44     double ttft_ms = 0.0;
45     double decode_step_p95_ms = 0.0;
46     double tpot_ms = 0.0;
47     std::int32_t completion_tokens = 0;
48     std::string_view finish_reason;
49     double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53     {"req_short", 32, 4, false},
54     {"req_rag", 512, 8, false},
55     {"req_cancel", 128, 16, true},
56     {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60     const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
        ↪ 2.0 *
61         LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62         LCQI_PROBE_KV_ELEMENT_BYTES;
63     return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67     if (values.empty()) {
68         return 0.0;
69     }
70     std::sort(values.begin(), values.end());
71     const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72     const std::size_t index =
73         std::min(max_index,
74             static_cast<std::size_t>(
75                 std::ceil(fraction * static_cast<double>(max_index))));
76     return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80     std::vector<ProbeResult> results;
81     double reserved_kv_mib = 0.0;
82     std::int32_t accepted_before = 0;
83     for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84         ProbeResult result;
85         result.request_id = request.request_id;
86         result.prompt_tokens = request.prompt_tokens;
87         result.reserved_tokens = request.prompt_tokens + request.max_new_tokens
        ↪ ;
88         result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
89         const double static_session_mib =
90             result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
91             LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92         if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB
        ↪ ) {

```

```

93         result.accepted = 0;
94         result.rejection_reason = "memory_budget_exceeded";
95         result.finish_reason = "rejected";
96         results.push_back(result);
97         continue;
98     }
99
100     reserved_kv_mib += result.kv_reserved_mib;
101     result.accepted = 1;
102     result.rejection_reason = "none";
103     result.queue_wait_ms = static_cast<double>(accepted_before) *
104                             LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105     result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106                             LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107     result.ttft_ms = result.queue_wait_ms + result.prefill_ms + ↵
↵ LCQI_PROBE_DECODE_STEP_MS;
108     result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
109                                 static_session_mib / 512.0;
110     result.tpot_ms = result.decode_step_p95_ms;
111     if (request.cancel_after_prefill) {
112         result.completion_tokens = 0;
113         result.finish_reason = "cancelled";
114         result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
115         reserved_kv_mib -= result.kv_reserved_mib;
116     } else {
117         result.completion_tokens = request.max_new_tokens;
118         result.finish_reason = "max_tokens";
119     }
120     ++accepted_before;
121     results.push_back(result);
122 }
123 return results;
124 }
125
126 void print_request_rows(const std::vector<ProbeResult>& results) {
127     std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,↵
↵ reserved_tokens,"
128                 "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,↵
↵ decode_step_p95_ms,"
129                 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,↵
↵ metric_name,"
130                 "metric_value\n";
131     for (const ProbeResult& result : results) {
132         std::cout << "request," << result.request_id << ',' <<
133                     << result.accepted << ',' <<
134                     << result.rejection_reason << ',' <<
135                     << result.prompt_tokens << ',' <<
136                     << result.reserved_tokens << ',' <<
137                     << result.kv_reserved_mib << ',' <<
138                     << result.queue_wait_ms << ',' <<
139                     << result.prefill_ms << ',' <<
140                     << result.ttft_ms << ',' <<

```

```

141         << result.decode_step_p95_ms << ', '
142         << result.tpot_ms << ', '
143         << result.completion_tokens << ', '
144         << result.finish_reason << ', '
145         << result.cancel_reclaim_ms << ",,\n";
146     }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150     std::vector<double> ttft_values;
151     std::vector<double> queue_values;
152     double accepted = 0.0;
153     double rejected = 0.0;
154     double cancelled = 0.0;
155     double reserved_peak_mib = 0.0;
156     double running_reserved_mib = 0.0;
157     for (const ProbeResult& result : results) {
158         if (result.accepted == 0) {
159             rejected += 1.0;
160             continue;
161         }
162         accepted += 1.0;
163         ttft_values.push_back(result.ttft_ms);
164         queue_values.push_back(result.queue_wait_ms);
165         running_reserved_mib += result.kv_reserved_mib;
166         reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167         if (result.finish_reason == "cancelled") {
168             cancelled += 1.0;
169             running_reserved_mib -= result.kv_reserved_mib;
170         }
171     }
172     const double total = accepted + rejected;
173     const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
175         << "accepted_count," << accepted << '\n';
176     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,"
177         << LCQI_PROBE_CANCEL_RECLAIM_MS << "cancel_count," << cancelled <<
178         << '\n';
179     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
180         << "rejected_count," << rejected << '\n';
181     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
182         << "rejection_rate," << rejection_rate << '\n';
183     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
184         << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
185     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
186         << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n' <<
187         ;
188     std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
189         << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
190 }
191 } // namespace

```

```

191
192 int main() {
193     try {
194         std::cout << std::fixed << std::setprecision(3);
195         const std::vector<ProbeResult> results = run_probe();
196         print_request_rows(results);
197         print_summary_row(results);
198         return EXIT_SUCCESS;
199     } catch (const std::exception& error) {
200         std::cerr << "error: " << error.what() << '\n';
201         return EXIT_FAILURE;
202     }
203 }

```

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing 16.6 src/onednn_bench.cpp

```

1 #include <dnnl.hpp>
2
3 #include <chrono>
4 #include <cstdint>
5 #include <cstdlib>
6 #include <iostream>
7 #include <numeric>
8 #include <stdexcept>
9 #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;
18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {
30     if (argc <= LCQI_REPEAT_ARG_INDEX) {
31         return LCQI_ONEDNN_DEFAULT_REPEAT;
32     }
33     const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));

```

```

34     if (repeat <= 0) {
35         throw std::runtime_error("repeat must be positive");
36     }
37     return repeat;
38 }
39
40 std::vector<float> make_input() {
41     std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42     for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43         input[static_cast<std::size_t>(index)] =
44             static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER) *
45             LCQI_INPUT_SCALE;
46     }
47     return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51     std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * LCQI_ONEDNN_0));
52     for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
53         for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
54             const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
55             const float value =
56                 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
57                                     LCQI_WEIGHT_CENTER) *
58                 LCQI_WEIGHT_SCALE;
59             weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
60         }
61     }
62     return weights;
63 }
64
65 float checksum(const std::vector<float>& values) {
66     return std::accumulate(values.begin(), values.end(), 0.0F);
67 }
68
69 } // namespace
70
71 int main(int argc, char** argv) {
72     try {
73         const std::int32_t repeat = parse_repeat(argc, argv);
74         std::vector<float> input = make_input();
75         std::vector<float> weights = make_weights_kn();
76         std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * LCQI_ONEDNN_0),
77                                     0.0F);
78
79         dnnl::engine engine(dnnl::engine::kind::cpu, 0);
80         dnnl::stream stream(engine);
81
82         const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, LCQI_ONEDNN_0};

```



```

↪ LCQI_ONEDNN_K};
83     const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_0};
84     const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↪
↪ LCQI_ONEDNN_0};

85
86     const dnnl::memory::desc source_desc(
87         source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
88     const dnnl::memory::desc weight_desc(
89         weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
90     const dnnl::memory::desc output_desc(
91         output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↪
↪ ::ab);
92
93     dnnl::memory source_memory(source_desc, engine, input.data());
94     dnnl::memory weight_memory(weight_desc, engine, weights.data());
95     dnnl::memory output_memory(output_desc, engine, output.data());
96
97     dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↪
↪ weight_desc, output_desc);
98     dnnl::matmul matmul(matmul_desc);
99     const std::unordered_map<int, dnnl::memory> arguments = {
100         {DNNL_ARG_SRC, source_memory},
101         {DNNL_ARG_WEIGHTS, weight_memory},
102         {DNNL_ARG_DST, output_memory},
103     };
104
105     for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106         matmul.execute(stream, arguments);
107         stream.wait();
108     }
109
110     const auto begin = std::chrono::steady_clock::now();
111     for (std::int32_t index = 0; index < repeat; ++index) {
112         matmul.execute(stream, arguments);
113         stream.wait();
114     }
115     const auto end = std::chrono::steady_clock::now();
116     const std::chrono::duration<double> elapsed = end - begin;
117     const double average_us =
118         elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>↪
↪ >(repeat);
119
120     std::cout << "library,input_size,output_size,dtype,repeat,average_us,↪
↪ checksum\n";
121     std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0
122         << ",f32," << repeat << ',' << average_us << ','
123         << checksum(output) << '\n';
124     return EXIT_SUCCESS;
125 } catch (const std::exception& error) {
126     std::cerr << "error: " << error.what() << '\n';

```

```

127         return EXIT_FAILURE;
128     }
129 }

```

外部模型 Smoke 与对比脚本

`run_smollm2_small_smoke.py` 下载并校验 SmolLM2-135M-Instruct 的 GGUF 量化文件，构建固定 commit 的 `llama-simple-chat`，在 CPU 上运行一次真实开源 small 模型生成，并输出可复查报告。它是外部真实模型 smoke，不替代 LCQI tiny golden trace。

Listing 16.7 `tools/run_smollm2_small_smoke.py`

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import sys
13 from dataclasses import dataclass
14 from pathlib import Path
15
16
17 MODEL_SOURCE_ID = "HuggingFaceTB/SmolLM2-135M-Instruct"
18 MODEL_GGUF_REPO_ID = "bartowski/SmolLM2-135M-Instruct-GGUF"
19 MODEL_FILE_NAME = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
20 MODEL_URL = (
21     "https://huggingface.co/bartowski/SmolLM2-135M-Instruct-GGUF"
22     f"/resolve/main/{MODEL_FILE_NAME}"
23 )
24 MODEL_EXPECTED_BYTES = 105454432
25 MODEL_EXPECTED_SHA256 = (
26     "2e8040ceae7815abe0dcb3540b9995eaa1fa0d2ca9e797d0a635ae4433c68c2d"
27 )
28 LLAMA_CPP_REPO_URL = "https://github.com/ggml-org/llama.cpp.git"
29 LLAMA_CPP_COMMIT = "b3fed31b99f9bd37725833674252bccb429bb183"
30 LLAMA_TARGET = "llama-simple-chat"
31 DEFAULT_PROMPT = "What is 2+3? Answer in one short sentence."
32 DEFAULT_CONTEXT_TOKENS = 512
33 DEFAULT_TIMEOUT_SECONDS = 180
34 DEFAULT_BUILD_JOBS = 2
35
36
37 @dataclass(frozen=True)
38 class CommandResult:
39     args: list[str]
40     returncode: int

```

```

41     stdout: str
42     stderr: str
43
44
45 def script_path() -> Path:
46     return Path(__file__).resolve()
47
48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def repo_dir() -> Path:
54     return script_path().parents[3]
55
56
57 def default_cache_dir() -> Path:
58     explicit_cache = os.environ.get("LCQI_SMOLLM2_CACHE_DIR")
59     if explicit_cache:
60         return Path(explicit_cache).expanduser()
61     xdg_cache = os.environ.get("XDG_CACHE_HOME")
62     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↵
↵ .cache"
63     return cache_root / "lcqi-smolllm2-small-smoke"
64
65
66 def default_report_path() -> Path:
67     return volume_dir() / "results" / "lcqi-smolllm2-small-model-smoke.txt"
68
69
70 def command_line(args: list[str]) -> str:
71     return " ".join(shlex.quote(arg) for arg in args)
72
73
74 def require_tool(name: str) -> str:
75     path = shutil.which(name)
76     if path is None:
77         raise RuntimeError(f"required tool not found in PATH: {name}")
78     return path
79
80
81 def run_command(
82     args: list[str],
83     *,
84     cwd: Path | None = None,
85     input_text: str | None = None,
86     timeout_seconds: int | None = None,
87 ) -> CommandResult:
88     try:
89         completed = subprocess.run(
90             args,
91             cwd=str(cwd) if cwd else None,

```

```

92         input=input_text,
93         text=True,
94         capture_output=True,
95         timeout=timeout_seconds,
96         check=False,
97     )
98 except subprocess.TimeoutExpired as exc:
99     stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100     stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101     return CommandResult(args=args, returncode=124, stdout=stdout, stderr=↵
↵ stderr)
102
103 return CommandResult(
104     args=args,
105     returncode=completed.returncode,
106     stdout=completed.stdout,
107     stderr=completed.stderr,
108 )
109
110
111 def ensure_success(result: CommandResult, action: str) -> None:
112     if result.returncode == 0:
113         return
114     raise RuntimeError(
115         f"{action} failed with exit code {result.returncode}\n"
116         f"command: {command_line(result.args)}\n"
117         f"stdout tail:\n{tail(result.stdout)}\n"
118         f"stderr tail:\n{tail(result.stderr)}"
119     )
120
121
122 def tail(text: str, max_chars: int = 4000) -> str:
123     if len(text) <= max_chars:
124         return text
125     return text[-max_chars:]
126
127
128 def sha256_file(path: Path) -> str:
129     digest = hashlib.sha256()
130     with path.open("rb") as handle:
131         for block in iter(lambda: handle.read(1024 * 1024), b''):
132             digest.update(block)
133     return digest.hexdigest()
134
135
136 def verify_model_file(path: Path) -> tuple[bool, str, int]:
137     if not path.exists():
138         return False, "", 0
139     actual_bytes = path.stat().st_size
140     actual_sha = sha256_file(path)
141     return (
142         actual_bytes == MODEL_EXPECTED_BYTES

```

```

143         and actual_sha == MODEL_EXPECTED_SHA256,
144         actual_sha,
145         actual_bytes,
146     )
147
148
149 def download_model(model_path: Path, *, force_download: bool, no_download: bool ↵
    ↵ ) -> None:
150     model_path.parent.mkdir(parents=True, exist_ok=True)
151
152     if force_download and model_path.exists():
153         model_path.unlink()
154
155     is_valid, actual_sha, actual_bytes = verify_model_file(model_path)
156     if is_valid:
157         print(f"[lcqi-smoke] model cache hit: {model_path}")
158         return
159
160     if model_path.exists():
161         print(
162             "[lcqi-smoke] cached model hash mismatch, redownloading: "
163             f"bytes={actual_bytes}, sha256={actual_sha}"
164         )
165         model_path.unlink()
166
167     if no_download:
168         raise RuntimeError(
169             f"model file is missing or invalid and --no-download was set: {↵
    ↵ model_path}"
170         )
171
172     require_tool("curl")
173     part_path = model_path.with_suffix(model_path.suffix + ".part")
174     if part_path.exists():
175         part_path.unlink()
176
177     print(f"[lcqi-smoke] downloading small GGUF model: {MODEL_GGUF_REPO_ID}/{↵
    ↵ MODEL_FILE_NAME}")
178     result = run_command(
179         [
180             "curl",
181             "-L",
182             "--fail",
183             "--retry",
184             "3",
185             "-o",
186             str(part_path),
187             MODEL_URL,
188         ]
189     )
190     ensure_success(result, "download SmolLM2 GGUF")
191

```

```

192     downloaded_ok, downloaded_sha, downloaded_bytes = verify_model_file(↵
↵     part_path)
193     if not downloaded_ok:
194         raise RuntimeError(
195             "downloaded model did not match expected identity: "
196             f"bytes={downloaded_bytes}, sha256={downloaded_sha}"
197         )
198     part_path.replace(model_path)
199     print(f"[lcqi-smoke] model verified: sha256={MODEL_EXPECTED_SHA256}")
200
201
202 def ensure_llama_simple_chat(cache_dir: Path, *, jobs: int, skip_build: bool) ↵
↵     -> Path:
203     cached_binary = cache_dir / "llama.cpp" / "build" / "bin" / LLAMA_TARGET
204     source_dir = cache_dir / "llama.cpp"
205     build_dir = source_dir / "build"
206     if cached_binary.exists() and os.access(cached_binary, os.X_OK) and ↵
↵     is_pinned_llama_checkout(source_dir):
207         print(f"[lcqi-smoke] llama.cpp binary cache hit: {cached_binary}")
208         return cached_binary
209
210     if skip_build:
211         raise RuntimeError(f"llama.cpp binary is missing and --skip-build was ↵
↵     set: {cached_binary}")
212
213     require_tool("git")
214     require_tool("cmake")
215
216     cache_dir.mkdir(parents=True, exist_ok=True)
217
218     if not (source_dir / ".git").exists():
219         print(f"[lcqi-smoke] cloning llama.cpp into cache: {source_dir}")
220         result = run_command(
221             [
222                 "git",
223                 "clone",
224                 "--filter=blob:none",
225                 "--no-checkout",
226                 LLAMA_CPP_REPO_URL,
227                 str(source_dir),
228             ]
229         )
230         ensure_success(result, "clone llama.cpp")
231
232     print(f"[lcqi-smoke] checking out pinned llama.cpp commit: {↵
↵     LLAMA_CPP_COMMIT}")
233     fetch = run_command(
234         ["git", "-C", str(source_dir), "fetch", "--depth", "1", "origin", ↵
↵     LLAMA_CPP_COMMIT]
235     )
236     ensure_success(fetch, "fetch pinned llama.cpp commit")
237     checkout = run_command(["git", "-C", str(source_dir), "checkout", "--detach↵

```

```

238     ↪ ", LLAMA_CPP_COMMIT])
239     ensure_success(checkout, "checkout pinned llama.cpp commit")
240
241     print(f"[lcqi-smoke] configuring llama.cpp build: {build_dir}")
242     configure = run_command(
243         [
244             "cmake",
245             "-S",
246             str(source_dir),
247             "-B",
248             str(build_dir),
249             "-DCMAKE_BUILD_TYPE=Release",
250             "-DLLAMA_BUILD_TESTS=OFF",
251             "-DLLAMA_BUILD_EXAMPLES=ON",
252             "-DLLAMA_BUILD_SERVER=OFF",
253             "-DGGML_NATIVE=OFF",
254         ]
255     )
256     ensure_success(configure, "configure llama.cpp")
257
258     print(f"[lcqi-smoke] building {LLAMA_TARGET} with -j{jobs}")
259     build = run_command(
260         ["cmake", "--build", str(build_dir), "--target", LLAMA_TARGET, "-j", ↪
261         ↪ str(jobs)]
262     )
263     ensure_success(build, f"build {LLAMA_TARGET}")
264
265     if not cached_binary.exists():
266         raise RuntimeError(f"expected binary was not produced: {cached_binary}" ↪
267         ↪ )
268     return cached_binary
269
270
271 def is_pinned_llama_checkout(source_dir: Path) -> bool:
272     if not (source_dir / ".git").exists():
273         return False
274     result = run_command(["git", "-C", str(source_dir), "rev-parse", "HEAD"])
275     if result.returncode != 0:
276         return False
277     return result.stdout.strip() == LLAMA_CPP_COMMIT
278
279
280 def strip_ansi(text: str) -> str:
281     return re.sub(r"\x1b\[([0-?]*[ -/]*[@-~])", "", text)
282
283
284 def extract_response(stdout: str) -> str:
285     cleaned = strip_ansi(stdout).replace("\r", "")
286     chunks = [chunk.strip() for chunk in re.split(r"(?:^|\n)> ?", cleaned) if ↪
287     ↪ chunk.strip()]
288     if not chunks:
289         return ""

```

```

286     return chunks[0]
287
288
289 def run_model_smoke(
290     llama_binary: Path,
291     model_path: Path,
292     *,
293     prompt: str,
294     context_tokens: int,
295     timeout_seconds: int,
296 ) -> tuple[CommandResult, str]:
297     command = [
298         str(llvm_binary),
299         "-m",
300         str(model_path),
301         "-ngl",
302         "0",
303         "-c",
304         str(context_tokens),
305     ]
306     print("[lcqi-smoke] running real small model generation")
307     result = run_command(
308         command,
309         input_text=prompt.rstrip("\n") + "\n\n",
310         timeout_seconds=timeout_seconds,
311     )
312     ensure_success(result, "run small model smoke")
313
314     response = extract_response(result.stdout)
315     if not response:
316         raise RuntimeError(
317             "small model smoke exited successfully but produced no assistant ←
↵ text\n"
318             f"stdout:\n{result.stdout}\n"
319             f"stderr:\n{result.stderr}"
320         )
321     return result, response
322
323
324 def current_repo_commit() -> str:
325     result = run_command(["git", "-C", str(repo_dir()), "rev-parse", "--short",
↵ "HEAD"])
326     if result.returncode != 0:
327         return "unknown"
328     return result.stdout.strip()
329
330
331 def current_uname() -> str:
332     result = run_command(["uname", "-a"])
333     if result.returncode != 0:
334         return "unknown"
335     return result.stdout.strip()

```



```

336
337
338 def write_report(
339     report_path: Path,
340     *,
341     cache_dir: Path,
342     llama_binary: Path,
343     model_path: Path,
344     prompt: str,
345     result: CommandResult,
346     response: str,
347 ) -> None:
348     report_path.parent.mkdir(parents=True, exist_ok=True)
349     clean_stdout = strip_ansi(result.stdout).strip()
350     report = "\n".join(
351         [
352             "LCQI SmolLM2 Small Model Smoke",
353             "",
354             f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
355             f"repo_commit={current_repo_commit()}",
356             f"host={current_uname()}",
357             f"python={sys.version.split()[0]}",
358             f"cache_dir={cache_dir}",
359             "",
360             f"model_source_id={MODEL_SOURCE_ID}",
361             f"model_gguf_repo_id={MODEL_GGUF_REPO_ID}",
362             f"model_file={MODEL_FILE_NAME}",
363             f"model_url={MODEL_URL}",
364             f"model_expected_bytes={MODEL_EXPECTED_BYTES}",
365             f"model_expected_sha256={MODEL_EXPECTED_SHA256}",
366             f"model_path={model_path}",
367             "",
368             f"llama_cpp_repo={LLAMA_CPP_REPO_URL}",
369             f"llama_cpp_commit={LLAMA_CPP_COMMIT}",
370             f"llama_target={LLAMA_TARGET}",
371             f"llama_binary={llama_binary}",
372             "",
373             f"prompt={prompt}",
374             f"command={command_line(result.args)}",
375             f"exit_code={result.returncode}",
376             "",
377             "validation=PASS: model hash matched, llama-simple-chat exited 0, "
378             "assistant response was non-empty",
379             "",
380             "assistant_response:",
381             response,
382             "",
383             "stdout_clean_tail:",
384             tail(clean_stdout, 2000),
385             "",
386             "stderr_tail:",
387             tail(result.stderr.strip(), 2000),

```

```

388         "",
389     ]
390 )
391 report_path.write_text(report, encoding="utf-8")
392
393
394 def parse_args() -> argparse.Namespace:
395     parser = argparse.ArgumentParser(
396         description=(
397             "Run a real local small open-source model smoke for CPU Volume 3. "
398             "The script caches llama.cpp and the GGUF model outside the Git ↵
↵ repository."
399         )
400     )
401     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
402     parser.add_argument("--model-path", type=Path, default=None)
403     parser.add_argument("--llama-bin", type=Path, default=None)
404     parser.add_argument("--report", type=Path, default=default_report_path())
405     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
406     parser.add_argument("--ctx", type=int, default=DEFAULT_CONTEXT_TOKENS)
407     parser.add_argument("--timeout", type=int, default=DEFAULT_TIMEOUT_SECONDS)
408     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
409     parser.add_argument("--force-download", action="store_true")
410     parser.add_argument("--no-download", action="store_true")
411     parser.add_argument("--skip-build", action="store_true")
412     return parser.parse_args()
413
414
415 def main() -> int:
416     args = parse_args()
417     cache_dir = args.cache_dir.expanduser().resolve()
418     model_path = (
419         args.model_path.expanduser().resolve()
420         if args.model_path
421         else cache_dir / "models" / MODEL_FILE_NAME
422     )
423     report_path = args.report.expanduser().resolve()
424
425     try:
426         download_model(
427             model_path,
428             force_download=args.force_download,
429             no_download=args.no_download,
430         )
431         if args.llama_bin:
432             llama_binary = args.llama_bin.expanduser().resolve()
433             if not llama_binary.exists() or not os.access(llama_binary, os.X_OK)↵
↵ ):
434                 raise RuntimeError(f"--llama-bin is not executable: {↵
↵ llama_binary}")
435         else:
436             llama_binary = ensure_llama_simple_chat(

```

```

437         cache_dir,
438         jobs=args.jobs,
439         skip_build=args.skip_build,
440     )
441
442     result, response = run_model_smoke(
443         llama_binary,
444         model_path,
445         prompt=args.prompt,
446         context_tokens=args.ctx,
447         timeout_seconds=args.timeout,
448     )
449     write_report(
450         report_path,
451         cache_dir=cache_dir,
452         llama_binary=llama_binary,
453         model_path=model_path,
454         prompt=args.prompt,
455         result=result,
456         response=response,
457     )
458     except RuntimeError as exc:
459         print(f"[lcqi-smoke] ERROR: {exc}", file=sys.stderr)
460         return 1
461
462     print(f"report={report_path}")
463     print(f"model_sha256={MODEL_EXPECTED_SHA256}")
464     print(f"assistant_response={response}")
465     return 0
466
467
468 if __name__ == "__main__":
469     raise SystemExit(main())

```

`run_gpt2_smoke.py` 下载标准 GPT-2 的 `config.json`、`vocab.json`、`merges.txt` 和 `model.safetensors`，再调用 LCQI 自研 `lcqi_gpt2` reference path 生成 token。它证明自研 `safetensors`、BPE、GPT-2 forward、KV cache decode 和 greedy generation 已连成闭环。

Listing 16.8 tools/run_gpt2_smoke.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import hashlib
7  import os
8  import shlex
9  import subprocess
10 import sys
11 import urllib.request
12 from dataclasses import dataclass
13 from pathlib import Path

```

```

14
15
16 MODEL_REPO_ID = "openai-community/gpt2"
17 MODEL_REVISION = "main"
18 MODEL_FILE_IDENTITIES = {
19     "config.json": (
20         665,
21         "0daed7749b4f02b8f76240d5444551d7b08712dab4d0adb8239c56ba823bb7b4",
22     ),
23     "vocab.json": (
24         1042301,
25         "196139668be63f3b5d6574427317ae82f612a97c5d1cdaf36ed2256dbf636783",
26     ),
27     "merges.txt": (
28         456318,
29         "1ce1664773c50f3e0cc8842619a93edc4624525b728b188a9e0be33b7726adc5",
30     ),
31     "model.safetensors": (
32         548105171,
33         "248dfc3911869ec493c76e65bf2fcf7f615828b0254c12b473182f0f81d3a707",
34     ),
35 }
36 DEFAULT_PROMPT = "Hello, my name is"
37 DEFAULT_MAX_NEW_TOKENS = 1
38 DEFAULT_TIMEOUT_SECONDS = 300
39 DEFAULT_BUILD_JOBS = 2
40
41
42 @dataclass(frozen=True)
43 class CommandResult:
44     args: list[str]
45     returncode: int
46     stdout: str
47     stderr: str
48
49
50 def script_path() -> Path:
51     return Path(__file__).resolve()
52
53
54 def volume_dir() -> Path:
55     return script_path().parents[1]
56
57
58 def repo_dir() -> Path:
59     return script_path().parents[3]
60
61
62 def default_cache_dir() -> Path:
63     explicit_cache = os.environ.get("LCQI_GPT2_CACHE_DIR")
64     if explicit_cache:
65         return Path(explicit_cache).expanduser()

```

```

66     xdg_cache = os.environ.get("XDG_CACHE_HOME")
67     cache_root = Path(xdg_cache).expanduser() if xdg_cache else Path.home() / "↵
↵ .cache"
68     return cache_root / "lcqi-gpt2-smoke" / MODEL_REPO_ID.replace("/", "--")
69
70
71 def default_build_dir() -> Path:
72     return volume_dir() / "build" / "lcqi-release"
73
74
75 def default_report_path() -> Path:
76     return volume_dir() / "results" / "lcqi-gpt2-smoke.txt"
77
78
79 def command_line(args: list[str]) -> str:
80     return " ".join(shlex.quote(arg) for arg in args)
81
82
83 def run_command(
84     args: list[str],
85     *,
86     cwd: Path | None = None,
87     timeout_seconds: int | None = None,
88 ) -> CommandResult:
89     try:
90         completed = subprocess.run(
91             args,
92             cwd=str(cwd) if cwd else None,
93             text=True,
94             capture_output=True,
95             timeout=timeout_seconds,
96             check=False,
97         )
98     except subprocess.TimeoutExpired as exc:
99         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
100         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
101         return CommandResult(args=args, returncode=124, stdout=stdout, stderr=↵
↵ stderr)
102     return CommandResult(
103         args=args,
104         returncode=completed.returncode,
105         stdout=completed.stdout,
106         stderr=completed.stderr,
107     )
108
109
110 def ensure_success(result: CommandResult, action: str) -> None:
111     if result.returncode == 0:
112         return
113     raise RuntimeError(
114         f"{action} failed with exit code {result.returncode}\n"
115         f"command: {command_line(result.args)}\n"

```

```

116         f"stdout tail:\n{tail(result.stdout)}\n"
117         f"stderr tail:\n{tail(result.stderr)}"
118     )
119
120
121 def tail(text: str, max_chars: int = 4000) -> str:
122     if len(text) <= max_chars:
123         return text
124     return text[-max_chars:]
125
126
127 def sha256_file(path: Path) -> str:
128     digest = hashlib.sha256()
129     with path.open("rb") as handle:
130         for block in iter(lambda: handle.read(1024 * 1024), b''):
131             digest.update(block)
132     return digest.hexdigest()
133
134
135 def model_url(file_name: str) -> str:
136     return (
137         f"https://huggingface.co/{MODEL_REPO_ID}/resolve/"
138         f"{MODEL_REVISION}/{file_name}"
139     )
140
141
142 def download_file(url: str, target: Path, timeout_seconds: int) -> None:
143     part_path = target.with_suffix(target.suffix + ".part")
144     if part_path.exists():
145         part_path.unlink()
146     request = urllib.request.Request(url, headers={"User-Agent": "lcqi-gpt2-↵
↵ smoke"})
147     with urllib.request.urlopen(request, timeout=timeout_seconds) as response:
148         with part_path.open("wb") as output:
149             while True:
150                 block = response.read(1024 * 1024)
151                 if not block:
152                     break
153                 output.write(block)
154     part_path.replace(target)
155
156
157 def file_identity_matches(path: Path, expected_bytes: int, expected_sha256: str↵
↵ ) -> bool:
158     return (
159         path.exists()
160         and path.stat().st_size == expected_bytes
161         and sha256_file(path) == expected_sha256
162     )
163
164
165 def ensure_model_files(cache_dir: Path, *, no_download: bool, timeout_seconds: ↵

```

```

↪ int) -> None:
166 cache_dir.mkdir(parents=True, exist_ok=True)
167 missing = [
168     name
169     for name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.↪
↪ items()
170     if not file_identity_matches(cache_dir / name, expected_bytes, ↪
↪ expected_sha256)
171 ]
172 if not missing:
173     print(f"[lcqi-gpt2] model cache hit: {cache_dir}")
174     return
175 if no_download:
176     raise RuntimeError(
177         f"missing GPT-2 model files and --no-download was set: {missing}"
178     )
179 for file_name in missing:
180     target = cache_dir / file_name
181     if target.exists():
182         target.unlink()
183     print(f"[lcqi-gpt2] downloading {MODEL_REPO_ID}/{file_name}")
184     download_file(model_url(file_name), target, timeout_seconds)
185     expected_bytes, expected_sha256 = MODEL_FILE_IDENTITIES[file_name]
186     if not file_identity_matches(target, expected_bytes, expected_sha256):
187         raise RuntimeError(
188             f"downloaded GPT-2 file identity mismatch: {file_name}"
189         )
190
191
192 def build_lcqi_gpt2(build_dir: Path, jobs: int) -> Path:
193     configure = run_command(
194         [
195             "cmake",
196             "-S",
197             str(volume_dir()),
198             "-B",
199             str(build_dir),
200             "-DCMAKE_BUILD_TYPE=Release",
201         ],
202         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
203     )
204     ensure_success(configure, "configure LCQI")
205     build = run_command(
206         [
207             "cmake",
208             "--build",
209             str(build_dir),
210             "--target",
211             "lcqi_gpt2",
212             "-j",
213             str(jobs),
214         ],

```

```

215         timeout_seconds=DEFAULT_TIMEOUT_SECONDS,
216     )
217     ensure_success(build, "build lcqi_gpt2")
218     binary = build_dir / "labs" / "linux_cpu_inference" / "lcqi_gpt2"
219     if not binary.exists():
220         raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
221     return binary
222
223
224 def write_report(
225     report_path: Path,
226     *,
227     cache_dir: Path,
228     binary: Path,
229     prompt: str,
230     max_new_tokens: int,
231     engine: str,
232     result: CommandResult,
233 ) -> None:
234     report_path.parent.mkdir(parents=True, exist_ok=True)
235     lines = [
236         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
237         f"model_repo_id={MODEL_REPO_ID}",
238         f"model_revision={MODEL_REVISION}",
239         f"model_cache_dir={cache_dir}",
240     ]
241     for file_name, (expected_bytes, expected_sha256) in MODEL_FILE_IDENTITIES.items():
242         path = cache_dir / file_name
243         lines.append(f"file.{file_name}.bytes={path.stat().st_size}")
244         lines.append(f"file.{file_name}.sha256={sha256_file(path)}")
245         lines.append(f"file.{file_name}.expected_bytes={expected_bytes}")
246         lines.append(f"file.{file_name}.expected_sha256={expected_sha256}")
247     lines.extend(
248         [
249             f"binary={binary}",
250             f"prompt={prompt}",
251             f"max_new_tokens={max_new_tokens}",
252             f"engine={engine}",
253             f"command={command_line(result.args)}",
254             f"exit_code={result.returncode}",
255             "stdout_begin",
256             result.stdout.rstrip(),
257             "stdout_end",
258             "stderr_begin",
259             tail(result.stderr).rstrip(),
260             "stderr_end",
261         ]
262     )
263     passed = result.returncode == 0 and "generated_text " in result.stdout
264     lines.append(f"validation={'PASS' if passed else 'FAIL'}")
265     report_path.write_text("\n".join(lines) + "\n", encoding="utf-8")

```



```

266
267
268 def parse_args() -> argparse.Namespace:
269     parser = argparse.ArgumentParser(
270         description="Run LCQI self-owned GPT-2 smoke on a HuggingFace ↵
↵ safetensors directory."
271     )
272     parser.add_argument("--cache-dir", type=Path, default=default_cache_dir())
273     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
274     parser.add_argument("--report", type=Path, default=default_report_path())
275     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
276     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
277     parser.add_argument("--engine", choices=("cached", "full"), default="cached↵
↵ ")
278     parser.add_argument("--benchmark", action="store_true")
279     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
280     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
281     parser.add_argument("--no-download", action="store_true")
282     return parser.parse_args()
283
284
285 def main() -> int:
286     args = parse_args()
287     try:
288         if args.max_new_tokens < 0:
289             raise RuntimeError("--max-new-tokens cannot be negative")
290         ensure_model_files(
291             args.cache_dir,
292             no_download=args.no_download,
293             timeout_seconds=args.timeout_seconds,
294         )
295         binary = build_lcqi_gpt2(args.build_dir, args.jobs)
296         command = [
297             str(binary),
298             "--engine",
299             args.engine,
300             str(args.cache_dir),
301             args.prompt,
302             str(args.max_new_tokens),
303         ]
304         if args.benchmark:
305             command.insert(1, "--benchmark")
306         result = run_command(command, cwd=repo_dir(), timeout_seconds=args.↵
↵ timeout_seconds)
307         write_report(
308             args.report,
309             cache_dir=args.cache_dir,
310             binary=binary,
311             prompt=args.prompt,
312             max_new_tokens=args.max_new_tokens,

```

```

313         engine=args.engine,
314         result=result,
315     )
316     print(f"[lcqi-gpt2] report: {args.report}")
317     print(tail(result.stdout, max_chars=2000))
318     return 0 if result.returncode == 0 else result.returncode
319 except Exception as error:
320     print(f"[lcqi-gpt2] error: {error}", file=sys.stderr)
321     return 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_benchmark_compare.py` 用同一个 GPT-2 F32 safetensors 模型分别运行 LCQI full-prefix 与 cached-KV 路径，并在本机存在 llama.cpp/SmolLM2 缓存时附带成熟引擎参考。它的报告说明算法差距、kernel 差距和模型口径差异。

Listing 16.9 tools/run_gpt2_benchmark_compare.py

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import os
7  import re
8  import shlex
9  import subprocess
10 import sys
11 from dataclasses import dataclass
12 from pathlib import Path
13
14 import run_gpt2_smoke
15
16
17 DEFAULT_PROMPT = "Hello, my name is"
18 DEFAULT_MAX_NEW_TOKENS = 4
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21 DEFAULT_SMOLLM2_CACHE_DIR = Path.home() / ".cache" / "lcqi-smolllm2-small-smoke"
22 SMOLLM2_GGUF = "SmolLM2-135M-Instruct-Q4_K_M.gguf"
23
24
25 @dataclass(frozen=True)
26 class EngineRun:
27     name: str
28     command: list[str]
29     returncode: int
30     stdout: str
31     stderr: str
32     metrics: dict[str, str]
33

```

```

34
35 def volume_dir() -> Path:
36     return Path(__file__).resolve().parents[1]
37
38
39 def repo_dir() -> Path:
40     return Path(__file__).resolve().parents[3]
41
42
43 def default_report_path() -> Path:
44     return volume_dir() / "results" / "lcqi-gpt2-benchmark-compare.txt"
45
46
47 def default_build_dir() -> Path:
48     return volume_dir() / "build" / "lcqi-release"
49
50
51 def command_line(args: list[str]) -> str:
52     return " ".join(shlex.quote(arg) for arg in args)
53
54
55 def tail(text: str, max_chars: int = 4000) -> str:
56     return text if len(text) <= max_chars else text[-max_chars:]
57
58
59 def run_command(args: list[str], *, timeout_seconds: int) -> tuple[int, str, ←
    ↪ str]:
60     try:
61         completed = subprocess.run(
62             args,
63             cwd=repo_dir(),
64             text=True,
65             capture_output=True,
66             timeout=timeout_seconds,
67             check=False,
68         )
69     except subprocess.TimeoutExpired as exc:
70         stdout = exc.stdout if isinstance(exc.stdout, str) else ""
71         stderr = exc.stderr if isinstance(exc.stderr, str) else ""
72         return 124, stdout, stderr
73     return completed.returncode, completed.stdout, completed.stderr
74
75
76 def parse_lcqi_metrics(stdout: str) -> dict[str, str]:
77     metrics: dict[str, str] = {}
78     for line in stdout.splitlines():
79         if not line:
80             continue
81         key, _, value = line.partition(" ")
82         if key in {
83             "engine",
84             "generated_text",

```

```

85         "benchmark_load_ms",
86         "benchmark_tokenize_ms",
87         "benchmark_prefill_ms",
88         "benchmark_decode_ms",
89         "benchmark_generate_ms",
90         "benchmark_total_ms",
91         "benchmark_prompt_tokens",
92         "benchmark_generated_tokens",
93         "benchmark_prefill_steps",
94         "benchmark_decode_steps",
95         "benchmark_generate_tokens_per_second",
96         "benchmark_decode_tokens_per_second",
97         "benchmark_kv_cache_bytes",
98     }:
99         metrics[key] = value
100     return metrics
101
102
103 def run_lcqi(
104     binary: Path,
105     model_dir: Path,
106     *,
107     engine: str,
108     prompt: str,
109     max_new_tokens: int,
110     timeout_seconds: int,
111 ) -> EngineRun:
112     command = [
113         str(binary),
114         "--benchmark",
115         "--engine",
116         engine,
117         str(model_dir),
118         prompt,
119         str(max_new_tokens),
120     ]
121     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)
122     return EngineRun(
123         name=f"lcqi_{engine}",
124         command=command,
125         returncode=returncode,
126         stdout=stdout,
127         stderr=stderr,
128         metrics=parse_lcqi_metrics(stdout),
129     )
130
131
132 def llama_bench_binary(cache_dir: Path) -> Path:
133     return cache_dir / "llama.cpp" / "build" / "bin" / "llama-bench"
134
135

```

```

136 def smollm2_model_path(cache_dir: Path) -> Path:
137     return cache_dir / "models" / SMOLLM2_GGUF
138
139
140 def parse_llama_bench_metrics(stdout: str) -> dict[str, str]:
141     metrics: dict[str, str] = {}
142     lines = [line.strip() for line in stdout.splitlines() if line.strip()]
143     for line in lines:
144         if not line.startswith("|"):
145             continue
146         columns = [column.strip() for column in line.strip("|").split("|")]
147         if len(columns) != 7 or columns[0] == "model" or columns[0].startswith(↵
↵ "-"):
148             continue
149         metrics["llama_model"] = columns[0]
150         metrics["llama_size"] = columns[1]
151         metrics["llama_params"] = columns[2]
152         metrics["llama_backend"] = columns[3]
153         metrics["llama_threads"] = columns[4]
154         test_name = columns[5]
155         tokens_per_second = re.sub(r"\s+.*$", "", columns[6])
156         if test_name.startswith("pp"):
157             metrics["llama_prefill_test"] = test_name
158             metrics["llama_prefill_tps"] = tokens_per_second
159         elif test_name.startswith("tg"):
160             metrics["llama_decode_test"] = test_name
161             metrics["llama_decode_tps"] = tokens_per_second
162     return metrics
163
164
165 def run_llama_bench(cache_dir: Path, *, timeout_seconds: int) -> EngineRun | ↵
↵ None:
166     binary = llama_bench_binary(cache_dir)
167     model = smollm2_model_path(cache_dir)
168     if not binary.exists() or not os.access(binary, os.X_OK) or not model.↵
↵ exists():
169         return None
170     command = [
171         str(binary),
172         "-m",
173         str(model),
174         "-ngl",
175         "0",
176         "-p",
177         "5",
178         "-n",
179         "4",
180         "-r",
181         "1",
182     ]
183     returncode, stdout, stderr = run_command(command, timeout_seconds=↵
↵ timeout_seconds)

```

```

184     return EngineRun(
185         name="llama_cpp_smolllm2_q4_k_m",
186         command=command,
187         returncode=returncode,
188         stdout=stdout,
189         stderr=stderr,
190         metrics=parse_llama_bench_metrics(stdout),
191     )
192
193
194 def write_report(path: Path, runs: list[EngineRun], *, prompt: str, ←
    ↪ max_new_tokens: int) -> None:
195     path.parent.mkdir(parents=True, exist_ok=True)
196     lines = [
197         "LCQI GPT-2 Benchmark Compare",
198         "",
199         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
200         f"repo_commit={current_commit()}",
201         f"working_tree_dirty={working_tree_dirty()}",
202         f"python={sys.version.split()[0]}",
203         f"prompt={prompt}",
204         f"max_new_tokens={max_new_tokens}",
205         "",
206         "scope=LCQI full_prefix and cached_kv run the same GPT-2 F32 ←
    ↪ safetensors model. "
207         "llama.cpp uses SmolLM2-135M-Instruct Q4_K_M as an external mature-←
    ↪ engine reference, "
208         "so its numbers are engineering context, not same-model quality ranking←
    ↪ .",
209         "",
210     ]
211     for run in runs:
212         lines.extend(
213             [
214                 f"[{run.name}]",
215                 f"returncode={run.returncode}",
216                 f"command={command_line(run.command)}",
217             ]
218         )
219         for key in sorted(run.metrics):
220             lines.append(f"{key}={run.metrics[key]}")
221         lines.extend(
222             [
223                 "stdout_tail:",
224                 tail(run.stdout.strip(), 2000),
225                 "stderr_tail:",
226                 tail(run.stderr.strip(), 2000),
227                 "",
228             ]
229         )
230     while lines and lines[-1] == "":
231         lines.pop()

```

```

232     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
233
234
235 def current_commit() -> str:
236     result = subprocess.run(
237         ["git", "-C", str(repo_dir()), "rev-parse", "--short", "HEAD"],
238         text=True,
239         capture_output=True,
240         check=False,
241     )
242     return result.stdout.strip() if result.returncode == 0 else "unknown"
243
244
245 def working_tree_dirty() -> str:
246     result = subprocess.run(
247         ["git", "-C", str(repo_dir()), "status", "--porcelain"],
248         text=True,
249         capture_output=True,
250         check=False,
251     )
252     if result.returncode != 0:
253         return "unknown"
254     return "true" if result.stdout.strip() else "false"
255
256
257 def parse_args() -> argparse.Namespace:
258     parser = argparse.ArgumentParser(
259         description="Compare LCQI GPT-2 full-prefix and KV-cache paths, with ↵
↵ optional llama.cpp context."
260     )
261     parser.add_argument("--cache-dir", type=Path, default=run_gpt2_smoke.↵
↵ default_cache_dir())
262     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
263     parser.add_argument("--report", type=Path, default=default_report_path())
264     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
265     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
266     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
267     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
268     parser.add_argument("--no-download", action="store_true")
269     parser.add_argument("--skip-llama", action="store_true")
270     parser.add_argument("--smollm2-cache-dir", type=Path, default=↵
↵ DEFAULT_SMOLLM2_CACHE_DIR)
271     return parser.parse_args()
272
273
274 def main() -> int:
275     args = parse_args()
276     if args.max_new_tokens < 0:
277         print("[lcqi-compare] --max-new-tokens cannot be negative", file=sys.↵
↵ stderr)

```

```

278         return 1
279     try:
280         run_gpt2_smoke.ensure_model_files(
281             args.cache_dir,
282             no_download=args.no_download,
283             timeout_seconds=args.timeout_seconds,
284         )
285         binary = run_gpt2_smoke.build_lcqi_gpt2(args.build_dir, args.jobs)
286         runs = [
287             run_lcqi(
288                 binary,
289                 args.cache_dir,
290                 engine="full",
291                 prompt=args.prompt,
292                 max_new_tokens=args.max_new_tokens,
293                 timeout_seconds=args.timeout_seconds,
294             ),
295             run_lcqi(
296                 binary,
297                 args.cache_dir,
298                 engine="cached",
299                 prompt=args.prompt,
300                 max_new_tokens=args.max_new_tokens,
301                 timeout_seconds=args.timeout_seconds,
302             ),
303         ]
304         if not args.skip_llama:
305             llama_run = run_llama_bench(
306                 args.smollm2_cache_dir.expanduser().resolve(),
307                 timeout_seconds=args.timeout_seconds,
308             )
309             if llama_run is not None:
310                 runs.append(llama_run)
311             write_report(args.report, runs, prompt=args.prompt, max_new_tokens=args↵
↵ .max_new_tokens)
312         except Exception as error:
313             print(f"[lcqi-compare] error: {error}", file=sys.stderr)
314             return 1
315
316         for run in runs:
317             print(f"{run.name}: returncode={run.returncode}")
318             for key in sorted(run.metrics):
319                 print(f"    {key}={run.metrics[key]}")
320             print(f"report={args.report}")
321         return 0 if all(run.returncode == 0 for run in runs) else 1
322
323
324 if __name__ == "__main__":
325     raise SystemExit(main())

```

`run_gpt2_optimization_ab.py` 专门比较 LCQI 自身改动前后。它从 baseline commit 建临时 worktree，baseline 和当前工作区都通过实践卷 CMake 入口构建 Release，再多轮运行同一个

GPT-2 F32 模型。报告中的 `median/min/max` 用来约束机器波动，`generated_text` 用来确认优化前后输出没有漂移，`benchmark_worker_count` 用来确认 cached decoder 到底是串行、自动线程数还是固定线程数。

Listing 16.10 `tools/run_gpt2_optimization_ab.py`

```

1  #!/usr/bin/env python3
2  from __future__ import annotations
3
4  import argparse
5  import datetime as dt
6  import shutil
7  import statistics
8  import subprocess
9  import sys
10 import tempfile
11 from dataclasses import dataclass
12 from pathlib import Path
13
14
15 DEFAULT_BASELINE_REF = "4febf3f"
16 DEFAULT_PROMPT = "Hello, my name is"
17 DEFAULT_MAX_NEW_TOKENS = 4
18 DEFAULT_ROUNDS = 3
19 DEFAULT_BUILD_JOBS = 2
20 DEFAULT_TIMEOUT_SECONDS = 300
21
22 METRIC_KEYS = [
23     "benchmark_prefill_ms",
24     "benchmark_decode_ms",
25     "benchmark_generate_ms",
26     "benchmark_worker_count",
27     "benchmark_generate_tokens_per_second",
28     "benchmark_decode_tokens_per_second",
29 ]
30
31
32 @dataclass(frozen=True)
33 class RunSample:
34     variant: str
35     engine: str
36     round_id: int
37     metrics: dict[str, float]
38     generated_text: str
39
40
41 def script_path() -> Path:
42     return Path(__file__).resolve()
43
44
45 def repo_dir() -> Path:
46     return script_path().parents[3]
47

```

```

48
49 def volume_dir() -> Path:
50     return script_path().parents[1]
51
52
53 def practice_dir(root: Path) -> Path:
54     return root / "books" / "cpu-volume-3-practice"
55
56
57 def default_report_path() -> Path:
58     return volume_dir() / "results" / "lcqi-gpt2-optimization-ab.txt"
59
60
61 def default_model_dir() -> Path:
62     return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"
63     ↪ "
64
65 def run_command(
66     args: list[str],
67     *,
68     cwd: Path,
69     timeout_seconds: int,
70 ) -> subprocess.CompletedProcess[str]:
71     return subprocess.run(
72         args,
73         cwd=cwd,
74         text=True,
75         capture_output=True,
76         timeout=timeout_seconds,
77         check=False,
78     )
79
80
81 def ensure_success(
82     completed: subprocess.CompletedProcess[str],
83     action: str,
84 ) -> None:
85     if completed.returncode == 0:
86         return
87     command = (
88         " ".join(completed.args)
89         if isinstance(completed.args, list)
90         else str(completed.args)
91     )
92     raise RuntimeError(
93         f"{action} failed with exit code {completed.returncode}\n"
94         f"command: {command}\n"
95         f"stdout tail:\n{completed.stdout[-2000:]}\n"
96         f"stderr tail:\n{completed.stderr[-2000:]}"
97     )
98

```

```

99
100 def git_text(args: list[str]) -> str:
101     completed = run_command(["git", *args], cwd=repo_dir(), timeout_seconds=60)
102     ensure_success(completed, "git command")
103     return completed.stdout.strip()
104
105
106 def working_tree_dirty() -> bool:
107     completed = run_command(
108         ["git", "status", "--porcelain"],
109         cwd=repo_dir(),
110         timeout_seconds=60,
111     )
112     ensure_success(completed, "git status")
113     return bool(completed.stdout.strip())
114
115
116 def build_binary(root: Path, build_dir: Path, jobs: int, timeout_seconds: int) ←
    ↪ -> Path:
117     configure = run_command(
118         [
119             "cmake",
120             "-S",
121             str(practice_dir(root)),
122             "-B",
123             str(build_dir),
124             "-DCMAKE_BUILD_TYPE=Release",
125         ],
126         cwd=root,
127         timeout_seconds=timeout_seconds,
128     )
129     ensure_success(configure, f"configure {root}")
130     build = run_command(
131         [
132             "cmake",
133             "--build",
134             str(build_dir),
135             "--target",
136             "lcqi_gpt2",
137             "-j",
138             str(jobs),
139         ],
140         cwd=root,
141         timeout_seconds=timeout_seconds,
142     )
143     ensure_success(build, f"build {root}")
144     binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
145     if not binary.exists():
146         raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
147     return binary
148
149

```

```

150 def parse_metrics(stdout: str) -> tuple[dict[str, float], str]:
151     metrics: dict[str, float] = {}
152     generated_text = ""
153     for line in stdout.splitlines():
154         key, _, value = line.partition(" ")
155         if key in METRIC_KEYS:
156             metrics[key] = float(value)
157         elif key == "generated_text":
158             generated_text = value
159     metrics.setdefault("benchmark_worker_count", 1.0)
160     missing = sorted(set(METRIC_KEYS) - metrics.keys())
161     if missing:
162         raise RuntimeError(f"missing benchmark metrics: {missing}")
163     return metrics, generated_text
164
165
166 def run_sample(
167     binary: Path,
168     *,
169     variant: str,
170     engine: str,
171     round_id: int,
172     model_dir: Path,
173     prompt: str,
174     max_new_tokens: int,
175     threads: int,
176     timeout_seconds: int,
177 ) -> RunSample:
178     command = [
179         str(binary),
180         "--benchmark",
181         "--engine",
182         engine,
183     ]
184     if variant == "current":
185         command.extend(["--threads", str(threads)])
186     command.extend([
187         str(model_dir),
188         prompt,
189         str(max_new_tokens),
190     ])
191     completed = run_command(
192         command,
193         cwd=repo_dir(),
194         timeout_seconds=timeout_seconds,
195     )
196     ensure_success(completed, f"run {variant} {engine}")
197     metrics, generated_text = parse_metrics(completed.stdout)
198     return RunSample(
199         variant=variant,
200         engine=engine,
201         round_id=round_id,

```

```

202     metrics=metrics,
203     generated_text=generated_text,
204 )
205
206
207 def summarize(samples: list[RunSample]) -> list[str]:
208     lines: list[str] = []
209     variants = sorted({sample.variant for sample in samples})
210     engines = sorted({sample.engine for sample in samples})
211     for variant in variants:
212         for engine in engines:
213             selected = [
214                 sample for sample in samples
215                 if sample.variant == variant and sample.engine == engine
216             ]
217             if not selected:
218                 continue
219             lines.append(f"[{variant} {engine}]")
220             for key in METRIC_KEYS:
221                 values = [sample.metrics[key] for sample in selected]
222                 lines.append(
223                     f"{key}=median:{statistics.median(values):.6g},"
224                     f"min:{min(values):.6g},max:{max(values):.6g}"
225                 )
226             texts = sorted({sample.generated_text for sample in selected})
227             lines.append(f"generated_text={texts[0] if len(texts) == 1 else ↵
↵ texts}")
228             lines.append("")
229     return lines
230
231
232 def write_report(
233     path: Path,
234     samples: list[RunSample],
235     *,
236     baseline_ref: str,
237     baseline_commit: str,
238     current_commit: str,
239     prompt: str,
240     max_new_tokens: int,
241     rounds: int,
242     current_threads: int,
243 ) -> None:
244     path.parent.mkdir(parents=True, exist_ok=True)
245     lines = [
246         "LCQI GPT-2 Optimization A/B",
247         "",
248         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
249         f"baseline_ref={baseline_ref}",
250         f"baseline_commit={baseline_commit}",
251         f"current_commit={current_commit}",
252         f"current_working_tree_dirty={str(working_tree_dirty()).lower()}",

```

```

253     f"prompt={prompt}",
254     f"max_new_tokens={max_new_tokens}",
255     f"rounds={rounds}",
256     f"baseline_threads=1",
257     f"current_threads={current_threads}",
258     "",
259     "scope=Both variants are built through books/cpu-volume-3-practice with↵
↵ "
260     "CMAKE_BUILD_TYPE=Release and run on the same GPT-2 F32 safetensors ↵
↵ model. "
261     "The llama.cpp comparison remains in lcqi-gpt2-benchmark-compare.txt; ↵
↵ this "
262     "report isolates LCQI before/after code changes.",
263     "",
264     "[samples]",
265 ]
266 for sample in samples:
267     metric_text = ",".join(
268         f"{key}:{sample.metrics[key]:.6g}" for key in METRIC_KEYS
269     )
270     lines.append(
271         f"round={sample.round_id},variant={sample.variant},engine={sample.↵
↵ engine},"
272         f"{metric_text},generated_text={sample.generated_text}"
273     )
274     lines.append("")
275     lines.extend(summarize(samples))
276     while lines and lines[-1] == "":
277         lines.pop()
278     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
279
280
281 def parse_args() -> argparse.Namespace:
282     parser = argparse.ArgumentParser(description="Run LCQI GPT-2 before/after A↵
↵ /B benchmark.")
283     parser.add_argument("--baseline-ref", default=DEFAULT_BASELINE_REF)
284     parser.add_argument("--model-dir", type=Path, default=default_model_dir())
285     parser.add_argument("--report", type=Path, default=default_report_path())
286     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
287     parser.add_argument("--max-new-tokens", type=int, default=↵
↵ DEFAULT_MAX_NEW_TOKENS)
288     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
289     parser.add_argument("--threads", type=int, default=0)
290     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
291     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
292     return parser.parse_args()
293
294
295 def main() -> int:
296     args = parse_args()
297     if args.rounds <= 0:

```

```

298     print("[lcqi-ab] --rounds must be positive", file=sys.stderr)
299     return 1
300 if args.max_new_tokens < 0:
301     print("[lcqi-ab] --max-new-tokens cannot be negative", file=sys.stderr)
302     return 1
303 if args.threads < 0:
304     print("[lcqi-ab] --threads cannot be negative", file=sys.stderr)
305     return 1
306 if not args.model_dir.exists():
307     print(f"[lcqi-ab] model directory not found: {args.model_dir}", file=
↵ sys.stderr)
308     return 1
309
310 baseline_commit = git_text(["rev-parse", "--short", args.baseline_ref])
311 current_commit = git_text(["rev-parse", "--short", "HEAD"])
312 with tempfile.TemporaryDirectory(prefix="lcqi-gpt2-ab-") as tmp:
313     tmp_path = Path(tmp)
314     baseline_root = tmp_path / "baseline"
315     current_build = tmp_path / "current-build"
316     baseline_build = tmp_path / "baseline-build"
317     ensure_success(
318         run_command(
319             ["git", "worktree", "add", "--detach", str(baseline_root), args
↵ .baseline_ref],
320             cwd=repo_dir(),
321             timeout_seconds=args.timeout_seconds,
322         ),
323         "create baseline worktree",
324     )
325     try:
326         baseline_binary = build_binary(
327             baseline_root,
328             baseline_build,
329             args.jobs,
330             args.timeout_seconds,
331         )
332         current_binary = build_binary(
333             repo_dir(),
334             current_build,
335             args.jobs,
336             args.timeout_seconds,
337         )
338
339     samples: list[RunSample] = []
340     for round_id in range(1, args.rounds + 1):
341         for variant, binary in [
342             ("baseline", baseline_binary),
343             ("current", current_binary),
344         ]:
345             for engine in ["full", "cached"]:
346                 sample = run_sample(
347                     binary,

```

```

348         variant=variant,
349         engine=engine,
350         round_id=round_id,
351         model_dir=args.model_dir,
352         prompt=args.prompt,
353         max_new_tokens=args.max_new_tokens,
354         threads=1 if variant == "baseline" else args.
↪ threads,
355         timeout_seconds=args.timeout_seconds,
356     )
357     samples.append(sample)
358     print(
359         f"round={round_id} variant={variant} engine={engine}
↪ } "
360         f"generate_ms={sample.metrics['
↪ benchmark_generate_ms']:.3f} "
361         f"workers={sample.metrics['benchmark_worker_count
↪ ']:.0f} "
362         f"gen_tps={sample.metrics['
↪ benchmark_generate_tokens_per_second']:.5f}"
363     )
364     write_report(
365         args.report,
366         samples,
367         baseline_ref=args.baseline_ref,
368         baseline_commit=baseline_commit,
369         current_commit=current_commit,
370         prompt=args.prompt,
371         max_new_tokens=args.max_new_tokens,
372         rounds=args.rounds,
373         current_threads=args.threads,
374     )
375     finally:
376         subprocess.run(
377             ["git", "worktree", "remove", "--force", str(baseline_root)],
378             cwd=repo_dir(),
379             text=True,
380             capture_output=True,
381             check=False,
382         )
383     print(f"report={args.report}")
384     return 0
385
386
387 if __name__ == "__main__":
388     raise SystemExit(main())

```

`run_gpt2_hotspot_profile.py` 专门收集 cached decode 内部热点。它构建 Release `lcqi_gpt2`, 用 `--profile-hotspots` 输出 `hotspot_lm_head_pct`、`hotspot_mlp_fc_pct`、`hotspot_mlp_projection_pct`、`hotspot_qkv_projection_pct`、`hotspot_attention_projection_pct`、`hotspot_attention_pct` 和 `benchmark_worker_count`, 再按 token 数和轮次计算中位数。这个脚本回答“优化应该打哪里”，不是回答“某次提交快了多少”。本轮线程

优化就是靠它发现第一次 auto 阈值漏掉了 attention output projection，随后把行数和列数阈值一起纳入并行判定。

Listing 16.11 tools/run_gpt2_hotspot_profile.py

```
1 #!/usr/bin/env python3
2 from __future__ import annotations
3
4 import argparse
5 import datetime as dt
6 import statistics
7 import subprocess
8 from dataclasses import dataclass
9 from pathlib import Path
10
11
12 DEFAULT_PROMPT = "Hello, my name is"
13 DEFAULT_TOKEN_COUNTS = "4,16"
14 DEFAULT_ROUNDS = 3
15 DEFAULT_BUILD_JOBS = 2
16 DEFAULT_TIMEOUT_SECONDS = 900
17
18 METRIC_PREFIXES = ("benchmark_", "hotspot_")
19
20
21 @dataclass(frozen=True)
22 class HotspotSample:
23     max_new_tokens: int
24     round_id: int
25     metrics: dict[str, float]
26     generated_text: str
27
28
29 def script_path() -> Path:
30     return Path(__file__).resolve()
31
32
33 def repo_dir() -> Path:
34     return script_path().parents[3]
35
36
37 def volume_dir() -> Path:
38     return script_path().parents[1]
39
40
41 def practice_dir() -> Path:
42     return repo_dir() / "books" / "cpu-volume-3-practice"
43
44
45 def default_build_dir() -> Path:
46     return volume_dir() / "build" / "lcqi-hotspot-profile"
47
48
```

```

49 def default_model_dir() -> Path:
50     return Path.home() / ".cache" / "lcqi-gpt2-smoke" / "openai-community--gpt2"
51
52
53 def default_report_path() -> Path:
54     return volume_dir() / "results" / "lcqi-gpt2-hotspot-profile.txt"
55
56
57 def run_command(
58     args: list[str],
59     *,
60     cwd: Path,
61     timeout_seconds: int,
62 ) -> subprocess.CompletedProcess[str]:
63     return subprocess.run(
64         args,
65         cwd=cwd,
66         text=True,
67         capture_output=True,
68         timeout=timeout_seconds,
69         check=False,
70     )
71
72
73 def ensure_success(completed: subprocess.CompletedProcess[str], action: str) -> None:
74     if completed.returncode == 0:
75         return
76     command = (
77         " ".join(completed.args)
78         if isinstance(completed.args, list)
79         else str(completed.args)
80     )
81     raise RuntimeError(
82         f"{action} failed with exit code {completed.returncode}\n"
83         f"command: {command}\n"
84         f"stdout tail:\n{completed.stdout[-2000:]]}\n"
85         f"stderr tail:\n{completed.stderr[-2000:]]}"
86     )
87
88
89 def build_binary(build_dir: Path, jobs: int, timeout_seconds: int) -> Path:
90     configure = run_command(
91         [
92             "cmake",
93             "-S",
94             str(practice_dir()),
95             "-B",
96             str(build_dir),
97             "-DCMAKE_BUILD_TYPE=Release",
98         ],

```

```

99     cwd=repo_dir(),
100     timeout_seconds=timeout_seconds,
101 )
102 ensure_success(configure, "configure hotspot profile build")
103 build = run_command(
104     [
105         "cmake",
106         "--build",
107         str(build_dir),
108         "--target",
109         "lcqi_gpt2",
110         "-j",
111         str(jobs),
112     ],
113     cwd=repo_dir(),
114     timeout_seconds=timeout_seconds,
115 )
116 ensure_success(build, "build lcqi_gpt2")
117 binary = build_dir / "linux_cpu_inference" / "lcqi_gpt2"
118 if not binary.exists():
119     raise RuntimeError(f"lcqi_gpt2 binary not found: {binary}")
120 return binary
121
122
123 def parse_numeric_list(value: str) -> list[int]:
124     result: list[int] = []
125     for part in value.split(","):
126         stripped = part.strip()
127         if not stripped:
128             continue
129         parsed = int(stripped)
130         if parsed <= 0:
131             raise RuntimeError("--token-counts entries must be positive")
132         result.append(parsed)
133     if not result:
134         raise RuntimeError("--token-counts must contain at least one positive ↵
↵ integer")
135     return result
136
137
138 def parse_stdout(stdout: str) -> tuple[dict[str, float], str]:
139     metrics: dict[str, float] = {}
140     generated_text = ""
141     for line in stdout.splitlines():
142         key, _, value = line.partition(" ")
143         if key.startswith(METRIC_PREFIXES):
144             metrics[key] = float(value)
145         elif key == "generated_text":
146             generated_text = value
147     required = {
148         "benchmark_generate_ms",
149         "benchmark_worker_count",

```

```

150     "hotspot_total_step_ms",
151     "hotspot_lm_head_pct",
152     "hotspot_mlp_fc_pct",
153     "hotspot_mlp_projection_pct",
154     "hotspot_qkv_projection_pct",
155     "hotspot_attention_pct",
156 }
157 missing = sorted(required - metrics.keys())
158 if missing:
159     raise RuntimeError(f"missing hotspot metrics: {missing}")
160 if not generated_text:
161     raise RuntimeError("missing generated_text")
162 return metrics, generated_text
163
164
165 def run_sample(
166     binary: Path,
167     *,
168     model_dir: Path,
169     prompt: str,
170     max_new_tokens: int,
171     round_id: int,
172     threads: int,
173     timeout_seconds: int,
174 ) -> HotspotSample:
175     completed = run_command(
176         [
177             str(binary),
178             "--profile-hotspots",
179             "--engine",
180             "cached",
181             "--threads",
182             str(threads),
183             str(model_dir),
184             prompt,
185             str(max_new_tokens),
186         ],
187         cwd=repo_dir(),
188         timeout_seconds=timeout_seconds,
189     )
190     ensure_success(completed, f"run hotspot profile tokens={max_new_tokens}")
191     metrics, generated_text = parse_stdout(completed.stdout)
192     return HotspotSample(
193         max_new_tokens=max_new_tokens,
194         round_id=round_id,
195         metrics=metrics,
196         generated_text=generated_text,
197     )
198
199
200 def summarize(samples: list[HotspotSample]) -> list[str]:
201     lines: list[str] = []

```

```

202     metric_keys = [
203         "benchmark_load_ms",
204         "benchmark_generate_ms",
205         "benchmark_generate_tokens_per_second",
206         "benchmark_decode_tokens_per_second",
207         "hotspot_total_step_ms",
208         "benchmark_worker_count",
209         "hotspot_lm_head_pct",
210         "hotspot_mlp_fc_pct",
211         "hotspot_mlp_projection_pct",
212         "hotspot_qkv_projection_pct",
213         "hotspot_attention_projection_pct",
214         "hotspot_attention_pct",
215     ]
216     for token_count in sorted({sample.max_new_tokens for sample in samples}):
217         selected = [sample for sample in samples if sample.max_new_tokens == ↵
↵ token_count]
218         lines.append(f"[summary max_new_tokens={token_count}]")
219         for key in metric_keys:
220             values = [sample.metrics[key] for sample in selected if key in ↵
↵ sample.metrics]
221             if not values:
222                 continue
223             lines.append(
224                 f"{key}=median:{statistics.median(values):.6g},"
225                 f"min:{min(values):.6g},max:{max(values):.6g}"
226             )
227             texts = sorted({sample.generated_text for sample in selected})
228             lines.append(f"generated_text={texts[0] if len(texts) == 1 else texts}"↵
↵ )
229             lines.append("")
230     return lines
231
232
233 def write_report(
234     path: Path,
235     samples: list[HotspotSample],
236     *,
237     model_dir: Path,
238     prompt: str,
239     token_counts: list[int],
240     rounds: int,
241     threads: int,
242 ) -> None:
243     path.parent.mkdir(parents=True, exist_ok=True)
244     lines = [
245         "LCQI GPT-2 Hotspot Profile",
246         "",
247         f"timestamp_utc={dt.datetime.now(dt.UTC).isoformat()}",
248         f"model_dir={model_dir}",
249         f"prompt={prompt}",
250         f"token_counts='{','.join(str(value) for value in token_counts)}",

```

```

251     f"rounds={rounds}",
252     f"requested_threads={threads}",
253     "engine=cached",
254     "build=CMAKE_BUILD_TYPE=Release through books/cpu-volume-3-practice",
255     "",
256     "[samples]",
257 ]
258 for sample in samples:
259     metric_text = ",".join(
260         f"{key}:{sample.metrics[key]:.6g}"
261         for key in sorted(sample.metrics)
262         if key.startswith(METRIC_PREFIXES)
263     )
264     lines.append(
265         f"round={sample.round_id},max_new_tokens={sample.max_new_tokens},"
266         f"{metric_text},generated_text={sample.generated_text}"
267     )
268     lines.append("")
269     lines.extend(summarize(samples))
270     while lines and lines[-1] == "":
271         lines.pop()
272     path.write_text("\n".join(lines) + "\n", encoding="utf-8")
273
274
275 def parse_args() -> argparse.Namespace:
276     parser = argparse.ArgumentParser(description="Run LCQI GPT-2 cached hotspot↵
↵ profile.")
277     parser.add_argument("--model-dir", type=Path, default=default_model_dir())
278     parser.add_argument("--build-dir", type=Path, default=default_build_dir())
279     parser.add_argument("--report", type=Path, default=default_report_path())
280     parser.add_argument("--prompt", default=DEFAULT_PROMPT)
281     parser.add_argument("--token-counts", default=DEFAULT_TOKEN_COUNTS)
282     parser.add_argument("--rounds", type=int, default=DEFAULT_ROUNDS)
283     parser.add_argument("--threads", type=int, default=0)
284     parser.add_argument("--jobs", type=int, default=DEFAULT_BUILD_JOBS)
285     parser.add_argument("--timeout-seconds", type=int, default=↵
↵ DEFAULT_TIMEOUT_SECONDS)
286     return parser.parse_args()
287
288
289 def main() -> int:
290     args = parse_args()
291     if args.rounds <= 0:
292         print("[lcqi-hotspot] --rounds must be positive")
293         return 1
294     if args.threads < 0:
295         print("[lcqi-hotspot] --threads cannot be negative")
296         return 1
297     if not args.model_dir.exists():
298         print(f"[lcqi-hotspot] model directory not found: {args.model_dir}")
299         return 1
300     token_counts = parse_numeric_list(args.token_counts)

```

```

301     binary = build_binary(args.build_dir, args.jobs, args.timeout_seconds)
302     samples: list[HotspotSample] = []
303     for round_id in range(1, args.rounds + 1):
304         for token_count in token_counts:
305             sample = run_sample(
306                 binary,
307                 model_dir=args.model_dir,
308                 prompt=args.prompt,
309                 max_new_tokens=token_count,
310                 round_id=round_id,
311                 threads=args.threads,
312                 timeout_seconds=args.timeout_seconds,
313             )
314             samples.append(sample)
315             print(
316                 f"round={round_id} max_new_tokens={token_count} "
317                 f"generate_ms={sample.metrics['benchmark_generate_ms']:.3f} "
318                 f"workers={sample.metrics['benchmark_worker_count']:.0f} "
319                 f"lm_head_pct={sample.metrics['hotspot_lm_head_pct']:.3f} "
320                 f"mlp_pct={sample.metrics['hotspot_mlp_fc_pct']} + sample.↵
↵ metrics['hotspot_mlp_projection_pct']:.3f"
321             )
322     write_report(
323         args.report,
324         samples,
325         model_dir=args.model_dir,
326         prompt=args.prompt,
327         token_counts=token_counts,
328         rounds=args.rounds,
329         threads=args.threads,
330     )
331     print(f"report={args.report}")
332     return 0
333
334
335 if __name__ == "__main__":
336     raise SystemExit(main())

```

`run_lcqi_benchmark.sh` 是本地 benchmark 收集脚本。它适合在固定机器上反复运行，把同一组 shape 和 backend 的结果落到报告文件。

Listing 16.12 tools/run_lcqi_benchmark.sh

```

1  #!/usr/bin/env bash
2  set -euo pipefail
3
4  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
5  VOLUME_DIR="$(cd "${SCRIPT_DIR}/.." && pwd)"
6  REPO_DIR="$(cd "${VOLUME_DIR}/../.." && pwd)"
7  BUILD_DIR="${VOLUME_DIR}/build/lcqi-release"
8  RESULT_DIR="${VOLUME_DIR}/results"
9  REPEAT="${1:-200}"
10 STAMP="$(date -u +%Y%m%dT%H%M%S)"

```

```

11 CSV_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.csv"
12 META_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.meta.txt"
13 PERF_PATH="${RESULT_DIR}/lcqi-bench-${STAMP}.perf.txt"
14
15 mkdir -p "${RESULT_DIR}"
16
17 cmake -S "${VOLUME_DIR}" -B "${BUILD_DIR}" -DCMAKE_BUILD_TYPE=Release
18 cmake --build "${BUILD_DIR}"
19 ctest --test-dir "${BUILD_DIR}" --output-on-failure
20
21 {
22     echo "timestamp_utc=${STAMP}"
23     echo "repeat=${REPEAT}"
24     echo "commit=$(git -C "${REPO_DIR}" rev-parse --short HEAD)"
25     echo "uname=$(uname -a)"
26     echo "compiler=$((${CXX:-c++}) --version | sed -n '1p')"
27     echo "cmake=$(cmake --version | sed -n '1p')"
28     echo "bench=${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench"
29 } > "${META_PATH}"
30
31 "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" > "${CSV_PATH}"
32
33 if command -v perf >/dev/null 2>&1; then
34     perf stat \
35         -e cycles,instructions,cache-misses,branches,branch-misses \
36         "${BUILD_DIR}/labs/linux_cpu_inference/lcqi_bench" "${REPEAT}" \
37         > /dev/null 2> "${PERF_PATH}" || true
38 else
39     echo "perf_unavailable=1" > "${PERF_PATH}"
40 fi
41
42 echo "csv=${CSV_PATH}"
43 echo "meta=${META_PATH}"
44 echo "perf=${PERF_PATH}"

```

完整测试

`lcqi_tests.cpp` 把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、GPT-2 tiny path、trace、ledger 和 serving probe 串成验收网。GPT-2 优化不是只看速度：测试会把 full-prefix logits、旧 cached logits、优化 decoder logits、强制两 worker 的并行 decoder logits 和 logits-free greedy token 放在同一个 tiny prompt 下对齐，确认 predicted token、logits、KV filled token 数和 generated ids 不漂移。新增优化或 shape 时，必须先扩展这里的 golden 和边界样例。

Listing 16.13 tests/lcqi_tests.cpp

```

1 #include <lcqi/gpt2_reference.hpp>
2 #include <lcqi/f32_kernels.hpp>
3 #include <lcqi/inference.hpp>
4 #include <lcqi/int8_kernels.hpp>
5 #include <lcqi/model.hpp>
6 #include <lcqi/reference_decoder.hpp>
7 #include <lcqi/safetensors.hpp>

```



```

8 #include <lcqi/tokenizer.hpp>
9
10 #include <array>
11 #include <cmath>
12 #include <cstring>
13 #include <cstdlib>
14 #include <filesystem>
15 #include <fstream>
16 #include <iostream>
17 #include <span>
18 #include <sstream>
19 #include <stdexcept>
20 #include <string>
21 #include <utility>
22 #include <vector>
23
24 namespace {
25
26 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
27 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
28 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
29 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
30 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
31 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
32 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
33 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
34 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
35 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
36 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;
37 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
38 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
39 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
40 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
41 constexpr float LCQI_F32_KERNEL_MAX_DIFF = 1.0e-4F;
42 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
43 constexpr std::size_t LCQI_F32_DOT_TEST_SIZE = 257;
44 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
45 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
46 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↵
    ↵ LCQI_REFERENCE_DECODER_LOGITS{
47     0.352516979F,
48     -0.126884326F,
49     0.0797837451F,
50     -0.29797405F,
51     0.127030417F,
52 };
53 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
54 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
55 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
56 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
57 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
58 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;

```

```

59 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
60 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
61 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
62 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
63 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
64 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
65 constexpr std::int32_t LCQI_GPT2_PREDICTED_TOKEN = 3;
66 constexpr std::size_t LCQI_GPT2_VOCAB_SIZE = 6;
67 constexpr std::array<float, LCQI_GPT2_VOCAB_SIZE> LCQI_GPT2_LOGITS{
68     0.407358F,
69     -0.504049F,
70     -0.107834F,
71     0.840337F,
72     -0.450123F,
73     -0.156763F,
74 };
75 constexpr std::int32_t LCQI_GPT2_GENERATED_LENGTH = 5;
76 constexpr std::int32_t LCQI_GPT2_TOKENIZER_HELLO_ID = 7;
77 constexpr std::uint16_t LCQI_TEST_F16_ONE = 0x3C00U;
78 constexpr std::uint16_t LCQI_TEST_F16_NEGATIVE_TWO = 0xC000U;
79 constexpr std::uint16_t LCQI_TEST_BF16_ONE_POINT_FIVE = 0x3FC0U;
80 constexpr std::uint16_t LCQI_TEST_BF16_NEGATIVE_HALF = 0xBF00U;
81
82 struct KernelShapeCase {
83     std::int32_t input_size = 0;
84     std::int32_t output_size = 0;
85     std::int32_t output_block_size = 0;
86 };
87
88 struct RawTensorFixture {
89     std::string name;
90     std::string dtype;
91     std::vector<std::int64_t> shape;
92     std::vector<std::uint8_t> bytes;
93 };
94
95 struct FloatTensorFixture {
96     std::string name;
97     std::vector<std::int64_t> shape;
98     std::vector<float> values;
99 };
100
101 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
102     {17, 19, 8},
103     {33, 7, 8},
104     {64, 32, 16},
105     {65, 31, 16},
106 }};
107
108 void require(bool condition, const char* message) {
109     if (!condition) {
110         throw std::runtime_error(message);

```

```

111     }
112 }
113
114 void require_close(float actual, float expected, const char* message) {
115     if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
116         throw std::runtime_error(message);
117     }
118 }
119
120 std::filesystem::path test_model_path() {
121     return std::filesystem::path(__FILE__).parent_path().parent_path() /
122         "models" / "tiny_mlp_i8.txt";
123 }
124
125 std::filesystem::path test_tokenizer_path() {
126     return std::filesystem::path(__FILE__).parent_path().parent_path() /
127         "models" / "tiny_tokenizer.txt";
128 }
129
130 std::filesystem::path temp_file_path(const std::string& name) {
131     return std::filesystem::temp_directory_path() / name;
132 }
133
134 void write_le_u64(std::ofstream& output, std::uint64_t value) {
135     for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
136         const char byte = static_cast<char>(
137             (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
138         output.write(&byte, 1);
139     }
140 }
141
142 void append_le_u16(std::vector<std::uint8_t>& output, std::uint16_t value) {
143     output.push_back(static_cast<std::uint8_t>(value & LCQI_BYTE_MASK));
144     output.push_back(static_cast<std::uint8_t>((value >> LCQI_TEST_BYTE_BITS) &
145 ↪ LCQI_BYTE_MASK));
146 }
147
148 void append_le_f32(std::vector<std::uint8_t>& output, float value) {
149     std::uint32_t bits = 0;
150     std::memcpy(&bits, &value, sizeof(bits));
151     for (std::int32_t i = 0; i < 4; ++i) {
152         output.push_back(static_cast<std::uint8_t>(
153             (bits >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK));
154     }
155 }
156
157 void write_safetensors_fixture(const std::filesystem::path& path,
158                                const std::string& header,
159                                std::uint64_t payload_bytes) {
160     std::ofstream output(path, std::ios::binary);
161     if (!output) {
162         throw std::runtime_error("cannot create safetensors test fixture");
163     }

```

```

162     }
163     write_le_u64(output, static_cast<std::uint64_t>(header.size()));
164     output.write(header.data(), static_cast<std::streamsize>(header.size()));
165     for (std::uint64_t i = 0; i < payload_bytes; ++i) {
166         const char byte = static_cast<char>(i & 0xFFU);
167         output.write(&byte, 1);
168     }
169 }
170
171 std::string json_shape(std::span<const std::int64_t> shape) {
172     std::ostringstream stream;
173     stream << '[';
174     for (std::size_t index = 0; index < shape.size(); ++index) {
175         if (index != 0) {
176             stream << ',';
177         }
178         stream << shape[index];
179     }
180     stream << ']';
181     return stream.str();
182 }
183
184 void write_safetensors_raw_fixture(const std::filesystem::path& path,
185                                   std::span<const RawTensorFixture> tensors) {
186     std::ostringstream header;
187     header << "{\"__metadata__\":{\"format\":\"lcqi-test\"}";
188     std::uint64_t offset = 0;
189     for (const RawTensorFixture& tensor : tensors) {
190         header << ",\"\" << tensor.name << "\":{\"dtype\":\"\" << tensor.dtype
191             << "\",\"shape\":\"\" << json_shape(tensor.shape)
192             << "\",\"data_offsets\":[\" << offset << ', '
193             << offset + tensor.bytes.size() << "]}\"";
194         offset += tensor.bytes.size();
195     }
196     header << '}';
197
198     std::ofstream output(path, std::ios::binary);
199     if (!output) {
200         throw std::runtime_error("cannot create safetensors raw test fixture");
201     }
202     const std::string header_text = header.str();
203     write_le_u64(output, static_cast<std::uint64_t>(header_text.size()));
204     output.write(header_text.data(), static_cast<std::streamsize>(header_text.↵
↵ size()));
205     for (const RawTensorFixture& tensor : tensors) {
206         output.write(reinterpret_cast<const char*>(tensor.bytes.data()),
207                     static_cast<std::streamsize>(tensor.bytes.size()));
208     }
209 }
210
211 std::vector<std::uint8_t> f32_payload(std::span<const float> values) {
212     std::vector<std::uint8_t> bytes;

```

```

213     bytes.reserve(values.size() * 4);
214     for (const float value : values) {
215         append_le_f32(bytes, value);
216     }
217     return bytes;
218 }
219
220 std::vector<float> transpose_linear_to_hf_conv1d(const lcqi::Gpt2LinearF32& ↵
    ↵ linear) {
221     std::vector<float> transposed(
222         static_cast<std::size_t>(linear.input_size) *
223         static_cast<std::size_t>(linear.output_size),
224         0.0F);
225     for (std::int32_t out = 0; out < linear.output_size; ++out) {
226         for (std::int32_t in = 0; in < linear.input_size; ++in) {
227             transposed[static_cast<std::size_t>(in) *
228                 static_cast<std::size_t>(linear.output_size) +
229                 static_cast<std::size_t>(out)] =
230                 linear.weights[static_cast<std::size_t>(out) *
231                     static_cast<std::size_t>(linear.input_size) ↵
    ↵ +
232                     static_cast<std::size_t>(in)];
233         }
234     }
235     return transposed;
236 }
237
238 void add_f32_tensor(std::vector<RawTensorFixture>& tensors,
239                     std::string name,
240                     std::vector<std::int64_t> shape,
241                     std::span<const float> values) {
242     tensors.push_back(RawTensorFixture{
243         std::move(name),
244         "F32",
245         std::move(shape),
246         f32_payload(values),
247     });
248 }
249
250 void write_text_file(const std::filesystem::path& path, const std::string& text ↵
    ↵ ) {
251     std::ofstream output(path);
252     if (!output) {
253         throw std::runtime_error("cannot create text fixture");
254     }
255     output << text;
256 }
257
258 void write_tiny_gpt2_safetensors(const std::filesystem::path& path,
259                                  const lcqi::Gpt2ReferenceModel& model) {
260     std::vector<RawTensorFixture> tensors;
261     add_f32_tensor(tensors,

```

```

262         "transformer.wte.weight",
263         {model.config.vocab_size, model.config.hidden_size},
264         model.token_embedding);
265     add_f32_tensor(tensors,
266         "transformer.wpe.weight",
267         {model.config.max_positions, model.config.hidden_size},
268         model.position_embedding);
269     for (std::int32_t layer_id = 0; layer_id < model.config.layer_count; ++
↪ layer_id) {
270         const lcqi::Gpt2LayerWeightsF32& layer =
271             model.layers[static_cast<std::size_t>(layer_id)];
272         const std::string prefix = "transformer.h." + std::to_string(layer_id)
↪ + ".";
273         add_f32_tensor(tensors, prefix + "ln_1.weight", {model.config.
↪ hidden_size}, layer.ln_1_weight);
274         add_f32_tensor(tensors, prefix + "ln_1.bias", {model.config.hidden_size
↪ }, layer.ln_1_bias);
275         const std::vector<float> c_attn_hf = transpose_linear_to_hf_conv1d(
↪ layer.c_attn);
276         add_f32_tensor(tensors,
277             prefix + "attn.c_attn.weight",
278             {model.config.hidden_size,
279              model.config.hidden_size * 3},
280             c_attn_hf);
281         add_f32_tensor(tensors,
282             prefix + "attn.c_attn.bias",
283             {model.config.hidden_size * 3},
284             layer.c_attn.bias);
285         const std::vector<float> c_proj_hf = transpose_linear_to_hf_conv1d(
↪ layer.c_proj);
286         add_f32_tensor(tensors,
287             prefix + "attn.c_proj.weight",
288             {model.config.hidden_size, model.config.hidden_size},
289             c_proj_hf);
290         add_f32_tensor(tensors,
291             prefix + "attn.c_proj.bias",
292             {model.config.hidden_size},
293             layer.c_proj.bias);
294         add_f32_tensor(tensors, prefix + "ln_2.weight", {model.config.
↪ hidden_size}, layer.ln_2_weight);
295         add_f32_tensor(tensors, prefix + "ln_2.bias", {model.config.hidden_size
↪ }, layer.ln_2_bias);
296         const std::vector<float> c_fc_hf = transpose_linear_to_hf_conv1d(layer.
↪ c_fc);
297         add_f32_tensor(tensors,
298             prefix + "mlp.c_fc.weight",
299             {model.config.hidden_size, model.config.
↪ intermediate_size},
300             c_fc_hf);
301         add_f32_tensor(tensors,
302             prefix + "mlp.c_fc.bias",
303             {model.config.intermediate_size},

```

```

304         layer.c_fc.bias);
305     const std::vector<float> mlp_proj_hf =
306         transpose_linear_to_hf_conv1d(layer.mlp_c_proj);
307     add_f32_tensor(tensors,
308         prefix + "mlp.c_proj.weight",
309         {model.config.intermediate_size, model.config.↵
↵ hidden_size},
310         mlp_proj_hf);
311     add_f32_tensor(tensors,
312         prefix + "mlp.c_proj.bias",
313         {model.config.hidden_size},
314         layer.mlp_c_proj.bias);
315 }
316 add_f32_tensor(tensors,
317     "transformer.ln_f.weight",
318     {model.config.hidden_size},
319     model.final_ln_weight);
320 add_f32_tensor(tensors,
321     "transformer.ln_f.bias",
322     {model.config.hidden_size},
323     model.final_ln_bias);
324 write_safetensors_raw_fixture(path, tensors);
325 }
326
327 void test_tiny_mlp() {
328     const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
329     require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
330     require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↵
↵ ;
331     require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↵
↵ ;
332
333     const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
334     const lcqi::InferenceResult result = lcqi::run_inference(model, input);
335
336     require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↵
↵ predicted class");
337     require(result.logits.size() == static_cast<std::size_t>(↵
↵ LCQI_TEST_OUTPUT_SIZE),
338         "unexpected logits size");
339     require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
340     require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
341
342     std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↵
↵ 0.0F);
343     lcqi::linear_i8(model.hidden, input, hidden);
344     lcqi::relu_inplace(hidden);
345     require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
346     require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
347     require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
348 }
349

```

```

350 void test_tokenizer_contract() {
351     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
352     require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
353     require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
354     require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
355     require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
356             "tokenizer template hash mismatch");
357
358     const std::vector<std::int32_t> token_ids =
359         lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
360     const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
361     require(token_ids == expected, "tokenizer prompt ids mismatch");
362     require(lcqi::tokenizer_contract_hash(tokenizer) == ↵
↵ LCQI_TOKENIZER_HASH_EXPECTED,
363             "tokenizer contract hash mismatch");
364 }
365
366 void test_tokenizer_rejects_unknown_without_unk() {
367     const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↵
↵ test_tokenizer_path());
368     bool threw = false;
369     try {
370         static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↵
↵ ;
371     } catch (const std::runtime_error&) {
372         threw = true;
373     }
374     require(threw, "tokenizer should reject unknown token when unk id is absent↵
↵ ");
375 }
376
377 void test_safetensors_manifest_contract() {
378     const std::filesystem::path path =
379         temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
380     const std::string header =
381         R"({"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight↵
↵ ":{ "dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.↵
↵ attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets"↵
↵ :[24,48]}})";
382     write_safetensors_fixture(path, header, ↵
↵ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
383
384     const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↵
↵ path);
385     std::filesystem::remove(path);
386
387     require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
388             "safetensors header size mismatch");
389     require(manifest.data_start_offset ==
390             static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
391             static_cast<std::uint64_t>(header.size()),

```



```

392     "safetensors data start mismatch");
393     require(manifest.metadata.size() == 1, "safetensors metadata count mismatch"
↪     );
394     require(manifest.metadata[0].first == "format" &&
395             manifest.metadata[0].second == "lcqi-fixture",
396             "safetensors metadata mismatch");
397     require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
398
399     const lcqi::SafeTensorEntry* embedding =
400         manifest.find_tensor("model.embed_tokens.weight");
401     require(embedding != nullptr, "safetensors embedding tensor missing");
402     require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
403     require(embedding->shape == std::vector<std::int64_t>({2, 3}),
404             "safetensors embedding shape mismatch");
405     require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
406             embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
407             "safetensors embedding offset mismatch");
408     require(embedding->byte_size() ==
409             LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
410             "safetensors embedding byte size mismatch");
411
412     const lcqi::SafeTensorEntry* q_proj =
413         manifest.find_tensor("model.layers.0.attn.q_proj.weight");
414     require(q_proj != nullptr, "safetensors q projection tensor missing");
415     require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
416     require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
417             q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
418             "safetensors q projection offset mismatch");
419 }
420
421 void test_safetensors_rejects_bad_offsets() {
422     const std::filesystem::path path =
423         temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
424     const std::string header =
425         R("{"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,32]}}");
426     write_safetensors_fixture(path, header, 4);
427
428     bool threw = false;
429     try {
430         static_cast<void>(lcqi::load_safetensors_manifest(path));
431     } catch (const std::runtime_error&) {
432         threw = true;
433     }
434     std::filesystem::remove(path);
435     require(threw, "safetensors parser should reject offsets beyond payload");
436 }
437
438 void test_safetensors_rejects_bad_dtype_shape_size() {
439     const std::filesystem::path path =
440         temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
441     const std::string header =
442         R("{"tensor":{"dtype":"F32","shape":[4],"data_offsets":[0,8]}}");

```

```

443     write_safetensors_fixture(path, header, 8);
444
445     bool threw = false;
446     try {
447         static_cast<void>(lcqi::load_safetensors_manifest(path));
448     } catch (const std::runtime_error&) {
449         threw = true;
450     }
451     std::filesystem::remove(path);
452     require(threw, "safetensors parser should reject dtype/shape byte mismatch"↵
↵ );
453 }
454
455 void test_safetensors_reads_float_tensors() {
456     const std::filesystem::path path =
457         temp_file_path("lcqi_safetensors_float_read.safetensors");
458     std::vector<std::uint8_t> f16_bytes;
459     append_le_u16(f16_bytes, LCQI_TEST_F16_ONE);
460     append_le_u16(f16_bytes, LCQI_TEST_F16_NEGATIVE_TWO);
461     std::vector<std::uint8_t> bf16_bytes;
462     append_le_u16(bf16_bytes, LCQI_TEST_BF16_ONE_POINT_FIVE);
463     append_le_u16(bf16_bytes, LCQI_TEST_BF16_NEGATIVE_HALF);
464
465     const std::array<RawTensorFixture, 3> tensors{{
466         {"f32", "F32", {2}, f32_payload(std::array<float, 2>{1.25F, -2.5F})},
467         {"f16", "F16", {2}, f16_bytes},
468         {"bf16", "BF16", {2}, bf16_bytes},
469     }};
470     write_safetensors_raw_fixture(path, tensors);
471
472     const lcqi::SafeTensorFile file = lcqi::load_safetensors_file(path);
473     std::filesystem::remove(path);
474     const std::vector<float> f32 = lcqi::read_safetensor_f32_tensor(file, "f32"↵
↵ );
475     const std::vector<float> f16 = lcqi::read_safetensor_f32_tensor(file, "f16"↵
↵ );
476     const std::vector<float> bf16 = lcqi::read_safetensor_f32_tensor(file, "↵
↵ bf16");
477
478     require_close(f32[0], 1.25F, "safetensors F32 value 0 mismatch");
479     require_close(f32[1], -2.5F, "safetensors F32 value 1 mismatch");
480     require_close(f16[0], 1.0F, "safetensors F16 value 0 mismatch");
481     require_close(f16[1], -2.0F, "safetensors F16 value 1 mismatch");
482     require_close(bf16[0], 1.5F, "safetensors BF16 value 0 mismatch");
483     require_close(bf16[1], -0.5F, "safetensors BF16 value 1 mismatch");
484 }
485
486 void test_gpt2_tiny_forward_and_generation() {
487     const lcqi::Gpt2ReferenceModel model = lcqi::make_tiny_gpt2_reference_model↵
↵ ();
488     const std::vector<std::int32_t> tokens{1, 2, 3};
489     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(model, tokens↵

```

```

↪ );
490
491 require(result.logits.size() == LCQI_GPT2_VOCAB_SIZE, "GPT-2 logits size ↪
↪ mismatch");
492 require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
493         "GPT-2 predicted token mismatch");
494 for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
495     require_close(result.logits[index], LCQI_GPT2_LOGITS[index], "GPT-2 ↪
↪ logit mismatch");
496 }
497
498 const std::vector<std::int32_t> generated =
499     lcqi::gpt2_generate_greedy(model, tokens, 2);
500 const std::vector<std::int32_t> expected{1, 2, 3, 3, 3};
501 require(generated == expected, "GPT-2 greedy generation mismatch");
502 require(static_cast<std::int32_t>(generated.size()) == ↪
↪ LCQI_GPT2_GENERATED_LENGTH,
503         "GPT-2 generated length mismatch");
504
505 lcqi::Gpt2KvCache cache(model.config);
506 lcqi::Gpt2ForwardResult cached_result;
507 for (const std::int32_t token : tokens) {
508     cached_result = lcqi::run_gpt2_forward_cached(model, cache, token);
509 }
510 require(cache.filled_tokens() == static_cast<std::int32_t>(tokens.size()),
511         "GPT-2 KV cache filled token count mismatch");
512 require(cache.byte_size() > 0, "GPT-2 KV cache byte size should be non-zero↪
↪ ");
513 require(cached_result.predicted_token == result.predicted_token,
514         "cached GPT-2 predicted token mismatch");
515 require(cached_result.logits.size() == result.logits.size(),
516         "cached GPT-2 logits size mismatch");
517 for (std::size_t index = 0; index < result.logits.size(); ++index) {
518     require_close(cached_result.logits[index],
519                 result.logits[index],
520                 "cached GPT-2 logit mismatch");
521 }
522
523 lcqi::Gpt2CachedGreedyDecoder decoder_with_logits(model);
524 lcqi::Gpt2ForwardResult decoder_result;
525 for (const std::int32_t token : tokens) {
526     decoder_result = decoder_with_logits.step_with_logits(token);
527 }
528 require(decoder_with_logits.filled_tokens() == static_cast<std::int32_t>(↪
↪ tokens.size()),
529         "GPT-2 optimized decoder filled token count mismatch");
530 require(decoder_with_logits.kv_cache_bytes() == cache.byte_size(),
531         "GPT-2 optimized decoder KV byte size mismatch");
532 require(decoder_result.predicted_token == result.predicted_token,
533         "optimized cached GPT-2 predicted token mismatch");
534 require(decoder_result.logits.size() == result.logits.size(),
535         "optimized cached GPT-2 logits size mismatch");

```

```

536     for (std::size_t index = 0; index < result.logits.size(); ++index) {
537         require_close(decoder_result.logits[index],
538             result.logits[index],
539             "optimized cached GPT-2 logit mismatch");
540     }
541
542     lcqi::Gpt2ExecutionOptions parallel_options;
543     parallel_options.worker_count = 2;
544     parallel_options.parallel_min_rows = 1;
545     lcqi::Gpt2CachedGreedyDecoder parallel_decoder(model, nullptr, ←
    ↪ parallel_options);
546     lcqi::Gpt2ForwardResult parallel_result;
547     for (const std::int32_t token : tokens) {
548         parallel_result = parallel_decoder.step_with_logits(token);
549     }
550     require(parallel_decoder.worker_count() == parallel_options.worker_count,
551         "parallel GPT-2 decoder worker count mismatch");
552     require(parallel_result.predicted_token == result.predicted_token,
553         "parallel cached GPT-2 predicted token mismatch");
554     require(parallel_result.logits.size() == result.logits.size(),
555         "parallel cached GPT-2 logits size mismatch");
556     for (std::size_t index = 0; index < result.logits.size(); ++index) {
557         require_close(parallel_result.logits[index],
558             result.logits[index],
559             "parallel cached GPT-2 logit mismatch");
560     }
561
562     lcqi::Gpt2CachedGreedyDecoder greedy_decoder(model);
563     std::int32_t greedy_predicted = 0;
564     for (const std::int32_t token : tokens) {
565         greedy_predicted = greedy_decoder.step(token);
566     }
567     require(greedy_predicted == result.predicted_token,
568         "optimized logits-free GPT-2 predicted token mismatch");
569
570     lcqi::Gpt2HotspotProfile hotspot_profile;
571     lcqi::Gpt2CachedGreedyDecoder profiled_decoder(model, &hotspot_profile);
572     std::int32_t profiled_predicted = 0;
573     for (const std::int32_t token : tokens) {
574         profiled_predicted = profiled_decoder.step(token);
575     }
576     require(profiled_predicted == result.predicted_token,
577         "profiled GPT-2 predicted token mismatch");
578     require(hotspot_profile.decoder_steps == static_cast<std::int64_t>(tokens.←
    ↪ size()),
579         "profiled GPT-2 decoder step count mismatch");
580     require(hotspot_profile.layer_steps ==
581         static_cast<std::int64_t>(tokens.size()) * model.config.←
    ↪ layer_count,
582         "profiled GPT-2 layer step count mismatch");
583     require(hotspot_profile.total_step_ms >= hotspot_profile.lm_head_ms,
584         "profiled GPT-2 total time should include lm head time");

```

```

585
586     const std::vector<std::int32_t> cached_generated =
587         lcqi::gpt2_generate_greedy_cached(model, tokens, 2);
588     require(cached_generated == expected, "cached GPT-2 greedy generation ↵
↵ mismatch");
589 }
590
591 void test_gpt2_byte_bpe_tokenizer() {
592     const std::filesystem::path vocab_path = temp_file_path("lcqi_gpt2_vocab.↵
↵ json");
593     const std::filesystem::path merges_path = temp_file_path("lcqi_gpt2_merges.↵
↵ txt");
594     const std::string space_marker = "\xC4\xA0";
595     const std::string vocab =
596         "{ \"Hello\":7, \" \":8, \"\" + space_marker + "world\":9, \"!\":10}";
597     const std::string merges =
598         "#version: 0.2\n"
599         "H e\n"
600         "He l\n"
601         "Hel l\n"
602         "Hell o\n" +
603         space_marker + " w\n" +
604         space_marker + "w o\n" +
605         space_marker + "wo r\n" +
606         space_marker + "wor l\n" +
607         space_marker + "worl d\n";
608     write_text_file(vocab_path, vocab);
609     write_text_file(merges_path, merges);
610
611     const lcqi::Gpt2Tokenizer tokenizer =
612         lcqi::load_gpt2_tokenizer(vocab_path, merges_path, -1, -1);
613     std::filesystem::remove(vocab_path);
614     std::filesystem::remove(merges_path);
615
616     const std::vector<std::int32_t> ids =
617         lcqi::gpt2_encode(tokenizer, "Hello, world!");
618     const std::vector<std::int32_t> expected{
619         LCQI_GPT2_TOKENIZER_HELLO_ID,
620         8,
621         9,
622         10,
623     };
624     require(ids == expected, "GPT-2 BPE ids mismatch");
625     require(lcqi::gpt2_decode(tokenizer, ids) == "Hello, world!",
626         "GPT-2 BPE decode mismatch");
627 }
628
629 void test_gpt2_loads_hf_style_tiny_checkpoint() {
630     const std::filesystem::path directory =
631         temp_file_path("lcqi_gpt2_tiny_checkpoint_dir");
632     std::filesystem::remove_all(directory);
633     std::filesystem::create_directories(directory);

```

```

634
635     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
636     write_text_file(
637         directory / "config.json",
638         R("{\"model_type\":\"gpt2\",\"n_embd\":4,\"n_head\":2,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5})");
639     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
640
641     const lcqi::Gpt2ReferenceModel loaded = lcqi::load_gpt2_from_directory(↵
↵ directory);
642     const std::vector<std::int32_t> tokens{1, 2, 3};
643     const lcqi::Gpt2ForwardResult result = lcqi::run_gpt2_forward(loaded, ↵
↵ tokens);
644     std::filesystem::remove_all(directory);
645
646     require(result.predicted_token == LCQI_GPT2_PREDICTED_TOKEN,
647         "loaded GPT-2 predicted token mismatch");
648     for (std::size_t index = 0; index < LCQI_GPT2_LOGITS.size(); ++index) {
649         require_close(result.logits[index],
650             LCQI_GPT2_LOGITS[index],
651             "loaded GPT-2 logit mismatch");
652     }
653 }
654
655 void test_gpt2_loader_rejects_bad_shape() {
656     const std::filesystem::path directory =
657         temp_file_path("lcqi_gpt2_bad_shape_dir");
658     std::filesystem::remove_all(directory);
659     std::filesystem::create_directories(directory);
660
661     const lcqi::Gpt2ReferenceModel original = lcqi::↵
↵ make_tiny_gpt2_reference_model();
662     write_text_file(
663         directory / "config.json",
664         R("{\"model_type\":\"gpt2\",\"n_embd\":5,\"n_head\":1,\"n_layer\":1,\"n_positions\"↵
↵ :8,\"n_ctx\":8,\"vocab_size\":6,\"n_inner\":8,\"layer_norm_epsilon\":0.00001,\"↵
↵ activation_function\":\"gelu_new\",\"bos_token_id\":0,\"eos_token_id\":5})");
665     write_tiny_gpt2_safetensors(directory / "model.safetensors", original);
666
667     bool threw = false;
668     try {
669         static_cast<void>(lcqi::load_gpt2_from_directory(directory));
670     } catch (const std::runtime_error&) {
671         threw = true;
672     }
673     std::filesystem::remove_all(directory);
674     require(threw, "GPT-2 loader should reject bad tensor shape");
675 }
676
677 void test_reference_rms_norm() {

```

```

678     const std::vector<float> input{3.0F, 4.0F};
679     const std::vector<float> weight{1.0F, 0.5F};
680     std::vector<float> output(2, 0.0F);
681     lcqi::rms_norm(input, weight, 0.0F, output);
682     require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
683     require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
684 }
685
686 void test_reference_rope() {
687     std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
688     lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
689     require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
690     require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
691     require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
    ↪ mismatch");
692     require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
693 }
694
695 void test_reference_kv_attention() {
696     lcqi::DecoderConfig config;
697     config.hidden_size = 4;
698     config.query_heads = 2;
699     config.kv_heads = 1;
700     config.head_dim = 2;
701     config.intermediate_size = 4;
702     config.vocab_size = 4;
703     config.max_context = 4;
704
705     lcqi::ReferenceKVCache cache(config, 1);
706     const std::vector<float> key{1.0F, 0.0F};
707     const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
    ↪ LCQI_ATTENTION_EXPECTED_1};
708     cache.append(0, 0, key, value);
709     require(cache.filled_tokens() == 1, "KV filled token count mismatch");
710     require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
711
712     const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
713     std::vector<float> output(4, 0.0F);
714     lcqi::attention_decode(config, cache, 0, 0, query, output);
715     require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0 ←
    ↪ mismatch");
716     require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1 ←
    ↪ mismatch");
717     require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0 ←
    ↪ mismatch");
718     require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1 ←
    ↪ mismatch");
719 }
720
721 void test_reference_decoder_end_to_end() {
722     const lcqi::ReferenceDecoderModel model = lcqi::←
    ↪ make_tiny_reference_decoder_model();

```



```

723     const std::vector<std::int32_t> tokens{1, 2, 3};
724     const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model ←
    ↪ , tokens);
725     require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
726             "reference decoder logits size mismatch");
727     require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
728             "reference decoder predicted token mismatch");
729     for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ←
    ↪ ++index) {
730         require_close(result.logits[index],
731                       LCQI_REFERENCE_DECODER_LOGITS[index],
732                       "reference decoder golden logit mismatch");
733     }
734 }
735
736 void test_reference_decoder_trace_contract() {
737     const lcqi::ReferenceDecoderModel model = lcqi:: ←
    ↪ make_tiny_reference_decoder_model();
738     const std::vector<std::int32_t> tokens{1, 2, 3};
739     const lcqi::ReferenceDecodeTraceResult trace =
740         lcqi::run_reference_decode_with_trace(model, tokens);
741
742     require(trace.result.predicted_token == ←
    ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
743             "reference trace predicted token mismatch");
744     require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
745
746     const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
747     require(first.name == "embedding", "first checkpoint should be embedding");
748     require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
749             "first checkpoint token position mismatch");
750     require(first.dtype == "f32", "first checkpoint dtype mismatch");
751     require(first.layout == "row_major", "first checkpoint layout mismatch");
752     require(first.shape.size() == 1 &&
753             first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
754             "first checkpoint shape mismatch");
755     require(first.stride.size() == 1 && first.stride[0] == 1,
756             "first checkpoint stride mismatch");
757
758     bool saw_logits = false;
759     bool saw_kv_slot = false;
760     for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) ←
    ↪ {
761         if (checkpoint.name == "kv_cache_key_slot") {
762             saw_kv_slot = true;
763             require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch" ←
    ↪ );
764             require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout ←
    ↪ mismatch");
765         }
766         if (checkpoint.name == "logits") {
767             saw_logits = true;

```



```

768         require(checkpoint.token_position == ↵
↵ LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
769             "logits checkpoint token position mismatch");
770         require(checkpoint.shape.size() == 1 &&
771             checkpoint.shape[0] ==
772                 static_cast<std::int32_t>(↵
↵ LCQI_REFERENCE_DECODER_VOCAB_SIZE),
773             "logits checkpoint shape mismatch");
774         require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(),
775             "logits checkpoint value size mismatch");
776         for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.↵
↵ size(); ++index) {
777             require_close(checkpoint.values[index],
778                 LCQI_REFERENCE_DECODER_LOGITS[index],
779                 "trace logits checkpoint mismatch");
780         }
781     }
782 }
783 require(saw_kv_slot, "KV checkpoint missing");
784 require(saw_logits, "logits checkpoint missing");
785 }
786
787 void test_packed_i8_kernel_matches_scalar() {
788     for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
789         const lcqi::QuantizedLinearLayer layer =
790             lcqi::make_deterministic_i8_layer(shape_case.input_size,
791                 shape_case.output_size,
792                 0.03125F);
793         const lcqi::PackedLinearI8 packed =
794             lcqi::pack_linear_i8(layer, shape_case.output_block_size);
795         const std::vector<float> input = lcqi::make_deterministic_input(layer.↵
↵ input_size);
796         std::vector<float> scalar_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
797         std::vector<float> packed_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
798         lcqi::linear_i8(layer, input, scalar_output);
799         lcqi::linear_i8_packed(packed, input, packed_output);
800         require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
801             "packed int8 kernel output differs from scalar");
802
803         const lcqi::PackedLinearI8 simd_packed =
804             lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
805         if (lcqi::linear_i8_packed_avx2_available()) {
806             std::vector<float> avx2_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
807             lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
808             require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
809                 "AVX2 int8 kernel output differs from scalar");

```

```

810     }
811 }
812 }
813
814 void test_f32_dot_kernel_matches_scalar() {
815     std::vector<float> lhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
816     std::vector<float> rhs(LCQI_F32_DOT_TEST_SIZE, 0.0F);
817     for (std::size_t index = 0; index < LCQI_F32_DOT_TEST_SIZE; ++index) {
818         lhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 19U) ←
↵ + 1) *
819             0.03125F;
820         rhs[index] = static_cast<float>(static_cast<std::int32_t>(index % 23U) ←
↵ - 11) *
821             0.0625F;
822     }
823
824     const float scalar =
825         lcqi::dot_f32_scalar_unchecked(lhs.data(), rhs.data(), lhs.size());
826     const float dispatched = lcqi::dot_f32(lhs, rhs);
827     require(std::fabs(scalar - dispatched) <= LCQI_F32_KERNEL_MAX_DIFF,
828         "F32 dot kernel output differs from scalar");
829
830     if (lcqi::dot_f32_avx2_available()) {
831         const float avx2 =
832             lcqi::dot_f32_avx2_unchecked(lhs.data(), rhs.data(), lhs.size());
833         require(std::fabs(scalar - avx2) <= LCQI_F32_KERNEL_MAX_DIFF,
834             "AVX2 F32 dot kernel output differs from scalar");
835     }
836
837     bool rejected_mismatch = false;
838     try {
839         (void)lcqi::dot_f32(std::span<const float>(lhs.data(), lhs.size() - 1U) ←
↵ , rhs);
840     } catch (const std::runtime_error&) {
841         rejected_mismatch = true;
842     }
843     require(rejected_mismatch, "F32 dot kernel accepted mismatched sizes");
844 }
845
846 } // namespace
847
848 int main() {
849     try {
850         test_tiny_mlp();
851         test_tokenizer_contract();
852         test_tokenizer_rejects_unknown_without_unk();
853         test_safetensors_manifest_contract();
854         test_safetensors_rejects_bad_offsets();
855         test_safetensors_rejects_bad_dtype_shape_size();
856         test_safetensors_reads_float_tensors();
857         test_gpt2_tiny_forward_and_generation();
858         test_gpt2_byte_bpe_tokenizer();

```

```
859     test_gpt2_loads_hf_style_tiny_checkpoint();
860     test_gpt2_loader_rejects_bad_shape();
861     test_reference_rms_norm();
862     test_reference_rope();
863     test_reference_kv_attention();
864     test_reference_decoder_end_to_end();
865     test_reference_decoder_trace_contract();
866     test_packed_i8_kernel_matches_scalar();
867     test_f32_dot_kernel_matches_scalar();
868
869     std::cout << "lcqi tests passed\n";
870     return EXIT_SUCCESS;
871 } catch (const std::exception& error) {
872     std::cerr << "lcqi test failure: " << error.what() << "\n";
873     return EXIT_FAILURE;
874 }
875 }
```

实践与代码总索引

本卷已经把实践任务、代码走读路线和完整代码清单合并到同一套内容目录。目录中不再存在一个独立的“源码卷部分”：构建入口、调用链、公共合同、tiny MLP、int8 kernel、reference decoder、GPT-2、工具脚本和测试文件都插入到相应主题之后，共同解释同一份 `books/cpu-volume-3/labs/linux_cpu_inference` 源码。

源码阅读任务

读者不需要再在实践卷和源码卷之间来回切换，也不需要先读完 16 个实践章再去末尾翻代码。实践章节提出任务，紧邻的代码走读解释完整实现，测试和 benchmark 章节给出回归证据；三者已经合并到这一本实践与代码卷里，共同构成一条可复查的工程证据链。

一、实践主题到代码走读的合并位置

- 第 1 章之后 插入“代码走读：构建入口、项目边界与调用链总图”，先把 CMake、README、模型资产、target、CLI、benchmark、trace、ledger、serving probe 和测试入口放到一张可复现地图里。
- 第 4 章之后 插入“代码走读：公共合同、Tiny MLP 与 Int8 Kernel”，把模型、推理、tokenizer、safetensors、reference decoder、GPT-2 头文件，以及 tiny MLP、基础 CLI、scalar/packed/AVX2 kernel 连起来。
- 第 10 章之后 插入“代码走读：Reference Decoder、Tokenizer、Safetensors 与 GPT-2”，把真实模型加载、byte-level BPE、safetensors、GPT-2 forward、cached KV 和 greedy generation 放在模型导入主题内部讲清楚。
- 第 16 章之后 插入“代码走读：工具、Smoke、Benchmark 与回归测试”，把 benchmark、trace、ledger、serving probe、SmolLM2 smoke、GPT-2 smoke、benchmark compare 和 CTest 回归证据集中收束。

二、最小复现命令

```
1 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
   ↪ DCMMAKE_BUILD_TYPE=Debug
2 cmake --build books/cpu-volume-3/build/lcqi-debug
3 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
4 make cpu3p-pdf
5 make cpu3p-epub
```

完成这组命令后，读者应同时检查相邻实践章节的作业结果、对应代码走读、测试断言和报告字段，确认代码、测试、报告字段和章节讲解是一条证据链。

术语表

术语	实践含义
manifest	模型资产清单，记录 tensor name、dtype、shape、offset、checksum 和 tokenizer/配置版本。
shape verifier	在 kernel 运行前检查 shape、dtype、layout、量化 descriptor 和 state edge 是否匹配。
reference decoder	以清楚正确为目标的 decoder 实现，用来生成 trace 和 golden。
oracle	比较候选实现与 reference 的判定器，可使用精确相等或误差界。
trace checkpoint	推理链路中的可复查节点，至少包含 shape、stride、dtype、layout、checksum 和摘要值。
KV cache	attention 历史 K/V 状态，跨 token 存活，不能被普通 activation planner 复用。
packing	把权重改写成更适合热循环和 SIMD 的布局。
backend	执行计划落到机器的实现，例如 scalar、AVX2、AVX-512、GPU 或库调用。
fallback	后端能力不满足时显式降级到可验证路径，并记录原因。
SLO	服务目标，例如 TTFT、TPOT、queue wait、拒绝率和取消回收时间。